CASTLES TECHNOLOGY

SATURN Series EFT-POS Terminal

Castles Android API Reference Manual

Castles Technology

Confidential

Version 4.9

Jan. 2025

MAN-S1-00-04-V4.9

Castles Technology Co., Ltd.

6F, No. 207-5, Sec. 3, Beixin Rd., Xindian District,

New Taipei City 23143, Taiwan R.O.C.

http://www.castech.com.tw

WARNING

Information in this document is subject to change without prior notice.

No part of this publication may be reproduced, transmitted, stored in a retrieval system, nor translated into any human or computer language, in any form or by any means, electronic, mechanical, magnetic, optical, chemical, manual, or otherwise, without the prior written permission of **Castles Technology Co., Ltd.**

All trademarks mentioned are proprietary of their respective owners.

Castles Technology

Contents

REVISION HISTORY 7

1. SYSTEM 23

1.1 System Application Programming Interfaces **23**
1.1.1 Public class CtSystem 23
1.1.2 Public class CtPrint 35
1.1.3 Public class CtSC 49
1.1.4 Public class CtExtraStorage 61
1.1.5 Public class CtBarcodeReader 63
1.1.6 Public class CtLoader 79
1.1.7 Public class CtMDB 87
1.1.8 Public class CtCL 97
1.1.9 Public class CtMSR 156
1.1.10 Public class CtSettings 161
1.1.11 Public class CtCradle 179
1.1.12 Public class CtDEX 181
1.1.13 Public class CtCertificate 185

2. KMS 188

2.1 KMS Framework **188**

2.2 Key Management Specifications **189**
2.2.1 Methods of Key Management 189
2.2.2 Methods of Key Injection 189
2.2.3 Key Identification and Attributes 190
2.2.3.1 Key Owner 190
2.2.3.2 Key Usage 191
2.2.3.3 Algorithm 191
2.2.3.4 Mode of Use 191
2.2.4 Single Purpose and Unique Value 192

2.3 Key Injection Methods **192**
2.3.1 Notation 192
2.3.2 Key Block Binding Method Using Key Derivation 192
2.3.2.1 Key Block Header (KBH) 193
2.3.2.2 Encrypted Confidential Data 195
2.3.2.3 MAC 196
2.3.3 Castles Proprietary Key Binding Method 196
2.3.3.1 Header 196
2.3.3.2 Key Data Block 198
2.3.3.3 Hash Block 199
2.3.3.4 CKBB_ENC and CKBB_MAC Key Derivation Method 200
2.3.3.4.1 Constant for Key Derivation 201
2.3.3.4.2 Secret Key Generation 201
2.3.3.4.3 Secret Data Generation 202
2.3.3.4.4 One Way Function 204
2.3.3.4.5 Derived Key Generation 204

2.4 SBKMS Application Programming Interfaces **207**
2.4.1 Class: CtKMS2System 208
2.4.2 Class: CtKMS2UserData 211

Castles Technology

2.4.3 Class: CtKMS2Key 212
2.4.4 Class: CtKMS2InjectionKey 214
2.4.4.1 Inherited Methods 214
2.4.4.2 Public Methods 214
2.4.5 Class: CtKMS2TR31 214
2.4.6 Class: CtKMS2CKBB 216
2.4.7 Class: CtKMS2SymmetricKey 217
2.4.8 Class: CtKMS2SymmetryKey.Inner 217
2.4.8.1 Inherited Fields 217
2.4.8.2 Inherited Methods 217
2.4.8.3 Public Methods 217
2.4.9 Class: CtKMS2FixedKey 220
2.4.10 Class:CtKMS2MKSK 222
2.4.11 CtKMS2Dukpt 223
2.4.12 Class: CtKMS2AESDukpt 224
2.4.12.1 Inherited Methods 224
2.4.12.2 Inherited Methods 224
2.4.12.3 Fields 224
2.4.12.4 Public Constructors 224
2.4.12.5 Public Methods 225
2.4.12.6 Example 226
2.4.13 Class: CtKMS2AsymmetricKey 226
2.4.13.1 Inherited Methods 226
2.4.13.2 Inherited Methods 226
2.4.14 Class: CtKMS2RSAKey 227
2.4.14.1 Inherited Methods 227
2.4.14.2 Fields 227
2.4.14.3 Public Constructors 227
2.4.14.4 Public Methods 228
2.4.14.5 Example 230
2.4.15 Class: CtKMS2ECKey 231
2.4.15.1 Inherited Methods 231
2.4.15.2 Fields 231
2.4.15.3 Public Constructors 232
2.4.15.4 Public Methods 232
2.4.15.5 Example 233
2.4.16 Class: CtKMS2CustomPINPadbyImage 235
2.4.16.1 Public Constructors 235
2.4.16.2 Public Methods 235
2.4.16.3 Example 235
2.4.17 Class: CtKMS2OfflinePIN 237
2.4.17.1 Public Constructors 238
2.4.17.2 Public Methods 238
2.4.17.3 Example 240
2.4.18 Class: CtKMS2SwipePINPad 243
2.4.18.1 Public Constructors 244
2.4.18.2 Public Methods 244
2.4.18.3 Example 245
2.4.19 Class: CtKMS2Callback 248
2.4.20 Class: CtKMS2Exception 250

2.5 CRYPTO Application Programming Interfaces 252
2.5.1 Class: CtCrypto 252
2.5.2 Class: CtCryptoException 253

3. EMV 255

3.1 CAEMV Application Programming Interfaces 255

Castles Technology

3.1.1 Class: CtEMV 257
3.1.1.1.1 Interface: IEMVSpecialEvent 276
3.1.1.1.2 Interface: IEventFirstGenACTermDecision 277
3.1.1.1.3 Interface: IEventTxnForcedOnline 278
3.1.1.1.4 Interface: IEventShowVirtualPIN 279
3.1.1.1.5 Interface: IEventGetPINDone 280
3.1.1.1.6 Interface: IEventShowVirtualPINEx 281
3.1.1.1.7 Interface: IEventOnlinePinBlockGet 282
3.1.1.1.8 Interface: IEventOfflinePINVerifyResult 283
3.1.1.1.9 Interface: IEventPINBypass 284
3.1.1.1.10 Interface: IEventForcedAcceptance 285
3.1.1.1.11 Interface: IEventAppListEx 286
3.1.1.2 Class: EMVEvent 287
3.1.1.3 Class: EMVAppInfo 290
3.1.1.4 Class: EMVOnlineResponseData 291
3.1.1.5 Class: EMVTxnData 292
3.1.1.6 Class: EMVAppListExData 293
3.1.1.7 Class: EMVGetPINFuncPara 294
3.1.1.8 Class: EMVVirtualPINPara 295
3.1.1.9 Class: EMVCandidateList 295
3.1.1.10 Class: EMVCandidateData 296
3.1.1.11 Class: EMVTxnPSERsp 297
3.1.1.12 Class: EMVCandidateDirEntry 298
3.1.1.13 Class: EMVSecureDataInfo 299
3.1.1.14 Class: TlvData 302
3.1.1.15 Class: EMVDataGetExPara 303
3.1.1.16 Class: EMVOnlinePinData 304
3.1.1.17 Class: EMVApplicationPara 305
3.1.1.18 Class: CAPublicKey 306
3.1.1.19 Class: EMVByteBufData 307
3.1.1.20 Class: EMVCardAcquisitionData 308
3.1.1.21 Class: EMVConfigData 309
3.1.1.22 Class: CAPublicKeyEx 310
3.1.2 Class: CtEMVMSR 311
3.1.2.1 Class: EMVMSRTracksLen 314
3.1.2.2 Class: EMVMSREncryptedTracks 315
3.1.2.3 Class: EMVMSRMaskedTracks 316
3.1.3 Class: CtEMVManualEntry 317
3.1.3.1 Class: EMVManualEntryEvent 322
3.1.3.2 Class: EMVManualEntryEncryptedData 325

4. EMVCL 326

4.1 CAEMVCL Application Programming Interfaces 326
4.1.1 Class: CtEMVCL 327
4.1.1.1 Interface: ISpecialEvent 341
4.1.1.1.1 Interface: IEventEMVCLLEDPicShow 342
4.1.1.1.2 Interface: IEventEMVCLAudioIndication 343
4.1.1.1.3 Interface: IEventEMVCLCancelTransaction 344
4.1.1.1.4 Interface: IEventEMVCLShowMessage 345
4.1.1.1.5 Interface: IEventEMVCLAppListV2 346
4.1.1.1.7 Interface: IEventEMVCLPreNonEMVCard 347
4.1.1.1.8 Interface: IEventEMVCLRMRDataExchange 349
4.1.1.1.9 Interface: IEventEMVCLAppListV3 350
4.1.1.1.10 Interface: IEventEMVCLAppListV4 351
4.1.1.1.11 Interface: IEventEMVCLShowVirtualPIN 352
4.1.1.1.12 Interface: IEventEMVCLShowPINDigit 353
4.1.1.1.13 Interface: IEventEMVCLGetPINDone 354

Castles Technology

4.1.1.1.14 Interface: IEventEMVCLShowPinBypass 355
4.1.1.1.15 Interface: IEventEMVCLPinBypass 356
4.1.1.1.16 Interface: IEventEMVCLCancelPinConfirm 357
4.1.1.1.17 Interface: IEventEMVCLOfflinePinVerifyResult 358
4.1.1.1.18 Interface: IEventEMVCLPinTryCounter 359
4.1.1.2 Class: EMVCLActData 360
4.1.1.3 Class: EMVCLRcDataEx 361
4.1.1.4 Class: EMVCLRcDataAnalyze 363
4.1.1.5 Class: EMVCLDetectTxnData 364
4.1.1.6 Class: EMVCLGetCardData 365
4.1.1.7 Class: EMVCLAidSetTagData 366
4.1.1.8 Class: EMVCLAidGetTagData 367
4.1.1.9 Class: EMVCLAidCombinationList 368
4.1.1.10 Class: EMVCLCAPublicKey 369
4.1.1.11 Class: EMVCLParameterData 370
4.1.1.12 Class: EMVCLUIReqData 371
4.1.1.13 Class: EMVCLSecureDataInfo 372
4.1.1.14 Class: EMVCLAppInfo 375
4.1.1.15 Class: EMVCLAppListV2 377
4.1.1.16 Class: EMVCLByteBufData 378
4.1.1.17 Class: EMVCLCardAcquisitionData 379
4.1.1.18 Class: EMVCLConfigData 380
4.1.1.19 Class: EMVCLAppListV3 381
4.1.1.20 Class: EMVCLAppListV4 382

5. EMVEDL 383

5.1 CAEMVEDL Application Programming Interfaces 383
5.1.1 Class: CtEMVEDL 384
5.1.1.1 Class: EMVEDLBinInfo 392
5.1.1.2 Class: EMVEDLEncryptedData 397

6. APPENDIX 398

6.1 Error Code List 398
6.1.1 System、EMV and security error code 398

Castles Technology

Revision History

Version	Date	Description
1.0	2018.05.04	Intial Creation
1.1	2018.05.23	1. Add classes of CtEMV below. • EMVApplicationPara • CAPublicKey 2. Add methods of CtEMV below • applicationListSet • capkSet • capkDelete • capkDeleteAll • terminalConfigSet • terminalConfigDelete • terminalConfigDeleteAll • appConfigSet • appConfigDelete • appConfigDeleteAll 3. Add error return value of CtEMV below. • d_EMVAPLIB_ERR_SC_NOT_PRESENT • d_EMVAPLIB_ERR_SECURE_DATA • d_EMVAPLIB_ERR_APP_CONFIRM_FAIL • d_EMVAPLIB_ERR_ON_APP_LIST_FAIL 4. Modify the method detail of EMVEvent. 5. Add Methods of CtEMVManualEntry. • keyinADR • keyinZIP • getAdditionalEntry 6. Add Methods of EMVManualEntryEvent. • onShowADR • onShowZIP 7. Add error return value of EMVManualEntryEvent. • d_EMVMKE_ERR_NOT_KEY_IN_DONE • d_EMVMKE_ERR_INVALID_INPUT_PARA • d_EMVMKE_ERR_ENTRY_ENC_ERR • d_EMVMKE_ERR_ENTRY_CHECKSUM_ERR • d_EMVMKE_ERR_ENTRY_GEN_LRC_ERR • d_EMVMKE_ERR_ENCRYPT_DATA_NOT_READ 8. Add class CtLoader(1.1.8) in System.
1.2	2018.07.17	1. Modify the chapter number of EMV.
1.3	2018.09.21	1. Add Methods of CtPrint • printerSearch • printerSelect 2. Add Sample of CtPrint for Saturn 7000
1.4	2018.11.16	1. Add method of CtBarcodeReader • setParameter

Castles Technology

		2. Modify the description of Public class CtLoader 3. Remove method of CtPrint • setWorkTime • setHeatLevel
1.5	2018.12.03	1. Add Methods of CtSystem • getModelName • shutdown
1.6	2019.05.10	1. Add Method of ctEMV below • txnDataSetEx • txnPreDataSet • partialTransactionSet • partialTransactionGet • getLastError 2. Add Interfaces of ctEMV below • IEventOfflinePINVerifyResult • IEventPINBypass 3. Add event id of CtEMV below • d_EVENTID_OFFLINE_PIN_VERIFY_RESULT • d_EVENTID_PIN_BYPASS 4. Add error return value of CtEMV below. • d_EMVAPLIB_ERR_INVALID_PARA • d_EMVAPLIB_ERR_OFFLINE_PIN_TIMEOUT • d_EMVAPLIB_ERR_OFFLINE_PIN_CANCEL 5. Add return value of Offline Pin Verify Result • d_PIN_RESULT_OK • d_PIN_RESULT_FAIL • d_PIN_RESULT_BLOCKED • d_PIN_RESULT_FAILBLOCKED 6. Add Method of Castles Proprietary Key Binding 7. Modify the chapter CAKMS to SBKMS 8. Add Method of CtKMS2System • setPINSound • getPINSound • setPINPADMoving • setPINScrambling • getPINScrambling 9. Modify Mothod CtKMS2Dukpt • startVirtualPin 10. Remove CtKMS2VirtualPINPad 11. Add classes of CtKMS2 below • CtKMS2RSAKey • CtKMS2Callback • CtKMS2Exception
1.7	2019.09.24	1. Add Method of CtKMS2System below • getaAllKeyResult • putMultipleCMDintoBuff_getKMS2LibVersion • putMultipleCMDintoBuff_init • putMultipleCMDintoBuff_checkAllKey

Castles Technology

		• putMultipleCMDintoBuff_checkKey • putMultipleCMDintoBuff_deleteAllKey • putMultipleCMDintoBuff_deleteKey • putMultipleCMDintoBuff_swapKey 2. Add Method of CtKMS2UserData below • putMultipleCMDintoBuff_read • putMultipleCMDintoBuff_write 3. Add Method of CtKMS2Key below • putMultipleCMDintoBuff_getInfo 4. Add Method of CtKMS2CKBB below • putMultipleCMDintoBuff_writeKey 5. Add Method of CtKMS2TR31 below • putMultipleCMDintoBuff_writeKey 6. Add Method of CtKMS2FixedKey below • startVirtualPin • putMultipleCMDintoBuff_MAC • putMultipleCMDintoBuff_dataEncrypt • putMultipleCMDintoBuff_dataDecrypt 7. Add Method detail of CtKMS2MKSK 8. Add Method of CtKMS2Dukpt below • isEnable • getisEnable • putMultipleCMDintoBuff_isEnable 9. Add Class CtKMS2AESDukpt 10. Add Method of CtKMS2RSAKey below • getCSR 11. Add Class CtKMS2CustomPINPadbyImage 12. Add Class CtKMS2MultipleCMD 13. Add Method of CtKMS2Exception below • getErrorBlockId 14. Add Method of CtEMV below. • setConfiguration with different value • setConfigurationFile • setConfigurationEx • delConfigurationFileAll • delConfigurationFile • enableNullPIN • disableNullPIN • enablePINSound • disablePINSound • setPINADMoving • setPINScrambling • preferredOrderSet • preferredOrderGet • appFilteringParaSet • appFilteringParaGet • txnCardAcquisitionRead • txnCardAcquisitionAppNumGet

Castles Technology

		• txnCardAcquisitionAppDataGet • cancelVirtualPIN • changeWDBPath • getWDBPath • getRtnString • resetWDBPath • verifyCommandSW12Get 15. Add Interface of CtEMV below • IEventForceAcceptance • IEventAppListEx 16. Add classes of CtEMV below • EMVByteBufData • EMVCardAcquisitionData • EMVConfigData 17. Add error return value of CtEMV below • d_EMVAPLIB_ERR_VIRTUAL_PIN_NOT_PROCESSING • d_EMVAPLIB_ERR_CAS_GAC2_WRONG_CID • d_EMVAPLIB_ERR_FINAL_SELECTION_SW_6283 • d_EMVAPLIB_ERR_FINAL_SELECTION_SW_NON_9000_NON_6283 • d_EMVAPLIB_ERR_FINAL_SELECTION_RSP_FORMAT_ERROR • d_EMVAPLIB_ERR_GPO_SW_6985 18. Add return value of Verify Command Status • d_VERIFY_CMD_PERFORMED • d_VERIFY_CMD_NOT_PERFORMED • d_VERIFY_CMD_APDU_FAIL 19. Add return value of EMVManualEntryEvent below • d_EMVMKE_ERR_KEY_IN_CANCEL • d_EMVMKE_ERR_PIN_PAD_MALFUCTION 20. Add Additional values in class: EMVManualEntryEncryptedData 21. Add classes of CtEMVCL below • EMVCLAppInfo • EMVCLAppListV2 • EMVCLByteBufData • EMVCLCardAcquisitionData • EMVCLConfigData 22. Add Method of CtEMVCL below • setConfiguration with different values • setConfigurationFile • setConfigurationEx • delConfigurationFileAll • delConfigurationFile • getLastError • detectCard • performCardAcquisition • getCardAcquisitionAppNum • getCardAcquisitionAppData • changeWDBPath • getWDBPath • getRtnString

Castles Technology

		• resetWDBPath 23. Add Interface of CtEMVCL below • IEventEMVCLAppListV2 24. Add return value of Audio Tone • d_EMVCL_EVENT_AUDIO_TONE_CARD_READ_OK • d_EMVCL_EVENT_AUDIO_TONE_ALERT 25. Add new chapter EMVEDL 26. Add Method of CtEMVEDL below • initialize • setWhiteListFile • deleteWhiteListFile • getWhiteListMatchedBinInfo • setEncMode • setEncKey • setCipherMethod • setICV • setSessionKey • setTransportKey • requestSensitiveEmvData • requestToken • getEncryption • changeWDBPath • getWDBPath • getRtnString • resetWDBPath 27. Add classes of CtEMVEDL below • EMVEDLBinInfo • EMVEDLEncryptedData
1.8	2019.10.18	1. Add Method of CtPrint below • lineEx 2. Clarify return code for Method txnPerform of CtEMV 3. Clarify parameters for Method dataGet of CtEMV 4. Clarify parameters for Method txnDataSet of CtEMV 5. Add Method of CtEMVCL below • listAllCAPKId • setReadyToReadBehavior 6. Add Interface of CtEMVCL below • IEventEMVCLNonEMVCard • IEventEMVCLPreNonEMVCard
1.9	2019.10.29	1. Modify Method of CtPrint below • drawText 2. Modify description for CtKMS2MKSK • setSK 3. Remove Method of CtKMS2RSAKey • getCSR 4. Modify parameter name on CtKMS2RSAKey.setInputData.
2.0	2019.12.11	1. Update Class of CtEMV below • EMVSecureDataInfo

Castles Technology

		2. Update Class of CtEMVCL below • EMVCLSecureDataInfo 3. Remove CtExtraApp Class
2.1	2020.01.14	1. Modify Class of CtEMV below • EMVSecureDataInfo
2.2	2020.02.21	1. Add Method of CtSystem below • gotoSleep
2.3	2020.04.01	1. Add parameters of CtSystem below • ID_EMV • ID_EMVCL
2.4	2020.04.16	1. Add Method of CtSystem below • sc_led • cl_led
2.5	2020.05.15	1. Add Interface of CtEMVCL below • IEventEMVCLRMRDataExchange 2. Modify the value of the MAC Key key usage value 3. Modify description for CtKMS2FixedKey • setPinInfo 4. Modify description for CtKMS2RSAKey • SetInputData 5. Add Method of CtCrypto below • des_ecb • aes_ecb 6. Delete Method of CtKMS2AESDukpt below • getisEnable • putMultipleCMDintoBuff_isEnable
2.6	2020.07.27	1. Add parameter of CtEMV below • onTxnResult 2. Add Return description of CtEMV below • getLastError 3. Remove class CtKMS2MultipleCMD and related function.code 4. Add class CtKMS2InjectionKey, CtKMS2SymmetryKey, CtKMS2SymmetryKey.Inner and CtKMS2AsymmetryKey. 5. Add Method of CtSystem • getFactorySNEx
2.7	2020.08.18	1. Add Method of CtMDB below • open • close • configSet • configGet • respondSend • commandGet • fwinfoGet
2.8	2020.08.26	1. Add Method of CtSC below • powerOff • resetEMV • resetSO

Castles Technology

		• warmResetEMV • warmResetSO • commonReset • getATRLen • getCardType • sendAPDU • getRLen 2. Add Method of CtCL below • powerOn • powerOff • ppPolling • GetPPPolling • ppActivation • ppAPDU • GetPPAPDURxLen • ppRemoval • felicaPolling • felicaReadWrite • GetRxLen 3. Add Method of CtMSR below • getTk1Len • getTk2Len • getTk3Len • getTk1Err • getTk2Err • getTk3Err • read • getLastErr 4. Add Method of CtSettings below • getApn • setApn
2.9	2020.11.04	1. Add Method of CtCL below • powerStatus • getStatus
3.0	2020.12.02	1. Add description of below • CtKMS2System.swapKey • CtKMS2UserData 2. Remove Method of CtEMV of below • setLed • startIdleLEDBehavior • stopIdleLEDBehavior
3.1	2020.12.16	1. Add Method of CtSettings below • openBluetooth • closeBluetooth • bondBtDevice • connectBtDevice • disconnectBtDevice • getSystemTime • setSystemTime

Castles Technology

		• getSystemZone • setSystemZone • set24HourFormat • setAutoRebootTime • getBrightness • setBrightness • getScreenTimeout • setScreenTimeout • setNavigation • getMediaVolumn • setMediaVolumn • getRingVolumn • setRingVolumn • getAlarmVolumn • setAlarmVolumn • getNotificationVolumn • setNotificationVolumn • openWifi • closeWifi • setDhcpWifi • setStaticWifi • getWifiConfig • setEth • setDHCPEth • setStaticEth • getEthConfig • getDebugModeState • setDebugModeState • getBatteryThreshold • setBatteryThreshold • setLoginPassword • setUserPassword • setFactoryPassword • setDefaultApp • getDefaultApp • factoryReset • virtualKeyboard • setLanguage • setNTPSetting • screenLock • getUserPasswordState • setUserPasswordState • setNTPServer • setNTPIntervalTime • setNTPAutoUpdateTime • openBatteryPercentage 2. Modify Method Detail of CtMDB • responseSend • commandGet 3. Modify Method Detail of CtSystem

Castles Technology

		• shutdown 4. Modify Incorrect function name: • updateNTPSetting
3.2   3.3

		Modify inherited Methods Modify Example • 5. CtKMS2TR31 Change to Capter 2.4.5 Modify extends Remove all Method Summery and Method Detail Add inherited Methods Modify inherited Methods Modify Example • 6. CtKMS2SysmmetryKey Modify Field Summery remove Constant Field Values • 7. CtKMS2Symetry.Inner Add setInputData Add startVirtualPin Add startVirtualPin Add startVirtualPin Remove startVirtualPin Remove startVirtualPin Remove startVirtualPin Modify inherited Methods • 8. CtKMS2FixedKey Modify extends Add Field Summary Remove Method Summery Modify inherited Methods Add inherited Methods Remove Constant Field Values Remove Method Detail Modify Example • 9. CtKMS2MKSK Modify extends Modify inherited Fields Modify inherited Methods Remove Method Detail of dataEncrypt, dataDecrypt, generateMAC, startVirtualPin Modify Example • 10. CtKMS2Dukpt Modify extends Modify inherited Methods Add Method of chooseResponseKey Remove getisEnable Modify Method Detail of useCurrentKey Modify Example • 11. CtKMS2AE2Dukpt Modify inherited Fields Remove Method of isUseCurrentKey, getKSN, isEnable Modify inherited Methods Remove Constant Field Values Add Method Detail of setDerivedKeyType, setDerivedKeyUsage.

Castles Technology

		Modify Example • 12. CtKMS2AsymmetryKey Modify inherited Method • 13. CtKMS2RSAKey Modify extends Modify ingerited Methods Remove methods of setKeyLocation, setKeyVersion, setKeyAttribute, setInputData, encryptByPublicKey, decryptByPrivateKey. Modify Example • 14. CtKMS2CustomerPINPadbyImage Modify Example • 15. CtKMS2Callback Modify Example • 16. Remove 2.4.17 Interface: IKMS2CallBack • 17. CtKMS2Exception Remove extends Add Field Summary Add method of getErrorBlock Add Method Detail of getErrCode, getMessage, toString Remove methods of showStatus, getError, log. Remove inherited Method Modify Example 7. Modify contends of 2.5 CRYPTO Application Programming Interfaces. • 1. CtCrypto Modify Fieild Summary Removed Constant Field Values. Modify Method Detail of rng • 2. Add 2.5.2 Class: CtCryptoException
3.5	2021.02.26	1. Modify Patameter Content of CtLoader • onUpdateInstallStatus 2. Add note of CtEMV • startTransactionEx 3. Remove Method of CtSystem • setULDPassword • getULDPasswordHash 4. Modify Parameters and Command Description of CtMDB • responesSend • commandGet 5. Add Method of CtMDB • icConfigSet • icConfigGet 6. Modify Parameters of CtSettings • setEth 7. Modify Methods of CtSystem • setStatusBar 8. Modify Content of KMS 2.2.3 Key Identification andAttributes
3.6	2021.03.15	1. Add Method of CtDEX • open

Castles Technology

		• close • send • getRawData • getFieldData • getData
3.7	2021.03.25	1. Add Method of CtCL • typeAActiveFromIdle • RATS • APDU • getCSNLen • getATSLen • getRLen • tclTransparentRaw • tclSetTimeout • GetRLastBits • mifareAUTH2Ex • mifareREADBLOCK • mifareWRITEBLOCK • mifareINCREASE • mifareDECREASE • mifareRESTORE • mifareTRANSFER 2. Modify Example of CtSettings
3.8	2021.04.01	1. Add Method of CtSystem • screenCap 2. Add Method detail of CtKMS2RSAKey • getCSR
3.9	2021.09.07	1. Add classes of CtEMV below. • EMVCLAppListV3 • EMVCLAppListV4 2. Add Interface of CtEMVCL • IEventEMVCLAppListV3 • IEventEMVCLAppListV4 • IEventEMVCLShowVirtualPIN • IEventEMVCLShowPINDigit • IEventEMVCLGetPINDone • IEventEMVCLShowPinBypass • IEventEMVCLPinBypass • IEventEMVCLCancelPinConfirm • IEventEMVCLOfflinePinVerifyResult • IEventEMVCLPinTryCounter 3. Add Field Detail of CtEMVCL • d_EMVCL_EVENTID_APP_LIST_V3 = 0x15; • d_EMVCL_EVENTID_APP_LIST_V4 = 0x1D; • d_EMVCL_EVENTID_SHOW_VIRTUAL_PIN_EX = 0xC2; • d_EMVCL_EVENTID_SHOW_PIN_DIGIT = 0x16; • d_EMVCL_EVENTID_GET_PIN_DONE = 0x17; • d_EMVCL_EVENTID_SHOW_PIN_BYPASS = 0x18; • d_EMVCL_EVENTID_PIN_BYPASS = 0x19;

Castles Technology

		• d_EMVCL_EVENTID_CANCEL_PIN_CONFIRM = 0x1A; • d_EMVCL_EVENTID_OFFLINE_PIN_VERIFY_RESULT = 0x1B; • d_EMVCL_EVENTID_PIN_TRY_COUNTER = 0x1C;
4.0	2022.08.10	1. Add public class CtCertificate • getEncodedCaCertificate • getUserCaAliases • deleteCaCertificate • installCaCert 2. Add Method of CtCL • getATQBLen • getRep • setPowerModeParm • getPowerModeParm • mifareLOADKEY • mifareAUTHEx • REQA • WUPA • antia • selecta • typeAHalt • typeAActiveFromHalt • REQB • WUPB • attrib • haltb • typeBActive • typeBActiveEx • RATSEx • deselect • nakPoll • tclTransparent • tclTransparentEx • ecpPollingV2 3. Add Method of CtSystem • beep • powerSource • setPowerKey • simSelect • setAutoRebootTime • setBackLight 4. Add Method of CtSettings • getApn • getCurrentApn • selectApn • setDhcpWifi • setStaticWifi • setNetworkSelectionMode • getAvailableNetworkList • setAirplaneMode • setLockPattern

Castles Technology

		• getCurrentLockPattern • factoryReset • setBluetoothBle • setBluetoothJustwork 5. Add Method of CtEMVManualEntry • setEXPKeyInFormat • setPANBackspaceMode • setEXPBackspaceMode • setCSCBackspaceMode • setADRBackspaceMode • setZIPBackspaceMode 6. Modify description for maskEXPDigits1To4 of CtEMVManualEntry 7. Add parameter for below events of CtEMVManualEntry • onShowPAN • onShowEXP • onShowCSC • onShowADR • onShowZIP 8. Add Method of CtPrint • getHeatLevel • setHeatLevel • connect • disconnect • setAlign • printPageEx • savePNG • saveJPEG • saveBmp • setConfig 9. Add Method of CtSC • SyncICCReset • sc44x2ReadMainMemory • sc44x2WriteMainMemory • sc44x2ReadProtectionMemory • sc44x2SetProtectionBit • sc44x2ReadSecurityMemory • sc44x2ChangePSC • sc44x2VerifyPSC 10. Modify 4.1.1.3 Class: EMVCLRcDataEx 11. Modify Value of public int getStatus() Add Error Code List 12. Remove method of CtfingerprintManager
4.1	2022.12.28	1. Modify 4.1.1 Class: CtEMVCL • public int setParamete 2. Add Method of CtCradle • hideCradleWifi
4.2	2023.08.31	1. Modify 1.1.2 Class: CtPrint • setHeatLevel

Castles Technology

		2. Remove Method of CTSC • commonReset • getATRLen 3. Add 1.1.2 Class: CtPrint • setPrintCallback • Interface PrintCallback 4. Add 1.1.7 Public class CtMDB • icConfigSet • icConfigGet 5. Update Class of CtEMV below • EMVSecureDataInfo 6. Update Class of CtEMVCL below • EMVCLSecureDataInfo 7. Add class of CtKMS2OfflinePIN below • setMovingRange • setPinControl • setTapbehavior • setCallback • startSwipePin • startVirtualPin • getPinFromKbd 8. Add class of CtKMS2SwipePINPad below • setMode • setPINPad • setControl • setTypeface • showPINPad • removePINPad 9. Add class of CtKMS2Callback below • onGetBlindModeStatus • onSwipePINPad • onNonOperation 10. Modify example of CtKMS2Callback
4.3	2023.11.03	1. Add 1.1.5 Public class CtBarcodeReader below • open • close • read • readCancel • configSet • configGet 2. Add 1.1.9 Public class CtMSR below • dualRead • dualGetLastErr
4.4	2023.12.12	1. Modify 2.3.2. Key Block Binding Method Using Key Derivation 2. Modify 2.3.2.2 Encrypted Confidential Data 3. Modify 2.3.2.3 MAC 4. Modify 2.3.3.2 Key Data Block 5. Modify 2.4 SBKMS Application Programming Interfaces

Castles Technology

		6. Modify 2.4.5 Class: CtKMS2TR31
4.5	2023.12.14	1. Change name. Change ANSI TR-31 to ANSI X9.143
4.6	2024.01.03	1. Modify getHetlevel 1~6 >1~5 2. Modify 2.3.2. Key Block Binding Method Using Key Derivation
4.7	2024.05.28	1. Modify 2.4 SBKMS Application Programming Interfaces 2. Update 4.1.1 Class: CtEMVCL • public int performTransactionEx 3. Update 4.1.1.3 Class: EMVCLRcDataEx • Add description for chipData and additionalData 4. Add 1.1.8 Public class CtCL • cts512bReqt • cts512bIdentify • cts512bSelect • cts512bRead • cts512bMultiRead • cts512bWrite • cts512bUpdate • cts512bDesactivate 5. Add 1.1.1 Public class CtSystem • getStorageDeviceStatus • screenCap
4.8	2024.09.13	1. Modify the typo in chapter 6. 2. Add 1.1.1 Public class CtSystem • selectSimCardBySlot 3. Add 1.1.10 Public class CtSettings • setMobileDataEnabled • enableEnhancedLteMode • isEnhancedLteEnabled • setTalkBack
4.9	2025.01.02	1. Add 1.1.8 Public class CtCL • cmStartEmulation • cmStopEmulation • cmGetStatus • cmReadApduCommand • cmWriteApduCommand 2. Add 2.4.15 Public class CtKMS2ECKey 3. Add 2.4.20 Class: CtKMS2Exception • PERMISSION_DENIED

Castles Technology

1. System

1.1 System Application Programming Interfaces

1.1.1 Public class CtSystem

The **CtSystem** class is used to provide the System setting.

Constructor Summary	
public	CtSystem() Constructs a binder connected with service.

Method Summary	
String	getmoduleVersion(int moduleName) throws CtSystemException Get module version
int	setAutoRebootTime(int hour, int minute) Set time when will you want to reboot.
int	setAutoRebootTime(int hour, int minute, boolean onOff) Set Auto Reboot Time
Int	• setSystemTime(int year, int month, int day, int hour, int minute, int second) • setSystemTime(Date date) Set the system time
int	getSerialNumber(byte[] buf) Retrieve terminal CPU hard-coded unique serial number.
String	getKeyHash(int keyIndex) throws CtSystemException Retrieve the hash value of a key.
Int	getDeviceModel() throws CtSystemException Retrieve the model of the terminal.
Int	setBackLight(boolean onOff) Turn on / off the backlight of LCD or keypad.
int	setBackLight(int device, boolean onOff) Turn on / off the backlight.
int	setTimeZone(String timeZone) Set Time Zone
int	updateNTPSetting(String server, String port, boolean autoUpdate, int time) sync NTP server time
byte[]	getFactorySN() throws CtSystemException Retrieve terminal factory serial number. It matches with the stick on the back part of the terminal.
String	getFactorySNEx() throws CtSystemException Retrieve terminal factory serial number in String. It matches with the stick on the back part of the terminal.
String	getModelName() Retrieve the model of the terminal
int	shutdown(byte bmode) Execute device reboot or shutdown
Int	gotoSleep() Go to sleep.
int	sc_led() Turn on / off Smart Card LED Light.
int	cl_led() Turn on / off Contactless LED Light.
boolean	changePassword(String oldPassword1, String oldPassword2, String newPassword1, String newPassword2) Change systemPanel password
String	getDefaultApp() Get default app package name
void	setDefaultApp(String packname) Set default app
boolean	setLanguage(String language, String country) Set system language
int	setNavBarIcon(boolean Home, boolean Back, boolean Search) Set navigation bar icon
int	setNavigationbar(boolean enable) Set navigation bar enable/disable
int	setStatusBar(int enable) Set status bar enable/disable
int	killAPP(in String pkgName) Disable app with package name

Castles Technology

boolean	setWifiAp(String SSID, String Passwd, boolean isOpen) Set connect wifi (Android 6 not support)
boolean	wifiApEnable(boolean isOpen) Open/Close wifi (Android 6 not support)
int	screenCap(String path) Save the screen file as a PNG.
int	beep() Make a beep sound
int	powerSource(byte[] bSrc) Check the power source
void	setPowerKey(boolean enable) Set Power key Lock/Unlock
int	simSelect(int sim) Select Sim card
int	getStorageDeviceStatus() Get /sys/kernel/debug/mmc0/mmc0:0001 status
int	screenCap(String path) Screencap the current screen to /sdcard/test.png
boolean	boolean selectSimCardBySlot(int slotIndex) Switch to the designated SIM card slot.

Constructor Detail

public CtSystem()
Create a CtSystem object and connect with a binder to invoke service.

Method Detail

public String getModuleVersion(int moduleName) throws CtSystemException

Description
Security chip version information for the specific system component.

Parameters
moduleName : Get the module version

ID_BOOTSULD 0
ID_CRYPTO_HAL 1
ID_LINUX_KERNEL 3
ID_SECURITY_KO 4
ID_SYSUPD_KO 5
ID_KMS 6
ID_CADRV_KO 7
ID_CAUSB_KO 8
ID_LIBCAUART_SO 9
ID_LIBCAUSBH_SO 10
ID_LIBCAMODEM_SO 11
ID_LIBCAETHERNET_SO 12
ID_LIBCAFONT_SO 13
ID_LIBCALCD_SO 14
ID_LIBCAPRT_SO 15
ID_LIBCARTC_SO 16
ID_LIBCAULDPM_SO 17
ID_LIBCAPMODEM_SO 18
ID_LIBCAGSM_SO 19
ID_LIBCAEMVL2_SO 20
ID_LIBCAKMS_SO 21
ID_LIBCAFS_SO 22
ID_LIBCABARCODE_SO 23
ID_CRADLE_MP 24
ID_LIBTLS_SO 25
ID_LIBCLVW_SO 26
ID_LIBCTOSAPI_SO 27
ID_SAM_KO 28
ID_CLVWM_MP 29
ID_ROOTFS 30
ID_BIOS 31
ID_CIF_KO 32
ID_CLDRV_KO 33
ID_TMS 34
ID_ULDPM 35
ID_SC_KO 36
ID_EMV = 37;
ID_EMVCL = 38;

Castles Technology

Return Value
String to store the version information.

Throws:
CtSystemException- if the object obtains the Security chip version error code.

public int setAutoRebootTime (int hour, int minutes)

Description
When you want to reboot the time

Parameters
Set the reboot time
hour : 0~23
minute: 0~59
hour = -1 or minute = -1 , close auto reboot

Return Value

Constants	Value
Success	0
Fail	!= 0

public int setAutoRebootTime(int hour, int minute, boolean onOff)

Description
Set Auto Reboot Time

Parameters
[IN]hour
Set auto reboot hour
[IN]minute
Set auto reboot min
[IN]onOff
true : on
false : off

Return Value

d_OK	0x0000
d_SYSTEM_INVALID_PARA	0x0201

Example

```
void system()
{
        CtSystem sys = new CtSystem();
        sys.setAutoRebootTime(0,1,true);
}
```

public int setSystemTime(int year, int month, int day, int hour, int minute, int second)
public int setSystemTime(Date date)

Description
Set the system time and security chip system time

Parameters
year : year - 1900
month : the month between 0-11
day : the day of the month between 1-31
hour : the hours between 0-23
minute : the minutes between 0-59
second : the seconds between 0-59
date: a Date object

Return Value

Constants	Value
Success	0
Fail	!= 0

public int getSerialNumber(byte[] buf)

Description
Retrieve terminal CPU hard-coded unique serial number.

Parameters

Castles Technology

[OUT] buf : Buffer to store the serial number.

Return Value

Constants	***Value***
d_OK	0000h
d_SYSTEM_INVALID_PARA	0201h
d_SYSTEM_HALT_FAULT	0202h
d_SYSTEM_SYS_PARA_ABSENT	0203h

Note
Minimum output buffer size is 16 bytes.

public String getKeyHash(int keyIndex) throws CtSystemException

Description
Retrieve the hash value of a key.

Parameters
[IN] keyIndex*:* Key index.

Return Value
string to the hash value of the key (SHA-1, 20 bytes)

Throws:
CtSystemException- if the object obtains the Security chip version error code.

public int getDeviceModel()

Description
Retrieve the model of the terminal.

Return Value

Constants	**Value**
d_MODEL_VEGA9000	1
d_MODEL_VEGA7000	2
d_MODEL_VEGA5000	3
d_MODEL_VEGA5000S	4
d_MODEL_VEGA3000	5
d_MODEL_VEGA3000P	6
d_MODEL_QPAD	7
d_MODEL_MP200	8
d_MODEL_S1	12

Throws:
CtSystemException- if the object obtains the Security chip version error code.

public int setBackLight(boolean onOff)

Description
Turn on / off the backlight.

Parameters
[IN] onOff
ON True
OFF False

public int setBackLight(int device, boolean onOff)

Description
Turn on / off the backlight.

Parameters
[IN] device
0 : LCD

[IN] onOff
true : on
false : off

Return Value

d_OK	0x0000
d_BKLIT_NOT_SUPPORT	0x1F01
d_BKLIT_INVALID_PARA	0x1F02
d_SYSTEM_INVALID_PARA	0x0201

Example
void system()

Castles Technology

```
{
        CtSystem sys = new CtSystem();
        sys.setBackLight(0,true);
}
```

public int setTimeZone(String timeZone)

Description
Set TimeZone

Parameters
[IN]timeZone : Android TimeZone

Return Value

Constants	Value
Success	0
Fail	!= 0

public int updateNTPSetting(String server, String port, boolean autoUpdate, int time)

Parameters
[IN] server: NTP Server
[IN] port : Server port
[IN] autoUpdate : open automatic updates (true or false)
[IN]time : How many minutes to update again

Return Value

Constants	Value
Success	0
Fail	!= 0

public byte[] getFactorySN() throws CtSystemException

Description
Retrieve terminal factory serial number. It matches with the stick on the back part of the terminal.

Return Value
Buffer to store the factory serial number. E.g: 0 0 0 0 0 1 0 5 0 0 1 1 1 8 0 =

Throws:
CtSystemException- if the object obtains the Security chip version error code.

Public String getFactorySNEx() throws CtSystemException

Description
Retrieve terminal factory serial number in string. It matches with the stick on the back part of the terminal.

Return Value
String of Factory serial number.

Throws:
CtSystemException- if the object obtains the Security chip version error code.

public String getModelName()

Description
Retrieve the model of the terminal.

Return Value
String to store the model information, Ex Saturn1000, Saturn1000_Elite.

public int shutdown(byte bmode)

Description
Execute device reboot or shutdown.

Parameters
[IN] bmode
d_PWR_REBOOT reboot
d_PWR_POWER_OFF shutdown

Return Value

Constants	Value
Success	0
INVALID_PARA	0x0601

public int gotoSleep()

Castles Technology

Description
The device into sleep.

Parameters
none

Exception

Constants	**String**
Success	d_OK 0
Fail	d_POWER_INSUFFICIENT_RESOURCE 0x3003

public int sc_led(int onOff)

Description
Turn on / off Smart Card LED Light.

Parameters
[IN] onOff
ON 1
OFF 0

Return Value

Constants	**Value**
d_LED_INVALID_PARA	0E01h
d_LED_HAL_FAULT	0E02h
d_BKLIT_NOT_SUPPORT	1F01h
d_BKLIT_INVALID_PARA	1F02h

public int cl_led(int onOff)

Description
Turn on / off Contactless LED Light.

Parameters
[IN] onOff
ON 1
OFF 0

Return Value

Constants	**Value**
Success	0
Fail	!= 0

public boolean changePassword(String oldPassword1, String oldPassword2, String newPassword
1, String newPassword2)

Description
Change systemPanel password

Parameters
oldPassword 1 First old password
oldPassword 2 Second old password
newPassword1 First new password
newPassword2 Second new password

Return Value
change state

public String getDefaultApp()

Description
Get default app package name

Parameters
none

Return Value
default app Default app package name

public void setDefaultApp(String packname)

Description
Set default app package name

Parameters
package name App package name

Return Value

Castles Technology

none

public boolean setLanguage(String language,String country)

Description
Set System Language

Parameters
language Set system language
country Set system country

Returns
state Change language state

public int setNavBarIcon (boolean Home, boolean Back, boolean Search)

Description
Set Navigation Bar hide or open

Parameters
home Set navigation home button state
back Set navigation back button state
search Set navigation search button state

Returns
set state 0 -> set success
-1 -> set fail

public int setNavigationbar(boolean enable)

Description
Set Navigation Bar hide or open

Parameters
state enable/disable

Return Value
set state 0 -> set success
-1 -> set fail

public int setStatusBar(int enable)

Description
Set status bar enable/disable

Parameters
[IN]enable
0 : disable
1 : enable

Return Value
d_OK 0x0000
d_SYSTEM_INVALID_PARA 0x0201

Example
```
void system()
{
	CtSystem sys = new CtSystem();
	sys.setStatusBar(1);
}
```

public int killAPP(String pkgName)

Description
Disable app with package name

Parameters
package name App package name

Return Value
state

public boolean setWifiAp(String SSID, String Passwd, boolean isOpen)

Description
Set connect wifi

Castles Technology

Parameters
SSID Connect wifi ssid
password Connect wifi password
state Open/Close wifi

Return Value
set state

public boolean wifiApEnable(boolean isOpen)

Description
Set wifi enable/disable

Parameters
state true -> Open wifi
false -> Close wifi

Return Value
change state

public int screenCap(Sting Path)

Description
Save the screen file as png.

Parameters
[IN] path
File absolute path. The path requires under "/data/data/pub/" folder.

Return Value

Constants	**Value**
Success	0
Fail	

public int beep()

Description
Make a beep sound

Parameters
None

Return Value
Success 0

Example
```
CtSystem ctSystem = new CtSystem();
ctSystem.beep();
```

public int powerSource(byte[] bSrc)

Description
Check the power source

Parameters
The bSrc is output parameters
bSrc = 0 -> Power by DCJack
bSrc = 1 -> Power by cradle
bSrc = 2 -> Power by battery
bSrc = 3 -> Power by usb

Return Value
Success 0

Example
```
CtSystem ctSystem = new CtSystem();
ctSystem.powerSource(new byte[]{});
```

public void setPowerKey(boolean enable)

Description
Set Power key Lock/Unlock

Parameters
lock/unlock true -> unlock power key
false -> lock power key

Return Value
None

Castles Technology

Example
CtSystem ctSystem = new CtSystem();
ctSystem.setPowerKey(true);

public int simSelect(int sim)

Description
Select Sim card

Parameters
sim card 0 -> sim 1
1 -> sim 2

Return Value
ret 0 -> success

Note
Only support Android6 & Android7 FW

Example
CtSystem ctSystem = new CtSystem();
ctSystem.simSelect(1);

public int getStorageDeviceStatus()

Description
Get /sys/kernel/debug/mmc0/mmc0:0001 status

Parameters
None

Return Value
int
0: Could not access the information
1: 0%-10% device life time used
2: 10%-20% device life time used
3: 20%-30% device life time used
4: 30%-40% device life time used
5: 40%-50% device life time used
6: 50%-60% device life time used
7: 60%-70% device life time used
8: 70%-80% device life time used
9: 80%-90% device life time used
10: 90%-100% device life time used
11: Exceeded its maximum estimated device life time used

Note
None

Example
private CtSystem cst;

cst = new CtSystem();

@Override
public void onClick(View view) {
Log.d(TAG, "onClick: " + cst.getStorageDeviceStatus());

}

public int screenCap(String path)

Description
Screencap the current screen to /sdcard/test.png

Parameters
[IN] path
Input “/sdcard/test.png”

Return Value
d_OK

Note
None

Example
public class MainActivity extends AppCompatActivity {

Castles Technology

```
        CtSystem sys = new CtSystem();

        ret = sys.screenCap(path);
        Log.d(TAG, "screenCap = " + ret);
}
```

boolean selectSimCardBySlot(int slotIndex)

Description
Switch to the designated SIM card slot.

Parameters
slotIndex sim card slot(From 0)

Return Value
d_OK

Note
Do not use with simselect.
Do not support the model with Android 6 & 7.

Example

```
import CTOS.CtSettings;
import android.util.Log;

--------------------

CtSystem mCtSystem = new CtSystem();

boolean ret = false;
try {
    ret = mCtSystem.selectSimCardBySlot(0);
    Log.d(TAG, String.format("CtSystem$selectSimCardBySlot(%b), result=%b ..... ok", 0, ret));
} catch (Exception e) {
    e.printStackTrace();
}
```

Example

```
package com.system_demo;

import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;

import CTOS.CtSystem;
import CTOS.CtSystemException;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        String TAG = "Saturn1000 System demo";

        CtSystem system = new CtSystem();

        String setTimeZone = "Asia/Taipei";
        String passwordString = "123456";
        String server = "tock.stdtime.gov.tw";
        String port = "123";
        int SULD = 0;
        int Kernel = 3;
        int Rootfs = 30;
        int BIOS = 31;
        byte buf[] = new byte[16];
        byte bmode = 0; // 0, 1
```

Castles Technology

```
        // setAutoRebootFlag
        int h = 1; // 0 ~ 23
        int m = 0; // 0 ~ 59
        Log.d(TAG, "setAutoRebootTime return = " + Integer.toString(system.setAutoRebootTime(h, m)));

        // getModuleVersion
        try {
            Log.d(TAG, "SULD Version = " + system.getModuleVersion(SULD));
            Log.d(TAG, "KERNEL Version = " + system.getModuleVersion(Kernel));
            Log.d(TAG, "ROOTFS Version = " + system.getModuleVersion(Rootfs));
            Log.d(TAG, "BIOS Version = " + system.getModuleVersion(BIOS));
        } catch (CtSystemException e) {
            e.showStatus();
        }

        // setSystemTime
        Log.d(TAG, "setSystemTime return = " + Integer.toString(system.setSystemTime(2017, 1, 16, 13, 13, 13)));

        // setSystemTime
        SimpleDateFormat simpleDateFormat = new SimpleDateFormat("yyyy/MM/dd HH:mm:ss");
        try {
            Date date = simpleDateFormat.parse("2017/01/07 23:49:00");
            Log.d(TAG, "setSystemTime return = " + Integer.toString(
                    system.setSystemTime(date)));
        } catch (ParseException e) {

        }

        // getSerialNumber
        Log.d(TAG, " return = " + Integer.toString(system.getSerialNumber(buf)));
        Log.d(TAG, String.format("%x%x%x%x%x%x%x%x%x%x%x%x%x%x%x%x"
                , buf[0], buf[1], buf[2], buf[3], buf[4], buf[5], buf[6]
                , buf[7], buf[8], buf[9], buf[10], buf[11], buf[12]
                , buf[13], buf[14], buf[15]));

        // getKeyHash
        try {
            Log.d(TAG, "getKeyHash = " + system.getKeyHash(0));
            Log.d(TAG, "getKeyHash = " + system.getKeyHash(1));
        } catch (CtSystemException e) {
            e.showStatus();
        }

        // getDeviceModel
        try {
            Log.d(TAG, String.format("Model = %d", system.getDeviceModel()));
        } catch (CtSystemException e) {
            e.showStatus();
        }

        // setBackLight
        Log.d(TAG, " return = " + Integer.toString(system.setBackLight(true)));

        //

        Log.d(TAG, " return = " + Integer.toString(system.setULDPassword(passwordString)));

        // setTimeZone
        Log.d(TAG, " return = " + Integer.toString(system.setTimeZone(setTimeZone)));

        // updateNTPSetting
        Log.d(TAG, " return = " + Integer.toString(system.updateNTPSetting(server, port, true, 1)));

        // getFactorySN
        try {
            buf = system.getFactorySN();
            Log.d(TAG, String.format("FactorySN = %c%c%c%c%c%c%c%c%c%c%c%c%c%c%c%c", buf[0],
buf[1], buf[2]
                    , buf[3], buf[4], buf[5], buf[6], buf[7], buf[8], buf[9]
                    , buf[10], buf[11], buf[12], buf[13], buf[14], buf[15]));
```

Castles Technology

```
        } catch (CtSystemException e) {
            e.showStatus();
        }

        // getModelName
        Log.d(TAG, String.format("Model = %s", system.getModelName()));

        // shutdown
        Log.d(TAG, String.format("return = %X", system.shutdown(bmode)));
    }
}
```

```
import CTOS.CtSystem;
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        CtSystem system = new CtSystem();
        system.gotoSleep();
    }
}
```

Castles Technology

1.1.2 Public class CtPrint

The **CtPrint** class is used to provide the printer.

Constructor Summary	
public	CtPrint() Constructs a binder connected with service.

Method Summary	
void	initPage(int h) Initialize the painting
void	printPage() Print buffer data
int	printPageEx Print out the picture
void	• drawText(int x, int y, String text, int fontSize, int lineWidth, int color, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, String familyName) • drawText(int x, int y, String text, int fontSize, int lineWidth, int color, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, Typeface font) • drawText(int x, int y, String text, int Xsize, int Ysize, int lineWidth, int color, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, Typeface font) • drawText (int x, int y, String text, int fontSize, int lineWidth, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, String familyName) • drawText(int x, int y, String text, int fontSize, int lineWidth, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText) • drawText(int x, int y, String text, int fontSize, int lineWidth, String familyName) • drawText(int x, int y, String text, int fontSize, int lineWidth) • drawText(int x, int y, String text, int fontSize, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, String familyName) • drawText(int x, int y, String text, int fontSize, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText) • drawText(int x, int y, String text, int fontSize, String familyName) • drawText(int x, int y, String text, int fontSize) • drawText(int x, int y, String text, int fontSize, int lineWidth, int color, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, boolean reverseText, String familyName) • drawText(int x, int y, String text, int fontSize, int lineWidth, int color, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, boolean reverseText, Typeface font) • drawText(int x, int y, String text, int Xsize, int Ysize, int lineWidth, int color, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, boolean reverseText, Typeface font) • drawText(int x, int y, String text, int fontSize, int lineWidth, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, boolean reverseText, String familyName) • drawText(int x, int y, String text, int fontSize, int lineWidth, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, boolean reverseText) • drawText(int x, int y, String text, int fontSize, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, boolean reverseText, String familyName) • drawText(int x, int y, String text, int fontSize, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, boolean reverseText) Draw the text
void	drawImage(Bitmap bitmap, int x, int y) Draw the bitmap using the specified matrix

Castles Technology

void	drawImage(Bitmap bitmap, Rect src, Rect dst) Draw the specified bitmap, scaling/translating automatically to fill the destination rectangle.
Bitmap	Bitmap encodeToBitmap Create barcode
int	roll(int Dots) Roll out (Saturn 1000 Only)
int	status() Get printer status (Saturn 1000 Only)
JSONArray	printerSearch() throws Exception Search WIFI printer. (Saturn 7000 Only)
Int	lineEx () Print user's buffer data
void	printerSelect(JSONObject printer) throws Exception Select WIFI printer. (Saturn 7000 Only)
int	getHeatLevel() throws CtPrinterException Get the heat level of the printer. The higher the level, the darker the printing quality.
void	setHeatLevel(int level) Set the heat level of the printer. The higher the level, the darker the printing quality
void	connect(int conn_interface) throws InterruptedException, ExecutionException, Exception Connect to a specific Wi-Fi cradle
void	disconnect() throws IOException Disconnect from Wi-Fi cradle
void	setAlign(int align) throws InterruptedException, ExecutionException, Exception Align the string of the drawtext
void	savePNG Save the bitmap as png format file
void	saveJPEG Save the bitmap as JPEG format file
void	saveBmp Save the bitmap as bmp format file
void	setConfig Set cradle config
int	setPrintCallback Sets print callback object to print service. After the printer service has finished printing, it will call the callback method to notify the listener the pinter buffer, width, and height.
Interface	PrintCallback The interface is used to implement callback function for print. The print uses the callback method to notify the application to get the printer buffer, width, and height.

Constructor Detail

public CtPrint()
Create a CtPrint object and connect with a binder to invoke service.

Method Detail

public void initPage(int h)

Description
Create a painting set height

Parameters
h : painting set height

Note :
height = (h % 8 != 0 ? (h / 8 + 1) * 8 : h);
Printer buffer size = ((384 * height) / 8) byte
please do not over

public void printPage()

Description
Print out the picture

Note :
height = (h % 8 != 0 ? (h / 8 + 1) * 8 : h);
Printer buffer size = ((384 * height) / 8) byte

Castles Technology

please do not over

public int printPageEx()

Description
Print out the picture

Parameters
None

Return Value

d_OK	0x00
d_PRINTER_HEAD_OVERHEAT	0x02
d_PRINTER_PAPER_OUT	0x03

Example

```
void Print()
{
        CtPrint print = new CtPrint();
        String print_font = "Test Printer";
        int print_y = 36;
        int Currently_high = 5;

        print.initPage(320);
        print.drawText(0, print_y + Currently_high, print_font, print_y, 1,0xFF000000, true, (float) 0, false,
        false,Typeface.DEFAULT);
        print.printPageEx();
}
```

public void drawText (int x, int y, String text, int fontSize, int lineWidth, boolean fakeBoldText, float textSkewX, boolean underlineText, boolean strikeThruText, boolean reverseText, String familyName)

Description
This function can be written text in bold italic font size and height

Parameters
x : The position of the left side of the bitmap being drawn
y : The position of the top side of the bitmap being drawn
text : To write the words
lineWidth : Set the paint's stroke width, used whenever the paint's style is Stroke or StrokeAndFill
fakeBoldText : True to set the fakeBoldText bit in the paint's flags, false to clear it
textSkewX : Set the paint's skew factor in X for drawing text
underlineText : True to set the underlineText bit in the paint's flags, false to clear it
strikeThruText : True to set the strikeThruText bit in the paint's flags, false to clear it
reverseText : True to set the reverseText bit in the paint's flags, false to clear it
familyName : Set the font EX : monospace, serif, sans ...

public void drawImage(Bitmap bitmap, int x, int y)

Description
Draw the specified bitmap, with its top/left corner at (x,y), using the specified paint, transformed by the current matrix.

ParametersBitmap: The bitmap to be drawn
x : The position of the left side of the bitmap being drawn
y : The position of the top side of the bitmap being drawn

Note:
If the paint contains a mask filter that generates a mask which extends beyond the bitmap's original width/height (e.g. BlurMaskFilter), then the bitmap will be drawn as if it were in a Shader with CLAMP mode. Thus the color outside of the original width/height will be the edge color replicated.
If the bitmap and canvas have different densities, this function will take care of automatically scaling the bitmap to draw at the same density as the canvas.

public void drawImage(Bitmap bitmap, Rect src, Rect dst)

Description
Draw the specified bitmap, scaling/translating automatically to fill the destination rectangle. If the source rectangle is not null, it specifies the subset of the bitmap to draw.

Parameters
Bitmap: The bitmap to be drawn
Rect: May be null. The subset of the bitmap to be drawn
Rect: The rectangle that the bitmap will be scaled/translated to fit into

Note:

Castles Technology

If the paint contains a maskfilter that generates a mask which extends beyond the bitmap's original width/height (e.g. BlurMaskFilter), then the bitmap will be drawn as if it were in a Shader with CLAMP mode. Thus the color outside of the original width/height will be the edge color replicated.
This function. Ignores the density associated with the bitmap. This is because the source and destination rectangle coordinate spaces are in their respective densities, so must already have the appropriate scaling factor applied.

public Bitmap encodeToBitmap(String contents, BarcodeFormat format, int img_width, int img_height)

Description
Create barcode

Parameters
contents: Barcode contents
BarcodeFormat:

CODE_128
Code 128 1D format.
CODE_39
Code 39 1D format.
CODE_93
Code 93 1D format.
EAN_13
EAN-13 1D format.
EAN_8
EAN-8 1D format.
QR_CODE
QR Code 2D barcode format.

img_width: barcode image width
mg_height: barcode image height

Return Value

Constants	**Value**
Success	Bitmap
Fail	NULL

Note:
Please note that the length of barcode EAN-13 and EAN-8

public int roll(int dots)

Description
Roll the paper of printer without printing.

Parameters
dots : Number of dot line to roll. **(** 0.125mm)

public int status()

Description
Get the current status of the printer.

public JSONArray printerSearch() throws Exception

Description
Search for printers via WIFI.

Parameters
[OUT]JSONArray
This JSONArray contains printer list with name, IP and port information.

Exception

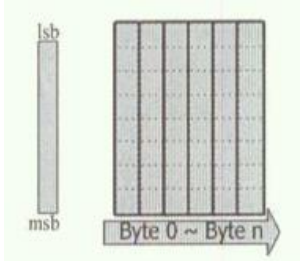
Constants	**String**
Success	none
Fail	“Service Unconnected” “printSearch NULL”

public int lineEx()

Description
Print out the user’s buffer

Note :
The buffer format is as follows:

Castles Technology



height = (h % 8 != 0 ? (h / 8 + 1) * 8 : h);
Printer buffer size = ((384 * height) / 8) byte
please do not over

public void printerSelect(JSONObject printer) throws Exception

Description
Select one of printer by the JSONObject group.

Parameters
[IN] printer
Selected printer JSONObject.

Exception

Constants	**String**
Success	none
Fail	"Service Unconnected"
	"JSONObject Error"

public int getHeatLevel() throws CtPrinterException

Description
Get the heat level of the printer.
The higher the level, the darker the printing quality.

Parameters
None

Return Value
The depth of the printing
Heatlevel 1~5

Throws:
CtSystemException - Return the printer error code.

Constants	Value
d_OK	0000h
d_PRINTER_HEAD_OVERHEAT	1602h
d_PRINTER_PAPER_OUT	1603h
d_PRINTER_BARCODE_GENERATE_ERR	1604h
d_PRINTER_BARCODE_CONTENT_ERR	1605h
d_PRINTER_BARCODE_CONTENT_LEN_ERR	1606h
d_PRINTER_BARCODE_OUTSIDE_PAPER	1607h
d_PRINTER_PARA_ERR	1608h
d_PRINTER_PIC_FORMAT_NOT_SUPPORT	1609h
d_PRINTER_PIC_OPEN_FAILED	160Ah
d_PRINTER_PIC_OVER_SIZE	160Bh
d_PRINTER_HAL_FAULT	160Ch
d_PRINTER_NOT_SUPPORT	1612h
d_PRINTER_LANGUAGE_NOT_SUPPORT	1613h
d_PRINTER_MOTOR_OVERHEAT	1614h

Note
Get the default level is 3.

Example

```
import CTOS.CtPrint;
import CTOS.CtPrinterException;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
```

Castles Technology

```
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);
            String TAG = "Saturn1000 Print demo";

            CtPrint print = new CtPrint();

            int iheatlevel;

try{
                        iheatlevel= print.getHeatLevel();
                    }catch (CtPrinterException e) {
                        e.showStatus();
                            System.out.println(e.getError());
                    }
        }
}
```

public void setHeatLevel(int level)

Description
Set the heat level of the printer.
The higher the level, the darker the printing quality.

Parameters
[IN] level
Set printer heat level

Return Value
None

Example

```
import CTOS.CtPrint;
import CTOS.CtPrinterException;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        String TAG = "Saturn1000 Print demo";

        CtPrint print = new CtPrint();

                print.setHeatLevel(3);

    }
}
```

public void connect(int conn_interface) throws InterruptedException, ExecutionException, Exception

Description
Connect to a specific Wi-Fi cradle.

Parameters
[IN] conn_interface
useless

Return Value
None

Note
Only for S1EP cradle

Example

```
void Print()
{
        String print_font = "Test Printer";
        int print_y = 36;
        int Currently_high = 5;

        try {
                print.connect(0);
        }catch (Exception e){
                e.printStackTrace();
        }
```

Castles Technology

```
            print.initPage(320);
            print.setAlign(print.PRT_ALIGNLEFT);
            print.drawText(0, print_y + Currently_high, print_font,    print_y, 1,0xFF000000, true, (float) 0, false,
            false,Typeface.DEFAULT);
            print.printPage();
            try {
                print.disconnect();
            }catch (Exception e){
                e.printStackTrace();
            }

}
```

public void disconnect() throws IOException

Description
Disconnect from Wi-Fi cradle

Parameters
None

Return Value
None

Note
Only for S1EP cradle

Example

```
void Print()
{
            String print_font = "Test Printer";
            int print_y = 36;
            int Currently_high = 5;

            try {
                        print.connect(0);
            }catch (Exception e){
                        e.printStackTrace();
            }

            print.initPage(320);
            print.setAlign(print.PRT_ALIGNLEFT);
            print.drawText(0, print_y + Currently_high, print_font,    print_y, 1,0xFF000000, true, (float) 0, false,
            false,Typeface.DEFAULT);
            print.printPage();

            try {
                        print.disconnect();
            }catch (Exception e){
                        e.printStackTrace();
            }

}
```

public void setAlign(int align) throws InterruptedException, ExecutionException, Exception

Description
Align the string of the drawtext.

Parameters
[IN] align

PRT_ALIGNNORMAL	0
PRT_ALIGNLEFT	1
PRT_ALIGNCENTER	2
PRT_ALIGNRIGHT	3

Return Value
None

Example

```
void Print()
{
            String print_font = "Test Printer";
            int print_y = 36;
            int Currently_high = 5;

            print.initPage(320);
            print.setAlign(print.PRT_ALIGNLEFT);
```

Castles Technology

```
            print.drawText(0, print_y + Currently_high, print_font,    print_y, 1,0xFF000000, true, (float) 0, false,
            false,Typeface.DEFAULT);
            Currently_high += print_y;
            print.setAlign(print.PRT_ALIGNCENTER);
            print.drawText(0, print_y + Currently_high, print_font,    print_y, 1,0xFF000000, true, (float) 0, false,
            false,Typeface.DEFAULT);
            Currently_high += print_y;
            print.setAlign(print.PRT_ALIGNRIGHT);
            print.drawText(0, print_y + Currently_high, print_font,    print_y, 1,0xFF000000, true, (float) 0, false,
            false,Typeface.DEFAULT);

            print.printPage();
}
```

public void savePNG(String fileName)

Description
Save the bitmap as png format file

Parameters
[IN]fileName
The absolute path of the file

Return Value
none

Example

```
void Print()
{
            CtPrint print = new CtPrint();
            String print_font = "Test Printer";
            int print_y = 36;
            int Currently_high = 5;

            print.initPage(320);
            print.drawText(0, print_y + Currently_high, print_font, print_y, 1,0xFF000000, true, (float) 0, false,
            false,Typeface.DEFAULT);
            print.savePNG("/sdcard/Pictures/Test.png");
}
```

public void saveJPEG(String fileName)

Description
Save the bitmap as JPEG format file

Parameters
[IN]fileName
The absolute path of the file

Return Value
none

Example

```
void Print()
{
            CtPrint print = new CtPrint();
            String print_font = "Test Printer";
            int print_y = 36;
            int Currently_high = 5;

            print.initPage(320);
            print.drawText(0, print_y + Currently_high, print_font, print_y, 1,0xFF000000, true, (float) 0, false,
            false,Typeface.DEFAULT);
            print.saveJPEG("/sdcard/Pictures/Test.jpg");
}
```

public void saveBmp(String fileName)

Description
Save the bitmap as bmp format file

Parameters
[IN]fileName
The absolute path of the file

Return Value

Castles Technology

none

Example

```
void Print()
{
	CtPrint print = new CtPrint();
	String print_font = "Test Printer";
	int print_y = 36;
	int Currently_high = 5;

	print.initPage(320);
	print.drawText(0, print_y + Currently_high, print_font, print_y, 1,0xFF000000, true, (float) 0, false,
	false,Typeface.DEFAULT);
		Currently_high += print_y;
	print.saveBmp("/sdcard/Pictures/Test.bmp");
}
```

public void setConfig(int tag, byte[] value, int len) throws ExecutionException, InterruptedException, Exception

Description

Set cradle config

Parameters

[IN]tag

CONFIG_CUTPAPER	1

[IN]value

VALUE_CUTFULL	1
VALUE_CUTPARTIAL	2

[IN]len

The value buffer length

Return Value

none

Example

```
void Print()
{
	CtPrint print = new CtPrint();
	String print_font = "Test Printer";
	int print_y = 36;
	int Currently_high = 5;
	byte[] value;
	value = new byte[1];

	try {
		print.connect(0);
	}catch (Exception e){
		e.printStackTrace();
	}
	value[0] = print.VALUE_CUTFULL;
	try {
	print.setConfig(print.CONFIG_CUTPAPER, value, value.length);
	}catch (Exception e){
		e.printStackTrace();
	}

	print.initPage(320);
	print.setAlign(print.PRT_ALIGNLEFT);
	print.drawText(0, print_y + Currently_high, print_font,    print_y, 1,0xFF000000, true, (float) 0, false,
	false,Typeface.DEFAULT);
	print.printPage();

	try {
		print.disconnect();
	}catch (Exception e){
		e.printStackTrace();
	}
}
```

public int setPrintCallback(CtPrint.PrintCallback callback)

Description

Sets print callback object to print service. After the printer service has finished printing, it will call the callback method to notify the listener the pinter buffer, width, and height.

Parameters

[IN] callback

Castles Technology

PrintCallback object

Return Value

OK	0
NOT_SUPPORT	0xFF70

Example

```
public class MainActivity extends AppCompatActivity implements CtPrint.PrintCallback{
    private static final String TAG = "Print";
    private static final int WHITE = 0xFFFFFFFF;
    private static final int BLACK = 0xFF000000;

    int[] bmp_pixels;
    CtPrint prt = new CtPrint();
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        prt.setPrintCallback(this);
    }

    private void setPixel(int x,int y, int width, int color) {
        bmp_pixels[x + y * width] = color;
    }

    public void onPrintBuffer(int width, int height, byte[] Buf) {
        int x, y, z;
        int Start = 0;
        byte [] mask = {(byte)0x01, (byte)0x02, (byte)0x04, (byte)0x08, (byte)0x10, (byte)0x20, (byte)0x40, (byte)0x80};

        Log.d(TAG,String.format("height = %d", height));
        Log.d(TAG,String.format("width = %d", width));
        Log.d(TAG,String.format("Buf.length = %d", Buf.length));

        bmp_pixels = new int[width * height];

        Bitmap bmp = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_8888);
        for(y = 0; y < height ; y += 8) {
            for(x = 0; x < width; x ++) {
                for (z = 0 ; z < 8 ; z ++) {
                    if((Buf[Start] & mask[z]) == mask[z]){
                        setPixel(x, y + z, width, BLACK);
                    }else{
                        setPixel(x, y + z, width, WHITE);
                    }
                }
                if(Start == Buf.length) {
                    break;
                }else {
                    Start++;
                }
            }
        }
        bmp.setPixels(bmp_pixels, 0, width, 0, 0, width, height);
        File externalFilesDir = getExternalFilesDir(null);
        try {
            bmp.compress(Bitmap.CompressFormat.JPEG, 100, new FileOutputStream(new File(externalFilesDir   +
"/test.jpg")));
        } catch (Exception ex) {
            Log.d(TAG,String.format("jpg fail"));
        }
    }
}
```

Interface PrintCallback

Description

The interface is used to implement callback function for print. The print uses the callback method to notify the application to get the printer buffer, width, and height.

Parameters

[OUT] width
The width of the image. (width is 384)
[OUT] height
The height of the image. (Buf.length/width*8)
[OUT] Buf

Castles Technology

The buffer of the printer

Return Value
none

Example

```
public class MainActivity extends AppCompatActivity implements CtPrint.PrintCallback{
    private static final String TAG = "Print";
    private static final int WHITE = 0xFFFFFFFF;
    private static final int BLACK = 0xFF000000;

    int[] bmp_pixels;
    CtPrint prt = new CtPrint();
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        prt.setPrintCallback(this);
    }

    private void setPixel(int x,int y, int width, int color) {
        bmp_pixels[x + y * width] = color;
    }

    public void onPrintBuffer(int width, int height, byte[] Buf) {
        int x, y, z;
        int Start = 0;
        byte [] mask = {(byte)0x01, (byte)0x02, (byte)0x04, (byte)0x08, (byte)0x10, (byte)0x20, (byte)0x40, (byte)0x80};

        Log.d(TAG,String.format("height = %d", height));
        Log.d(TAG,String.format("width = %d", width));
        Log.d(TAG,String.format("Buf.length = %d", Buf.length));

        bmp_pixels = new int[width * height];

        Bitmap bmp = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_8888);
        for(y = 0; y < height ; y += 8) {
            for(x = 0; x < width; x ++) {
                for (z = 0 ; z < 8 ; z ++) {
                    if((Buf[Start] & mask[z]) == mask[z]){
                        setPixel(x, y + z, width, BLACK);
                    }else{
                        setPixel(x, y + z, width, WHITE);
                    }
                }
                if(Start == Buf.length) {
                    break;
                }else {
                    Start++;
                }
            }
        }
        bmp.setPixels(bmp_pixels, 0, width, 0, 0, width, height);
        File externalFilesDir = getExternalFilesDir(null);
        try {
            bmp.compress(Bitmap.CompressFormat.JPEG, 100, new FileOutputStream(new File(externalFilesDir +
"/test.jpg")));
        } catch (Exception ex) {
            Log.d(TAG,String.format("jpg fail"));
        }
    }
}
```

Methods:
onPrintBuffer: notify listener after print completely.

Field Detail	
Constant Field	Value
d_PRINTER_HEAD_OVERHEAT	0x02
d_PRINTER_PAPER_OUT	0x03
d_PRINTER_BARCODE_GENERATE_ERR	0x04
d_PRINTER_BARCODE_CONTENT_ERR	0x05
d_PRINTER_BARCODE_CONTENT_LEN_ERR	0x06

Castles Technology

d_PRINTER_BARCODE_OUTSIDE_PAPER	0x07
d_PRINTER_PARA_ERR	0x08
d_PRINTER_PIC_FORMAT_NOT_SUPPORT	0x09
d_PRINTER_PIC_OPEN_FAILED	0x0A
d_PRINTER_PIC_OVER_SIZE	0x0B
d_PRINTER_HAL_FAULT	0x0C
d_PRINTER_NOT_SUPPORT	0x12
d_PRINTER_LANGUAGE_NOT_SUPPORT	0x13

Example

```
//For Saturn 1000
package com.print_demo;

import android.graphics.Bitmap;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;

import CTOS.CtPrint;


public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        String TAG = "Saturn1000 Print demo";

        CtPrint print = new CtPrint();

        String print_font;
        int print_x = 0;
        int print_y = 36;
        int Currently_high = 20;
        int ret = 0;
        Bitmap bitmap = null;

        print.initPage(200);
        bitmap = print.encodeToBitmap("12ASDFSS34", print.QR_CODE, 150, 150);
        print.drawImage(bitmap, 0, 0);
        print.printPage();

        ret = print.roll(10);
        Log.d(TAG, String.format("Roll ret = %d", ret));

        ret = print.status();
        Log.d(TAG, String.format("status ret = %d", ret));

        print.initPage(100);

        print_font = "PRINT TESTING";
        print.drawText(0, print_y + Currently_high, print_font, print_y, 1, true, (float) 0, false, false);
        Currently_high += print_y;

        //Bitmap image = BitmapFactory.decodeFile("/data/" + "test.jpg");
        //print.drawImage(image, 0, Currently_high);

        print.printPage();
    }
}
```

```
//For Saturn 7000
package castles.printer_sample;

import android.content.res.AssetManager;
import android.graphics.Bitmap;
import android.graphics.Typeface;
import android.os.IBinder;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.util.Log;
import android.content.Intent;
import android.content.Context;
import java.io.IOException;

import android.view.View;
```

Castles Technology

```
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.content.ServiceConnection;

import org.json.JSONObject;
import org.json.JSONArray;
import org.json.JSONException;

import CTOS.CtPrint;

public class MainActivity extends AppCompatActivity implements View.OnClickListener
{
	String TAG = "CtPrint";

	Context mContext = this;
	CtPrint print = new CtPrint();

	private Button button, button1, button2;
	private TextView textView, textView1, textView2;

	JSONObject json_obj = new JSONObject();
	JSONArray json_arr = new JSONArray();

	@Override
	protected void onCreate(Bundle savedInstanceState)
	{
		super.onCreate(savedInstanceState);
		setContentView(R.layout.activity_main);

		textView = (TextView) findViewById(R.id.textView);
		textView1 = (TextView) findViewById(R.id.textView1);
		textView2 = (TextView) findViewById(R.id.textView2);

		button = (Button)findViewById(R.id.button);
		button1 = (Button)findViewById(R.id.button1);
		button2 = (Button)findViewById(R.id.button2);

		button.setOnClickListener(this);
		button1.setOnClickListener(this);
		button2.setOnClickListener(this);

		String print_font;
		int print_x = 0;
		int print_y = 36;
		int Currently_high = 20;
		int ret = 0;

		Bitmap bitmap = null;

		Typeface font;
		AssetManager assetManager = getAssets();
		font = Typeface.createFromAsset(assetManager, String.format("fonts/%s", "bokutachi.otf"));

		print.initPage(200);

		print_font = "PRINT TESTING";

		print.drawText(0, print_y + Currently_high, print_font, print_y, 1,0xFF000000, true, (float) 0, false,
false,font);
		Currently_high += print_y;
		print.drawText(0, print_y + Currently_high, print_font, print_y/2, print_y, 1,0xFF000000, true, (float) 0, false,
false,font);
		Currently_high += print_y;
		print.drawText(0, print_y + Currently_high, print_font, print_y*2, print_y, 1,0xFF000000, true, (float) 0, false,
false,font);
	}

	@Override
	public void onClick(View v)
	{
		switch (v.getId())
		{
			case R.id.button:
				try
				{
					json_arr = print.printerSearch();
```

Castles Technology

```
                                        }catch(Exception e)
                                        {
                                                Log.d(TAG, e.getMessage());
                                                textView.setText(e.getMessage());
                                        }

                                        Log.d(TAG, "AP json_arr = "+json_arr);

                                        String json_printname = "";
                                        String json_printip = "";
                                        String json_printport = "";

                                        try{
                                                json_obj = json_arr.getJSONObject(0);
                                                json_printname = json_obj.getString("PrinterName");
                                                json_printip = json_obj.getString("PrinterIP");
                                                json_printport = json_obj.getString("PrinterPort");

                                        }catch(JSONException e)
                                        {
                                                e.printStackTrace();
                                                Log.d(TAG, "AP json_obj.getString fail");
                                        }
                                        Log.d(TAG, "AP json_printname = "+json_printname);
                                        Log.d(TAG, "AP json_printip = "+json_printip);
                                        Log.d(TAG, "AP json_printport = "+json_printport);

                                        break;

                                case R.id.button1:
                                        try{
                                                print.printerSelect(json_obj);
                                        }catch(Exception e){
                                                Log.d(TAG, e.getMessage());
                                                textView1.setText(e.getMessage());
                                        }
                                        break;

                                case R.id.button2:
                                        try{
                                                print.printPage();
                                        }catch(Exception e){
                                                Log.d(TAG, e.getMessage());
                                                textView2.setText(e.getMessage());
                                        }
                                        break;

                        }
                }
}
```

Castles Technology

1.1.3 Public class CtSC

The **CtSC** class is used to the SC card.

Constructor Summary	
public	CtSC() Constructs a binder connected with service.

Method Summary	
int	status(int ID) Check the ICC status.
int	getStatus() Get result of status function.
int	powerOff(int ID) Turn off the power of ICC.
int	resetEMV(int ID, int Volt, byte[] ATR) Power on the ICC and return the ATR which meets the requirement of EMV specification.
int	resetISO(int ID, int Volt, byte[] ATR) Power on the ICC and return the ATR which meets ISO-7816 specification.
int	warmResetEMV(int ID, int Volt, byte[] ATR) Warm reset the ICC and return the ATR which meets the requirement of EMV specification.
int	warmResetISO(int ID, int Volt, byte[] ATR) Warm reset the ICC and return the ATR and return the ATR which meets ISO-7816 specification.
int	getCardType() Get card type of power functions.
int	sendAPDU(int ID, byte[] SBuf, byte[] RBuf) Send an APDU command to ICC and get the response from ICC.
int	getRLen() Get RBuf length of sendAPDU() parameter.
int	SyncICCReset(byte[] Buf, int[] Len) Reset a SLE44x2, SLE44x4 SLE44x6 or SLE44x8 memory card.
int	sc44x2ReadMainMemory(int Addr, byte[] Buf, int[] Len) Read the data from main memory.
int	sc44x2WriteMainMemory(int Addr, byte[] Buf) Write data to main memory.
int	sc44x2ReadProtectionMemory(byte[] Buf) Read the data of protection memory. The function read all four bytes at once.
int	sc44x2SetProtectionBit(int Addr) Set the protection bit in protection memory.
int	sc44x2ReadSecurityMemory(byte[] Buf) Read out the data of security memory. The function read all four bytes at once.
int	sc44x2ChangePSC(byte[] Buf) Change the PSC. Must successfully verify the PSC before changing it.
int	sc44x2VerifyPSC(byte[] Buf) Verify the PSC.

Constructor Detail
public CtSC() Create a CtSC object and connect with binder to invoke service

Method Detail
public int status(int ID) **Description** Check the ICC status. **Parameters** [IN] ID Socket number of ICC module. **d_SC_USER 0** **d_SC_SAM1 1** **d_SC_SAM2 2** **d_SC_SAM3 3** **d_SC_SAM4 4** **Return Value**

Castles Technology

Constants	**Value**	
Success	0	
Fail	Fail	0x14xxh (Field Detail)

Note
Call this function to check whether the ICC is present or absent and whether the ICC is active or de-active.
d_SC_USER refers to the chip (ISO7816) reader of the terminal.

public int getStatus()

Description
Get status of status function.

Parameters
None

Return Value
Constants Value
ICC insert / not insert 1 --> 1 means the first bit
ICC active / not active 2 --> 2 means the second bit

If the ICC card is insert and active, will return 3(11).
If the ICC card is insert but not active, will return 2(10).

Note
For all SAM card, it is assumed always exist, so that the **d_MK_SC_PRESENT** is always set.

public int powerOff(int ID)

Description
Turn off the power of ICC.

Parameters
[IN] ID
Socket number of ICC module.

d_SC_USER	**0**
d_SC_SAM1	**1**
d_SC_SAM2	**2**
d_SC_SAM3	**3**
d_SC_SAM4	**4**

Return Value

Constants	**Value**	
Success	0	
Fail	Fail	0x14xxh (Field Detail)

Note
d_SC_USER refers to the chip (ISO7816) reader of the terminal.

public int resetEMV(int ID, int Volt, byte[] ATR)

Description
Power on the ICC and return the ATR which meets the requirement of EMV specification.

Parameters
[IN] ID
Socket number of ICC module.

d_SC_USER	**0**
d_SC_SAM1	**1**
d_SC_SAM2	**2**
d_SC_SAM3	**3**
d_SC_SAM4	**4**

[IN] Volt
Voltage level to apply to ICC.

d_SC_5V	**1**
d_SC_3V	**2**
d_SC_1_8V	**3**

[OUT] ATR
Buffer to store ATR return from ICC.

Castles Technology

Return Value

Constants	Value	
Success	0	
Fail	Fail	0x14xxh (Field Detail)

Note
This function performs the ICC reset and parse the ATR according to the EMV specification. If the ATR do not meet the requirement of EMV specification, the function will return with error.

public int resetISO(int ID, int Volt, byte[] ATR)

Description
Power on the ICC and return the ATR which meets ISO-7816 specification.

Parameters
[IN] ID
Socket number of ICC module.

d_SC_USER	**0**
d_SC_SAM1	**1**
d_SC_SAM2	**2**
d_SC_SAM3	**3**
d_SC_SAM4	**4**

[IN] Volt
Voltage level to apply to ICC.

d_SC_5V	**1**
d_SC_3V	**2**
d_SC_1_8V	**3**

[OUT] ATR
Buffer to store ATR return from ICC.

Return Value

Constants	Value	
Success	0	
Fail	Fail	0x14xxh (Field Detail)

Note
This function performs the ICC reset and parse the ATR according to the ISO-7816 specification. Generally ISO-7816 specification has less restriction than EMV specification.

public int warmResetEMV(int ID, int Volt, byte[] ATR)

Description
Warm reset the ICC and return the ATR which meets the requirement of EMV specification.

Parameters
[IN] ID
Socket number of ICC module.

d_SC_USER	**0**
d_SC_SAM1	**1**
d_SC_SAM2	**2**
d_SC_SAM3	**3**
d_SC_SAM4	**4**

[IN] Volt
Voltage level to apply to ICC.

d_SC_5V	**1**
d_SC_3V	**2**
d_SC_1_8V	**3**

[OUT] ATR
Buffer to store ATR return from ICC.

Return Value

Constants	Value	
Success	0	
Fail	Fail	0x14xxh (Field Detail)

Note

Castles Technology

Before calling warmResetEMV(), power on the ICC by resetEMV().
This function performs the ICC reset and parse the ATR according to the EMV specification. If the ATR do not meet the requirement of EMV specification, the function will return with error.

public int warmResetISO(int ID, int Volt, byte[] ATR)

Description
Warm reset the ICC and return the ATR and return the ATR which meets ISO-7816 specification.

Parameters
[IN] ID
Socket number of ICC module.

d_SC_USER	**0**
d_SC_SAM1	**1**
d_SC_SAM2	**2**
d_SC_SAM3	**3**
d_SC_SAM4	**4**

[IN] Volt
Voltage level to apply to ICC.

d_SC_5V	**1**
d_SC_3V	**2**
d_SC_1_8V	**3**

[OUT] ATR
Buffer to store ATR return from ICC.

Return Value

Constants	**Value**		
Success	0		
Fail		Fail	0x14xxh (Field Detail)

Note
Before calling warmResetISO(), power on the ICC by resetISO().
This function performs the ICC reset and parse the ATR according to the ISO-7816 specification. Generally ISO-7816 specification has less restriction than EMV specification.

public int getCardType()

Description
Get card type of reset functions.

Parameters
none

Return Value

Constants	**Value**
Type of the ICC in the socket.	
d_SC_TYPE_T0	0
T=0 asynchronous ICC	
d_SC_TYPE_T1	1
T=1 asynchronous ICC	

public int sendAPDU(int ID, byte[] SBuf, byte[] RBuf)

Description
Send an APDU command to ICC and get the response from ICC.

Parameters
[IN] ID
Socket number of ICC module.

d_SC_USER	**0**
d_SC_SAM1	**1**
d_SC_SAM2	**2**
d_SC_SAM3	**3**
d_SC_SAM4	**4**

Castles Technology

[IN] SBuf
Buffer storing APDU command to send to ICC.

[OUT] RBuf
Buffer to store the response from ICC.

Return Value

Constants	**Value**	
Success	0	
Fail	Fail	0x14xxh (Field Detail)

Note
Before calling warmResetISO(), power on the ICC by resetISO(). This function performs the ICC reset and parse the ATR according to the ISO-7816 specification. Generally ISO-7816 specification has less restriction than EMV specification.

---

public int getRLen()

Description
Get RBuf length of sendAPDU() parameter.

Parameters
none

Return Value
The number of bytes of RBuf length.

d_SC_USER refers to the chip (ISO7816) reader of the terminal.

---

public int SyncICCReset(byte[] Buf, int[] Len)

Description:
Reset a SLE44x2, SLE44x4 SLE44x6 or SLE44x8 memory card.

Parameters:
[OUT] Buf
Buffer to store the response data from a memory card.
[OUT] Len
Length of response data.

Return Value:

d_OK	0000h
d_MC_NOT_PRESENT	1501h
d_MC_NOT_ACTIVATED	1502h
d_MC_MUTE	1504h
d_MC_PARITY_ERROR	1505h
d_MC_EDC_ERROR	1506h
d_MC_BUFFER_OVERRUN	1507h
d_MC_HW_ERROR	1508h
d_MC_DEACTIVATED_PROTOCOL	1509h
d_MC_POWER_FAILED	1510h
d_MC_INVALID_PARAMETER	1511h
d_MC_SOCKET_NOT_EXIST	1530h
d_MC_BUF_TOO_SMALL	1531h
d_MC_COMM_ERROR	15A3h
d_MC_COMMAND_ERROR	15A4h
d_MC_LOCKED_Memory	15A5h
d_MC_VERIFY_FAIL	15A6h
d_MC_WRITE_ERROR	15A7h
d_MC_NOT_SUPPORTED	15ABh

Note:
After calling this function successfully, the card type information will be put in the first byte of baBuf.

0x02 : SLE memory card
0x03 : I2C card

Example:

```
void sample()
{
        CtSC sc = new CtSC();
        byte [] retBuf = new byte[256];
        byte [] sampleBuf = new byte[256];
```

Castles Technology

```
		byte [] baATR = new byte[128];
		byte [] Buf = new byte[3];
		int [] Len = new int[1];
		int iRtn, i;

		for (i = 0; i < sampleBuf.length; i++)
			sampleBuf[i] = (byte) i;

		for(i = 0; i < Buf.length; i++)
			Buf[i] = (byte)0xFF;

		Len[0] = baATR.length;
		iRtn = sc.SyncICCReset(baATR, Len);
		Log.d(ACTIVITY_TAG, String.format("SyncICCReset = 0x%02X", iRtn));
		if(iRtn == 0)
		{
			iRtn = sc.sc44x2VerifyPSC(Buf);
			if(iRtn == 0)
			{
				iRtn = sc.sc44x2WriteMainMemory(0x00, sampleBuf);
				if(iRtn == 0)
				{
					iRtn = sc.sc44x2ReadMainMemory(0x00, retBuf, Len);
					Log.d(ACTIVITY_TAG, String.format("sc44x2ReadMainMemory = %d", iRtn));
					for(i=0;i<Len[0];i++)
					{
						Log.d(ACTIVITY_TAG, String.format("retBuf[%d] = 0x%02x", i, retBuf[i]));
					}
				}
			}
		}
}
```

public int sc44x2ReadMainMemory(int Addr, byte[] Buf, int[] Len)

Description:
Read the data from main memory.

Parameters:
[IN] Addr
Start address of main memory to read. The range is from 0x00 to 0xFF.
[OUT] Buf
Buffer to store the data from main memory.
[OUT] Len
Length of data returned into the buffer.

Return Value:

d_OK	0000h
d_MC_NOT_PRESENT	1501h
d_MC_NOT_ACTIVATED	1502h
d_MC_MUTE	1504h
d_MC_PARITY_ERROR	1505h
d_MC_EDC_ERROR	1506h
d_MC_BUFFER_OVERRUN	1507h
d_MC_HW_ERROR	1508h
d_MC_DEACTIVATED_PROTOCOL	1509h
d_MC_POWER_FAILED	1510h
d_MC_INVALID_PARAMETER	1511h
d_MC_SOCKET_NOT_EXIST	1530h
d_MC_BUF_TOO_SMALL	1531h
d_MC_COMM_ERROR	15A3h
d_MC_COMMAND_ERROR	15A4h
d_MC_LOCKED_Memory	15A5h
d_MC_VERIFY_FAIL	15A6h
d_MC_WRITE_ERROR	15A7h
d_MC_NOT_SUPPORTED	15ABh

Note:
None.

Example:

```
void sample()
{
	CtSC sc = new CtSC();
	byte [] retBuf = new byte[256];
	byte [] sampleBuf = new byte[256];
	byte [] baATR = new byte[128];
```

Castles Technology

```
		byte [] Buf = new byte[3];
		int [] Len = new int[1];
		int iRtn, i;

		for (i = 0; i < sampleBuf.length; i++)
			sampleBuf[i] = (byte) i;

		for(i = 0; i < Buf.length; i++)
			Buf[i] = (byte)0xFF;

		Len[0] = baATR.length;
		iRtn = sc.SyncICCReset(baATR, Len);
		Log.d(ACTIVITY_TAG, String.format("SyncICCReset = 0x%02X", iRtn));
		if(iRtn == 0)
		{
			iRtn = sc.sc44x2VerifyPSC(Buf);
			if(iRtn == 0)
			{
				iRtn = sc.sc44x2WriteMainMemory(0x00, sampleBuf);
				if(iRtn == 0)
				{
					iRtn = sc.sc44x2ReadMainMemory(0x00, retBuf, Len);
					Log.d(ACTIVITY_TAG, String.format("sc44x2ReadMainMemory = %d", iRtn));
					for(i=0;i<Len[0];i++)
					{
						Log.d(ACTIVITY_TAG, String.format("retBuf[%d] = 0x%02x", i, retBuf[i]));
					}
				}
			}
		}
}
```

public int sc44x2WriteMainMemory(int Addr, byte[] Buf)

Description:
Write data to main memory.

Parameters:
[IN] Addr
Start address of main memory to write. The range is from 0x00 to 0xFF.
[IN] Buf
Data to write.

Return Value:

d_OK	0000h
d_MC_NOT_PRESENT	1501h
d_MC_NOT_ACTIVATED	1502h
d_MC_MUTE	1504h
d_MC_PARITY_ERROR	1505h
d_MC_EDC_ERROR	1506h
d_MC_BUFFER_OVERRUN	1507h
d_MC_HW_ERROR	1508h
d_MC_DEACTIVATED_PROTOCOL	1509h
d_MC_POWER_FAILED	1510h
d_MC_INVALID_PARAMETER	1511h
d_MC_SOCKET_NOT_EXIST	1530h
d_MC_BUF_TOO_SMALL	1531h
d_MC_COMM_ERROR	15A3h
d_MC_COMMAND_ERROR	15A4h
d_MC_LOCKED_Memory	15A5h
d_MC_VERIFY_FAIL	15A6h
d_MC_WRITE_ERROR	15A7h
d_MC_NOT_SUPPORTED	15ABh

Note:
None.

Example:

```
void sample()
{
		CtSC sc = new CtSC();
		byte [] retBuf = new byte[256];
		byte [] sampleBuf = new byte[256];
		byte [] baATR = new byte[128];
		byte [] Buf = new byte[3];
		int [] Len = new int[1];
		int iRtn, i;
```

Castles Technology

```
        for (i = 0; i < sampleBuf.length; i++)
                sampleBuf[i] = (byte) i;

        for(i = 0; i < Buf.length; i++)
                Buf[i] = (byte)0xFF;

        Len[0] = baATR.length;
        iRtn = sc.SyncICCReset(baATR, Len);
        Log.d(ACTIVITY_TAG, String.format("SyncICCReset = 0x%02X", iRtn));
        if(iRtn == 0)
        {
                iRtn = sc.sc44x2VerifyPSC(Buf);
                if(iRtn == 0)
                {
                        iRtn = sc.sc44x2WriteMainMemory(0x00, sampleBuf);
                        if(iRtn == 0)
                        {
                                iRtn = sc.sc44x2ReadMainMemory(0x00, retBuf, Len);
                                Log.d(ACTIVITY_TAG, String.format("sc44x2ReadMainMemory = %d", iRtn));
                                for(i=0;i<Len[0];i++)
                                {
                                        Log.d(ACTIVITY_TAG, String.format("retBuf[%d] = 0x%02x", i, retBuf[i]));
                                }
                        }
                }
        }
}
```

public int sc44x2ReadProtectionMemory(byte[] Buf)

Description:
Read the data of protection memory.
The function read all four bytes at once.

Parameters:
[OUT] Buf
Buffer to store the data of protection memory, 4 bytes.

Return Value:

d_OK	0000h
d_MC_NOT_PRESENT	1501h
d_MC_NOT_ACTIVATED	1502h
d_MC_MUTE	1504h
d_MC_PARITY_ERROR	1505h
d_MC_EDC_ERROR	1506h
d_MC_BUFFER_OVERRUN	1507h
d_MC_HW_ERROR	1508h
d_MC_DEACTIVATED_PROTOCOL	1509h
d_MC_POWER_FAILED	1510h
d_MC_INVALID_PARAMETER	1511h
d_MC_SOCKET_NOT_EXIST	1530h
d_MC_BUF_TOO_SMALL	1531h
d_MC_COMM_ERROR	15A3h
d_MC_COMMAND_ERROR	15A4h
d_MC_LOCKED_Memory	15A5h
d_MC_VERIFY_FAIL	15A6h
d_MC_WRITE_ERROR	15A7h
d_MC_NOT_SUPPORTED	15ABh

Note:
None.

Example:
None

public int sc44x2SetProtectionBit(int Addr)

Description:
Set the protection bit in protection memory.

Parameters:
[IN] Addr
Address of protection bit to set.
The range is from 0x00 to 0x1F.

Return Value:

d_OK	0000h
d_MC_NOT_PRESENT	1501h

Castles Technology

d_MC_NOT_ACTIVATED	1502h
d_MC_MUTE	1504h
d_MC_PARITY_ERROR	1505h
d_MC_EDC_ERROR	1506h
d_MC_BUFFER_OVERRUN	1507h
d_MC_HW_ERROR	1508h
d_MC_DEACTIVATED_PROTOCOL	1509h
d_MC_POWER_FAILED	1510h
d_MC_INVALID_PARAMETER	1511h
d_MC_SOCKET_NOT_EXIST	1530h
d_MC_BUF_TOO_SMALL	1531h
d_MC_COMM_ERROR	15A3h
d_MC_COMMAND_ERROR	15A4h
d_MC_LOCKED_Memory	15A5h
d_MC_VERIFY_FAIL	15A6h
d_MC_WRITE_ERROR	15A7h
d_MC_NOT_SUPPORTED	15ABh

Note:
Once a bit in the protection memory is set, the corresponding bytes in the main memory will be protected.
For example, by calling CTOS_44x2SetProtectionBit(3), the data in address 3 of main memory is protected.

Example:
None

public int sc44x2ReadSecurityMemory(byte[] Buf)

Description:
Read out the data of security memory.
The function read all four bytes at once.

Parameters:
[OUT] Buf
Buffer to store the data of security memory, 4 bytes.
Byte1 : Error counter
Byte2~Byte4 : PSC.

Return Value:

d_OK	0000h
d_MC_NOT_PRESENT	1501h
d_MC_NOT_ACTIVATED	1502h
d_MC_MUTE	1504h
d_MC_PARITY_ERROR	1505h
d_MC_EDC_ERROR	1506h
d_MC_BUFFER_OVERRUN	1507h
d_MC_HW_ERROR	1508h
d_MC_DEACTIVATED_PROTOCOL	1509h
d_MC_POWER_FAILED	1510h
d_MC_INVALID_PARAMETER	1511h
d_MC_SOCKET_NOT_EXIST	1530h
d_MC_BUF_TOO_SMALL	1531h
d_MC_COMM_ERROR	15A3h
d_MC_COMMAND_ERROR	15A4h
d_MC_LOCKED_Memory	15A5h
d_MC_VERIFY_FAIL	15A6h
d_MC_WRITE_ERROR	15A7h
d_MC_NOT_SUPPORTED	15ABh

Note:
None

Example:
None

public int sc44x2ChangePSC(byte[] Buf)

Description:
Change the PSC.
Must successfully verify the PSC before changing it.

Parameters:
[IN] Buf
New PSC value to change, 3 bytes.

Return Value:

d_OK	0000h
d_MC_NOT_PRESENT	1501h

Castles Technology

d_MC_NOT_ACTIVATED	1502h
d_MC_MUTE	1504h
d_MC_PARITY_ERROR	1505h
d_MC_EDC_ERROR	1506h
d_MC_BUFFER_OVERRUN	1507h
d_MC_HW_ERROR	1508h
d_MC_DEACTIVATED_PROTOCOL	1509h
d_MC_POWER_FAILED	1510h
d_MC_INVALID_PARAMETER	1511h
d_MC_SOCKET_NOT_EXIST	1530h
d_MC_BUF_TOO_SMALL	1531h
d_MC_COMM_ERROR	15A3h
d_MC_COMMAND_ERROR	15A4h
d_MC_LOCKED_Memory	15A5h
d_MC_VERIFY_FAIL	15A6h
d_MC_WRITE_ERROR	15A7h
d_MC_NOT_SUPPORTED	15ABh

Note:
None

Example:
None

public int sc44x2VerifyPSC(byte[] Buf)

Description:
Verify the PSC.

Parameters:
[IN] Buf
PSC value to verify, 3 bytes

Return Value:

d_OK	0000h
d_MC_NOT_PRESENT	1501h
d_MC_NOT_ACTIVATED	1502h
d_MC_MUTE	1504h
d_MC_PARITY_ERROR	1505h
d_MC_EDC_ERROR	1506h
d_MC_BUFFER_OVERRUN	1507h
d_MC_HW_ERROR	1508h
d_MC_DEACTIVATED_PROTOCOL	1509h
d_MC_POWER_FAILED	1510h
d_MC_INVALID_PARAMETER	1511h
d_MC_SOCKET_NOT_EXIST	1530h
d_MC_BUF_TOO_SMALL	1531h
d_MC_COMM_ERROR	15A3h
d_MC_COMMAND_ERROR	15A4h
d_MC_LOCKED_Memory	15A5h
d_MC_VERIFY_FAIL	15A6h
d_MC_WRITE_ERROR	15A7h
d_MC_NOT_SUPPORTED	15ABh

Note:
None

Example:

```
void sample()
{
        CtSC sc = new CtSC();
        byte [] retBuf = new byte[256];
        byte [] sampleBuf = new byte[256];
        byte [] baATR = new byte[128];
        byte [] Buf = new byte[3];
        int [] Len = new int[1];
        int iRtn, i;

        for (i = 0; i < sampleBuf.length; i++)
                sampleBuf[i] = (byte) i;

        for(i = 0; i < Buf.length; i++)
                Buf[i] = (byte)0xFF;

        Len[0] = baATR.length;
        iRtn = sc.SyncICCReset(baATR, Len);
        Log.d(ACTIVITY_TAG, String.format("SyncICCReset = 0x%02X", iRtn));
```

Castles Technology

```
        if(iRtn == 0)
        {
                iRtn = sc.sc44x2VerifyPSC(Buf);
                if(iRtn == 0)
                {
                        iRtn = sc.sc44x2WriteMainMemory(0x00, sampleBuf);
                        if(iRtn == 0)
                        {
                                iRtn = sc.sc44x2ReadMainMemory(0x00, retBuf, Len);
                                Log.d(ACTIVITY_TAG, String.format("sc44x2ReadMainMemory = %d", iRtn));
                                for(i=0;i<Len[0];i++)
                                {
                                        Log.d(ACTIVITY_TAG, String.format("retBuf[%d] = 0x%02x", i, retBuf[i]));
                                }
                        }
                }
        }
}
```

Field Detail

Constant Field	Value
d_SC_NOT_PRESENT	0x1401
d_SC_NOT_ACTIVATED	0x1402
d_SC_COMM_ERROR	0x1403
d_SC_MUTE	0x1404
d_SC_PARITY_ERROR	0x1405
d_SC_EDC_ERROR	0x1406
d_SC_BUFFER_OVERRUN	0x1407
d_SC_HW_ERROR	0x1408
d_SC_DEACTIVATED_PROTOCOL	0x1409
d_SC_BAD_TS	0x140A
d_SC_ATR_TOO_LONG	0x140B
d_SC_ATR_INVALID	0x140C
d_SC_TA1_NOT_SUPPORTED	0x140D
d_SC_PTS_RESPONSE_ERROR	0x140E
d_SC_PROCEDURE_BYTE_CONFLICT	0x140F
d_SC_POWER_FAILED	0x1410
d_SC_INVALID_PARAMETER	0x1411
d_SC_INSUFFICIENT_BUF	0x1420
d_SC_SOCKET_NOT_EXIST	0x1430
d_SC_BUF_TOO_SMALL	0x1431

Example

```
import CTOS.*;

CtSC sc = new CtSC();

int socket = 0, status = 0, ATRLen = 0, ret = 0, i = 0;
byte baATR[] = new byte[128];
byte baRBuf[] = new byte[128];
byte baSBuf[] = new byte[5];

for(socket = 0 ; socket < 1; socket++) {
        Log.d(TAG, String.format("test socket:%d", socket));
        ret = sc.status(socket);
        status = sc.getStatus();
        Log.d(TAG, String.format("getStatus return code:0x%02x, status:0x%02x", ret, status));

        if(status == 0) {
                Log.d(TAG, String.format("card no exist"));
        } else {
                ret = sc.resetISO(socket, 1, baATR);
                Log.d(TAG, String.format("resetISO return code:0x%02x", ret));
                ATRLen = sc.getATRLen();
                Log.d(TAG, String.format("ATRLen = %d", ATRLen));
                Log.d(TAG, String.format("CardType = %d", sc.getCardType()));

                if(ret == 0) {
                        Log.d(TAG, "ATR :");
                        for(i = 0; i < ATRLen; i++) {
                                Log.d(TAG, String.format("0x%02x, ", baATR[i]));
                        }
```

Castles Technology

```
                                baSBuf[0] = (byte)0x00;
                                baSBuf[1] = (byte)0x86;
                                baSBuf[2] = (byte)0x00;
                                baSBuf[3] = (byte)0x00;
                                baSBuf[4] = (byte)0x08;

                                ret = sc.sendAPDU(socket, baSBuf, baRBuf);
                                Log.d(TAG, String.format("sendAPDU return code:0x%02x", ret));
                                if(ret == 0) {
                                        Log.d(TAG, "RX: ");
                                        for (i = 0; i < sc.getRLen(); i++) {
                                                Log.d(TAG, String.format("0x%02x, ", baRBuf[i]));
                                        }
                                }
                        }
                }
        }
}
```

Castles Technology

1.1.4 Public class CtExtraStorage

public class **CtExtraStorage**
The **CtExtraStorage** class is used to provide the extra storage.

Constructor Summary	
public	CtExtraStorage() Constructs a binder connected with service.

Method Summary	
int	read(String fileName, byte[] buf, int nByte, int offset) read extra storage
int	getBufLen() get read buffer length
int	write(String fileName, byte[] buf, int nByte, int offset) write extra storage

Constructor Detail

public CtExtraStorage()
Create a CtExtraStorage object and connect with binder to invoke service.

Method Detail

public int read(String fileName, byte[] buf, int nByte, int offset)

Description
read extra storage

Parameters
fileName : file path name
buf : places the characters read into buffer
nbyte : function attempts to read nbytes from the file associated with fileName
offset : The offset to the desired starting position in the file

Return Value

Constants	Value
OK	0x0000
FS_INVALID_PARAMETER	0x2001
FS_DATA_FULL	0x2007
FS_OPERATION_ERROR	0x20FF
LNX_FILE_OPEN_FAILED	0x2410
LNX_FILE_READ_FAILED	0x2411
LNX_FILE_WRITE_FAILED	0x2412

public int getBufLen()

Description
get read buffer length

Return Value
read buffer length

public int write(String fileName, byte[] buf, int nByte, int offset)

Description
write extra storage

Parameters
extraAppName : file path name
buf : places the characters write into buffer
nbyte : function attempts to write nbytes from the file associated with fileName
offset : The offset to the desired starting position in the file

Return Value

Constants	Value
OK	0x0000
FS_INVALID_PARAMETER	0x2001
FS_DATA_FULL	0x2007
FS_OPERATION_ERROR	0x20FF
LNX_FILE_OPEN_FAILED	0x2410
LNX_FILE_READ_FAILED	0x2411
LNX_FILE_WRITE_FAILED	0x2412

Castles Technology

Example

```
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;

import CTOS.CtExtraStorage;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        String TAG = "Saturn1000 extraStorage demo";

        CtExtraStorage extraStorage = new CtExtraStorage();
        byte wbuf[] = new byte[32], rbuf[] = new byte[32];

        int rnbyte = 0, wnbyte = 0, i = 0, ret = 0, offset = 0;

        for (i = 0; i < 5; i++) {
            wbuf[i] = (byte) (i + 5);
        }
        wnbyte = 5;
        ret = extraStorage.write("DATA.IN", wbuf, wnbyte, offset);
        if (ret == 0) {
            Log.d(TAG, "write ok");
        }

        rnbyte = 32;
        ret = extraStorage.read("DATA.IN", rbuf, rnbyte, offset);
        if (ret == 0) {
            Log.d(TAG, String.format("getBufLen %d", extraStorage.getBufLen()));
            for (i = 0; i < extraStorage.getBufLen(); i++)
                Log.d(TAG, String.format("%d. %d", i, rbuf[i]));
        }
    }
}
```

Castles Technology

1.1.5 Public class CtBarcodeReader

The **CtBarcodeReader** class is used to provide the barcode functions. The developer could use CtbarcodeReader to scan 1D and 2D bar code image and decode the data.

Constructor Summary	
public	CtBarcodeReader() Constructs the bar code reader.

Method Summary	
void	initialize(Context context) Starts the bar code service and initilize the bar code engine.
void	setParameter(int key , int value) Set the barcode engine parameters.
void	setPreviewSurface(Surface surface) Sets preview screen surface to bar code engine.
void	setDecodeTimeout(int timeout) Sets decoding timeout. The bar code engine will scan bar code in the timeout interval only.
interface	DecodeCallback Implements DecodeCallback interface and overide onDecodeComplete meothod to get scanning result.
void	setDecodeCallback(CtBarcodeReader.DecodeCallback callback) Sets DecodeCallback object to bar code service to notify decoding status and result.
void	startDecode() Starts to scan image and decode bar code.
void	StopDecode() Stops scanning bar code.
void	terminate() Stops the bar code service.
int	isBarcodeExist() Check device barcode exist
int	open() Open and initial barcode.
int	close() Close and deinitialize barcode.
int	read() The user calls the API to start scanning the barcode, if the barcode scanner scans any barcode, the user can call the API to get the barcode data.
int	readCancel() The user can call the API to stop the barcode scanner from scanning a barcode.
int	configSet() Setting the configuration to the barcode scanner.
int	configGet() Getting the configuration from the barcode scanner.

Constructor Detail
public CtBarcodeReader() Creates a CtBarcodeReader object.

Method Detail
public void Initialize(Context context) **Description** Starts the bar code service and initialize the bar code engine. **Parameters** context : Context. **Return Value** void
public void setParameter(int key, int value) **Description** Set the barcode engine parameters.

Castles Technology

Parameters
key : the parameter identifier.

1D Symbologies

Bookland EAN	83
Chinese 2 of 5	408
Codabar	7
Code 11	10
Code 128	8
Code 32	86
Code 39	0
Code 93	9
Composite CC-C	341
Composite CC-A/B	342
Composite TLC-39	371
Discrete 2 of 5	5
EAN-13/JAN 13	3
EAN-8/JAN 8	4
GS1 DataBar -14	338
GS1-128	14
Interleaved 2 of 5	6
Inverse 1D	586
ISBT 128	84
ISSN EAN	617
Korean 3 of 5	581
Matrix 2 of 5	618
MSI	11
Trioptic Code 39	13
UCC Coupon Code	85
UPC-A	1
UPC-E	2
UPC-E1	12

2D Symbologies

Aztec Inverse	589
Data Matrix	292
Data Matrix Inverse	588
Han Xin	1167
Han Xin Inverse	1168
Maxicode	294
MicroPDF417	227
MicroQR	573
PDF417	15
QR Code	293
QR Inverse	587

Postal Codes

Australian Postal	291
Japan Postal	290
Netherlands KIX Code	326
UK Postal	91
UPU FICS Postal	611
US Planet	90
US Postnet	89
USPS 4CB/One Code/Intelligent Mail	592

value : the new setting value.
1 (Enable)
0 (Disable)

Return Value

Void

public void setPreviewSurface(Surface surface)

Description

Sets preview screen surface to bar code engine. If the surface is null, the camera picture will not be displayed.

Castles Technology

Parameters

surface : The surface of view to display camera image.

Return Value

Void

public void setDecodeTimeout(int timeout)

Description

Sets decoding timeout. The bar code engine will scan bar code in the timeout interval only. The bar code engine will stop to decode image after timeout microseconds

Parameters

timeout : decoding timeout in microsecond.

Return Value

Void

interface DecodeCallback

Description

The interface is used to implement callback function for bar code engine. The bar code engine uses the callback method to notify the application to get the status and decode result.

Methods:

onDecodeComplete : notify listener after decoding completely.

public void setDecodeCallback(CtBarcodeReader.DecodeCallback callback)

Description

Sets decoding callback object to bar code service. After the bar code service scans image and decode data, it will call the callback method to notify the listener the decoding status and result

Parameters

callback : DecodeCallback object.

Return Value

void

public void startDecode()

Description

Starts bar code scanning process. The method will notify barcode service to start scanning image and decoding data. The method will return immediately, but the bar code service will continue to scan image iget the decoding result in the timeout interval.

Parameters

void.

Castles Technology

Return Value

void

public void stoptDecode()

Description

Stops bar code scanning process. The method will notify barcode service to stop scanning image.

Parameters

void.

Return Value

void

public void terminate()

Description

Stops the bar code service.

Parameters

void.

Return Value

void

public int isBarcodeExist()

Description

Check device barcode exist

Parameters

void.

Return Value

barcode state 0 -> device barcode isn't exist

1 -> device barcode is exist

-1 -> get device barcode timeout

public int open(byte var1)

Description

Only for S1U

Open and initial barcode.

Parameters

[IN]

var1

Communication console port number

Castles Technology

Return Value

d_OK	0x0000
d_ BARCODE_NOT_SUPPORT	0x3101
d_ BARCODE_HW_ERROR	0x3107

Note

Need use once at AP start

Example

```
/*
Sample code
*/
import CTOS.CtBarcodeScan;
tBarcodeScan barcodescan = new CtBarcodeScan();

byte[] data = new byte[1024];
short[] slen = {0};

//need use once at AP start
ret = barcodescan.open((byte)0);
System.out.println(">>>>>> barcodescan.open ret = " + ret);
if(ret != d_OK) {
System.out.println(">>>>>> barcodescan.open fail");
while(true);
}

while (true) {

slen[0] = 1024;
ret = barcodescan.read(data, slen);
System.out.println(">>>>>> barcodescan.read ret = " + ret);
if (ret == d_OK) {
break;
}

try {
Thread.currentThread().sleep(200);
} catch (Exception e) {
e.printStackTrace();
}
}

System.out.println("slen:" + slen[0]);
for (int i = 0; i < slen[0]; i++) {
System.out.println("data[" + i + "]:" + Integer.toHexString((int) data[i] & 0x000000ff));
}
for (int i = 0; i < slen[0]; i++) {
data[i] = (byte) 0;
}
```

Castles Technology

```
//need use at exit AP to close barcode
ret = barcodescan.close();
System.out.println(">>>>>> barcodescan.close ret = " + ret);
```

public int close()

Description

Only for S1U

Close and deinitialize barcode.

Parameters

None

Return Value

d_OK	0x0000
d_BARCODE_NOT_OPEN	0x3106

Note

Need use once at AP end.

Example

```
/*
Sample code
*/
import CTOS.CtBarcodeScan;
tBarcodeScan barcodescan = new CtBarcodeScan();

byte[] data = new byte[1024];
short[] slen = {0};

//need use once at AP start
ret = barcodescan.open((byte)0);
System.out.println(">>>>>> barcodescan.open ret = " + ret);
if(ret != d_OK) {
System.out.println(">>>>>> barcodescan.open fail");
while(true);
}

while (true) {

slen[0] = 1024;
ret = barcodescan.read(data, slen);
System.out.println(">>>>>> barcodescan.read ret = " + ret);
if (ret == d_OK) {
break;
}

try {
Thread.currentThread().sleep(200);
} catch (Exception e) {
e.printStackTrace();
```

Castles Technology

```
}
}

System.out.println("slen:" + slen[0]);
for (int i = 0; i < slen[0]; i++) {
System.out.println("data[" + i + "]:" + Integer.toHexString((int) data[i] & 0x000000ff));
}
for (int i = 0; i < slen[0]; i++) {
data[i] = (byte) 0;
}

//need use at exit AP to close barcode
ret = barcodescan.close();
System.out.println(">>>>>> barcodescan.close ret = " + ret);
```

public int read(byte[] var1, short[] var2)

Description

Only for S1U

The user calls the API to start scanning the barcode, if the barcode scanner scans any barcode, the user can call the API to get the barcode data.

Parameters

None

Return Value

d_OK	0x0000
d_BARCODE_NO_DATA	0x3102
d_BARCODE_INSUFFICIENT_CMD_BUF	0x3103
d_BARCODE_NOT_OPEN	0x3106

Note

[OUT]

Var1

The point for data buffer.

[IN/OUT]

Var2

Pass the length of data buffer, if have data, the variable means the barcode length.

Example

```
/*
Sample code
*/
import CTOS.CtBarcodeScan;
tBarcodeScan barcodescan = new CtBarcodeScan();

byte[] data = new byte[1024];
short[] slen = {0};

//need use once at AP start
ret = barcodescan.open((byte)0);
System.out.println(">>>>>> barcodescan.open ret = " + ret);
```

Castles Technology

```
if(ret != d_OK) {
System.out.println(">>>>>> barcodescan.open fail");
while(true);
}

while (true) {

slen[0] = 1024;
ret = barcodescan.read(data, slen);
System.out.println(">>>>>> barcodescan.read ret = " + ret);
if (ret == d_OK) {
break;
}

try {
Thread.currentThread().sleep(200);
} catch (Exception e) {
e.printStackTrace();
}
}

System.out.println("slen:" + slen[0]);
for (int i = 0; i < slen[0]; i++) {
System.out.println("data[" + i + "]:" + Integer.toHexString((int) data[i] & 0x000000ff));
}
for (int i = 0; i < slen[0]; i++) {
data[i] = (byte) 0;
}

//need use at exit AP to close barcode
ret = barcodescan.close();
System.out.println(">>>>>> barcodescan.close ret = " + ret);
```

public int readCancel()

Description

Only for S1U

The user can call the API to stop the barcode scanner from scanning a barcode.

Parameters

None

Return Value

d_OK	0x0000
d_BARCODE_NOT_OPEN	0x3106

Note

None

Example

```
/*
```

Castles Technology

```
Sample code
*/
import CTOS.CtBarcodeScan;
tBarcodeScan barcodescan = new CtBarcodeScan();

byte[] data = new byte[1024];
short[] slen = {0};

//need use once at AP start
ret = barcodescan.open((byte)0);
System.out.println(">>>>>> barcodescan.open ret = " + ret);
if(ret != d_OK) {
System.out.println(">>>>>> barcodescan.open fail");
while(true);
}

while (true) {

slen[0] = 1024;
ret = barcodescan.read(data, slen);
System.out.println(">>>>>> barcodescan.read ret = " + ret);

//stop the scanning barcode
barcodescan.readCancel();

//need use at exit AP to close barcode
ret = barcodescan.close();
System.out.println(">>>>>> barcodescan.close ret = " + ret);
```

public int configSet(byte var1, byte[] var2, byte var3)

Description

Only for S1U

Setting the configuration to the barcode scanner.

Parameters

[IN]

Var1

The item of configuration.

Customers can set following items:

BARCODE_CONFIG_READTIMEOUT:

The read timeout of barcode and the unit is a sec.

BARCODE_CONFIG_SYMBOLOGIES:

General barcode symbology can read or not.

BARCODE_CONFIG_INVERSE_BARCODE:

Inverse barcode symbology can read or not.

BARCODE_CONFIG_AZTEC_BARCODE:

Aztec barcode symbology can read or not.

[IN]

Castles Technology

Var2

The configuration data.

BARCODE_CONFIG_READTIMEOUT:

Data 1: customers want to seconds of time out, which can pass 1~65535.

BARCODE_CONFIG_SYMBOLOGIES:

Data 1:

Customers can set following parameter:

BARCODE_SYMBOLOGIES_ENABLE

BARCODE_SYMBOLOGIES_DISABLE

Data 2:

Customers can set following parameter:

BARCODE_ALL_SYMBOLOGIES

BARCODE_1D_SYMBOLOGIES

BARCODE_2D_SYMBOLOGIES

BARCODE_CONFIG_INVERSE_BARCODE:

Data 1:

Customers can set following parameter:

BARCODE_SYMBOLOGIES_ENABLE

BARCODE_SYMBOLOGIES_DISABLE

BARCODE_CONFIG_AZTEC_BARCODE:

Data 1:

Customers can set following parameter:

BARCODE_SYMBOLOGIES_ENABLE

BARCODE_SYMBOLOGIES_DISABLE

[IN]

Var3

The length of configuration data.

Return Value

d_OK	0x0000
d_BARCODE_INVALID_PARA	0x3104
d_BARCODE_IO_PROCESSING	0x3105
d_BARCODE_NOT_OPEN	0x3106

Note

None

Example

```
/*
Sample code
*/
import CTOS.CtBarcodeScan;
tBarcodeScan barcodescan = new CtBarcodeScan();
```

Castles Technology

```
short[] slen = {0};
byte[] ConfgData = new byte[8];

//need use once at AP start
ret = barcodescan.open((byte)0);
System.out.println(">>>>>> barcodescan.open ret = " + ret);
if(ret != d_OK) {
System.out.println(">>>>>> barcodescan.open fail");
while(true);
}

ConfgData [0] = (byte)barcodescan.BARCODE_SYMBOLOGIES_ENABLE;
slen[0] = 1; barcodescan.configSet((byte)barcodescan.BARCODE_CONFIG_INVERSE_BARCODE, ConfgData, (byte)(slen[0]));
if(ret != d_OK) {
  System.out.println(">>>>>> barcodescan.configSet fail");
}
//need use at exit AP to close barcode
ret = barcodescan.close();
System.out.println(">>>>>> barcodescan.close ret = " + ret);
```

```
public int configGet(byte var1, byte[] var2, byte[] var3)
```

Description

Only for S1U

Getting the configuration from the barcode scanner.

Parameters

[IN]

Var1

The item of configuration.

Customers can set following items:

BARCODE_CONFIG_READTIMEOUT:

The read timeout of barcode and the unit is a sec.

BARCODE_CONFIG_SYMBOLOGIES:

General barcode symbology can read or not.

BARCODE_CONFIG_INVERSE_BARCODE:

Inverse barcode symbology can read or not.

BARCODE_CONFIG_AZTEC_BARCODE:

Aztec barcode symbology can read or not.

[OUT]

Var2

The configuration data.

BARCODE_CONFIG_READTIMEOUT:

Data 1: means the seconds of time out

BARCODE_CONFIG_SYMBOLOGIES:

Data 1: whether support the ALL barcode

Data 2: whether support the 1D barcode

Castles Technology

Data 3: whether support the 2D barcode

BARCODE_CONFIG_INVERSE_BARCODE:

Data 1: whether support the inverse barcode

BARCODE_CONFIG_AZTEC_BARCODE:

Data 1: whether support the aztec barcode

[OUT]

Var3

The length of configuration data.

Return Value

d_OK	0x0000
d_BARCODE_INSUFFICIENT_CMD_BUF	0x3103
d_BARCODE_INVALID_PARA	0x3104
d_BARCODE_IO_PROCESSING	0x3105
d_BARCODE_NOT_OPEN	0x3106

Note

None

Example

```
/*
Sample code
*/
import CTOS.CtBarcodeScan;
tBarcodeScan barcodescan = new CtBarcodeScan();

byte[] slen = new byte[1];
byte[] ConfgData = new byte[8];

//need use once at AP start
ret = barcodescan.open((byte)0);
System.out.println(">>>>>> barcodescan.open ret = " + ret);
if(ret != d_OK) {
System.out.println(">>>>>> barcodescan.open fail");
while(true);
}

slen[0] = 8; barcodescan.configGet((byte)barcodescan.BARCODE_CONFIG_INVERSE_BARCODE, ConfgData, slen);
if(ret != d_OK) {
  System.out.println(">>>>>> barcodescan.configSet fail");
}
//need use at exit AP to close barcode
ret = barcodescan.close();
System.out.println(">>>>>> barcodescan.close ret = " + ret);
```

Example

```
package com.castles.myapplication;
```

Castles Technology

```
import android.graphics.PixelFormat;
import android.os.Bundle;
import android.os.StrictMode;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;
import android.view.SurfaceHolder;
import android.view.SurfaceView;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

import com.castles.ctbarcodereader.CtBarcodeReader;



public class MainActivity extends AppCompatActivity implements SurfaceHolder.Callback,
CtBarcodeReader.DecodeCallback {
    final private String TAG = "BarcodeApp";
    private SurfaceView surfaceView = null;
    private SurfaceHolder surfaceHolder = null;
    private TextView textDecode = null;

    CtBarcodeReader barcodeReader;
    barcodeReader.setParameter(8, 1); // Enable CODE128
    //-----------------------------------------------------
    // SurfaceHolder callbacks
    public void surfaceCreated(SurfaceHolder holder) {
        try {
            barcodeReader.setPreviewSurface(holder.getSurface());
            barcodeReader.startDecode();
        } catch (Exception e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        } // just set the preview display
    }

    //-----------------------------------------------------
    public void surfaceChanged(SurfaceHolder holder, int format, int width, int height) {
    }

    //-----------------------------------------------------
    public void surfaceDestroyed(SurfaceHolder holder) {
    }

    private String decodeData;

    public void onDecodeComplete(int result, int symbology, String data) {
```

Castles Technology

```
        decodeData = "Result:" + result + "    symbology:" + symbology + " data:" + data;
        Log.d(TAG, "onDecodeComplete: " + decodeData);

        //the activity provides this method to be able to run code in the UI Thread
        runOnUiThread(new Runnable() {
            @Override
            public void run() {
                if (textDecode != null) {
                    textDecode.setText(decodeData);
                } else {
                    mainScreen();
                    textDecode.setText(decodeData);
                }
            }
        });

    }

    private void setupPreview() {
        try {
            setContentView(R.layout.previewimage_tiny);
            getWindow().setFormat(PixelFormat.UNKNOWN);
            surfaceView = (SurfaceView) findViewById(R.id.SurfaceView1);

            surfaceHolder = surfaceView.getHolder();
            surfaceHolder.addCallback(this);
            surfaceHolder.setType(SurfaceHolder.SURFACE_TYPE_PUSH_BUFFERS);

            surfaceView.setOnClickListener(mImageClickListener);

            textDecode = null;

        } catch (Exception e) {
        }
    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        mainScreen();
        barcodeReader = new CtBarcodeReader();
    }

    @Override
```

Castles Technology

```
    protected void onStart() {
        Log.d(TAG, "start");
        super.onStart();
        barcodeReader.initialize(this);
        barcodeReader.setDecodeTimeout(10000);
        barcodeReader.setDecodeCallback(this);
    }

    @Override
    protected void onStop() {
        Log.d(TAG, "stop");
        barcodeReader.terminate();
        super.onStop();
    }



    private void mainScreen() {
        setContentView(R.layout.main);              // Inflate our UI from its XML layout description.

        // Hook up button presses to the appropriate event handler.
        ((Button) findViewById(R.id.buttonDec)).setOnClickListener(mDecodeListener);
        ((Button) findViewById(R.id.buttonHF)).setOnClickListener(mHandsFreeListener);

        textDecode = (TextView) findViewById(R.id.textDecode);

    }

    View.OnClickListener mDecodeListener = new View.OnClickListener() {
        public void onClick(View v) {
            doDecode();
        }
    };
    View.OnClickListener mHandsFreeListener = new View.OnClickListener() {
        public void onClick(View v) {
            barcodeReader.setPreviewSurface(null);
            barcodeReader.setDecodeTimeout(10000);
            barcodeReader.startDecode();

        }
    };
    View.OnClickListener mImageClickListener = new View.OnClickListener() {
        public void onClick(View v) {
            mainScreen();
        }
    };

    private void doDecode() {
```

Castles Technology

```
            setupPreview();
        }


}
```

Castles Technology

1.1.6 Public class CtLoader

The **CtLoader** class is for downloading and updating. The data of APK, OTA and security module FW&APP to be installed into the terminal must be wrapped into a .CAP container file before uploading. The CAP file is packaged by Castle's CAP tool.

The **CtLoader** provides interface for updating status. The developer needs to set all callback functions by **setLoaderCallback()** method first.

We provide the Ulder demo in following path C:\Program Files (x86)\Castles\SATURN1000\examples\Ulder_demo

Constructor Summary	
public	CtLoader() Constructs a Ctloader object.

Method Summary	
int	downloadStart() Start to download CAP file by CDC USB.
void	onUpdateDownloadStatus(int downloadIsRunning, String downloadInf, String downloadSpeed, String downloadProgress, String downloadTime, int downloadFileNum, String[] downloadFilePath) This function is callback function for updating download status.
int	downloadCancel() Cancel the downloading process.
int	installStart(final String filePath) Start to install CAP file.
int	uninstallStart(final String packageName) Start to uninstall APK by package name.
void	onUpdateInstallStatus(int installIsRunning, String installStage, String installCAPType, String installInf, int installRet, CtLoaderSecurityModuleInstallStatus securityInstallStatus) This function is callback function for updating install status.
void	onUpdateUninstallStatus(int uninstallIsRunning, String uninstallInf, int uninstallRet) This function is callback function for updating uninstall status.
void	setLoaderCallback(LoaderCallBack loadercb) Set Ctloader all of callback function.

Constructor Detail
public CtLoader() Create a CtLoader object.

Method Detail
public int downloadStart() **Description** Start to download CAP file by CDC USB. **Parameters** void **Return Value** **Constants** — **Value** Success — 0 Fail — != 0
public void onUpdateDownloadStatus(int downloadIsRunning, String downloadInf, String downloadSpeed, String downloadProgress, String downloadTime, int downloadFileNum, String[] downloadFilePath) **Description** This function is callback function for updating download status. **Parameters** [OUT] downloadIsRunning : If the value is 1, the downloading is running. If the value is 0, the downloading is stop. [OUT] downloadInf : The string is download information of error message. [OUT] downloadSpeed : The string is download speed. [OUT] downloadProgress : The string is download progress. [OUT] downloadTime : The string is download total time. The base unit of this time is second. [OUT] downloadFileNum : The integer is number of download file. [OUT] downloadFilePath : The string is path of download file.

Castles Technology

Return Value
void

public int downloadCancel()

Description
Cancel the downlading process.

Parameters
void

Return Value

Constants	Value
Success	0
Fail	!= 0

public int installStart(final String filePath)

Description
Start to install CAP file.

Parameters
[IN] filePath : Path of CAP file. The path must be the absolute path. The file must be in the “/mnt/sdcard/Download/” folder.

Return Value

Constants	Value
Success	0
Fail	!= 0

public int uninstallStart(final String packageName)

Description
Start to uninstall APK by package name.

Parameters
[IN] packageName : package name of APK.

Return Value

Constants	Value
Success	0
Fail	!= 0

public void onUpdateInstallStatus(int installIsRunning, String installStage, String installCAPType, String installInf, int installRet, CtLoaderSecurityModuleInstallStatus securityInstallStatus)

Description
This function is callback function for updating install status.

Parameters
[OUT] installIsRunning : If the value is 1, the installing is running. If the value is 0, the installing is stop.
[OUT] installStage : The string is install stage.(RESERVED)
[OUT] installCAPType : The string is CAP type.
CAP Type:
OTA "OTA"
APK "APK"
Security module fw "SMF"
Extra app “SMA”
[OUT] installInf : The string is install information of error message.
[OUT] installRet : The integer is install return code.
Install return code:
if ("SMF" || "SMA"||”SME”) {
    Success 0x64
    Security module need to reboot 0x65, 0x66
    Fail Other
} else {
    Success 0
    Fail != 0
}

[OUT] securityInstallStatus : The installing status of security module fw&app.(RESERVED)

Return Value
void

public void onUpdateUninstallStatus(int uninstallIsRunning, String uninstallInf, int uninstallRet)

Castles Technology

Description
This function is callback function for updating uninstall status.

Parameters
[OUT] uninstallIsRunning : If the value is 1, the uninstalling is running. If the value is 0, the uninstalling is stop.
[OUT] uninstallInf : The string is uninstall information of error message.
[OUT] uninstallRet : The integer is install return code.

Return Value

Constants	**Value**
Success	0
Fail	!= 0

public void setLoaderCallback(LoaderCallBack loadercb)

Description
Set Ctloader all of callback function.

Parameters
[IN] loadercb : loadercb is Inderface of LoaderCallBack.

```
public interface LoaderCallBack {
        void onUpdateDownloadStatus(int downloadIsRunning, String downloadInf, String downloadSpeed, String
downloadProgress, String downloadTime, int downloadFileNum, String[] downloadFilePath);
        void onUpdateInstallStatus(int installIsRunning, String installStage, String installCAPType, String installInf, int
installRet, CtLoaderSecurityModuleInstallStatus securityInstallStatus);
        void onUpdateUninstallStatus(int uninstallIsRunning, String uninstallInf, int uninstallRet);
}
```

Return Value
void

Field Detail

CAP Type	Description
APK	Android application package kit
SMA	BCM589x Security module application.
APF	Android application package kit and data file.
ADF	Android application data file.
SHF	Android application share file.
FNT	Andorid font.
CFG	Android config.
APC	Android application certificate, for directly-installing method.
KTS	KTS, the file for NTTD XLSX convert to JSON.
OTA	Over the air, android system image.
SMF	BCM589x Security module firmware.
SME	BCM589x Security module extension.
CMF	Android cradle firmware.
ADB	Android ADB firmware.
MDB	Android MDB firmware.
EMV	BCM589x EMV module firmware.
SBL	Andorid SBL firmware.
LYX	BCM5810x Security module firmware.
SBP	Andorid static boot patten.
DBI	Andorid dymatic boot image.

Example

```
/* The “Loader sample” contains in SDK.
** The blue part of blow sources code are about loader api; Others are about UI.
*/
import CTOS.*;


public class UlderActivity extends AppCompatActivity {

    private static final String TAG = "Ulder";

    private String download_speed;
    private String download_bar;
    private String download_time;
    private String[] download_path = new String[128];
    private int download_path_num;

    private TextView bar_textview;
```

Castles Technology

```
    private TextView flow_textview;
    private TextView speed_textview;
    private TextView inf_textview;
    private Button download_button;
    private Button cancel_button;

    //from ctosapi.h
    private static final int d_CAP_UPDATE_FINISHED = 0x0064;
    private static final int d_CAP_RESET_REQUIRED = 0x0065;
    private static final int d_CAP_REBOOT_REQUIRED = 0x0066;
    //from ctos_redirect.h
    private static final int d_FRD_RX_TIMEOUT = 0x9009;

    private int loaderInstallingCount = 0;

    private String[] loaderInstallPath;
    private int loaderInstallNum = 0;
    private int loaderInstallIndex = 0;

    CtSystem system = new CtSystem();
    CtLoader loader = new CtLoader();

    private Handler ViewHandler = new Handler() {
        @Override
        public void handleMessage(Message msg) {
            String msgStr = msg.getData().getString(TAG);
            switch (msg.what) {
                case 1:
                    bar_textview.setText(msgStr);
                    break;
                case 2:
                    flow_textview.setText(msgStr);
                    break;
                case 3:
                    speed_textview.setText(msgStr);
                    break;
                case 4:
                    inf_textview.setText(msgStr);
                    break;
            }
        }
    };

    private Handler ButtonHandler = new Handler() {
        @Override
        public void handleMessage(Message msg) {
            switch (msg.what) {
                case 1:
                    download_button.setEnabled(true);
                    break;
                case 2:
                    download_button.setEnabled(false);
                    break;
                case 3:
                    cancel_button.setEnabled(true);
                    break;
                case 4:
                    cancel_button.setEnabled(false);
                    break;
            }
        }
    };

    private Handler ScreenHandler = new Handler() {
        @Override
        public void handleMessage(Message msg) {
            switch (msg.what) {
                case 1:
                    getWindow().clearFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON);
                    break;
                case 2:
                    getWindow().addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON);
                    break;
            }
        }
    };
```

Castles Technology

```
    private void show_bar_textview(String s) {
        Message msg = new Message();
        Bundle b = new Bundle();
        b.putString(TAG, s);
        msg.what = 1;
        msg.setData(b);
        ViewHandler.sendMessage(msg);
    }
    private void show_flow_textview(String s) {
        Message msg = new Message();
        Bundle b = new Bundle();
        b.putString(TAG, s);
        msg.what = 2;
        msg.setData(b);
        ViewHandler.sendMessage(msg);
    }
    private void show_speed_textview(String s) {
        Message msg = new Message();
        Bundle b = new Bundle();
        b.putString(TAG, s);
        msg.what = 3;
        msg.setData(b);
        ViewHandler.sendMessage(msg);
    }
    private void show_inf_textview(String s) {
        Message msg = new Message();
        Bundle b = new Bundle();
        b.putString(TAG, s);
        msg.what = 4;
        msg.setData(b);
        ViewHandler.sendMessage(msg);
    }

    private void download_button_enable() {
        Message msg = new Message();
        msg.what = 1;
        ButtonHandler.sendMessage(msg);
    }
    private void download_button_disable() {
        Message msg = new Message();
        msg.what = 2;
        ButtonHandler.sendMessage(msg);
    }
    private void cancel_button_enable() {
        Message msg = new Message();
        msg.what = 3;
        ButtonHandler.sendMessage(msg);
    }
    private void cancel_button_disable() {
        Message msg = new Message();
        msg.what = 4;
        ButtonHandler.sendMessage(msg);
    }


    private void screen_sleep_enable() {
        Message msg = new Message();
        msg.what = 1;
        ScreenHandler.sendMessage(msg);
    }
    private void screen_sleep_disable() {
        Message msg = new Message();
        msg.what = 2;
        ScreenHandler.sendMessage(msg);
    }

    private void download_init() {
        download_bar = null;
        download_speed = null;
        show_inf_textview("");
        show_bar_textview("-");
        show_speed_textview("-");
    }
    private void download_view_update() {
                    show_flow_textview("Download mci" + download_path_num + "...");
        if (!(download_bar.equals("null") || download_bar == null))
            show_bar_textview(download_bar);
```

Castles Technology

```
        if (!(download_speed.equals("null") || download_speed == null))
            show_speed_textview(download_speed);
    }


    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        CtLoader.LoaderCallBack loadercb = new CtLoader.LoaderCallBack() {
            public void onUpdateDownloadStatus(int downloadIsRunning, String downloadInf, String downloadSpeed, String
downloadProgress
                    , String downloadTime ,final int downloadFileNum, final String[] downloadFilePath) {
                Log.e(TAG, "speed:"+downloadSpeed+" ,time:"+downloadTime+" ,progress:"+downloadProgress);
                if (downloadIsRunning == 1) {
                    download_bar = downloadProgress;
                    download_speed = downloadSpeed;
                    download_path_num = downloadFileNum;
                    download_view_update();
                } else if (downloadIsRunning == 0) {
                    cancel_button_disable();
                    if (downloadInf.length() > 0) {
                        show_flow_textview(downloadInf);
                    } else if (downloadFileNum > 0) {
                        show_flow_textview("Download success");
                        loaderInstallingCount = 0;
                        loaderInstallNum = downloadFileNum;
                        loaderInstallPath = downloadFilePath;
                        loaderInstallIndex = 0;
                        show_flow_textview("Install mci" + loaderInstallIndex + "...");
                        loader.installStart(loaderInstallPath[0]);
                    } else {
                        show_flow_textview("Download fail");
                        download_button_enable();
                        screen_sleep_enable();
                    }
                }
            }

            public void onUpdateInstallStatus(int installIsRunning, String installStage, String installCAPType
                , String installInf, int installRet, CtLoaderSecurityModuleInstallStatus securityInstallStatus) {
                if (installIsRunning == 0) {
                    int is_success = 0;
                    if(installCAPType.equals("SMF") || installCAPType.equals("SMA")) {
                        if (installRet == d_CAP_UPDATE_FINISHED) {
                          is_success = 1;
                            show_flow_textview("Install success.");
                        } else if ((installRet == d_CAP_RESET_REQUIRED)||(installRet == d_CAP_REBOOT_REQUIRED)) {
                            installRet = system.securityReboot();
                            Log.i(TAG, "system.securityReboot: " + installRet);
                            byte sn_buf[] = new byte[16];
                            while(true) {
                                installRet = system.getSerialNumber(sn_buf);
                                Log.i(TAG, "system.getSerialNumber: " + installRet);
                                if (installRet == 0) {
                                    show_flow_textview("Install success.");
                                    is_success = 1;
                                    break;
                                } else if (installRet == d_FRD_RX_TIMEOUT) {
                                    try {
                                        Thread.sleep(1000);
                                    } catch (InterruptedException e) {
                                        e.printStackTrace();
                                    }
                                } else {
                                    show_flow_textview("Install mci" + loaderInstallIndex + " fail..");
                                    break;
                                }
                            }
                        } else {
                            show_flow_textview("Install mci" + loaderInstallIndex + " fail." + installRet);
                            if (installInf != null && !installInf.equals("null"))
                                show_inf_textview(installInf);
                        }
                    } else if (installCAPType != null && !installCAPType.equals("null")) { //OTA, APK ...
                        if (installRet == 0) {
                          is_success = 1;
```

Castles Technology

```
                                        show_flow_textview("Install success.");
                                } else {
                                        show_flow_textview("Install mci" + loaderInstallIndex + " fail.." + installRet);
                                        if (installInf != null && !installInf.equals("null")) {
                                                show_inf_textview(installInf);
                                        }
                                }
                        } else {
                                show_flow_textview("Install mci" + loaderInstallIndex + " fail..." + installRet);
                                if (installInf != null && !installInf.equals("null")) {
                                        show_inf_textview(installInf);
                                }
                        }

                        loaderInstallIndex++;
                        if (is_success == 1 && loaderInstallIndex < loaderInstallNum) {
                            show_flow_textview("Install mci" + loaderInstallIndex + "...");
                            loader.installStart(loaderInstallPath[loaderInstallIndex]);
                        } else {
                            download_button_enable();
                            screen_sleep_enable();
                        }
                    } else {
                        if (loaderInstallingCount%4 == 0)
                            show_bar_textview(".");
                        else if (loaderInstallingCount%4 == 1)
                            show_bar_textview("..");
                        else if (loaderInstallingCount%4 == 2)
                            show_bar_textview("...");
                        else if (loaderInstallingCount%4 == 3)
                            show_bar_textview("....");
                        loaderInstallingCount++;
                        show_speed_textview("");
                    }
                }

                public void onUpdateUninstallStatus(int uninstallIsRunning, String uninstallInf, int uninstallRet) {
                    Log.e(TAG, "Run:"+uninstallIsRunning+" ,Inf:"+uninstallInf+" ,Ret:"+uninstallRet);

                }
            };

            loader.setLoaderCallback(loadercb);


getWindow().setFlags(WindowManager.LayoutParams.FLAG_FULLSCREEN,WindowManager.LayoutParams.FLAG_FULLSCREEN);
            setContentView(R.layout.activity_ulder);

            bar_textview = (TextView) findViewById(R.id.bar_textview);
            flow_textview = (TextView) findViewById(R.id.flow_textview);
            speed_textview = (TextView) findViewById(R.id.speed_textview);
            inf_textview = (TextView) findViewById(R.id.inf_textview);
            download_button = (Button) findViewById(R.id.download_button);
            cancel_button = (Button) findViewById(R.id.cancel_button);

            cancel_button_disable();
            download_button.setOnClickListener(new View.OnClickListener() {
                @Override

                public void onClick(View view) {
                    /*
                    new Thread(new Runnable() {
                        public void run() {
                            Log.e(TAG, "UNINSTALL START");
                            loader.uninstallStart("com.android");
                        }
                    }).start();
                    */
                    download_button_disable();
                    screen_sleep_disable();
                    download_init();
                    loader.downloadStart();
                    cancel_button_enable();

                }
            });
```

Castles Technology

```
        cancel_button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                new Thread(new Runnable() {
                    public void run() {
                        show_flow_textview("Download cancel...");
                        loader.downloadCancel();
                    }
                }).start();
            }
        });
    }
}
```

Castles Technology

1.1.7 Public class CtMDB

The **CtMDB** class is used to provide the cashless MDB functions. The developer could use CtMDB to communication with VMC.

Constructor Summary	
public	CtMDB() Communicate with VMC as Cashless.

Method Summary	
int	open() Open the communication port between MDB box and cashless.
int	close() Close the communication port between MDB box and cashless.
int	configSet(ctMDBConfig varl) Send the configuration of cashless to VMC via MDB box.
int	configGet(ctMDBConfig varl) Get the configuration data from MDB box.
int	responesSend(int varl, int varl2, int[] varl3) Send Response to VMC via MDB box.
int	commandGet(int[] varl, int[] varl2, int[] varl3) Get the command from VMC via MDB box.
int	fwinfoGet(int varl, int[] varl2, int[] varl3) Get the MDB FW Version from MDB box.
int	icConfigSet(int var1, int[] var2, int var3) Set the configuration of MDB IC
int	icConfigGet(int var1, int[] var2, int[] var3) Get the configuration of MDB IC

Constructor Detail
public CtMDB() Creates a CtMDB object.

Method Detail

public int open()

Description

Open the communication port between MDB box and cashless..

Parameters

void.

Return Value

Constants	Value	
Success	0	
Fail	Fail	0x35xxh (Field Detail)

public int close()

Description

Close the communication port between MDB box and cashless.

Parameters

void.

Return Value

Constants	Value	
Success	0	
Fail	Fail	0x35xxh (Field Detail)

Castles Technology

class CtMDBConfig

Description

The class is used to implement config function for MDB. The MDB uses the CtMDBConfig method to set or get config.

Methods:

Level: Indicates the feature level of the reader.
CountryCode: Country / Currency Code.
ScaleFactor: The multiplier used to scale all monetary values transferred between the.VMC and the reader
DecimalPlaces: The number of decimal places used to communicate monetary values between the VMC and the payment media reader.
ManufacturerCode: Identification code for the equipment supplier
SoftVer: Current software version.
RSPTime: Cashless Response Time - seconds.
Miscellaneous: Other item.

class CtMDBConfigV2

Description

This class is extended from class CtMDBConfig and add new configure options. The class is used to implement config function for MDB. The MDB uses the CtMDBConfigV2 method to set or get more config.

Methods:

SerialNumber: Factory assigned serial number(ASCII).
ModelNumber: Manufacturer assigned model number.(ASCII)
OptionalFeaturebits: Each of the 32 bits indicate an optional feature availability.
getSerialNumber: Get SerialNumber from class CtMDBConfigV2 object.
setSerialNumber: Set SerialNumber to class CtMDBConfigV2 object.
getModelNumber: Get ModelNumber from class CtMDBConfigV2 object.
setModelNumber: Set ModelNumber to class CtMDBConfigV2 object.

public int configSet(ctMDBConfig varl)

Description

Send the configuration of cashless to VMC via MDB box.

Parameters

[IN] varl : CtMDBConfig object or CtMDBConfigV2 object.

Return Value

Constants	Value	
Success	0	
Fail	Fail	0x35xxh (Field Detail)

public int configGet(ctMDBConfig varl)

Description

Get the configuration data from MDB box.

Parameters

[OUT] varl : CtMDBConfig object or CtMDBConfigV2 object.

Return Value

Constants	Value	
Success	0	
Fail	Fail	0x35xxh (Field Detail)

public int responseSend(int varl, int varl2, int[] varl3)

Castles Technology

Description

Send Response to VMC via MDB box.

Parameters

[IN] varl : Response type.

MDB_VMC_RSP_BEGIN_SESSION	1
MDB_VMC_RSP_SESSION_CANCEL_REQUEST	2
MDB_VMC_RSP_APPROVED	3
MDB_VMC_RSP_DENIED	4
MDB_VMC_RSP_CANCELLED	5
MDB_VMC_RSP_JUST_RESET	6
MDB_VMC_RSP_SELECTION_REQUEST	7

[IN] varl2 : Indicates the feature level of the reader, The level must same with level of configset.

[IN] varl3 : Response data.

Return Value

Constants	**Value**	
Success	0	
Fail	Fail	0x35xxh (Field Detail)

public int commandGet(int[] varl, int[] varl2, int[] varl3)

Description

Get the command from VMC via MDB box.

Parameters

[OUT] varl : the command type.

MDB_VMC_CMD_IDLE	0x01
MDB_VMC_CMD_VMC_CONFIG_AND_SETUP	0x02
MDB_VMC_CMD_SETUP_DISABLE	0x03
MDB_VMC_CMD_SETUP_ENABLE	0x04
MDB_VMC_CMD_VEND_REQUEST	0x05
MDB_VMC_CMD_VEND_SUCCESS	0x06
MDB_VMC_CMD_VEND_SESSION_COMPLETE	0x07
MDB_VMC_CMD_SETUP_RESET	0x08
MDB_VMC_CMD_VEND_FAILURE	0x09
MDB_VMC_CMD_CASH_SALE	0x0A
MDB_VMC_CMD_ITEM_NUMBER	0x0B
MDB_VMC_CMD_READER_CANCEL	0x0C
MDB_VMC_CMD_ENABLE_OPTIONS	0x0D
MDB_VMC_CMD_DISCONNECT	0x0E
MDB_VMC_CMD_VEND_CANCEL	0x0F
MDB_VMC_CMD_VEND_SELECTION_DENIED	0x10

[OUT] varl2 : command data.

[IN/OUT] varl3 :
IN : The size of data buffer.
OUT : The length of command data.

Command discription

bCmd	describe	baData	pbLen
IDLE	VMC only send poll command	The command not return data	0
CONFIG_AND_SETUP	VMC is sending its configuration data to reader.	BYTE1: VMC feature level. BYTE2~BYTE3: Maximum Price, 0xFFFF = Undefined BYTE4~BYTE5: Minimum Price BYTE5: Columns on Display	8

Castles Technology

		BYTE6: Rows on Display BYTE7: Display Information 0x0000 = Undefined	
DISABLE	This informs the payment media reader that it has been disabled	The command not return data	0
ENABLE	This informs the payment media reader that is has been enabled	The command not return data	0
VEND REQUEST	The patron has made a selection.	BYTE0~BYTE1: The price of the selected product. BYTE2~BYTE3: The item number of the selected product. 0xFFFF = Undefined	4
VEND SUCCESS	The selected product has been successfully dispensed.	The command not return data	0
VEND SESSION COMPLETE	This tells the payment media reader that the session is complete and to return to the Enabled state.	The command not return data	0
RESET	This informs the payment media reader that is has been reset.	The command not return data	0
VEND FAILURE	The selected product has been failure dispensed.	The command not return data	0
CASH SALE	Use cash to complete the sale.	BYTE0~BYTE1: The price of the sale product. 0xFFFF = Undefined BYTE2~BYTE3: The item number of the sale product. 0xFFFF = Undefined	4
ITEM NUMBER	The patron has made a selection.	BYTE0~BYTE1: The item number of the selected product. 0xFFFF = Undefined	2
READER CANCEL	This command is issued to abort payment media reader activities which occur in the Enabled state.	The command not return data	0
ENABLE OPTIONS	Individual expanded feature bits as sent by reader in response to the 17H-00H EXPANSION REQUEST ID command. To enable a feature, a bit is set to one.	BYTE0~BYTE3: Optional Feature Bits	4
DISCONNECT	VMC is disconnected	The command not return data	0
VEND CANCEL	This command can be issued by the VMC to cancel a VEND REQUEST Command before a VEND APPROVED/DENIED has been sent by the payment media reader.	The command not return data	0
SELECTION DENIED	A vend which was requested by SELECTION REQUEST could not be dispensed.	BYTE0~BYTE1: Item number. Should be equal to ITEM NUMBER of SELECTION REQUEST. BYTE2: Reason why it could not be dispensed.	3

Return Value

Constants	**Value**	
Success	0	
Fail	Fail	0x35xxh (Field Detail)

spublic int fwinfoGet(int varl, int[] varl2, int[] varl3)

Description

Get the MDB FW Version from MDB box.

Parameters

[IN] varl : MDB FW information to get.

MDB_FWINFO_VER 1

Castles Technology

[OUT] varl2 : MDB FW information data.

[IN/OUT] varl3 :
IN : The size of data buffer.
OUT : The length of command data.

Return Value

Constants	Value	
Success	0	
SYSTEM_NOT_SUPPORT	0x0205	
Fail	Fail	0x35xxh (Field Detail)

public int icConfigSet (int varl, int[] varl2, int varl3)

Description

Set the configuration of the MDB IC.

Parameters

[IN] varl : the config type.

MDB_IC_CONFIG_ADDRESS	0x01
MDB_IC_CONFIG_TIMING_JUSTRESET	0x10

[IN] varl2 : Setting data

for MDB_IC_CONFIG_ADDRESS: use address 0x10 or 0x60 to setting.

for MDB_IC_CONFIG_TIMING_JUSTRESET:

MDB_TIMING_PARA_AFTER_POLL	0x01
MDB_TIMING_PARA_IMMEDIATELY	0x02

[IN] varl3 : The length of setting data.

Return Value

Constants	Value	
Success	0	
SYSTEM_NOT_SUPPORT	0x0205	
Fail	Fail	0x35xxh (Field Detail)

public int icConfigGet (int varl, int[] varl2, int[] varl3)

Description

Get the configuration of the MDB IC.

Parameters

[IN] varl : the config type.

MDB_IC_CONFIG_ADDRESS	0x01

[OUT] varl2 : data buffer.

for MDB_IC_CONFIG_ADDRESS: return to the set address.

for MDB_IC_CONFIG_TIMING_JUSTRESET:

MDB_TIMING_PARA_AFTER_POLL	0x01
MDB_TIMING_PARA_IMMEDIATELY	0x02

[IN/OUT] varl3 :
IN : The size of data buffer.
OUT : The length of MDB IC config data.

Return Value

Constants	Value
Success	0
SYSTEM_NOT_SUPPORT	0x0205

Castles Technology

Fail	Fail	0x35xxh (Field Detail)

Field Detail

Constant Field	Value
MDB_HAL_FAULT	0x3501
MDB_TRANSACTION_FAIL	0x3502
MDB_INSUFFICIENT_CMD_BUF	0x3503
MDB_LEVEL_NOT_SUPPORT	0x3504
MDB_INVALID_PARA	0x3505
MDB_NOT_SUPPORT	0x3506

Example

```
package com.example.administrator.testmdb;

import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

import CTOS.CtMDB;
import CTOS.CtMDBConfig;

public class MainActivity extends AppCompatActivity {
    int nKey = 0;
    View.OnClickListener lis;
    public void createListener(){
        lis = new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                nKey = 1;
            }
        };
    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        new Thread(new Runnable() {
            @Override
            public void run() {
                int nRet = 0;
                int nCount = 0;
                int pNullValue[] = new int[1];
                int pValue[] = new int[1];
                int pNumber[] = new int[1];
                int pnCMD[] = new int[1];
                int baData[] = new int[32];
                int pnLen[] = new int[1];

                CtMDBConfig MDBConfigobj = new CtMDBConfig();
                CtMDB mdb = new CtMDB();

                Button button = findViewById(R.id.button);

                createListener();

                button.setOnClickListener(lis);

                runOnUiThread(new Runnable() {
                    @Override
                    public void run() {
                        TextView tvL1 = (TextView) findViewById(R.id.L1);
                        tvL1.setText("Test MDB");
                    }
                });
                mdb.open();

                MDBConfigobj.Level = mdb.MDB_CONFIG_LEVEL_1;
                MDBConfigobj.CountryCode = 0x1978; //country code of Taiwan for ISO 4217
                MDBConfigobj.ScaleFactor = 1;
                MDBConfigobj.DecimalPlaces = 2;
```

```
                MDBConfigobj.ManufacturerCode = 0x434153;   //CAS
                MDBConfigobj.SoftVer = 0x0001;
                MDBConfigobj.RSPTime = 5;
                MDBConfigobj.Miscellaneous =
mdb.MDB_CONFIG_MISC_RESTORE_FUNDS|mdb.MDB_CONFIG_MISC_MULTIVEND|mdb.MDB_CONFIG_MISC_SUPPO
RT_CASH_SALE;

                while(true)
                {
                    nRet = mdb.configSet(MDBConfigobj);
                    if(mdb.OK   == nRet)
                    {
                        break;
                    }

                    if(1 == nKey)
                    {
                        mdb.close();
                        return;
                    }
                }

                runOnUiThread(new Runnable() {
                    @Override
                    public void run() {
                        TextView tvL2 = (TextView) findViewById(R.id.L2);
                        tvL2.setText("Init MDB");
                    }
                });

                while(true)
                {
                    pnLen[0] = baData.length;
                    nRet = mdb.commandGet(pnCMD, baData, pnLen);
                    if(nRet == mdb.OK && pnCMD[0] != mdb.MDB_VMC_CMD_IDLE)
                    {
                        /* simulator command config and setup time */
                        if(pnCMD[0] == mdb.MDB_VMC_CMD_VMC_CONFIG_AND_SETUP)
                        {
                            break;
                        }
                    }
                    try {
                        Thread.sleep(10);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                    if(1 == nKey)
                    {
                        runOnUiThread(new Runnable() {
                            @Override
                            public void run() {
                                TextView tvL1 = (TextView) findViewById(R.id.L1);
                                tvL1.setText("");
                                TextView tvL2 = (TextView) findViewById(R.id.L2);
                                tvL2.setText("");
                            }
                        });
                        mdb.close();
                        return;
                    }
                }

                while(true)
                {
                    pnLen[0] = baData.length;
                    nRet = mdb.commandGet(pnCMD, baData, pnLen);
                    if(nRet == mdb.OK && pnCMD[0] != mdb.MDB_VMC_CMD_IDLE)
                    {
                        if(pnCMD[0] == mdb.MDB_VMC_CMD_SETUP_ENABLE)
                        {
                            break;
                        }
                    }
                    try {
                        Thread.sleep(10);
                    } catch (InterruptedException e) {
```

```
                    e.printStackTrace();
                }
                if(1 == nKey)
                {
                    runOnUiThread(new Runnable() {
                        @Override
                        public void run() {
                            TextView tvL1 = (TextView) findViewById(R.id.L1);
                            tvL1.setText("");
                            TextView tvL2 = (TextView) findViewById(R.id.L2);
                            tvL2.setText("");
                        }
                    });
                    mdb.close();
                    return;
                }
            }

            runOnUiThread(new Runnable() {
                @Override
                public void run() {
                    TextView tvL1 = (TextView) findViewById(R.id.L1);
                    tvL1.setText("");
                    TextView tvL2 = (TextView) findViewById(R.id.L2);
                    tvL2.setText("");
                }
            });
            boolean bFlag = true;
            while(bFlag)
            {
                runOnUiThread(new Runnable() {
                    @Override
                    public void run() {
                        TextView tvL3 = (TextView) findViewById(R.id.L3);
                        tvL3.setText(String.valueOf("Ples Pay Now"));
                    }
                });
                while(true)
                {
                    nCount++;
                    if(nCount > 200)
                    {
                        /* simulator pay time */
                        nCount = 0;
                        break;
                    }
                    try {
                        Thread.sleep(10);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                    if(1 == nKey)
                    {
                        runOnUiThread(new Runnable() {
                            @Override
                            public void run() {
                                TextView tvL3 = (TextView) findViewById(R.id.L3);
                                tvL3.setText("");
                            }
                        });
                        mdb.close();
                        return;
                    }
                }
                pValue[0] = 200;
                pNumber[0] = 0;
                mdb.responseSend(mdb.MDB_VMC_RSP_BEGIN_SESSION, mdb.MDB_CONFIG_LEVEL_1,
pValue);
                runOnUiThread(new Runnable() {
                    @Override
                    public void run() {
                        TextView tvL4 = (TextView) findViewById(R.id.L4);
                        tvL4.setText("Pls select item");
                        TextView tvL5 = (TextView) findViewById(R.id.L5);
                        tvL5.setText("Clear to Cancel");
                    }
                });
```

```
                    while(true)
                    {
                        nCount++;
                        pnLen[0] = baData.length;
                        nRet = mdb.commandGet(pnCMD, baData, pnLen);
                        if(nRet == mdb.OK && pnCMD[0] == mdb.MDB_VMC_CMD_VEND_REQUEST)
                        {
                            if(pnLen[0] >= 2) {
                                pValue[0] = ((baData[0] & 0xFF) << 8) + (baData[1] & 0xFF);
                                pNumber[0] = ((baData[2] & 0xFF) << 8) + (baData[3] & 0xFF);
                            }
                            nCount = 0;
                            break;
                        }


                        try {
                            Thread.sleep(10);
                        } catch (InterruptedException e) {
                            e.printStackTrace();
                        }
                        if(1 == nKey)
                        {
                            nCount = 0;
                            mdb.responseSend(mdb.MDB_VMC_RSP_SESSION_CANCEL_REQUEST,
mdb.MDB_CONFIG_LEVEL_1, pNullValue);
                            mdb.close();

                            return;
                        }
                    }
                    if(!(1 == nKey))
                    {
                        mdb.responseSend(mdb.MDB_VMC_RSP_APPEOVED,mdb.MDB_CONFIG_LEVEL_1, pValue);
                    }
                    while(true)
                    {
                        nCount++;
                        pnLen[0] = baData.length;
                        nRet = mdb.commandGet(pnCMD, baData, pnLen);
                        if(nRet == mdb.OK && pnCMD[0] == mdb.MDB_VMC_CMD_VEND_SUCCESS)
                        {
                            nCount = 0;
                            runOnUiThread(new Runnable() {
                                @Override
                                public void run() {
                                    TextView tvL6 = (TextView) findViewById(R.id.L6);
                                    tvL6.setText("Vend Success");
                                }
                            });
                            break;
                        }
                        if(nRet == mdb.OK && pnCMD[0] == mdb.MDB_VMC_CMD_VEND_FAILURE)
                        {
                            nCount = 0;
                            runOnUiThread(new Runnable() {
                                @Override
                                public void run() {
                                    TextView tvL6 = (TextView) findViewById(R.id.L6);
                                    tvL6.setText("Vend Failure");
                                }
                            });
                            break;
                        }
                        try {
                            Thread.sleep(10);
                        } catch (InterruptedException e) {
                            e.printStackTrace();
                        }
                        if(1 == nKey)
                        {
                            nCount = 0;
                            nKey = 0;
                            mdb.close();
                            return;
                        }
```

```
                        }
                        runOnUiThread(new Runnable() {
                            @Override
                            public void run() {
                                TextView tvL7 = (TextView) findViewById(R.id.L7);
                                tvL7.setText("Transaction OK");
                            }
                        });
                        try {
                            Thread.sleep(3000);
                        } catch (InterruptedException e) {
                            e.printStackTrace();
                        }
                        runOnUiThread(new Runnable() {
                            @Override
                            public void run() {
                                TextView tvL3 = (TextView) findViewById(R.id.L3);
                                tvL3.setText("");
                                TextView tvL4 = (TextView) findViewById(R.id.L4);
                                tvL4.setText("");
                                TextView tvL5 = (TextView) findViewById(R.id.L5);
                                tvL5.setText("");
                                TextView tvL6 = (TextView) findViewById(R.id.L6);
                                tvL6.setText("");
                                TextView tvL7 = (TextView) findViewById(R.id.L7);
                                tvL7.setText("");
                            }
                        });
//                          break;
                    }
                    mdb.close();
                }
            }).start();
    }
}
```

Castles Tech

1.1.8 Public class CtCL

The **CtCL** class is used to the contactless card which support ISO14443 .

Constructor Summary	
public	CtCL() Constructs a binder connected with service.

Constants Summary	
int	CM_OPTION_DEFAULT Card Mode read and write data default option.
int	CM_MAX_TX_DATA_SIZE Card Mode write data max bytes.
int	CM_MAX_RX_DATA_SIZE Card Mode read data max buffer size.
int	CM_STATUS_NONE Card Mode. Simulated card in Non-Active status, wait PCD emission RF and transmit data.
int	CM_STATUS_ACTIVE Card Mode. Simulated card in Active status, wait PCD transmit APDU data.
int	CM_STATUS_NO_RF Card Mode. if PCD turn off RF from Active of simulated card status, simulated card will entry this status.
int	CM_STATUS_DESELECT Card Mode. if PCD send ISO14443-4 DESELECT command from Active of simulated card status, simulated card will entry this status.
int	CM_STATUS_READ_DATA Card Mode. if PCD send APDU data from Active of simulated card status, simulated card will entry this status to instruct user read data out.
int	CM_STATUS_WRITE_DATA Card Mode. Simulated card entry this status to instruct user write data out to the PCD after read out data.

Method Summary	
int	powerOn() Turn on the power of contactless reader.
int	powerOff() Turn off the power of contactless reader.
int	ppPolling(byte[] CSN, int Timeout) To detect PICCs that are in the Operating Field.
int	GetPPPollingCSNLen() Get CSN length of ppPolling() parameter.
int	ppActivation(byte[] CSN) Activates the PICCs.
int	ppAPDU(byte[] TxBuf, byte[] RxBuf, int RxSize) Sending the APDU to the PICC and receive the response from PICC.
int	GetPPAPDURxLen() Get RxBuf length of ppAPDU() parameter.
int	ppRemoval(int Timeout) Removal PICCs's state.
int	felicaPolling(byte[] IDm, byte[] Pmm) Detect if any Felica card on the landing field.
int	felicaReadWrite(byte[] TxBuf, byte[] RxBuf, int Timeout) Write data to the Felica and read response from Felica.
int	GetRxLen() Get RxBuf length of felicaReadWrite() parameter.
int	powerStatus() Get currently RF status.
int	getStatus() Read out RF status.
int	typeAActiveFromIdle(int BaudRate, byte[] ATQA, byte[] SAK, byte[] CSN) Activate a TypeA PICC from its IDLE state.

Castles Technology

int	RATS(int AutoBR, byte[] ATS) Perform the Request Answer To Select (RATS) defined in ISO14443-4.
int	APDU(byte[] SBuf, byte[] RBuf) Send the APDU to the TypeA PICC and receive the response from TypeA PICC.
int	getCSNLen() Get valid data length stored in CSN parameter.
int	getATSLen() Get valid data length stored in ATS parameter.
int	getRLen() Get valid data length stored in RBuf parameter.
int	tclSetTimeout(int Timeout) Set the timeout for tclTransparent*() method.
int	tclTransparentRaw(byte[] Sbuf, int TxLastBits, byte[] RBuf, boolean TxCrcEnable, boolean RxCrcEnable, boolean ParityEnable) Send raw data directly to the PICC.
int	GetRLastBits() This method return number of valid bits in the last received byte after calling tclTransparentRaw(). If zero, the whole byte is valid.
int	mifareAUTH2Ex(int KeyType, int BlockNr, byte[] Key, byte[] CardSN, int CardSNLen) Perform the CRYPTO1 (Mifare Classic) card authentication with input key.
int	mifareREADBLOCK(int BlockNr, byte[] Buf) Read the content of the specific block (16 bytes) from the PICC.
int	mifareWRITEBLOCK(int BlockNr, byte[] Buf) Write data to the specific block (16 bytes) of the PICC.
int	mifareINCREASE(int BlockNr, byte[] Value) Increases the value of the block and stores the result into the internal data register.
int	mifareDECREASE(int BlockNr, byte[] Value) Decreases the value of the block and stores the result into the internal data register.
int	mifareRESTORE(int BlockNr) Read the contents of the block and stores into the internal data register.
int	mifareTRANSFER(int BlockNr) Write the contents of the internal data register into the block.
int	getATQBLen() Get valid data length stored in ATQB parameter.
int	getRep() Get response of ATTRIB command after performing the attrib().
int	setPowerModeParm(boolean Auto, int AutoOffTime, boolean LEDOn) This function control RF Auto-Closing mechanism
int	getPowerModeParm(boolean[] Auto, int[] AutoOffTime, boolean[] LEDOn) This function get setting of RF Auto-Closing mechanism. setNetworkSelectionMode
int	mifareLOADKEY(byte[] Key) Put key value into the internal key buffer of RC531.
int	mifareAUTHEx(int KeyType, int BlockNr, byte[] CardSN) Perform the CRYPTO1 (Mifare Classic) card authentication with the key in the internal key buffer
int	REQA(byte[] ATQA) Execute Type A Request command.
int	WUPA(byte[] ATQA) Execute Type A Wake Up command.
int	antia(int SelCode, byte[] SNR, int Cnt) Issue a Type A ANTICOLLISION command to the PICC.
int	selecta(int SelCode, byte[] SNR, byte[] SAK) Issue a type A SELECT command to the PICC.
int	typeAHalt() Put a type A PICC into HALT state.
int	typeAActiveFromHalt(int BaudRate, byte[] ATQA, byte[] SAK, byte[] CSN) Executes WUPA and SELECTA sequence for the PICC with the same serial number comparing to the input CSN.

Castles Technology

int	REQB(int AFI, byte[] ATQB) Execute Type B Request command.
int	WUPB(int AFI, byte[] ATQB) Execute Type B Wake Up command.
int	attrib(byte[] PUPI) Select the TypeB PICC. TypeB PICC goes to the ACTIVE state after performing the ATTRIB command.
int	haltb(byte[] PUPI) Put the PICC of Type B in the HALT state.
int	typeBActive(byte[] PUPI) Activate a Type B PICC.
int	typeBActiveEx(byte[] PUPI, int AFI, boolean WUPB, int FSDI, int NumAntiColSlot, int MaxRate) Activate a Type B PICC.
int	RATSEx(int AutoBR, byte[] ATS, int MaxRate, int FSDI) Perform the Request Answer To Select (RATS) defined in ISO14443-4.
int	deselect() Exit the ISO14443-4 state and enter the HALT state.
int	nakPoll() Issue a T=CL NAK command to the PICC.
int	tclTransparent(byte[] SBuf, byte[] RBuf) Send raw data directly to the PICC.
int	tclTransparentEx(byte[] SBuf, byte[] RBuf, boolean TxCrcEnable, boolean RxCrcEnable) Send raw data directly to the PICC.
int	ecpPollingV2(byte[] TypeVCmd, byte[] CSN) Perform ECP L1 Polling Loop and Collision Detction.
Int	cts512bReqt(byte[] data) Request instruction.
Int	cts512bIdentify(byte[] NB, byte[] SN) Anti-collision instruction.
Int	cts512bSelect(byte[] NB, byte[] SN) Anti-collision instruction.
int	cts512bRead(int Address, byte[] Data) Read one block instruction.
int	cts512bMultiRead(int Address, byte[] Data) Read four consecutive blocks in one instruction.
int	cts512bWrite(int Address, byte[] Data, byte[] WrittenData) WRITE one block instruction.
int	cts512bUpdate(int Address, byte[] Data) Erase, write then verify one block.
int	cts512bDesactivate(byte[] SN) After receiving this instruction, ticket goes in HALT state.
int	cmStartEmulation() Start card emulation and entry Card Mode.
int	cmStopEmulation() Stop card emulation and exit Card Mode.
int	cmGetStatus(int[] ulStatus) Get simulated card status.
int	cmReadApduCommand(int usOption, byte[] baRxBuf, int[] usRxLen) Receive the APDU data from PCD.
int	cmWriteApduCommand(int usOption, byte[] baTxBuf, int[] usWrittenLen) Transmit APDU response data to PCD.

Constructor Detail

public CtCL()
Create a CtCL object and connect with binder to invoke service.

Castles Technology

Method Detail

public int powerOn()

Description
Turn on the RF power of contactless reader, which provide energy to PICCs.

Parameters
none

Return Value

Constants	Value
Success	0
Fail	0x82xxh (Field Detail)

public int powerOff()

Description
Turn off the RF power of contactless reader, which stop to provide energy to PICCs.

Parameters
none

Return Value

Constants	Value
Success	0
Fail	0x82xxh (Field Detail)

public int ppPolling(byte[] CSN, int Timeout)

Description
To detect PICCs that are in the Operating Field, it performs EMVCo CL POLL and Collision Detection process.

Parameters
[OUT] CSN
Card serial number of the selected PICC. The maximum value is 10 bytes.

[IN] Timeout
RFU (set to 0).

Return Value

Constants	Value
Success	0
Fail	0x82xxh (Field Detail)

public int getPPPollingCSNLen()

Description
Get CSN length of ppPolling() parameter.

Parameters
none

Return Value
The number of bytes of CSN length.

public int ppActivation(byte[] CSN)

Description
Active PICC that are in the Operating Field, it performs EMVCo CL Activation process.

Parameters
[IN] CSN
Card serial number of the PICC about to activate.

Return Value

Constants	Value
Success	0

Castles Technology

Fail	0x82xxh (Field Detail)

public int ppAPDU(byte[] TxBuf, byte[] RxBuf, int RxSize)

Description
Send the APDU to the PICC and receive the response from PICC
according to the T=CL protocol defined in ISO14443-4.

Parameters
[IN] TxBuf
Buffer storing the APDU to transmit to PICC.
[OUT] RxBuf
Buffer to store the response received from PICC.

[IN] RxSize
Size of the RxBuf.

Return Value

Constants	**Value**
Success	0
Fail	0x82xxh (Field Detail)

public int getPPAPDURxLen ()

Description
Get RxBuf length of ppAPDU() parameter.

Parameters
none

Return Value
The number of bytes of RxBuf length.

public int ppRemoval(int Timeout)

Description
To removal PICC that are in the Operating Field, it performs EMVCo CL Removal process.
After reset card state, it returns the card's state which the card exist on the Operating Field or not.

Parameters
[IN] Timeout
RFU (set to 0).

Return Value

Constants	**Value**
Success	0
Fail	0x82xxh (Field Detail)

public int felicaPolling(byte[] IDm, byte[] PMm)

Description
Detect if any Felica card on the landing field.

Parameters
[OUT] IDm
Manufacture ID.

[OUT] PMm
Manufacture Parameter.

Return Value

Constants	Value
Success	0
Fail	0x83xxh (Field Detail)

public int felicaReadWrite(byte[] TxBuf, byte[] RxBuf, int Timeout)

Description
Write data to the Felica and read response from Felica.

Parameter TxBuf/RxBuf contains Fields are “Data Length(LEN)” + “Packet Data”.

Castles Technology

The “Data Length(LEN)” is 1 byte, the value is 1 (Byte length of LEN) + n (Byte length of Packet Data).
The “Packet Data” is n bytes, data are command send to or response from Felica.

Parameters
[IN] TxBuf
The write data.

[OUT] RxBuf
The return data.

[IN] Timeout
Set timeout value in 256x16/fc (0.302 ms, fc = 13.56MHz) basis.

Return Value

Constants	**Value**
Success	0
Fail	0x83xxh (Field Detail)

public int getRxLen()

Description
Get RxBuf length of felicaReadWrite() parameter.

Parameters
none

Return Value
The number of bytes of RxBuf length.

public int powerStatus()

Description
Get currently RF status.

Parameters
none

Return Value

Constants	**Value**
Success	0
Fail	0x83xxh (Field Detail)

public int getStatus()

Description
Read out RF status.

Parameters
none

Return Value

Constants	**Value**
RF on	1
RF Off	0

public int typeAActiveFromIdle(int BaudRate, byte[] ATQA, byte[] SAK, byte[] CSN)

Description
Activate a PICC from its IDLE state.
This command executes REQA, anti-collision loop, and SELECTA sequence for one of PICC in the field.
After execution of Active_IDLE command, the PICC goes from IDLE state to ACTIVE state, and is ready to enter ISO-14443-4 area.

Parameters
[IN] BaudRate
RFU (set to 0).

[OUT] ATQA
Two bytes ATQA of the selected PICC.

[OUT] SAK
One byte SAK of the selected PICC.

[OUT] CSN
CSN of the selected PICC. The maximum array size is 10 bytes.
The method getCSNLen() return the valid length of received CSN from selected PICC.

Castles Technology

Return Value

Constants	Value
Success	0
Fail	0x83xxh (Field Detail)

public int RATS(int AutoBR, byte[] ATS)

Description
Perform the Request Answer To Select (RATS) defined in ISO14443-4.
This command will send the RATS request, receive ATS from the PICC, parse the ATS, and setup the communication parameters of PICC.

Parameters
[IN] AutoBR
RFU (set to 0).

[OUT] ATS
Buffer to store ATS received from PICC.
The method GetATSLen() return the valid data length stored in ATS.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int APDU(byte[] SBuf, byte[] RBuf)

Description
Send the APDU to the PICC and receive the response from PICC according to the T=CL protocol defined in ISO14443-4. This command will send the APDU with the communication parameters retrieved from RATS.

Parameters
[IN] Sbuf
Buffer storing the APDU to transmit to PICC.
[OUT] Rbuf
Buffer to store the response received from PICC.
The method getRLen() return the valid data length stored in RBuf.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int getCSNLen()

Description
Get valid data length stored in CSN parameter.

Parameters
void

Return Value
CSN length.

public int getATSLen()

Description
Get valid data length stored in ATS parameter.

Parameters
void

Return Value
ATS length.

public int getRLen()

Description
Get valid data length stored in RBuf parameter.

Parameters
void

Return Value
RBuf length.

Castles Technology

public int tclSetTimeout(int Timeout)

Description
Set the timeout for tclTransparent*() method.

Parameters
[IN] Timeout
Timeout in millisecond.

Return Value

Constants	Value
Success	0
Fail	0x83xxh (Field Detail)

public int tclTransparentRaw(byte[] SBuf, int TxLastBits, byte[] RBuf, boolean TxCrcEnable, boolean RxCrcEnable, boolean ParityEnable)

Description
Send raw data directly to the PICC.
If this method return success or Incomplete byte, the method getRLen() show the valid length of Rbuf received from PICC. The method getRLastBits() show the valid bits of last received byte.

Note [1] About return value 0x837B (Incomplete byte), it means the last bytes of receiving data is incomplete bytes.

For example:
Return code is 0x837B, getRLen() is 1, and getRLastBits() = 4, that means the data buffer have one byte data, and this data byte only 4 bits (bit 0, bit 1, bit 2, bit 3) valid.

Return code is 0x837B, getRLen() 10, and getRLastBits() = 4, that means the data buffer have 10 bytes data, the first 9 bytes is complete data byte, the last data byte is incomplete bytes, only 4 bits (bit 0, bit 1, bit 2, bit 3) valid.

Note [2] The parameter TxLastBits is used for define the number of bits of the last byte that shall be transmitted.

For example:
If length of SBuf is 1 and TxLastBits is 2, that means transmit 2 bits (bit0, bit1)only.
If length of SBuf is 10 and TxLastBits is 2, that means transmit 9 data bytes and the 2 bits of last data byte (bit0, bit1).

Parameters
[IN] Sbuf
Data to send.

[IN] TxLastBits
It defines the number of bits of the last byte that shall be transmitted.
A zero indicates that all bits of the last byte shall be transmitted.

[OUT] RBuf
Buffer to store the response from PICC.
The method getRLen() show the valid length of Rbuf received from PICC.
The method getRLastBits() show the valid bits of last received byte.

[IN] TxCrcEnable
Whether to calculate and pad the CRC when sending data to PICC.

[IN] RxCrcEnable
Whether to regard the last byte of data read from PICC as CRC and calculate its validity.

[IN] ParityEnable
If set true, the generation of the Parity bit for transmission and the Parity-Check for receiving is switched on.
If set false, the generation of the Parity bit for transmission and the Parity-Check for receiving is switched off, the received Parity bit is handled like a data bit.

Return Value

Constants	Value
Success	0
Incomplete byte	0x837B (The last bytes of receiving data is incomplete bytes)
Fail	0x83xxh (Field Detail)

public int getRLastBits()

Description
This method return number of valid bits in the last received byte after calling tclTransparentRaw().
If zero, the whole byte is valid.

Parameters
void

Return Value
The valid bits of last received byte. If zero, the whole byte is valid.

Castles Technology

public int mifareAUTH2Ex(int KeyType, int BlockNr, byte[] Key, byte[] CardSN, int CardSNLen)

Description
Perform the CRYPTO1 (Mifare Classic) card authentication with input key.
If authentication is successful, user could operate the blocks in the same sector of the authenticated block.

Parameters
[IN] KeyType
The type off key.
= 0x60: Key A
= 0x61: Key B

[IN] BlockNr
The number of the block to be authenticated. (From #0 To # 63) 16 sectors with 4 blocks of 16 bytes each.
Blocks in the same sector would be authenticated with the same key stored in sector trailer (Block#3, #7, ..., #63).

[IN] Key
6 bytes of Mifare key to authenticate.

[IN] CardSN
Card serial number.

[IN] CardSNLen
Number of bytes of card serial number. Can be 4, 7, or 10 bytes.

Return Value

Constants	Value
Success	0
Fail	0x83xxh (Field Detail)

public int mifareREADBLOCK(int BlockNr, byte[] Buf)

Description
Read the content of the specific block (16 bytes) from the PICC.

Parameters
[IN] BlockNr
The block number.

[OUT] Buf
The data read from the block of PICC.

Return Value

Constants	Value
Success	0
Fail	0x83xxh (Field Detail)

public int mifareWRITEBLOCK(int BlockNr, byte[] Buf)

Description
Write data to the specific block (16 bytes) of the PICC.

Parameters
[IN] BlockNr
The block number.

[IN] Buf
The data will be written to the block of PICC.

Return Value

Constants	Value
Success	0
Fail	0x83xxh (Field Detail)

public int mifareINCREASE(int BlockNr, byte[] Value)

Description
Increases the value of the block and stores the result into the internal data register.

Parameters
[IN] BlockNr
The block number.

[IN] Value
The amount to be added in hexadecimal.

Return Value

Constants	Value
Success	0
Fail	0x83xxh (Field Detail)

Castles Technology

public int mifareDECREASE(int BlockNr, byte[] Value)

Description
Decreases the value of the block and stores the result into the internal data register.

Parameters
[IN] BlockNr
The block number.

[IN] Value
The amount to be reduced in hexadecimal.

Return Value

Constants	Value
Success	0
Fail	0x83xxh (Field Detail)

public int mifareRESTORE(int BlockNr)

Description
Read the contents of the block and stores into the internal data register.

Parameters
[IN] BlockNr
The block number.

Return Value

Constants	Value
Success	0
Fail	0x83xxh (Field Detail)

public int mifareTRANSFER(int BlockNr)

Description
Write the contents of the internal data register into the block.

Parameters
[IN] BlockNr
The block number.

Return Value

Constants	Value
Success	0
Fail	0x83xxh (Field Detail)

public int getATQBLen()

Description
Get valid data length stored in ATQB parameter.

Parameters
void

Return Value
ATQB length.

public int getRep()

Description
Get response of ATTRIB command after performing the attrib().

Parameters
void

Return Value
ATTRIB response.

public int setPowerModeParm(boolean Auto, int AutoOffTime, boolean LEDOn)

Description
Contactless support RF Auto-Closing mechanism, the time between each contactless functions which be called larger than 1.5 second (default time), the RF will be closed. This function control RF Auto-Closing mechanism.

Parameters
[IN] Auto
If set true, enable RF Auto-Closing mechanism.
If set false, disable RF Auto-Closing mechanism.
[IN] AutoOffTime
If parameter Auto setting true:
This value define the time of RF Auto-Closing mechanism in millisecond.

Castles Technology

[IN] LEDOn
RFU : false

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int getPowerModeParm(boolean[] Auto, int[] AutoOffTime, boolean[] LEDOn)

Description
Contactless support RF Auto-Closing mechanism, the time between each contactless functions which be called larger than 1.5 second (default time), the RF will be closed. This function get setting of RF Auto-Closing mechanism.

Parameters
[OUT] Auto
If set true, enable RF Auto-Closing mechanism.
If set false, disable RF Auto-Closing mechanism.
[OUT] AutoOffTime
If parameter Auto setting true:
This value define the time of RF Auto-Closing mechanism in millisecond.
[OUT] LEDOn
RFU : false

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int mifareLOADKEY(byte[] Key)

Description
Put key value into the internal key buffer of RC531.

Parameters
[IN] Key
Key value put into the internal key buffer of RC531.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int mifareAUTHEx(int KeyType, int BlockNr, byte[] CardSN)

Description
Perform the CRYPTO1 (Mifare Classic) card authentication with the key in the internal key buffer. If authentication is successful, RC531 could operate the blocks in the same sector of the authenticated block.

Parameters
[IN] KeyType
The type off key.
= 0x60: Key A
= 0x61: Key B

[IN] BlockNr
The number of the block to be authenticated. (From #0 To # 63) 16 sectors with 4 blocks of 16 bytes each.
Blocks in the same sector would be authenticated with the same key stored in sector trailer (Block#3, #7, ..., #63).

[IN] CardSN
Card serial number

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int REQA(byte[] ATQA)

Description
Execute Type A Request command.

Parameters
[OUT] ATQA
Two bytes ATQA of the selected PICC.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int WUPA(byte[] ATQA)

Castles Technology

Description
Execute Type A Wake Up command.

Parameters
[OUT] ATQA
Two bytes ATQA of the selected PICC.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int antia(int SelCode, byte[] SNR, int Cnt)

Description
Issue a Type A ANTICOLLISION command to the PICC.

Parameters
[IN] SelCode
SEL code which defines the cascade level of UID to issue.
(Can be one of the following value, 0x93, 0x95 or 0x97)

[IN] SNR
Preference UID sequence to perform the ANTICOLLISION.

[OUT] SNR
UID CLn, 4 bytes of ANTICOLLISION result. (excluding BCC)

[IN] Cnt
Number of preference UID in bits. Normally set to 0.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int selecta(int SelCode, byte[] SNR, byte[] SAK)

Description
Issue a type A SELECT command to the PICC.

Parameters
[IN] SelCode
SEL code which defines the cascade level of UID to issue.
(Can be one of the following value, 0x93, 0x95 or 0x97)

[IN] SNR
CSN of the selected PICC. The maximum value is 10 bytes.
4 bytes of UID CLn to select. Normally input with the result of ANTICOLLISION.

[OUT] SAK
One byte SAK of the selected PICC.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int typeAHalt(void)

Description
Put a type A PICC into HALT state. This function will send out HALTA command as defined in ISO14443.

Parameters
void

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int typeAActiveFromHalt(int BaudRate, byte[] ATQA, byte[] SAK, byte[] CSN)

Description
Activate a PICC from its HALT state.
This command executes WUPA, and SELECTA sequence for the PICC with the same serial number comparing to the input CSN. After execution of Active_HALT command, the PICC goes from HALT state to ACTIVE state and is ready to enter the ISO-14443-4 area.

Castles Technology

Parameters
[IN] BaudRate
RFU (set to 0).

[OUT] ATQA
Two bytes ATQA of the selected PICC.

[OUT] SAK
One byte SAK of the selected PICC.

[IN] CSN
CSN of the selected PICC. The array length is one of 4 / 7 / 10 bytes.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int REQB(int AFI, byte[] ATQB)

Description
Execute Type B Request command.

Parameters
[IN] AFI
Select AFI (Application Family Identifier) to pre-select PICC.
Only the PICC's matching the AFI will answer the REQB/WUPB command.
Normally we use 0 for selecting All families and sub-families.

[OUT] ATQB
ATQB of the selected PICC. The maximum array size is 13 bytes.
The method getATQBLen() return the valid length of received ATQB from selected PICC.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int WUPB(int AFI, byte[] ATQB)

Description
Execute Type B Wake Up command.

Parameters
[IN] AFI
Select AFI (Application Family Identifier) to pre-select PICC.
Only the PICC's matching the AFI will answer the REQB/WUPB command.
Normally we use 0 for selecting All families and sub-families.

[OUT] ATQB
ATQB of the selected PICC. The maximum array size is 13 bytes.
The method getATQBLen() return the valid length of received ATQB from selected PICC.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int attrib(byte[] PUPI)

Description
Select the PICC.
PICC goes to the ACTIVE state after performing the ATTRIB command.
The method getRep() return the ATTRIB response.

Parameters
[IN] PUPI
Four bytes Pseudo-Unique PICC Identifier from PICC.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int haltb(byte[] PUPI)

Description
Put the PICC of Type B in the HALT state.

Parameters

Castles Technology

[IN] PUPI
Four bytes Pseudo-Unique PICC Identifier from PICC.

Return Value

Constants	**Value**
Success	0
Fail	0x83xxh

public int typeBActive(byte[] PUPI)

Description
Activate a Type B PICC. After this command, the PICC directly enters ISO14443-4 T=CL domain.
No need to call RATS.

Parameters
[OUT] PUPI
Four bytes Pseudo-Unique PICC Identifier from PICC.

Return Value

Constants	**Value**
Success	0
Fail	0x83xxh

public int typeBActiveEx(byte[] PUPI, int AFI, boolean WUPB, int FSDI, int NumAntiColSlot, int MaxRate)

Description
Activate a Type B PICC. After this command, the PICC directly enter ISO14443-4 T=CL domain.
No need to call RATS. All related input parameter please refer to description as below.

Parameters
[OUT] PUPI
Four bytes Pseudo-Unique PICC Identifier from PICC.

[IN] AFI
Select AFI (Application Family Identifier) to pre-select PICC.
Only the PICC's matching the AFI will answer the REQB/WUPB command.
Normally we use 0 for selecting All families and sub-families.

[IN] FSDI
Select the maximum number of data can be transmitted to the PICC.

d_CL_FSDI_16	0
d_CL_FSDI_24	1
d_CL_FSDI_32	2
d_CL_FSDI_40	3
d_CL_FSDI_48	4
d_CL_FSDI_64	5
d_CL_FSDI_96	6
d_CL_FSDI_128	7
d_CL_FSDI_256	8

[IN] NumAntiColSlot
Select the slot number when performing the Slot-Marker anti-collision method.

d_CL_NUMSLOT_x1	0x00
d_CL_NUMSLOT_x2	0x01
d_CL_NUMSLOT_x4	0x02
d_CL_NUMSLOT_x8	0x03
d_CL_NUMSLOT_x16	0x04

[IN] MaxRate
Select the maximum baud rate that can be applied when communicating to the PICC.

d_CL_MAXRATE_106	106

Return Value

Constants	**Value**
Success	0
Fail	0x83xxh

public int RATSEx(int AutoBR, byte[] ATS, int MaxRate, int FSDI)

Description
Perform the Request Answer To Select (RATS) defined in ISO14443-4.
This command will send the RATS request, receive ATS from the PICC, parse the ATS, and setup the communication parameters of PICC.

Parameters
[IN] AutoBR
RFU (set to 0).

[OUT] ATS

Castles Technology

Buffer to store ATS received from PICC.

[IN] MaxRate
Select the maximum baud rate that can be applied when communicating to the PICC.

d_MAXRATE_106	106

[IN] FSDI
Select the maximum number of data can be transmitted to the PICC.

d_CL_FSDI_16	0
d_CL_FSDI_24	1
d_CL_FSDI_32	2
d_CL_FSDI_40	3
d_CL_FSDI_48	4
d_CL_FSDI_64	5
d_CL_FSDI_96	6
d_CL_FSDI_128	7
d_CL_FSDI_256	8

Return Value

Constants	**Value**
Success	0
Fail	0x83xxh

public int deselect()

Description
Exit the ISO14443-4 state and enter the HALT state.

Parameters
void.

Return Value

Constants	**Value**
Success	0
Fail	0x83xxh

public int nakPoll()

Description
Issue a T=CL NAK command to the PICC. This function is to prevent some PICC from power saturation.

Parameters
void.

Return Value

Constants	**Value**
Success	0
Fail	0x83xxh

public int tclTransparent(byte[] SBuf, byte[] RBuf)

Description
Send raw data directly to the PICC.

Parameters
[IN] SBuf
Data to send.

[OUT] RBuf
Buffer to store the response from PICC.

Return Value

Constants	**Value**
Success	0
Fail	0x83xxh

public int tclTransparentEx(byte[] SBuf, byte[] RBuf, boolean TxCrcEnable, boolean RxCrcEnable)

Description
Send raw data directly to the PICC.

Parameters
[IN] SBuf
Data to send.

[OUT] RBuf
Buffer to store the response from PICC.

[IN] TxCrcEnable
Whether to calculate and pad the CRC when sending data to PICC.

Castles Technology

[IN] RxCrcEnable
Whether to regard the last byte of data read from PICC as CRC and calculate its validity.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int ecpPollingV2(byte[] TypeVCmd, byte[] CSN)

Description
Perform ECP L1 Polling Loop and Collision Detction.
Implement this function by reference documentation [1].

Parameters
[IN] TypeVCmd
Data to transmit, format is following ISO14443A standard frame.

[OUT] CSN
Return CSN of the selected PICC. The maximum value is 10 bytes.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

Note
[1] “Enhanced Contactless Polling Specification, Version 2.0, Jul 3, 2018”

public int cts512bReqt(byte[] data)

Description
Request instruction
After power up, the chip is waiting for this instruction: request the ticket family.
Before this, the chip never executes any instructions.
After receiving REQT, chip answers and enter in ready state.

Parameters
[OUT] data
2 bytes output data.
The first byte is “Product code”.
The second byte is “Fa code”.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int cts512bSelect(byte[] NB, byte[] SN)

Description
Anti-collision instruction. If serial number is well understood by the reader, SELECT instruction active one chip by this unique serial number. All others ticket stay in ready mode.

Parameters
[IN] NB
2 bytes input data.
The first byte is “NB lsb”.
The second byte is “NB msb”.

[OUT] SN
2 bytes output data.
The first byte is “Serial lsb”.
The second byte is “Serial msb”

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int cts512bRead(int Address, byte[] Data)

Description
Read one block instruction. Allow a reading from block 0 until block 31.
Ticket must be selected.

Parameters
[IN] Address
The block address, which valid less or equal than 0x1F.

Castles Technology

[OUT] Data
2 bytes output data.
The first byte is “DATA1”.
The second byte is “DATA2”

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int cts512bMultiRead(int Address, byte[] Data)

Description
Read four consecutive blocks in one instruction or speed up read sequence.

Parameters
[IN] Address
The block address, which valid less or equal than 0x1F.

[OUT] Data
8 bytes output data.
The first byte is “DATA1” … and so on to “DATA8”.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int cts512bWrite(int Address, byte[] Data, byte[] WrittenData)

Description
Depend on system bits, write is performed from block 0 to 31 or not.
Write instruction put at "1" or keep the state depend on data transmitted.
Erased state is 0000 Hexa and full written state is FFFF Hexa.
After write operation, ticket answers written data.
WRITE instruction is not allowed in counter blocks.

Parameters
[IN] Address
The block address, which valid less or equal than 0x1F.

[IN] Data
2 bytes input data.
The first byte is “DATA1”.
The second byte is “DATA2”

[OUT] WrittenData
2 bytes output data.
Ticket answers the written data.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int cts512bUpdate(int Address, byte[] Data)

Description
Erase, write then verify one block. System bits manage security and update access.
This instruction is the only way to increase counter configured blocks.

Parameters
[IN] Address
The block address, which valid less or equal than 0x1F.

[IN] Data
Two bytes input data.
The first byte is “DATA1”.
The second byte is “DATA2”

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int cts512bDesactivate(byte[] SN)

Description
After receiving this instruction, ticket goes in HALT state. In this state, ticket no more executes any instruction.

Castles Technology

Chip could be re-selected by SELECT instruction.

Parameters
[OUT] SN
2 bytes output data.
The first byte is “Serial lsb”.
The second byte is “Serial msb”

Return Value

Constants	Value
Success	0
Fail	0x83xxh

public int cmStartEmulation()

Description
Start card emulation and entry Card Mode.

Parameters

Return Value

Constants	**Value**
Success	0
Fail	0x83xx

public int cmStopEmulation()

Description
Stop card emulation and exit Card Mode

Parameters

Return Value

Constants	**Value**
Success	0
Fail	0x83xxh

public int cmGetStatus(int[] ulStatus)

Description
Get simulated card status.

Parameters

[OUT] ulStatus
1 of int size output data.
The data is currently simulated card status, it is one of constants:
CM_STATUS_NONE, CM_STATUS_ACTIVE, CM_STATUS_NO_RF, CM_STATUS_DESELECT,
CM_STATUS_READ_DATA, CM_STATUS_WRITE_DATA.

Return Value

Constants	**Value**
Success	0
Fail	0x83xxh

public int cmReadApduCommand(int usOption, byte[] baRxBuf, int[] usRxLen)

Description
Receiving APDU data send from PCD.

Parameters
[IN] usOption
RFU : Using constant CM_OPTION_DEFAULT.

[OUT] baRxBuf
Byte array with max size CM_MAX_RX_DATA_SIZE
The APDU data send from PCD.

[OUT]
1 of int size output data, valid data length of baRxBuf.

Return Value

Constants	**Value**
Success	0
Fail	0x83xxh

public int cmWriteApduCommand(int usOption, byte[] baTxBuf, int[] usWrittenLen)

Castles Technology

Description
Transmit APDU response data to PCD.

Parameters
[IN] usOption
RFU : Using constant CM_OPTION_DEFAULT.

[IN] baTxBuf
Byte array with max size CM_MAX_TX_DATA_SIZE
The APDU data send to PCD.

[OUT]
1 of int size output data, the actually transmit data length of baTxBuf to PCD.

Return Value

Constants	Value
Success	0
Fail	0x83xxh

Field Detail

Constant Field	Value
d_CLPP_TIMEOUT_ERR	0x8201
d_CLPP_COLLISION_ERR	0x8202
d_CLPP_TYP_COLLISION_ERR	0x8203
d_CLPP_TRANSMISSION_ERR	0x8204
d_CLPP_PROTOCOL_ERR	0x8205
d_CLPP_AP_TIMEOUT_ERR	0x8206
d_CLPP_PICC_NOT_READY	0x8207
d_CLPP_PICC_STILL_EXIST	0x8208
d_CLPP_PICC_NOT_DETECTED	0x8209
d_CL_UNDEFINE_ERR	0x83FF
d_CL_CRC_ERR	0x83FE
d_CL_AUTH_ERR	0x83FC
d_CL_PARITY_ERR	0x83FB
d_CL_CODE_ERR	0x83FA
d_CL_SERNR_ERR	0x83F8
d_CL_KEY_ERR	0x83F7
d_CL_NOTAUTH_ERR	0x83F6
d_CL_BITCOUNT_ERR	0x83F5
d_CL_BYTECOUNT_ERR	0x83F4
d_CL_WRITE_ERR	0x83F1
d_CL_OVFL_ERR	0x83ED
d_CL_FRAMING_ERR	0x83EB
d_CL_ACCESS_ERR	0x83EA
d_CL_UNKNOWN_COMMAND	0x83E9
d_CL_COLL_ERR	0x83E8
d_CL_INIT_ERR	0x83E7
d_CL_INTERFACE_ERR	0x83E6
d_CL_ACCESS_TIMEOUT	0x83E5
d_CL_NO_BITWISE_ANTICOLL	0x83E4
d_CL_CARD_CODING_ERR	0x83E1
d_CL_CASC_LEV_EX	0x83CC
d_CL_BAUDRATE_NOT_SUPPORTED	0x83CA
d_CL_WRONG_PARAMETER_VALUE	0x83C4
d_CL_NOT_SUPPORTED	0x83BC
d_CL_NOT_IMPLEMENT	0x839C
d_CL_NAK	0x8384
d_CL_TCL_ERR	0x8383
d_CL_TCL_TIMEOUT	0x8382
d_CL_TCL_ABORT	0x8381
d_CL_INCOMPLETE_BYTE	0x837B

Example

```
import CTOS.*;

CtCL cl = new CtCL();

public void CL_Delay(){
        long mCreationTime;

        mCreationTime = SystemClock.elapsedRealtime();
```

Castles Technology

```
		while(SystemClock.elapsedRealtime()<(mCreationTime+10));
}

public int CL_ResetRF(){
		int ret;

		ret = cl.powerOff();
		if(ret != 0){
			Log.d(TAG, String.format("[cl.powerOff] PowerOff Fail."));
			Log.d(TAG, String.format("[cl.powerOff] ret = 0x%04X.", ret));
			return ret;
		}

		CL_Delay();

		ret = cl.powerOn();
		if(ret != 0){
			Log.d(TAG, String.format("[cl.powerOn] PowerOn Fail."));
			Log.d(TAG, String.format("[cl.powerOn] ret = 0x%04X.", ret));
			return ret;
		}
		return 0;
}

public int CL_FindCard(byte[] baCSN, int TimeOut){
		long mCreationTime;
		int ret = 0;

		mCreationTime = SystemClock.elapsedRealtime();
		while(SystemClock.elapsedRealtime()<(mCreationTime+TimeOut)){
			ret = cl.ppPolling(baCSN, 0);
			if(ret == 0){
				break;
			}
		}
		if(ret != 0){
			return ret;
		}
		return 0;
}

public int clpp_test(){
		int ret = 0;
		byte baCSN[] = new byte[10];
		byte baTxBuf[] = {(byte)0x00, (byte)0xA4, (byte)0x02, (byte)0x00,
				(byte)0x02, (byte)0x00, (byte)0xB0};
		byte baRxBuf[] = new byte[256];
		int RxLen = 0;
		int baCSNLen = 0;
		int i = 0;

		// Reset contactless card sensing area.
		ret = CL_ResetRF();
		if(ret != 0){
			Log.d(TAG, String.format("[cl.powerOn] Rest Fail."));
			Log.d(TAG, String.format("[cl.powerOn] ret = 0x%04X.", ret));
			return ret;
		}

		// Start search contactless card, timeout is 1 second.
		ret = CL_FindCard(baCSN, 1000);
		if(ret != 0){
			Log.d(TAG, String.format("[cl.ppPolling] Find Card Fail"));
			Log.d(TAG, String.format("[cl.ppPolling] ret = 0x%04X.", ret));
			return ret;
		}
		baCSNLen = cl.getPPPollingCSNLen();
		Log.d(TAG, String.format("[cl.ppPolling] baCSNLen : %d", baCSNLen));
		for(i=0; i<baCSNLen; i++){
			Log.d(TAG, String.format("[cl.ppPolling] baCSN : %d", baCSN[i]));
		}

		// Activate contactless card.
		ret = cl.ppActivation(baCSN);
		if(ret != 0){
			Log.d(TAG, String.format("[cl.ppActivation] Active Card Fail"));
			Log.d(TAG, String.format("[cl.ppActivation] ret = 0x%04X.", ret));
```

Castles Technology

```
                return ret ;
        }

        // Send APDU command.
        ret = cl.ppAPDU(baTxBuf, baRxBuf, 256);
        if (ret != 0) {
                Log.d(TAG, String.format("[cl.ppAPDU] APDU Fail"));
                Log.d(TAG, String.format("[cl.ppAPDU] ret = 0x%04X.", ret));
                return ret ;
        }
        RxLen = cl.getPPAPDURxLen();
        Log.d(TAG, String.format("[cl.ppAPDU] RxLen : %d", RxLen));
        for(i=0; i<RxLen; i++){
                Log.d(TAG, String.format("[cl.ppAPDU] RxBuf : %d", baRxBuf[i]));
        }

        ret = cl.ppRemoval(0);
        if(ret == 0x8208){
                Log.d(TAG, String.format("[cl.ppRemoval] Card Still on the Operating Field "));
                Log.d(TAG, String.format("[cl.ppRemoval] ret = 0x%04X.", ret));
                return ret;
        }
                if(ret == 0x8207){
                Log.d(TAG, String.format("[cl.ppRemoval] Card not ready. "));
                Log.d(TAG, String.format("[cl.ppRemoval] ret = 0x%04X.", ret));
                return ret;
        }
                Log.d(TAG, String.format("[cl.ppRemoval] Card has beed removed from the Operating Field "));
                Log.d(TAG, String.format("[cl.ppRemoval] ret = 0x%04X.", ret));

        Log.d(TAG, String.format("[cl] Test success!"));
        return ret;
}
```

Example : Felica

```
import CTOS.CtCL;

CtCL cl = new CtCL();

  /**
   * Perfrom Felica Read Without Encryption command.
   *
   * @param IDm The card Manufacture ID.
   * @param PMm The card Manufacture Parameters.
   * @param numberOfService Number of service.
   * @param serviceCodeList List of services.
   * @param numberOfBlock    Number of block to send.
   * @param blockList List of block to read.
   * @return block data received from card.
   * @throws IOException if an I/O failure, transmission error, or invalid received data.
   */
  public byte[] felicaCommandReadWithoutEncryption(
          byte [] IDm,
          byte [] PMm,
          byte numberOfService,
          byte [] serviceCodeList,
          byte numberOfBlock,
          byte [] blockList) throws IOException
  {
      int index;
      int ret;

      int realPartA;
      int readPartB;
      int exponenitalPartE;
      int n;
      int timeout;

      byte []tx;
      int txLen;
      byte []rx;
      int rxLen;

      /**
       * Calculate timeout of Read Without Encrtption Command.
       * Timeout = T * [(B+1) * n + (A + 1)] * 4^E
       * T = 256 * 16 /fc
       *
```

Castles Technology

```
             * parameter timeout of felicaReadWrite() is in 256 * 16 /fc basis.
             */
            realPartA = (PMm[5] & 0x07);
            readPartB = (PMm[5] >> 3) & 0x07;
            exponenitalPartE = (PMm[5] >> 6) & 0x03;
            n = numberOfBlock;
            timeout = (int)(((readPartB + 1) * n + (realPartA + 1)) * Math.pow(4, exponenitalPartE));

            /**
             * Tx data contains fields "Data Length, 1 byte" + "Packet Data, n bytes".
             */
            tx = new byte[1 + (1 + 8 + 1 + (2 * numberOfService) + 1 + (2 * numberOfBlock))];
            txLen = 0;

            /* Field "Data Length" */
            tx[txLen++] = (byte)tx.length;

            /* Field "Packet Data" */
            // Command Code.
            tx[txLen++] = 0x06;
            // IDm
            for (index = 0 ; index < 8 ; index++) {
                tx[txLen++] = IDm[index];
            }
            // Number of service.
            tx[txLen++] = numberOfService;
            // Service code list.
            for (index = 0 ; index < (2 * numberOfService) ; index++) {
                tx[txLen++] = serviceCodeList[index];
            }
            // Number of block.
            tx[txLen++] = numberOfBlock;
            // BLock list.
            for (index = 0 ; index < (2 * numberOfBlock) ; index++) {
                tx[txLen++] = blockList[index];
            }

            /**
             * Rx data contains fields "Data Length, 1 byte" + "Packet Data, n bytes".
             */
            rx = new byte[256];

            ret = cl.felicaReadWrite(tx, rx, timeout);
            if (ret != 0x0000) {
                throw new IOException(String.format("[0x%04X] felicaReadWrite", ret));
            }

            rxLen = cl.getRxLen();
            if (rxLen < 12) {
                throw new IOException("receive length fail.");
            }
            if ((rx[10] != 0x00) || (rx[11] != 0x00)) {
                throw new IOException(String.format("Status Flags 0x%02X, 0x%02X", rx[10], rx[11]));
            }
            if (rxLen != (13 + (16 * numberOfBlock))) {
                throw new IOException("receive data length fail.");
            }

            return Arrays.copyOfRange(rx, 13, rxLen);
        }

        /**
         * Felica example.
         */
        private Runnable runExampleFelica = new Runnable() {
            public void run() {
                String log = "";
                int ret;
                int index;
                int block;

                byte[] IDm = new byte[8];
                byte[] PMm = new byte[8];

                byte numberOfService;
                byte[] serviceCodeList;
                byte numberOfBlock;
```

Castles Technology

```
                byte[] blockList;

                byte[] blockData;

                log = "";

                try {
                    ret = cl.felicaPolling(IDm, PMm);
                    if (ret != 0x0000) {
                        throw new IOException(String.format("[0x%04X] felicaPolling", ret));
                    }

                    numberOfService = 1;
                    serviceCodeList = new byte[2 * numberOfService];
                    serviceCodeList[0] = (byte)0x09;
                    serviceCodeList[1] = (byte)0x10;
                    numberOfBlock = 1;
                    blockList = new byte[2 * numberOfBlock];
                    blockList[0] = (byte)0x80;
                    blockList[1] = (byte)0x00;

                    blockData = felicaCommandReadWithoutEncryption(IDm, PMm, numberOfService, serviceCodeList,
numberOfBlock, blockList);

                }  catch (IOException e) {
                    log += e.getMessage() + "\n";
                    return ;
                }

                for (block = 0 ; block < (blockData.length / 16) ; block++) {
                    for (index = 0; index < blockData.length; index++) {
                        log += String.format("0x%02X, ", blockData[(block * 16) + index]);
                    }
                    log += "\n";
                }
            }
        };
```

Example : Get RF Status

```
import CTOS.*;

public int getCurrentRFStatus() throws IOException {
        int ret;
        int rf;

        CtCL cl = new CtCL();

        ret = cl.powerStatus();
        if (ret != 0) {
                throw new IOException(String.format("powerStatus 0x%04X.", ret));
        }

        rf = cl.getStatus();
        return rf;
}
```

Example : Mifare Classic

```
import java.io.IOException;
import java.util.Arrays;

import CTOS.CtCL;

/**
 * Provides access to MIFARE Classic Card.
 */
public class MifareClassic {
    private final static String TAG = "MifareClassic";

    private int mValueAddressData = 0;

    /** Mifare Classic Authentication Key A Value. */
    private final static int KEYA = 0x60;
    /** Mifare Classic Authentication Key B Value. */
    private final static int KEYB = 0x61;

    /** Length of a Mifare Classic authentication key. */
```

Castles Technology

```
    public final static int AUTH_KEY_LENGTH       = 6;
    /** Length of a Mifare Classic data block. */
    public final static int DATA_BLOCK_LENGTH    = 16;

    final CtCL mTech;

    /** @hide */
    public MifareClassic(CtCL tech) {
        this.mTech = tech;
    }

    /** @hide */
    private byte[] valueToByteArray(int value) {
        byte[] data = new byte[4];
        data[0]   = (byte)(value & 0xFF);
        data[1]   = (byte)((value >>    8) & 0xFF);
        data[2]   = (byte)((value >> 16) & 0xFF);
        data[3]   = (byte)((value >> 24) & 0xFF);
        return data;
    }

    /** @hide */
    private int byteArrayToValue(byte[] data) {
        int value;
        value = data[0] & 0xFF;
        value |= (data[1] & 0xFF) << 8;
        value |= (data[2] & 0xFF) << 16;
        value |= (data[3] & 0xFF) << 24;
        return value;
    }

    /** @hide */
    private boolean checkValueBlockFormat(byte[] valueBlockData){
        if ((valueBlockData[0] != (byte)valueBlockData[8]) ||
                    (valueBlockData[1] != (byte)valueBlockData[9]) ||
                    (valueBlockData[2] != (byte)valueBlockData[10]) ||
                    (valueBlockData[3] != (byte)valueBlockData[11]) ||
                    (valueBlockData[4] != (byte)((valueBlockData[0] & 0xFF) ^ 0xFF)) ||
                    (valueBlockData[5] != (byte)((valueBlockData[1] & 0xFF) ^ 0xFF)) ||
                    (valueBlockData[6] != (byte)((valueBlockData[2] & 0xFF) ^ 0xFF)) ||
                    (valueBlockData[7] != (byte)((valueBlockData[3] & 0xFF) ^ 0xFF)) ||
                    (valueBlockData[12] != (byte)valueBlockData[14]) ||
                    (valueBlockData[13] != (byte)valueBlockData[15]) ||
                    (valueBlockData[12] != (byte)((valueBlockData[13] & 0xFF) ^ 0xFF)))
        {
            return false;
        } else {
            return true;
        }
    }

    /** @hide */
    private byte[] createValueFormatBlock(int value, int addressData) {
        byte []valueBlock = new byte[DATA_BLOCK_LENGTH];

        valueBlock[0]   = (byte)(value & 0xFF);
        valueBlock[1]   = (byte)((value >>    8) & 0xFF);
        valueBlock[2]   = (byte)((value >> 16) & 0xFF);
        valueBlock[3]   = (byte)((value >> 24) & 0xFF);
        valueBlock[4]   = (byte)~(byte)(value & 0xFF);
        valueBlock[5]   = (byte)~(byte)((value >>    8) & 0xFF);
        valueBlock[6]   = (byte)~(byte)((value >> 16) & 0xFF);
        valueBlock[7]   = (byte)~(byte)((value >> 24) & 0xFF);
        valueBlock[8]   = valueBlock[0];
        valueBlock[9]   = valueBlock[1];
        valueBlock[10] = valueBlock[2];
        valueBlock[11] = valueBlock[3];
        valueBlock[12] = (byte)addressData;
        valueBlock[13] = (byte)~(byte)addressData;
        valueBlock[14] = (byte)addressData;
        valueBlock[15] = (byte)~(byte)addressData;

        return valueBlock;
    }

    /** @hide */
    private void authenticate(int keyType, int blockAddress, byte[] key, byte[] uid) throws IOException,
```

```
IllegalArgumentException, NullPointerException {
        int retval;

        if (key == null) {
            throw new NullPointerException(String.format("Invalid key array"));
        }
        if (uid == null) {
            throw new NullPointerException(String.format("Invalid uid array"));
        }
        if (keyType != KEYA && keyType != KEYB) {
            throw new IllegalArgumentException(String.format("Invalid key type"));
        }
        if (key.length != AUTH_KEY_LENGTH) {
            throw new IllegalArgumentException(String.format("Invalid length of key"));
        }
        if (uid.length != 4 && uid.length != 7 && uid.length != 10) {
            throw new IllegalArgumentException(String.format("Invalid length of uid"));
        }

        retval = mTech.mifareAUTH2Ex(keyType, blockAddress, key, uid, uid.length);
        if (retval != 0) {
            throw new IOException(String.format("[0x%04X] mifareAUTH2Ex", retval));
        }
    }

    /**
     * Perform MIFARE Classic Authenticate command with key A.
     *
     * <p>Applications must discovery MIFARE CLASSIC tag before call this method.
     *
     * @param blockAddress Block-Address on card to authenticate to.
     * @param key Key to be used in authentication.
     * @param uid Complete serial number of PICC, length is 4 or 7 or 10.
     * @throws IOException if there is an I/O failure, transmission error, or invalid card.
     * @throws IllegalArgumentException Input parameter illegal.
     * @throws NullPointerException Object reference is null.
     */
    public void authenticateBlockWithKeyA(int blockAddress, byte[] key, byte[] uid) throws IOException {
        this.authenticate(KEYA, blockAddress, key, uid);
    }

    /**
     * Perform MIFARE Classic Authenticate command with key B.
     *
     * <p>Applications must discovery MIFARE CLASSIC tag before call this method.
     *
     * @param blockAddress Block-Address on card to authenticate to.
     * @param key Key to be used in authentication.
     * @param uid Complete serial number of PICC, length is 4 or 7 or 10.
     * @throws IOException if there is an I/O failure, transmission error, or invalid card.
     * @throws IllegalArgumentException Input parameter illegal.
     * @throws NullPointerException Object reference is null.
     */
    public void authenticateBlockWithKeyB(int blockAddress, byte[] key, byte[] uid) throws IOException {
        this.authenticate(KEYB, blockAddress, key, uid);
    }

    /**
     * Perform Read command with MIFARE Classic PICC.
     *
     * <p>Applications must discovery MIFARE CLASSIC tag before call this method.
     *
     * @param blockAddress address of block to read, starting from 0.
     * @return 16 bytes block
     * @throws IOException if there is an I/O failure, transmission error.
     */
    public byte[] read(int blockAddress) throws IOException {
        int retval;
        byte[] data = new byte[DATA_BLOCK_LENGTH];

        retval = mTech.mifareREADBLOCK(blockAddress, data);
        if (retval != 0) {
            throw new IOException(String.format("[0x%04X] mifareREADBLOCK", retval));
        }

        return data;
    }
```

```
    /**
     * Perform Write command with MIFARE Classic PICC.
     *
     * <p>Applications must discovery MIFARE CLASSIC tag before call this method.
     *
     * @param blockAddress address of block to read, starting from 0.
     * @param data 16 bytes of data to write.
     * @throws IOException if there is an I/O failure, transmission error.
     * @throws IllegalArgumentException Input parameter illegal.
     */
    public void write(int blockAddress, byte[] data) throws IOException {
        int retval;

        if (data.length != DATA_BLOCK_LENGTH) {
            throw new IllegalArgumentException("Invalid length of data");
        }

        retval = mTech.mifareWRITEBLOCK(blockAddress, data);
        if (retval != 0) {
            throw new IOException(String.format("[0x%04X] mifareREADBLOCK", retval));
        }
    }

    /**
     * Perform Write Value command with MIFARE Classic PICC.
     *
     * <p>Applications must discovery MIFARE CLASSIC tag before call this method.
     *
     * @param blockAddress address of block to write, starting from 0.
     * @param value value to be written to the MIFARE Classic.
     * @param addressData containing address written to the MIFARE Classic value block.
     * @throws IOException if there is an I/O failure, transmission error.
     */
    public void writeValue(int blockAddress, int value, int addressData) throws IOException {
        byte[] valueBlockData = createValueFormatBlock(value, addressData);
        this.write(blockAddress, valueBlockData);
    }

    /**
     * Perform Read Value command with MIFARE Classic PICC.
     *
     * <p>Applications must discovery MIFARE CLASSIC tag before call this method.
     *
     * @param blockAddress address of Value block to read, starting from 0.
     * @return value to read from the MIFARE Classic.
     * @throws IOException if there is an I/O failure, transmission error, or
     * Value block read from MIFARE CLASSIC is invalid format.
     */
    public int readValue(int blockAddress) throws IOException {
        byte[] data = this.read(blockAddress);

        if (!checkValueBlockFormat(data)) {
            throw new IOException("Invalid value block.");
        }

        /* Retrieve Address Data. */
        mValueAddressData = data[12];

        return byteArrayToValue(Arrays.copyOf(data, 4));
    }

    /**
     * Return address data of Value block.
     *
     * <p>Applications should perform {@link #readValue} method first.
     *
     * @return value read from the MIFARE Classic
     */
    public int getValueAddressData() {
        return mValueAddressData;
    }

    /**
     * Perform Increment command with MIFARE Classic PICC.
     *
     * <p>Applications must discovery MIFARE CLASSIC tag before call this method.
```

```
     *
     * @param blockAddress address of block to increment, starting from 0.
     * @param value value to increment by.
     * @throws IOException if there is an I/O failure, transmission error.
     */
    public void increment(int blockAddress, int value) throws IOException {
        int retval;
        byte[] data = valueToByteArray(value);

        retval = mTech.mifareINCREASE(blockAddress, data);
        if (retval != 0) {
            throw new IOException(String.format("[0x%04X] mifareINCREASE", retval));
        }
    }

    /**
     * Perform Decrement command with MIFARE Classic PICC.
     *
     * <p>Applications must discovery MIFARE CLASSIC tag before call this method.
     *
     * @param blockAddress address of block to decrement, starting from 0
     * @param value value to decrement by.
     * @throws IOException if there is an I/O failure, transmission error.
     */
    public void decrement(int blockAddress, int value) throws IOException {
        int retval;
        byte[] data = valueToByteArray(value);

        retval = mTech.mifareDECREASE(blockAddress, data);
        if (retval != 0) {
            throw new IOException(String.format("[0x%04X] mifareDECREASE", retval));
        }
    }

    /**
     * Perform Transfer command with MIFARE Classic PICC.
     *
     * @param blockAddress address of block the transfer buffer shall be transferred to
     * @throws IOException if there is an I/O failure, transmission error.
     */
    public void transfer(int blockAddress) throws IOException {
        int retval;

        retval = mTech.mifareTRANSFER(blockAddress);
        if (retval != 0) {
            throw new IOException(String.format("[0x%04X] mifareTRANSFER", retval));
        }
    }

    /**
     * Perform Restore command with MIFARE Classic PICC.
     *
     * @param blockAddress address of block the transfer buffer shall be restored from
     * @throws IOException if there is an I/O failure, transmission error.
     */
    public void restore(int blockAddress) throws IOException {
        int retval;

        retval = mTech.mifareRESTORE(blockAddress);
        if (retval != 0) {
            throw new IOException(String.format("[0x%04X] mifareRESTORE", retval));
        }
    }
}

/**
 * Mifare Classic example.
 */
private final Runnable runMifareClassicCExample = new Runnable() {
    public void run() {
        String log = "";
        int rc;

        CtCL cl = new CtCL();
        MifareClassic mfc = new MifareClassic(cl);
        boolean foundCard = false;
```

```
/**
 * Depending on user's implement case.
 */
int mfcBlockRWAddr = 4;
int mfcBlockValueAddr = 5;
int mfcBlockBackupAddr = 6;
int price = 1;

try {
    /* Find ISO14443A Card. */
    byte[] ATQA = new byte[2];
    byte[] SAK = new byte[1];
    byte[] CSN = new byte[10];

    rc = cl.typeAActiveFromIdle(0, ATQA, SAK, CSN);
    if (rc != 0) {
        throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
    }
    foundCard = true;

    /* Found card information. */
    log += String.format("ATQA = 0x%02X, 0x%02X. SAK = 0x%02X.\n\n", ATQA[0], ATQA[1], SAK[0]);

    /* Copy uid data. */
    byte[] uid = Arrays.copyOf(CSN, cl.getCSNLen());

    /**
     * Set authentication key of Mifare(R) PICC.
     * The key is depending on user's Mifare Classic Card.
     */
    byte[] mfcAuthKey = new byte[MifareClassic.AUTH_KEY_LENGTH];
    Arrays.fill(mfcAuthKey, (byte) 0xFF);

    /* Perform Mifare Classic <Authenticate> command. */
    mfc.authenticateBlockWithKeyA(4, mfcAuthKey, uid);

    /* Generate random number which will write into data block. */
    byte[] wrData = new byte[MifareClassic.DATA_BLOCK_LENGTH];
    for (int i = 0; i < wrData.length; i++) {
        wrData[i] = (byte) (Math.random() * 255);
    }

    /* Write data to Mifare Classic PICC. */
    mfc.write(mfcBlockRWAddr, wrData);

    log += String.format("Write Block Address (0x%02X) : \n", mfcBlockRWAddr);
    log += byteArrayToHexString(wrData) + "\n\n";

    /* Read data from Mifare Classic PICC. */
    byte[] rdData = mfc.read(mfcBlockRWAddr);

    log += String.format("Read Block Address (0x%02X) : \n", mfcBlockRWAddr);
    log += byteArrayToHexString(rdData) + "\n\n";

    /* Compare Write and Read data to/from Mifare PICC. */
    if (!Arrays.equals(rdData, wrData)) {
        log += "[Compare] Fail\n\n";
    } else {
        log += "[Compare] success\n\n";
    }

    /**
     * Create a Value-Format block.
     * Setting value is @price.
     * Setting backup block address is @mfcBlockBackupAddr.
     */
    log += String.format("[writeValue] Init value %d.\n", price);
    mfc.writeValue(mfcBlockValueAddr, price, mfcBlockBackupAddr);

    /* Read value from Value-Format block. */
    int value = mfc.readValue(mfcBlockValueAddr);
    int addrData = mfc.getValueAddressData();

    /* Check stored value and address */
    if (value != price || addrData != mfcBlockBackupAddr) {
        log += "[Write] Value-Format block fail.\n";
    } else {
```

```
                    log += "[Write] Value-Format block success.\n";
                }

                /**
                 * Create a Value-Format block which is backup block.
                 * Setting value equal value of @mfcBlockValueAddr.
                 * Setting backup block address is itself @mfcBlockBackupAddr.
                 */
                mfc.writeValue(mfcBlockBackupAddr, value, mfcBlockBackupAddr);

                /* Increment 10. */
                mfc.increment(mfcBlockValueAddr, 10);

                /* Store result to block. */
                mfc.transfer(mfcBlockValueAddr);

                value = mfc.readValue(mfcBlockValueAddr);
                log += String.format("[Increment $10] Value is %d.\n", value);

                /* Decrement 5. */
                mfc.decrement(mfcBlockValueAddr, 5);

                /* Store result to block. */
                mfc.transfer(mfcBlockValueAddr);

                value = mfc.readValue(mfcBlockValueAddr);
                log += String.format("[Decrement $5] Value is %d.\n", value);

                /* Restore @mfcBlockValueAddr from @mfcBlockBackupAddr. */
                mfc.restore(mfcBlockBackupAddr);

                /* Store result to block. */
                mfc.transfer(mfcBlockValueAddr);

                value = mfc.readValue(mfcBlockValueAddr);
                log += String.format("[Restore] Value is %d.\n", value);

            } catch (IOException e) {
                log += "[IOException]" + e.getMessage() + "\n";
            } catch (Exception e) {
                log += "[Other Exception]" + e.getMessage() + "\n";
            }

            if (foundCard) {
                cl.powerOff();
            }
        }
};
```

Example : Mifare Ultralight, Ultralight C

```
import java.io.IOException;
import java.security.InvalidAlgorithmParameterException;
import java.security.InvalidKeyException;
import java.security.NoSuchAlgorithmException;
import java.util.Arrays;

import javax.crypto.BadPaddingException;
import javax.crypto.Cipher;
import javax.crypto.IllegalBlockSizeException;
import javax.crypto.NoSuchPaddingException;
import javax.crypto.SecretKey;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;

import CTOS.CtCL;

class MifareUltralight {
    private final static String TAG = "MifareUltralight";

    /** MIFARE product ACK    response code */
    private final byte MF_RESP_ACK    = (byte)0x0A;
    /** MIFARE product NAK0 response code */
    private final byte MF_RESP_NAK0 = (byte)0x00;
    /** MIFARE product NAK1 response code */
    private final byte MF_RESP_NAK1 = (byte)0x01;
    /** MIFARE product NAK4 response code */
    private final byte MF_RESP_NAK4 = (byte)0x04;
    /** MIFARE product NAK5 response code */
```

```
    private final byte MF_RESP_NAK5 = (byte)0x05;
    /** MIFARE product NAK6 response code */
    private final byte MF_RESP_NAK6 = (byte)0x06;
    /** MIFARE product NAK7 response code */
    private final byte MF_RESP_NAK7 = (byte)0x07;
    /** MIFARE product NAK9 response code */
    private final byte MF_RESP_NAK9 = (byte)0x09;

    /** Mifare Ultralight Read command code */
    private final static byte CMD_READ    = (byte)0x30;
    /** Mifare Ultralight Write command code */
    private final static byte CMD_WRITE = (byte)0xA2;
    /** Mifare UltralightC Authenticate command code */
    private final static byte CMD_AUTH    = (byte)0x1A;

    /** Length of a read MIFARE(R) Ultralight data block. */
    public final static int READ_BLOCK_LENGTH     = 16;
    /** Length of a write MIFARE(R) Ultralight data block. */
    public final static int WRITE_BLOCK_LENGTH    = 4;

    /** Length of an MIFARE(R) Ultralight DES block. */
    public final static int DES_BLOCK_SIZE    = 8;
    /** Length of a MIFARE(R) Ultralight DES authentication key. */
    public final static int DES_KEY_LENGTH    = 16;
    /** Max timeout value [in millisec] of receiving of MIFARE(R) Ultralight. */
    public final static int RX_TIMEOUT_MS     = 5;

    /** ISO14443-3 Type A CRC16 */
    private final static int CRC16_TYPE_ISO14443A = 0;
    /** ISO14443-3 Type B CRC16 */
    private final static int CRC16_TYPE_ISO14443B = 1;

    final CtCL mTech;
    private static int mExchangeI3Timeout = -1;

    /**
     * Calculate CRC of ISO14443 specification.
     * @param crcType Specification of {@link #CRC16_TYPE_ISO14443A}{@link #CRC16_TYPE_ISO14443B}.
     * @param data Input data.
     * @return two bytes CRC.
     * @throws IllegalArgumentException invalid specification.
     */
    private static byte[] calcISO14443Crc16(int crcType, byte[] data) throws IllegalArgumentException {
        short crc16;
        byte block;
        byte[] crc16Array;
        int i;

        switch(crcType) {
            case CRC16_TYPE_ISO14443A:
                /* ITU-V.41 */
                crc16 = (short)0x6363;
                break;

            case CRC16_TYPE_ISO14443B:
                /* ISO/IEC 13239 (formerly ISO/IEC 3309) */
                crc16 = (short)0xFFFF;
                break;

            default:
                throw new IllegalArgumentException(String.format("calcISO14443Crc16 doesn't support this type, %d.",
crcType));
        }

        for (i = 0 ; i < data.length ; i++) {
            block = data[i];
            block = (byte)((block & 0x00FF) ^ (crc16 & 0x00FF));
            block = (byte)((block & 0x00FF) ^ ((block & 0x00FF) << 4));
            crc16 = (short)(((crc16 & 0xFFFF) >> 8) ^ ((block & 0x00FF) << 8) ^ ((block & 0x00FF) << 3) ^ ((block & 0x00FF)
>> 4));
        }

        if (crcType == CRC16_TYPE_ISO14443B) {
            /* ISO/IEC 13239 (formerly ISO/IEC 3309) */
            crc16 = (short)(~crc16);
        }
```

```
        crc16Array = new byte[2];
        crc16Array[0] = (byte)(crc16 & 0x00FF);
        crc16Array[1] = (byte)(((crc16 & 0xFFFF) >> 8) & 0x00FF);

        return crc16Array;
    }

    /**
     * Decrypt data with 2KEY 3DES CBC mode.
     * @param dataToDecrypt input data.
     * @param iv initialization vector.
     * @param key decryption key.
     * @return decrypted data.
     * @throws IllegalArgumentException invalid data, value.
     */
    private static byte[] decrypt_2Key3DES_CBC(byte[] dataToDecrypt, byte[] iv, byte[] key) throws IllegalArgumentException {
        SecretKey secretKey;
        Cipher cipher;
        IvParameterSpec ivForCBC;
        byte[] plainText;

        if (((dataToDecrypt.length % 8) != 0) || (dataToDecrypt.length == 0)) {
            throw new IllegalArgumentException("Invalid data length.");
        }
        if (iv.length != 8) {
            throw new IllegalArgumentException("Invalid iv length.");
        }
        if (key.length != 16) {
            throw new IllegalArgumentException("Invalid key length.");
        }

        secretKey = new SecretKeySpec(key, 0, key.length, "DESede");

        try {
            cipher = Cipher.getInstance("DESede/CBC/NoPadding");
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalArgumentException(e.toString());
        } catch (NoSuchPaddingException e) {
            throw new IllegalArgumentException(e.toString());
        }

        ivForCBC = new IvParameterSpec(iv);

        try {
            cipher.init(Cipher.DECRYPT_MODE, secretKey, ivForCBC);
        } catch (InvalidKeyException e) {
            throw new IllegalArgumentException(e.toString());
        } catch (InvalidAlgorithmParameterException e) {
            throw new IllegalArgumentException(e.toString());
        }

        try {
            plainText = cipher.doFinal(dataToDecrypt);
        } catch (IllegalBlockSizeException e) {
            throw new IllegalArgumentException(e.toString());
        } catch (BadPaddingException e) {
            throw new IllegalArgumentException(e.toString());
        }

        System.arraycopy(dataToDecrypt, (dataToDecrypt.length - iv.length), iv, 0, iv.length);

        return Arrays.copyOf(plainText, dataToDecrypt.length);
    }

    /**
     * Encrypt data with 2KEY 3DES CBC mode.
     * @param dataToEncrypt
     * @param iv initialization vector.
     * @param key decryption key.
     * @return encrypted data.
     * @throws IllegalArgumentException invalid data, value.
     */
    private static byte[] encrypt_2Key3DES_CBC(byte[] dataToEncrypt, byte[] iv, byte[] key) throws IllegalArgumentException {
        SecretKey secretKey;
        Cipher cipher;
        IvParameterSpec ivForCBC;
        byte[] encryptedData;
```

```
        if (((dataToEncrypt.length % 8) != 0) || (dataToEncrypt.length == 0)) {
            throw new IllegalArgumentException("Invalid data length.");
        }
        if (iv.length != 8) {
            throw new IllegalArgumentException("Invalid iv length.");
        }
        if (key.length != 16) {
            throw new IllegalArgumentException("Invalid key length.");
        }

        secretKey = new SecretKeySpec(key, 0, key.length, "DESede");

        try {
            cipher = Cipher.getInstance("DESede/CBC/NoPadding");
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalArgumentException(e.toString());
        } catch (NoSuchPaddingException e) {
            throw new IllegalArgumentException(e.toString());
        }

        ivForCBC = new IvParameterSpec(iv);

        try {
            cipher.init(Cipher.ENCRYPT_MODE, secretKey, ivForCBC);
        } catch (InvalidKeyException e) {
            throw new IllegalArgumentException(e.toString());
        } catch (InvalidAlgorithmParameterException e) {
            throw new IllegalArgumentException(e.toString());
        }

        try {
            encryptedData = cipher.doFinal(dataToEncrypt);
        } catch (IllegalBlockSizeException e) {
            throw new IllegalArgumentException(e.toString());
        } catch (BadPaddingException e) {
            throw new IllegalArgumentException(e.toString());
        }

        System.arraycopy(encryptedData, (encryptedData.length - iv.length), iv, 0, iv.length);

        return encryptedData;
    }

    /**
     * Set transparent receiving timeout value.
     * @param millisec    timeout value with milli second
     * @throws IOException invalid value
     */
    private void setTimeout(int millisec) throws IOException {
        int retval;

        if (millisec <= 0) {
            throw new IOException("Invalid timeout.");
        }
        if (mExchangeI3Timeout == millisec) {
            return ;
        }

        retval = mTech.tclSetTimeout(millisec);
        if (retval != 0) {
            throw new IOException(String.format("[0x%04X] tclSetTimeout", retval));
        }

        mExchangeI3Timeout = millisec;
    }

    /**
     * Perform ISO14443-3 data exchange with Mifare product-based PICC.
     * @param data Data to transmit.
     * @param millisecTimeout timeout value with milli second.
     * @return received data.
     * @throws IOException if there is an I/O failure, transmission error, or invalid command.
     */
    private byte[] mifareExchangeI3(byte[] data, int millisecTimeout) throws IOException {
        int retval;
        byte[] rx = new byte[256];
```

```
        byte[] crc16;

        this.setTimeout(millisecTimeout);

        /* Exchange data. */
        retval = mTech.tclTransparentRaw(data, 0, rx,true,false, true);
        if (retval != 0 && retval != 0x837B) {
            throw new IOException(String.format("[0x%04X] tclTransparentRaw", retval));
        }

        /* ACK/NAK Handling */
        if (retval == 0x837B) {
            if (mTech.getRLen() != 1) {
                throw new IOException("Protocol error.");
            }
            /* Retrieve bit-count */
            if (mTech.getRLastBits() != 4) {
                throw new IOException("Protocol error.");
            }

            switch (rx[0]) {
                /* Get ACK. */
                case MF_RESP_ACK:
                    break;
                /* Mapping of NAK codes. */
                case MF_RESP_NAK0:
                case MF_RESP_NAK1:
                case MF_RESP_NAK4:
                case MF_RESP_NAK5:
                case MF_RESP_NAK6:
                case MF_RESP_NAK7:
                case MF_RESP_NAK9:
                    throw new IOException(String.format("MIFARE(R) NAK %d.", rx[0]));
                    /* Get Unknown response. */
                default:
                    throw new IOException(String.format("MIFARE(R) unknown respnse, %d.", rx[0]));
            }

            return null;
        } else {
            if (mTech.getRLen() < 3) {
                throw new IOException("Protocol error.");
            }

            /* Calculate CRC of received data */
            crc16 = this.calcISO14443Crc16(CRC16_TYPE_ISO14443A, Arrays.copyOf(rx, (mTech.getRLen() - 2)));

            /* CRC check */
            if ((crc16[0] != rx[mTech.getRLen() - 2]) || (crc16[1] != rx[mTech.getRLen() - 1])) {
                throw new IOException("Invalid CRC.");
            }

            /* Removal CRC bytes. */
            return Arrays.copyOf(rx, (mTech.getRLen() - 2));
        }
    }

    public MifareUltralight(CtCL tech) {
        this.mTech = tech;
    }

    /**
     * Perform Mifare Ultralight read command.
     * @param pageAddress Start page address.
     * @return data content of the addressed pages.
     * @throws IOException if there is an I/O failure, transmission error, or invalid card.
     */
    public byte[] read(int pageAddress) throws IOException {
        byte[] cmd;
        byte[] recv;

        cmd = new byte[2];
        cmd[0] = CMD_READ;
        cmd[1] = (byte)pageAddress;

        try {
            recv = this.mifareExchangeI3(cmd, RX_TIMEOUT_MS);
```

```
        } catch (IOException e) {
            throw e;
        }
        if (recv == null) {
            throw new IOException("Receive no data");
        }
        if (recv.length != READ_BLOCK_LENGTH) {
            throw new IOException("Invalid length of receive data");
        }

        return recv;
    }

    /**
     * Perform Mifare Ultralight write command.
     * @param pageAddress Page address write to.
     * @param data data write to page.
     * @throws IOException if there is an I/O failure, transmission error, or invalid card.
     */
    public void write(int pageAddress, byte[] data) throws IOException {
        byte[] cmd;
        byte[] recv;

        if (data.length != WRITE_BLOCK_LENGTH) {
            throw new IOException("Wrong length of writing data");
        }

        cmd = new byte[2 + data.length];
        cmd[0] = CMD_WRITE;
        cmd[1] = (byte)pageAddress;
        System.arraycopy(data, 0, cmd, 2, data.length);

        try {
            recv = this.mifareExchangeI3(cmd, RX_TIMEOUT_MS);
        } catch (IOException e) {
            throw e;
        }
        if (recv != null) {
            throw new IOException("Invalid length of receive data");
        }
    }

    /**
     * Perform Mifare Ultralight C authenticate command.
     * @param key 3DES Authentication Key.
     * @throws IOException if there is an I/O failure, transmission error, or invalid card.
     */
    public void ulcAuthenticate(byte[] key) throws IOException {
        byte[] cmd;
        byte[] recv;
        byte[] iv;
        byte[] rndA;
        byte[] rndB;
        byte[] ekRndABp;
        byte[] rndAFromPicc;

        /* Send Authentication Part-1 command */
        try {
            cmd = new byte[2];
            cmd[0] = CMD_AUTH;
            cmd[1] = 0x00;

            recv = this.mifareExchangeI3(cmd, RX_TIMEOUT_MS);
            if (recv == null) {
                throw new IOException("Receive no data");
            }
            if (recv.length != (1 + DES_BLOCK_SIZE)) {
                throw new IOException("Invalid length of receive data");
            }
            if (recv[0] != (byte)0xAF) {
                throw new IOException("Invalid protocol of receive data frame");
            }
        } catch (IOException e) {
            throw new IOException("Authentication part1, " + e.getMessage());
        }

        /* Init IV of 2K3DES CBC */
```

```
            iv = new byte [DES_BLOCK_SIZE];
            Arrays.fill(iv, (byte)0);

            /* Retrieve RndB from MIFARE PICC. */
            try {
                byte[] ekRndB = Arrays.copyOfRange(recv, 1, recv.length);
                rndB = this.decrypt_2Key3DES_CBC(ekRndB, iv, key);
            } catch (IllegalArgumentException e){
                throw new IOException(e.getMessage());
            }

            /* Generate RndA. */
            rndA = new byte[8];
            for (int i = 0 ; i < rndA.length ; i++) {
                rndA[i] = (byte)(Math.random() * 255);
            }

            /* Generate ek(RandA || RandB') */
            try {
                byte []rndABp = new byte[16];
                /* copy RndA.*/
                System.arraycopy(rndA, 0, rndABp, 0, rndA.length);
                /* Copy RndB' which is from rotating RndB.*/
                System.arraycopy(rndB, 1, rndABp, 8, rndB.length - 1);
                rndABp[15] = rndB[0];
                /* Encrypt (RandA || RandB') */
                ekRndABp = this.encrypt_2Key3DES_CBC(rndABp, iv, key);
            } catch (IllegalArgumentException e) {
                throw new IOException(e.getMessage());
            }

            /* Send Authentication Part-2 command */
            try {
                cmd = new byte[17];
                cmd[0] = (byte)0xAF;
                System.arraycopy(ekRndABp, 0, cmd, 1, ekRndABp.length);

                recv = this.mifareExchangeI3(cmd, RX_TIMEOUT_MS);
                if (recv == null) {
                    throw new IOException("Receive no data");
                }
                if (recv.length != (1 + DES_BLOCK_SIZE)) {
                    throw new IOException("Invalid length of receive data");
                }
                if (recv[0] != (byte)0x00) {
                    throw new IOException("Invalid protocol of receive data frame");
                }
            } catch (IOException e) {
                throw new IOException("Authentication part2, " + e.getMessage());
            }

            /* Retrieve RndA from MIFARE PICC. */
            try {
                /* Retrieve ek(RndA') from MIFARE PICC. */
                byte[] ekRndAp = Arrays.copyOfRange(recv, 1, recv.length);
                /* Decrypt RndA' from ek(RndA'). */
                byte[] rndAp = this.decrypt_2Key3DES_CBC(ekRndAp, iv, key);
                /* Retrieve RndA from RndA'. */
                rndAFromPicc = new byte[8];
                rndAFromPicc[0] = rndAp[7];
                System.arraycopy(rndAp, 0, rndAFromPicc, 1,rndAFromPicc.length - 1);

            } catch (IllegalArgumentException e){
                throw new IOException(e.getMessage());
            }

            /* Compare rndA and rndAFromPicc receiving from PICC*/
            if (Arrays.equals(rndAFromPicc, rndA) == false) {
                throw new IOException("Invalid authentication.");
            }
        }
    }

    /**
     * Mifare Ultralight example.
     */
    private Runnable runMifareULExample = new Runnable() {
```

```
    public void run() {
        String log = "";
        int rc;

        CtCL cl = new CtCL();
        MifareUltralight mful = new MifareUltralight(cl);
        boolean foundCard = false;

        /**
         * Depending on user implementation case.
         */
        int pageRW = 4;

        try {
            /* Find ISO14443A Card. */
            byte[] ATQA = new byte[2];
            byte[] SAK = new byte[1];
            byte[] CSN = new byte[10];

            rc = cl.typeAActiveFromIdle(0, ATQA, SAK, CSN);
            if (rc != 0) {
                throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
            }
            foundCard = true;

            /* Found card information. */
            log += String.format("ATQA = 0x%02X, 0x%02X. SAK = 0x%02X.\n\n", ATQA[0], ATQA[1], SAK[0]);

            /* Generate random number which will write into data block. */
            byte[] wrData = new byte[MifareUltralight.WRITE_BLOCK_LENGTH];
            for (int i = 0 ; i < wrData.length ; i++) {
                wrData[i] = (byte)(Math.random() * 255);
            }

            /* Write data to Mifare UltralightC PICC. */
            mful.write(pageRW, wrData);

            log += String.format("Write Page Address (0x%02X) : \n", pageRW);
            log += byteArrayToHexString(wrData) + "\n\n";

            /* Read data from Mifare UltralightC PICC. */
            byte[]rdData = mful.read(pageRW);

            log += String.format("Read Page Address (0x%02X) : \n", pageRW);
            log += byteArrayToHexString(rdData) + "\n\n";

            /* Compare Write and Read data to/from Mifare PICC. */
            if (!Arrays.equals(Arrays.copyOf(rdData, MifareUltralight.WRITE_BLOCK_LENGTH), wrData)) {
                log += "[Compare] Fail\n\n";
            } else {
                log += "[Compare] success\n\n";
            }

        } catch (IOException e) {
            log += "[IOException]" + e.getMessage() + "\n";
        } catch (Exception e) {
            log += "[Other Exception]" + e.getMessage() + "\n";
        }

        if (foundCard) {
            cl.powerOff();
        }
    }
};

/**
 * Mifare Ultralight C example.
 */
private Runnable runMifareULCExample = new Runnable() {
    public void run() {
        String log = "";
        int rc;

        CtCL cl = new CtCL();
        MifareUltralight mful = new MifareUltralight(cl);
        boolean foundCard = false;
```

```
            /**
             * Depending on user implementation case.
             */
            int pageRW = 4;

            try {
                /* Find ISO14443A Card. */
                byte[] ATQA = new byte[2];
                byte[] SAK = new byte[1];
                byte[] CSN = new byte[10];

                rc = cl.typeAActiveFromIdle(0, ATQA, SAK, CSN);
                if (rc != 0) {
                    throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
                }
                foundCard = true;

                /* Found card information. */
                log += String.format("ATQA = 0x%02X, 0x%02X. SAK = 0x%02X.\n\n", ATQA[0], ATQA[1], SAK[0]);

                /**
                 * Set authentication key of Mifare(R) UltralightC PICC.
                 * The key is depending on user's Mifare UltralightC Card.
                 */
                byte[] mfulcAuthKey = new byte[MifareUltralight.DES_KEY_LENGTH];
                String ulcStringKey = "IEMKAERB!NACUOYF";
                System.arraycopy(ulcStringKey.getBytes(), 0, mfulcAuthKey, 0, mfulcAuthKey.length);

                /* Perform Mifare UltralightC <Authenticate> command */
                mful.ulcAuthenticate(mfulcAuthKey);
                log += "Authenticate success!\n\n";

                /* Generate random number which will write into data block. */
                byte[] wrData = new byte[MifareUltralight.WRITE_BLOCK_LENGTH];
                for (int i = 0 ; i < wrData.length ; i++) {
                    wrData[i] = (byte)(Math.random() * 255);
                }

                /* Write data to Mifare UltralightC PICC. */
                mful.write(pageRW, wrData);

                log += String.format("Write Page Address (0x%02X) : \n", pageRW);
                log += byteArrayToHexString(wrData) + "\n\n";

                /* Read data from Mifare UltralightC PICC. */
                byte[]rdData = mful.read(pageRW);

                log += String.format("Read Page Address (0x%02X) : \n", pageRW);
                log += byteArrayToHexString(rdData) + "\n\n";

                /* Compare Write and Read data to/from Mifare PICC. */
                if (!Arrays.equals(Arrays.copyOf(rdData, MifareUltralight.WRITE_BLOCK_LENGTH), wrData)) {
                    log += "[Compare] Fail\n\n";
                } else {
                    log += "[Compare] success\n\n";
                }

            } catch (IOException e) {
                log += "[IOException]" + e.getMessage() + "\n";
            } catch (Exception e) {
                log += "[Other Exception]" + e.getMessage() + "\n";
            }

            if (foundCard) {
                cl.powerOff();
            }
        }
};
```

Example : CL General

```
/**
 * This example is used to demonstrate how to set RF Auto-Closing.
 */
import CTOS.CtCL;

   /** Contactless card reader. */
   CtCL cl = new CtCL();
```

```
    private static final int d_CL_POWER_STATUS_OFF = 0;
    private static final int d_CL_POWER_STATUS_ON = 1;

    private Runnable runClGeneralSample = new Runnable() {

    public void run() {
        String log = "";
        int rc;

        CtCL cl = new CtCL();

        try {
            rc = cl.powerOff();
            if (rc != 0) {
                throw new IOException(String.format("[0x%04X] powerOff.", rc));
            }

            /* Set RF Auto-Closing.*/
            Auto[0] = true;
            AutoOffTime[0] = 500;
            LEDOn[0] = false;
            rc = cl.setPowerModeParm(Auto[0], AutoOffTime[0], LEDOn[0]);
            if (rc != 0) {
                throw new IOException(String.format("[0x%04X] setPowerModeParm.", rc));
            }

            rc = cl.getPowerModeParm(Auto, AutoOffTime, LEDOn);
            if (rc != 0) {
                throw new IOException(String.format("[0x%04X] getPowerModeParm.", rc));
            }

            if (Auto[0] == true) {
                log += "RF will be auto-closed after "+ AutoOffTime[0] + " ms.\n\n";
            } else {
                log += "RF will be not auto-closed.\n\n";
            }

            rc = cl.powerOn();
            if (rc != 0) {
                throw new IOException(String.format("[0x%04X] powerOn.", rc));
            }

            /* Delay 500ms */
            SystemClock.sleep(500);

            rc = cl.powerStatus();
            if (rc != 0) {
                throw new IOException(String.format("[0x%04X] powerStatus.", rc));
            }

            int Status = cl.getStatus();

             if (Status == d_CL_POWER_STATUS_ON) {
                rc = cl.powerOff();
                if (rc != 0) {
                    throw new IOException(String.format("[0x%04X] powerOff.", rc));
                }
                    throw new IOException("RF is not closed.");
             }

            log += "RF had been closed." + "\n\n";
        } catch (IOException e) {
            log += "[IOException]" + e.getMessage() + "\n";
        } catch (Exception e) {
            log += "[Other Exception]" + e.getMessage() + "\n";
        }
        cl.powerOff();
    }
};
```

Example : Type A example I

```
/**
 * This example is used to demonstrate that the PICC is activated from its IDLE state to ACTIVE state
 * and then to PROTOCOL state to send APDU.
 */

    import CTOS.CtCL;
```

Castles Technology

```
/** Contactless card reader. */
CtCL cl = new CtCL();

public String byteArrayToHexString(byte[] data) {
   int i;
   String s = "";

   for (i = 0 ; i < data.length ; i++) {
      s += String.format("0x%02X, ", data[i]);
   }

    return s;
}

private Runnable runTypeASample = new Runnable() {
   public void run() {
      String log = "";
      int rc;

      CtCL cl = new CtCL();
      boolean foundCard = false;

      try {

         /* Find ISO14443A Card.
          * if success, Type-A card is in ACTIVE state of ISO14443-3.
          */
         byte[] ATQA = new byte[2];
         byte[] SAK = new byte[1];
         byte[] CSN = new byte[10];

         rc = cl.typeAActiveFromIdle(0, ATQA, SAK, CSN);
         if (rc != 0) {
            throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
         }
         foundCard = true;

         /* Check picc compliant with ISO14443-4 or not */
         boolean i14443p4Comp = false;
         if ((SAK[0] & 0x20) == 0x20) {
            i14443p4Comp = true;
         }

          /* Found card information. */
          log += String.format("ATQA = 0x%02X, 0x%02X. SAK = 0x%02X.\n\n", ATQA[0], ATQA[1], SAK[0]);
          log += String.format("i1443p4Comp = %b\n\n", i14443p4Comp);

          if (i14443p4Comp == false) {
             throw new IOException("This card doesn't support ISO14443-4 protocol.");
          }

             /* ISO14443-4A Card protocol activation.
              * if success, Type-A card is in PROTOCOL state of ISO14443-4.
              */
             byte[] ATS = new byte[256];
             rc = cl.RATS(0, ATS);
             if (rc != 0) {
                throw new IOException(String.format("[0x%04X] RATS", rc));
             }

             /* Read out valid length of ATS response */
             int ATSLen = cl.getATSLen();

             log += "ATS : " + byteArrayToHexString(Arrays.copyOf(ATS, ATSLen)) + "\n\n";

             /* Data exchange by ISO14443-4 APDU protocol. */
             byte SBuf[] = new byte [] {(byte)0x00, (byte)0xa4, (byte)0x04, (byte)0x00, (byte)0x07,
                   (byte)0xa0, (byte)0x00, (byte)0x00, (byte)0x00, (byte)0x04, (byte)0x10, (byte)0x10};
             byte RBuf[] = new byte[256];

             rc = cl.APDU(SBuf, RBuf);
             if (rc != 0) {
                throw new IOException(String.format("[0x%04X] APDU", rc));
             }
             int RBufLen = cl.getRLen();
```

Castles Technology

```
                log += "APDU Send : " + byteArrayToHexString(SBuf) + "\n\n";
                log += "APDU Recv : " + byteArrayToHexString(Arrays.copyOf(RBuf, RBufLen)) + "\n\n";

                /* Here may be more data exchange by APDU protocol ... */
            } catch (IOException e) {
                log += "[IOException]" + e.getMessage() + "\n";
            } catch (Exception e) {
                log += "[Other Exception]" + e.getMessage() + "\n";
            }

            if (foundCard) {
               cl.powerOff();
            }
        }
    };
```

Example : Type A example II

```
/**
 * This example is used to demonstrate the execution of the Type A Request command to change
 * the PICC from the IDLE status to the READY status, and issue the SELECT command and the ANTICOLLISION command
 * to obtain the CSN and enter the PROTOCOL status to send APDU.
 */

   import CTOS.CtCL;

   /** Contactless card reader. */
   CtCL cl = new CtCL();

   public String byteArrayToHexString(byte[] data) {
      int i;
      String s = "";

      for (i = 0 ; i < data.length ; i++) {
         s += String.format("0x%02X, ", data[i]);
      }

      return s;
   }

    private Runnable runTypeASample = new Runnable() {
       public void run() {
          String log = "";
          int rc;

          CtCL cl = new CtCL();
          boolean foundCard = false;

          try {
             byte[] ATQA = new byte[2];
             byte[] SNR = new byte[4];
                int[] SelCode = new int [] {0x93, 0x95, 0x97};
                byte[] SAK = new byte[1];
                int i;

                /* Execute Type A Request command, PICC will enter READY status from IDLE status. */
                rc = cl.REQA(ATQA);
                if (rc != 0) {
                   throw new IOException(String.format("[0x%04X] REQA", rc));
                }

                /* Execute ANTICOLLISION command and select PICC and, PICC will enter ACTIVE status from READY status. */
                for (i = 0; i < 3; i++) {
                   rc = cl.antia(SelCode[i], SNR, 0);
                   if (rc != 0) {
                      throw new IOException(String.format("[0x%04X] antia", rc));
                   }

                   rc = cl.selecta(SelCode[i], SNR, SAK);
                   if (rc != 0) {
                      throw new IOException(String.format("[0x%04X] selecta", rc));
                   }

                   if ((SAK[0] & 0x04) == 0x00) {
                      break;
                   } else if ((SAK[0] & 0x04) == 0x04) {
                      continue;
```

Castles Technology

```
                }
            }
            foundCard = true;

            /* Check picc compliant with ISO14443-4 or not */
            boolean i14443p4Comp = false;
            if ((SAK[0] & 0x20) == 0x20) {
                i14443p4Comp = true;
            }

            /* Found card information. */
            log += String.format("ATQA = 0x%02X, 0x%02X. SAK = 0x%02X.\n\n", ATQA[0], ATQA[1], SAK[0]);
            log += String.format("i1443p4Comp = %b\n\n", i14443p4Comp);

            if (i14443p4Comp == false) {
                throw new IOException("This card doesn't support ISO14443-4 protocol.");
            }

            /* ISO14443-4A Card protocol activation.
             * if success, Type-A card is in PROTOCOL state of ISO14443-4.
             */
            byte[] ATS = new byte[256];
            rc = cl.RATS(0, ATS);
            if (rc != 0) {
                throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
            }
            /* Read out valid length of ATS response */
            int ATSLen = cl.getATSLen();

            log += "ATS : " + byteArrayToHexString(Arrays.copyOf(ATS, ATSLen)) + "\n\n";

            /* Data exchange by ISO14443-4 APDU protocol. */
            byte SBuf[] = new byte [] {(byte)0x00, (byte)0xa4, (byte)0x04, (byte)0x00, (byte)0x07,
                  (byte)0xa0, (byte)0x00, (byte)0x00, (byte)0x00, (byte)0x04, (byte)0x10, (byte)0x10};
            byte RBuf[] = new byte[256];

            rc = cl.APDU(SBuf, RBuf);
            if (rc != 0) {
                throw new IOException(String.format("[0x%04X] APDU", rc));
            }
            int RBufLen = cl.getRLen();

            log += "APDU Send : " + byteArrayToHexString(SBuf) + "\n\n";
            log += "APDU Recv : " + byteArrayToHexString(Arrays.copyOf(RBuf, RBufLen)) + "\n\n";

            /* Here may be more data exchange by APDU protocol ... */
        } catch (IOException e) {
            log += "[IOException]" + e.getMessage() + "\n";
        } catch (Exception e) {
            log += "[Other Exception]" + e.getMessage() + "\n";
        }

        if (foundCard) {
            cl.powerOff();
        }
    }
};
```

Example : Type A example III

```
/**
* This example is used to demonstrate that the PICC is activated from IDLE state to ACTIVE state and then execute
* HLTAcommand to enter HALT status,
* next to activated PICC from its HALT state to ACTIVE state and enter PROTOCOL state to send APDU.
*/

  import CTOS.CtCL;

  /** Contactless card reader. */
  CtCL cl = new CtCL();

  public String byteArrayToHexString(byte[] data) {
    int i;
    String s = "";

    for (i = 0 ; i < data.length ; i++) {
      s += String.format("0x%02X, ", data[i]);
```

Castles Technology

```
    }

    return s;
}

private Runnable runTypeASample = new Runnable() {
    public void run() {
        String log = "";
        int rc;

        CtCL cl = new CtCL();
        boolean foundCard = false;

        try {
            /* Find ISO14443A Card.
             * if success, Type-A card is in ACTIVE state of ISO14443-3.
             */
            byte[] ATQA = new byte[2];
            byte[] SAK = new byte[1];
            byte[] CSN = new byte[10];

            rc = cl.typeAActiveFromIdle(0, ATQA, SAK, CSN);
            if (rc != 0) {
                throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
            }
            foundCard = true;

            int CSNLen = cl.getCSNLen();

            /* Check picc compliant with ISO14443-4 or not */
            boolean i14443p4Comp = false;
            if ((SAK[0] & 0x20) == 0x20) {
                i14443p4Comp = true;
            }

            /* Found card information. */
            log += String.format("ATQA = 0x%02X, 0x%02X. SAK = 0x%02X.\n\n", ATQA[0], ATQA[1], SAK[0]);
            log += String.format("i1443p4Comp = %b\n\n", i14443p4Comp);

            if (i14443p4Comp == false) {
                throw new IOException("This card doesn't support ISO14443-4 protocol.");
            }

            /* Enter HALT status. */
            rc = cl.typeAHalt();
            if (rc != 0) {
                throw new IOException(String.format("[0x%04X] typeAHalt", rc));
            }

            /* Activate PICC form HALT status to ACTIVE status. */
            rc = cl.typeAActiveFromHalt(0, ATQA, SAK, Arrays.copyOf(CSN, CSNLen));
             if (rc != 0) {
                 throw new IOException(String.format("[0x%04X] typeAActiveFromHalt", rc));
             }

             /* Check picc compliant with ISO14443-4 or not */
             i14443p4Comp = false;
             if ((SAK[0] & 0x20) == 0x20) {
                i14443p4Comp = true;
             }

             /* Found card information. */
             log += String.format("ATQA = 0x%02X, 0x%02X. SAK = 0x%02X.\n\n", ATQA[0], ATQA[1], SAK[0]);
             log += String.format("i1443p4Comp = %b\n\n", i14443p4Comp);

             if (i14443p4Comp == false) {
                 throw new IOException("This card doesn't support ISO14443-4 protocol.");
             }

             /* ISO14443-4A Card protocol activation.
              * if success, Type-A card is in PROTOCOL state of ISO14443-4.
              */
             byte[] ATS = new byte[256];
             rc = cl.RATS(0, ATS);
             if (rc != 0) {
                throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
             }
```

```
            /* Read out valid length of ATS response */
            int ATSLen = cl.getATSLen();

            log += "ATS : " + byteArrayToHexString(Arrays.copyOf(ATS, ATSLen)) + "\n\n";

            /* Data exchange by ISO14443-4 APDU protocol. */
            byte SBuf[] = new byte [] {(byte)0x00, (byte)0xa4, (byte)0x04, (byte)0x00, (byte)0x07,
                  (byte)0xa0, (byte)0x00, (byte)0x00, (byte)0x00, (byte)0x04, (byte)0x10, (byte)0x10};
            byte RBuf[] = new byte[256];

            rc = cl.APDU(SBuf, RBuf);
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] APDU", rc));
            }
            int RBufLen = cl.getRLen();

            log += "APDU Send : " + byteArrayToHexString(SBuf) + "\n\n";
            log += "APDU Recv : " + byteArrayToHexString(Arrays.copyOf(RBuf, RBufLen)) + "\n\n";

            /* Here may be more data exchange by APDU protocol ... */

         } catch (IOException e) {
            log += "[IOException]" + e.getMessage() + "\n";
         } catch (Exception e) {
            log += "[Other Exception]" + e.getMessage() + "\n";
         }

         if (foundCard) {
            cl.powerOff();
         }
      }
   };
```

Example : Mifare Classic example I

```
/**
 * This example is used to demonstrate that after the PICC is activated,
 * the authentication can be performed after the key is loaded,
 * Mifare Classic commands such as Read, Write could be performed after the authentication is passed.
 */
   import CTOS.CtCL;

   /** Contactless card reader. */
   CtCL cl = new CtCL();

   private static final int CL_MFC_AUTH_KEY_TYPEA = 0x60;
   private static final int CL_MFC_AUTH_KEY_TYPEB = 0x61;
   private static final int CL_MFC_AUTH_KEY_LEN = 6;
   private static final int CL_MFC_AUTH_LOADKEY_LEN = 12;
   private static final int CL_MFC_DATA_BLOCK_NUM = 4;
   private static final int CL_MFC_VALUE_BLOCK_NUM = 8;
   private static final int CL_MFC_RESTORE_BLOCK_NUM = 9;
   private static final int CL_MFC_DATA_LENGTH = 16;
   private static final int CL_MFC_VALUE_LENGTH = 4;

   public String byteArrayToHexString(byte[] data) {
      int i;
      String s = "";

      for (i = 0 ; i < data.length ; i++) {
         s += String.format("0x%02X, ", data[i]);
      }

      return s;
   }

   private Runnable runMFCSample = new Runnable() {
      public void run() {
         String log = "";
         int rc, i;

         CtCL cl = new CtCL();
         boolean foundCard = false;

         try {
            /* Find ISO14443A Card. */
            byte[] ATQA = new byte[2];
```

```
            byte[] SAK = new byte[1];
            byte[] CSN = new byte[10];
            byte[] ReadBuf = new byte[CL_MFC_DATA_LENGTH];
            byte[] WriteBuf = new byte[CL_MFC_DATA_LENGTH];

            Arrays.fill(WriteBuf, (byte)(Math.random() * 10 + 1));

            rc = cl.typeAActiveFromIdle(0, ATQA, SAK, CSN);
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
            }
            foundCard = true;

            int CSNLen = cl.getCSNLen();

            /* Found card information. */
            log += String.format("ATQA = 0x%02X, 0x%02X. SAK = 0x%02X.\n\n", ATQA[0], ATQA[1], SAK[0]);

            /* Perform Mifare Classic <Authenticate> command, Key is depending on user's Mifare Classic Card.
            * Execute mifareLOADKEY(LoadKey(12 bytes)) and mifareAUTHEx() is equal to mifareAUTH2Ex(Key(6 bytes)).
            */
            byte[] Key = new byte[CL_MFC_AUTH_KEY_LEN ];
            byte[] LoadKey = new byte[CL_MFC_AUTH_LOADKEY_LEN ];
            Arrays.fill(Key, (byte) 0xFF);

            /* Transform Key(6 bytes) to LoadKey(12 bytes). */
            for (i = 0 ; i < 6 ; i++) {
               LoadKey[i*2] = (byte)(((~Key[i]) & 0xF0) | ((Key[i] & 0xF0) >> 4));
               LoadKey[i*2 + 1] = (byte)((((~Key[i]) << 4) & 0xF0) | (Key[i] & 0x0F));
            }

            rc = cl.mifareLOADKEY(LoadKey);
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] mifareLOADKEY", rc));
            }

            rc = cl.mifareAUTHEx(CL_MFC_AUTH_KEY_TYPEA, CL_MFC_DATA_BLOCK_NUM, Arrays.copyOf(CSN, CSNLen));
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] mifareAUTHEx", rc));
            }

            /* Write data to Mifare Classic PICC. */
            rc = cl.mifareWRITEBLOCK(CL_MFC_DATA_BLOCK_NUM, WriteBuf);
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] mifareWRITEBLOCK", rc));
            }

            /* Read data from Mifare Classic PICC. */
            rc = cl.mifareREADBLOCK(CL_MFC_DATA_BLOCK_NUM, ReadBuf);
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] mifareREADBLOCK", rc));
               }

               log += "Write : " + byteArrayToHexString(WriteBuf) + "\n\n";
               log += "Read : " + byteArrayToHexString(ReadBuf) + "\n\n";

               /* Compare Write and Read data to/from Mifare PICC. */
               if (!Arrays.equals(WriteBuf, ReadBuf)) {
                  log += "[Compare] Fail\n\n";
               } else {
                  log += "[Compare] success\n\n";
               }
            } catch (IOException e) {
                log += "[IOException]" + e.getMessage() + "\n";
            } catch (Exception e) {
                log += "[Other Exception]" + e.getMessage() + "\n";
            }

            if (foundCard) {
               cl.powerOff();
            }
         }
      };
```

Example : Mifare Classic example II

```
/**
```

```
 * This example is used to demonstrate that after the PICC is activated,
 * the authentication can be performed after the key is loaded,
 * Mifare Classic commands such as Read, Write, Decrement, Increment, Restroe and Transefer could be performed after
 * the authentication is passed.
 */
    import CTOS.CtCL;

    /** Contactless card reader. */
    CtCL cl = new CtCL();

    private static final int CL_MFC_AUTH_KEY_TYPEA = 0x60;
    private static final int CL_MFC_AUTH_KEY_TYPEB = 0x61;
    private static final int CL_MFC_AUTH_KEY_LEN = 6;
    private static final int CL_MFC_AUTH_LOADKEY_LEN = 12;
    private static final int CL_MFC_DATA_BLOCK_NUM = 4;
    private static final int CL_MFC_VALUE_BLOCK_NUM = 8;
    private static final int CL_MFC_RESTORE_BLOCK_NUM = 9;
    private static final int CL_MFC_DATA_LENGTH = 16;
    private static final int CL_MFC_VALUE_LENGTH = 4;

    public String byteArrayToHexString(byte[] data) {
        int i;
        String s = "";

        for (i = 0 ; i < data.length ; i++) {
            s += String.format("0x%02X, ", data[i]);
        }

        return s;
    }

   /** Create an array that conforms to the value block format. */
    private byte[] createValueFormatBlock(int value, int addressData) {
        byte []valueBlock = new byte[DATA_BLOCK_LENGTH];

        valueBlock[0]  = (byte)(value & 0xFF);
        valueBlock[1]  = (byte)((value >>  8) & 0xFF);
        valueBlock[2]  = (byte)((value >> 16) & 0xFF);
        valueBlock[3]  = (byte)((value >> 24) & 0xFF);
        valueBlock[4]  = (byte)~(byte)(value & 0xFF);
        valueBlock[5]  = (byte)~(byte)((value >>  8) & 0xFF);
        valueBlock[6]  = (byte)~(byte)((value >> 16) & 0xFF);
        valueBlock[7]  = (byte)~(byte)((value >> 24) & 0xFF);
        valueBlock[8]  = valueBlock[0];
        valueBlock[9]  = valueBlock[1];
        valueBlock[10] = valueBlock[2];
        valueBlock[11] = valueBlock[3];
        valueBlock[12] = (byte)addressData;
        valueBlock[13] = (byte)~(byte)addressData;
        valueBlock[14] = (byte)addressData;
        valueBlock[15] = (byte)~(byte)addressData;

         return valueBlock;
    }

    /** Set "int" to LSB first array. */
    private byte[] valueToByteArray(int value) {
        byte[] data = new byte[4];
        data[0]  = (byte)(value & 0xFF);
        data[1]  = (byte)((value >>  8) & 0xFF);
        data[2]  = (byte)((value >> 16) & 0xFF);
        data[3]  = (byte)((value >> 24) & 0xFF);
        return data;
    }

    /** Set LSB first array to "int". */
    private int byteArrayToValue(byte[] data) {
        int value;
        value = data[0] & 0xFF;
        value |= (data[1] & 0xFF) << 8;
        value |= (data[2] & 0xFF) << 16;
        value |= (data[3] & 0xFF) << 24;
        return value;
    }

    private Runnable runMFCSample = new Runnable() {
        public void run() {
```

```
String log = "";
int rc;

CtCL cl = new CtCL();
boolean foundCard = false;

try {
   /* Find ISO14443A Card. */
   byte[] ATQA = new byte[2];
   byte[] SAK = new byte[1];
   byte[] CSN = new byte[10];
   byte[] ReadBuf = new byte[CL_MFC_DATA_LENGTH];
   int value;
   byte[] valueBlockData;
   byte[] data;

   rc = cl.typeAActiveFromIdle(0, ATQA, SAK, CSN);
   if (rc != 0) {
      throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
   }
   foundCard = true;

   int CSNLen = cl.getCSNLen();

   /* Found card information. */
   log += String.format("ATQA = 0x%02X, 0x%02X. SAK = 0x%02X.\n\n", ATQA[0], ATQA[1], SAK[0]);

   /**
    * Set authentication key of Mifare(R) PICC.
    * The key is depending on user's Mifare Classic Card.
    */
   byte[] Key = new byte[CL_MFC_AUTH_KEY_LEN];
   Arrays.fill(Key, (byte) 0xFF);

   /* Perform Mifare Classic <Authenticate> command. */
   rc = cl.mifareAUTH2Ex(CL_MFC_AUTH_KEY_TYPEA, CL_MFC_VALUE_BLOCK_NUM, Key, CSN, CSNLen);
   if (rc != 0) {
      throw new IOException(String.format("[0x%04X] mifareAUTH2Ex", rc));
   }

   /**
    * Create a Value-Format block.
    * Setting value is @value.
    * Setting backup block address is @CL_MFC_RESTORE_BLOCK_NUM.
    */
   /* Initial the value block. */
   value = 0;
   valueBlockData = createValueFormatBlock(value, CL_MFC_VALUE_BLOCK_NUM);
   rc = cl.mifareWRITEBLOCK(CL_MFC_VALUE_BLOCK_NUM, valueBlockData);
   if (rc != 0) {
      throw new IOException(String.format("[0x%04X] mifareWRITEBLOCK", rc));
    }

    /* Back up value block. */
    rc = cl.mifareWRITEBLOCK(CL_MFC_RESTORE_BLOCK_NUM, valueBlockData);
    if (rc != 0) {
       throw new IOException(String.format("[0x%04X] mifareWRITEBLOCK", rc));
    }

    log += "Initial value block and back up value." + "\n";
    log += "balance : " + value + "\n\n";

    /* Increment 10. */
    value = 10;
    log += "Add value : " + value + "\n";
    data = valueToByteArray(value);
    rc = cl.mifareINCREASE(CL_MFC_VALUE_BLOCK_NUM, data);
    if (rc != 0) {
       throw new IOException(String.format("[0x%04X] mifareINCREASE", rc));
    }

    /* Store result to block. */
    rc = cl.mifareTRANSFER(CL_MFC_VALUE_BLOCK_NUM);
    if (rc != 0) {
       throw new IOException(String.format("[0x%04X] mifareTRANSFER", rc));
    }
```

```
            rc = cl.mifareREADBLOCK(CL_MFC_VALUE_BLOCK_NUM, ReadBuf);
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] mifareREADBLOCK", rc));
            }

            value = byteArrayToValue(ReadBuf);
            log += "balance : " + value + "\n\n";

            /* Decrement 10. */
            value = 5;
            log += "Deduct value : " + value + "\n";
            data = valueToByteArray(value);
            rc = cl.mifareDECREASE(CL_MFC_VALUE_BLOCK_NUM, data);
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] mifareINCREASE", rc));
            }

            /* Store result to block. */
            rc = cl.mifareTRANSFER(CL_MFC_VALUE_BLOCK_NUM);
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] mifareTRANSFER", rc));
            }

            rc = cl.mifareREADBLOCK(CL_MFC_VALUE_BLOCK_NUM, ReadBuf);
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] mifareREADBLOCK", rc));
            }

            value = byteArrayToValue(ReadBuf);
            log += "balance : " + value + "\n\n";

            /* Restore @CL_MFC_VALUE_BLOCK_NUM from @CL_MFC_RESTORE_BLOCK_NUM. */
            rc = cl.mifareRESTORE(CL_MFC_RESTORE_BLOCK_NUM);
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] mifareRESTORE", rc));
            }

            /* Store result to block. */
            rc = cl.mifareTRANSFER(CL_MFC_VALUE_BLOCK_NUM);
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] mifareTRANSFER", rc));
            }

            rc = cl.mifareREADBLOCK(CL_MFC_VALUE_BLOCK_NUM, ReadBuf);
            if (rc != 0) {
               throw new IOException(String.format("[0x%04X] mifareREADBLOCK", rc));
            }

            value = byteArrayToValue(ReadBuf);

            log += "Restore value (balance : " + value + ")\n\n";

          } catch (IOException e) {
              log += "[IOException]" + e.getMessage() + "\n";
          } catch (Exception e) {
              log += "[Other Exception]" + e.getMessage() + "\n";
          }

          if (foundCard) {
             cl.powerOff();
          }
        }
      };
```

Example : Type B example I

```
/**
 * This example is used to demonstrate that the PICC is activated from its IDLE state to ACTIVE state
 * and then to PROTOCOL state to send APDU.
 */

    import CTOS.CtCL;

    /** Contactless card reader. */
    CtCL cl = new CtCL();

    private static final int CL_PUPI_LEN = 4;
```

```
    public String byteArrayToHexString(byte[] data) {
        int i;
        String s = "";

        for (i = 0 ; i < data.length ; i++) {
           s += String.format("0x%02X, ", data[i]);
        }

        return s;
    }

    private Runnable runTypeBSample = new Runnable() {
       public void run() {
           String log = "";
          int rc;

          CtCL cl = new CtCL();
          boolean foundCard = false;

          try {
             /* Find ISO14443B Card.
             * if success, Type-B card is in ACTIVE state of ISO14443-3.
             */
             byte[] PUPI = new byte[4];

             /* Activate PICC form IDLE status to ACTIVE status. */
             rc = cl.typeBActive(PUPI);
             if (rc != 0) {
                throw new IOException(String.format("[0x%04X] typeBActive", rc));
             }
             foundCard = true;

             log += "PUPI : " + byteArrayToHexString(Arrays.copyOf(PUPI, 4)) + "\n\n";

             /* Data exchange by ISO14443-4 APDU protocol. */
             byte SBuf[] = new byte [] {(byte)0x00, (byte)0xa4, (byte)0x04, (byte)0x00, (byte)0x07,
                   (byte)0xa0, (byte)0x00, (byte)0x00, (byte)0x00, (byte)0x04, (byte)0x10, (byte)0x10};
             byte RBuf[] = new byte[256];

             rc = cl.APDU(SBuf, RBuf);
             if (rc != 0) {
                throw new IOException(String.format("[0x%04X] APDU", rc));
             }
             int RBufLen = cl.getRLen();

             log += "APDU Send : " + byteArrayToHexString(SBuf) + "\n\n";
             log += "APDU Recv : " + byteArrayToHexString(Arrays.copyOf(RBuf, RBufLen)) + "\n\n";

             /* Here may be more data exchange by APDU protocol ... */

          } catch (IOException e) {
                log += "[IOException]" + e.getMessage() + "\n";
            } catch (Exception e) {
               log += "[Other Exception]" + e.getMessage() + "\n";
            }

            if (foundCard) {
               cl.powerOff();
            }
        }
     };
```

Example : Type B example II

```
/**
 * This example is used to demonstrate the execution of the Type B Request command to change the PICC from
 * the IDLE status to the READY status, and issue the ATTRIB command to enter the ACTIVE status to send APDU.
 */

    import CTOS.CtCL;

    /** Contactless card reader. */
    CtCL cl = new CtCL();

    private static final boolean CL_USE_REQB = false;
    private static final boolean CL_USE_WUPB = true;
```

```
public String byteArrayToHexString(byte[] data) {
    int i;
    String s = "";

    for (i = 0 ; i < data.length ; i++) {
       s += String.format("0x%02X, ", data[i]);
    }

    return s;
}

private Runnable runTypeBSample = new Runnable() {
    public void run() {
        String log = "";
       int rc;

       CtCL cl = new CtCL();
       boolean foundCard = false;

       try {
          /* Find ISO14443B Card.
           * if success, Type-B card is in ACTIVE state of ISO14443-3.
           */
           byte[] ATQB = new byte[14];
           byte[] PUPI = new byte[4];

           /* Execute Type B Request command, PICC will enter READY status from IDLE status. */
           rc = cl.REQB(0, ATQB);
           if (rc != 0) {
              throw new IOException(String.format("[0x%04X] REQB", rc));
           }
           foundCard = true;

           int ATQBLen = cl.getATQBLen();
           log += "ATQB : " + byteArrayToHexString(Arrays.copyOf(ATQB, ATQBLen)) + "\n\n";

           System.arraycopy(ATQB, 1, PUPI, 0, 4);
           log += "PUPI : " + byteArrayToHexString(Arrays.copyOf(PUPI, 4)) + "\n\n";

           /* Select the PICC. PICC goes to the ACTIVE status from READY status. */
           rc = cl.attrib(PUPI);
           if (rc != 0) {
              throw new IOException(String.format("[0x%04X] attrib", rc));
           }

           int Rep = cl.getRep();
           log += "CID : " + (Rep & 0x0F) + "\n\n";

           /* Data exchange by ISO14443-4 APDU protocol. */
           byte SBuf[] = new byte [] {(byte)0x00, (byte)0xa4, (byte)0x04, (byte)0x00, (byte)0x07,
                 (byte)0xa0, (byte)0x00, (byte)0x00, (byte)0x00, (byte)0x04, (byte)0x10, (byte)0x10};
              byte RBuf[] = new byte[256];

              rc = cl.APDU(SBuf, RBuf);
              if (rc != 0) {
                 throw new IOException(String.format("[0x%04X] APDU", rc));
              }
              int RBufLen = cl.getRLen();

              log += "APDU Send : " + byteArrayToHexString(SBuf) + "\n\n";
              log += "APDU Recv : " + byteArrayToHexString(Arrays.copyOf(RBuf, RBufLen)) + "\n\n";

              /* Here may be more data exchange by APDU protocol ... */

          } catch (IOException e) {
              log += "[IOException]" + e.getMessage() + "\n";
          } catch (Exception e) {
              log += "[Other Exception]" + e.getMessage() + "\n";
          }

          if (foundCard) {
             cl.powerOff();
          }
       }
};
```

Example : Type B example III

```
/**
 * This example is used to demonstrate the execution of the Type B Request command to change the PICC from the IDLE status to the READY status,
 * and issue the ATTRIB command to enter the ACTIVE status and then execute HLTB command to enter HALT status,
 * next to activated PICC from its HALT status to ACTIVE status and enter ACTIVE state to send APDU.
 */

  import CTOS.CtCL;

  /** Contactless card reader. */
  CtCL cl = new CtCL();

  private static final boolean CL_USE_REQB = false;
  private static final boolean CL_USE_WUPB = true;
  private static final int CL_PUPI_LEN = 4;
  private static final int d_CL_FSDI_16 = 0;
  private static final int d_CL_FSDI_24 = 1;
  private static final int d_CL_FSDI_32 = 2;
  private static final int d_CL_FSDI_40 = 3;
  private static final int d_CL_FSDI_48 = 4;
  private static final int d_CL_FSDI_64 = 5;
  private static final int d_CL_FSDI_96 = 6;
  private static final int d_CL_FSDI_128 = 7;
  private static final int d_CL_FSDI_256 = 8;

  private static final int d_CL_NUMSLOT_x1 = 0x00;
  private static final int d_CL_NUMSLOT_x2 = 0x01;
  private static final int d_CL_NUMSLOT_x4 = 0x02;
  private static final int d_CL_NUMSLOT_x8 = 0x03;
  private static final int d_CL_NUMSLOT_x16 = 0x04;
  private static final int d_CL_MAXRATE_106 = 106;

  public String byteArrayToHexString(byte[] data) {
      int i;
      String s = "";

      for (i = 0 ; i < data.length ; i++) {
          s += String.format("0x%02X, ", data[i]);
      }

      return s;
   }

   private Runnable runTypeBSample = new Runnable() {
      public void run() {
          String log = "";
          int rc;

          CtCL cl = new CtCL();
          boolean foundCard = false;

             try {
                /* Find ISO14443B Card.
                  * if success, Type-B card is in ACTIVE state of ISO14443-3.
                  */
                byte[] ATQB = new byte[14];
                int ATQBLen;
                byte[] PUPI = new byte[4];
                int i;

                /* Execute Type B Wake Up command, PICC will enter READY status from IDLE status. */
                rc = cl.WUPB(0, ATQB);
                if (rc != 0) {
                    throw new IOException(String.format("[0x%04X] REQB", rc));
                }
                foundCard = true;

                int ATQBLen = cl.getATQBLen();
                log += "ATQB : " + byteArrayToHexString(Arrays.copyOf(ATQB, ATQBLen)) + "\n\n";

                /* Enter HALT status. */
                System.arraycopy(ATQB, 1, PUPI, 0, 4);
                rc = cl.haltb(PUPI);
                if (rc != 0) {
                    throw new IOException(String.format("[0x%04X] attrib", rc));
                }

                /* Activate PICC form HALT status to ACTIVE status. */
```

```
                rc = cl.typeBActiveEx(PUPI, 0, CL_USE_WUPB, d_CL_FSDI_256, d_CL_NUMSLOT_x2, d_CL_MAXRATE_106);
                if (rc != 0) {
                    throw new IOException(String.format("[0x%04X] typeBActiveEx", rc));
                }

                for (i = 0; i < 4; i++) {
                    if(PUPI[i] != ATQB[i + 1]) {
                        throw new IOException(String.format("[0x%04X] typeBActiveEx", rc));
                    }
                }

                log += "PUPI : " + byteArrayToHexString(Arrays.copyOf(PUPI, CL_PUPI_LEN)) + "\n\n";

                /* Data exchange by ISO14443-4 APDU protocol. */
                byte SBuf[] = new byte [] {(byte)0x00, (byte)0xa4, (byte)0x04, (byte)0x00, (byte)0x07,
                        (byte)0xa0, (byte)0x00, (byte)0x00, (byte)0x00, (byte)0x04, (byte)0x10, (byte)0x10};
                byte RBuf[] = new byte[256];

                rc = cl.APDU(SBuf, RBuf);
                if (rc != 0) {
                    throw new IOException(String.format("[0x%04X] APDU", rc));
                }
                int RBufLen = cl.getRLen();

                log += "APDU Send : " + byteArrayToHexString(SBuf) + "\n\n";
                log += "APDU Recv : " + byteArrayToHexString(Arrays.copyOf(RBuf, RBufLen)) + "\n\n";

                /* Here may be more data exchange by APDU protocol ... */

            } catch (IOException e) {
                log += "[IOException]" + e.getMessage() + "\n";
            } catch (Exception e) {
                log += "[Other Exception]" + e.getMessage() + "\n";
            }

            if (foundCard) {
                cl.powerOff();
            }
        }
    };
```

Example : ISO14443-4 T=CL example

```
/**
 * This example is used to demonstrate that the PICC has been activated from its IDLE state to the ACTIVE state,
 * and then activated to the PROTOCOL state.
 * Next, issue the DESELECT command to let PICC exit the PROTOCOL state and enter the HALT state.
 * Finally, activate the PICC from its HALT state to the ACTIVE state, and enter the PROTOCOL state to send APDUs.
 */
  import CTOS.CtCL;

    /** Contactless card reader. */
    CtCL cl = new CtCL();

    public String byteArrayToHexString(byte[] data) {
        int i;
        String s = "";

        for (i = 0 ; i < data.length ; i++) {
            s += String.format("0x%02X, ", data[i]);
        }

        return s;
    }

    private Runnable runISO14443p4Sample = new Runnable() {
        public void run() {
            String log = "";
            int rc;

            CtCL cl = new CtCL();
            boolean foundCard = false;

            try {
                /* Find ISO14443A Card.
                 * if success, Type-A card is in ACTIVE state of ISO14443-3.
                 */
                byte[] ATQA = new byte[2];
```

```
byte[] SAK = new byte[1];
byte[] CSN = new byte[10];

rc = cl.typeAActiveFromIdle(0, ATQA, SAK, CSN);
if (rc != 0) {
   throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
}
foundCard = true;

int CSNLen = cl.getCSNLen();

/* Check picc compliant with ISO14443-4 or not */
boolean i14443p4Comp = false;
if ((SAK[0] & 0x20) == 0x20) {
   i14443p4Comp = true;
}

/* Found card information. */
log += String.format("ATQA = 0x%02X, 0x%02X. SAK = 0x%02X.\n\n", ATQA[0], ATQA[1], SAK[0]);
log += String.format("i1443p4Comp = %b\n\n", i14443p4Comp);

if (i14443p4Comp == false) {
   throw new IOException("This card doesn't support ISO14443-4 protocol.");
}

/* ISO14443-4A Card protocol activation.
  * if success, Type-A card is in PROTOCOL state of ISO14443-4.
  */
byte[] ATS = new byte[256];
rc = cl.RATS(0, ATS);
if (rc != 0) {
   throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
}

/* Read out valid length of ATS response */
int ATSLen = cl.getATSLen();

log += "ATS : " + byteArrayToHexString(Arrays.copyOf(ATS, ATSLen)) + "\n\n";

/* PICC exit the PROTOCOL state and enter the HALT state. */
rc = cl.deselect();
if (rc != 0) {
   throw new IOException(String.format("[0x%04X] deselect", rc));
}

byte[] CSN_Halt = new byte[CSNLen];
System.arraycopy(CSN, 0, CSN_Halt, 0, CSNLen);

/* Activate PICC from HALT state to the ACTIVE state */
rc = cl.typeAActiveFromHalt(0, ATQA, SAK, CSN_Halt);
   if (rc != 0) {
      throw new IOException(String.format("[0x%04X] typeAActiveFromHalt", rc));
   }

   /* ISO14443-4A Card protocol activation.
     * if success, Type-A card is in PROTOCOL state of ISO14443-4.
     */
   rc = cl.RATSEx(0, ATS, 106, 256);
   if (rc != 0) {
      throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
   }

   /* Check PICC in the PROTOCOL status. */
   rc = cl.nakPoll();
   if (rc != 0) {
      throw new IOException(String.format("[0x%04X] PICC is not in PROTOCOL status.", rc));
   }

   /* Data exchange by ISO14443-4 APDU protocol. */
   byte SBuf[] = new byte [] {(byte)0x00, (byte)0xa4, (byte)0x04, (byte)0x00, (byte)0x07,
         (byte)0xa0, (byte)0x00, (byte)0x00, (byte)0x00, (byte)0x04, (byte)0x10, (byte)0x10};
   byte RBuf[] = new byte[256];

   rc = cl.APDU(SBuf, RBuf);
   if (rc != 0) {
      throw new IOException(String.format("[0x%04X] APDU", rc));
   }
```

```
                int RBufLen = cl.getRLen();

                log += "APDU Send : " + byteArrayToHexString(SBuf) + "\n\n";
                log += "APDU Recv : " + byteArrayToHexString(Arrays.copyOf(RBuf, RBufLen)) + "\n\n";

                /* Here may be more data exchange by APDU protocol ... */

            } catch (IOException e) {
                log += "[IOException]" + e.getMessage() + "\n";
            } catch (Exception e) {
                log += "[Other Exception]" + e.getMessage() + "\n";
            }

            if (foundCard) {
               cl.powerOff();
            }
        }
    };
```

Example : Transparent CL Access example

```
/**
 * This example is used to demonstrate that after the PICC is activated,
 * send Mifare Ultralight commands to PICC.
 */

public String byteArrayToHexString(byte[] data) {
    int i;
    String s = "";

    for (i = 0 ; i < data.length ; i++) {
       s += String.format("0x%02X, ", data[i]);
    }

    return s;
}

private Runnable runCLAccessSample = new Runnable() {

   public void run() {
      String log = "";
      int rc;

      CtCL cl = new CtCL();
      boolean foundCard = false;

      try {
         /* Find ISO14443A Card.
          * if success, Type-A card is in ACTIVE state of ISO14443-3.
          */
         byte[] ATQA = new byte[2];
          byte[] SAK = new byte[1];
          byte[] CSN = new byte[10];
          byte[] SBuf = new byte[256];
          byte[] RBuf = new byte[256];

          rc = cl.typeAActiveFromIdle(0, ATQA, SAK, CSN);
          if (rc != 0) {
              throw new IOException(String.format("[0x%04X] typeAActiveFromIdle", rc));
          }
          foundCard = true;

          int CSNLen = cl.getCSNLen();
          log += "CSN : " + byteArrayToHexString(Arrays.copyOf(CSN, CSNLen)) + "\n\n";

          /* Set timeout for tclTransparentRaw(). */
          int Timeout = 5;
          rc = cl.tclSetTimeout(Timeout);
          if (rc != 0) {
             throw new IOException(String.format("[0x%04X] tclSetTimeout", rc));
          }

          int SLen = 0;
          SBuf[SLen++] = 0x30; //Cmd <Read>
          SBuf[SLen++] = 0x01; //Addr
          SBuf[SLen++] = 0x0A;
          SBuf[SLen++] = 0x40;
          SBuf[SLen++] = 0x05;
```

```
            rc = cl.tclTransparentRaw(Arrays.copyOf(SBuf, SLen), 4, RBuf, false, false, false);
            if (rc != 0x837B && rc != 0x00) {
               throw new IOException(String.format("[0x%04X] tclTransparentRaw", rc));
            }

            int RLen = cl.getRLen();
            int RLastBits = cl.getRLastBits();

            log += "Receive buffer length should be " + RLen + " bytes" + "\n\n";
            log += "Valid bits of last byte is " + RLastBits + " bits" + "\n\n";

            /* Here may be more data exchange by APDU protocol ... */
        } catch (IOException e) {
            log += "[IOException]" + e.getMessage() + "\n";
        } catch (Exception e) {
            log += "[Other Exception]" + e.getMessage() + "\n";
        }

        if (foundCard) {
            cl.powerOff();
        }

        sendStringToLogHandler(log);
    }
};
```

Example: CTS256B

```
/**
 * This example is used to demonstrate that the CTS512B basic instructions.
 */
private final Runnable runExample = new Runnable() {
        public void run() {
            String log = "";
            CtCL cl = new CtCL();
            byte []txData = new byte[2];
            byte []rxData = new byte[8];
            int rc = 0;
            byte address = 0x05;

            log += "Start polling cards\n\n";
            long millisec = System.currentTimeMillis() + 3000;
            while (millisec > System.currentTimeMillis()) {
                /* Refer document "CTS512B-DS-V1.3.pdf"
                 * 2.1 Instructions and responses <REQT> command
                 * */
                rc = cl.cts512bReqt(rxData);
                log += String.format("REQT = 0x%04X\n", rc);
                if (rc == 0) {
                    break;
                }
            }
            if (rc != 0) {
                return ;
            }

            /* Refer document "CTS512B-DS-V1.3.pdf"
             * 2.1 Instructions and responses <REQT> response
```

Castles Technology

```
 * */
log += String.format("Product code = 0x%02X\n", rxData[0]);
log += (String.format("Fab code         = 0x%02X\n", rxData[1]);

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <IDENTIFY> command
 * */
txData[0] = (byte)0xFF;
txData[1] = (byte)0xFF;
rc = cl.cts512bIdentify(txData, rxData);
log += String.format("\nIDENTIFY = 0x%04X\n", rc);
if (rc != 0) {
      return ;
}

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <IDENTIFY> response
 * */
log += String.format("Serial lsb = 0x%02X\n", rxData[0]);
log += String.format("Serial msb = 0x%02X\n", rxData[1]);

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <SELECT> command
 * */
txData[0] = rxData[0];
txData[1] = rxData[1];
rc = cl.cts512bSelect(txData, rxData);
log += String.format("\nSELECT = 0x%04X\n", rc);
if (rc != 0) {
      return ;
}

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <SELECT> response
 * */
log += String.format("Serial lsb = 0x%02X\n", rxData[0]);
log += String.format("Serial msb = 0x%02X\n", rxData[1]);

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <READ> command
 * */
rc = cl.cts512bRead(address, rxData);
log += String.format("\nREAD = 0x%04X\n", rc);
if (rc != 0) {
      return ;
}
```

Castles Technology

```
/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <READ> response
 * */
log += String.format("DATA 1 = 0x%02X\n", rxData[0]);
log += String.format("DATA 2 = 0x%02X\n", rxData[1]);

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <WRITE> command
 * */
txData[0] = (byte)0xAA;
txData[1] = (byte)0x55;
rc = cl.cts512bWrite(address, txData, rxData);
log += String.format("\nWRITE = 0x%04X\n", rc);
if (rc != 0) {
    return ;
}

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <WRITE> response
 * */
log += String.format("DATA 1 = 0x%02X\n", rxData[0]);
log += String.format("DATA 2 = 0x%02X\n", rxData[1]);

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <READ> command
 * */
rc = cl.cts512bRead(address, rxData);
log += String.format("\nREAD = 0x%04X\n", rc));
if (rc != 0) {
    return ;
}

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <READ> response
 * */
log += String.format("DATA 1 = 0x%02X\n", rxData[0]);
log += String.format("DATA 2 = 0x%02X\n", rxData[1]);

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <UPDATE> command
 * */
txData[0] = (byte)0;
txData[1] = (byte)0;
rc = cl.cts512bUpdate(address, txData);
log += String.format("\nUPDATE = 0x%04X\n", rc));
if (rc != 0) {
    return ;
```

Castles Technology

```
}

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <READ> command
 * */
rc = cl.cts512bRead(address, rxData);
log += String.format("\nREAD = 0x%04X\n", rc));
if (rc != 0) {
    return ;
}

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <READ> response
 * */
log += String.format("DATA 1 = 0x%02X\n", rxData[0]);
log += String.format("DATA 2 = 0x%02X\n", rxData[1]);

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <MULTIREAD> command
 * */
rc = cl.cts512bMultiRead(address, rxData);
log += String.format("\nMULTIREAD = 0x%04X\n", rc));
if (rc != 0) {
    return ;
}

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <MULTIREAD> response
 * */
for (int i = 0; i < 8; i++) {
    log += String.format("DATA %d = 0x%02X\n", (i + 1), rxData[i]);
}

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <DESACTIVATE> command
 * */
rc = cl.cts512bDesactivate(rxData);
log += String.format("\nDESACTIVATE = 0x%04X\n", rc);
if (rc != 0) {
    return ;
}

/* Refer document "CTS512B-DS-V1.3.pdf"
 * 2.1 Instructions and responses <DESACTIVATE> response
 * */
log += String.format("Serial lsb = 0x%02X\n", rxData[0]);
log += String.format("Serial msb = 0x%02X\n", rxData[1]);
```

Castles Technology

```
        }
```

Example: Card Emulation

```
/**
 * This example is used to demonstrate basic instructions for card mode.
 */
private Runnable runCardEmulationExample = new Runnable() {
    public void run() {
        int rc;
        int status_temp = -1;
        int []status = new int[1];
        byte[] rxBuf = new byte[CtCL.CM_MAX_RX_DATA_SIZE];
        int []rxLen = new int[1];
        byte []rxApdu;
        byte []txApdu = new byte[0];
        long start_time;

        /* Start card emulation */
        rc = mCL.cmStartEmulation();
        if (rc == 0x83BC) {
            /* This machine doesn't support this feature */
            return;
        }else if (rc != 0) {
            /* Other error reason */
            return;
        }

        start_time = System.currentTimeMillis();
        /* Set 30 second as timeout, if no C-APDU event */
        while (System.currentTimeMillis() < (start_time + (30 * 1000))) {
            rc = mCL.cmGetStatus(status);
            if (rc != 0) {
                continue;
            }

            switch (status[0]) {
                case CtCL.CM_STATUS_NO_RF:
                case CtCL.CM_STATUS_DESELECT:
                    if (status_temp != status[0]) {
                        onDeactivated(status[0]);
                    }
                    break;

                case CtCL.CM_STATUS_READ_DATA:
                    rc = mCL.cmReadApduCommand(CtCL.CM_OPTION_DEFAULT, rxBuf, rxLen);
                    if (rc == 0) {
                        rxApdu = Arrays.copyOf(rxBuf, rxLen[0]);
```

Castles Technology

```
                        /* Alert user process APDU data */
                        txApdu = processCommandApdu(rxApdu);
                    }

                    start_time = System.currentTimeMillis();
                    break;

                case CtCL.CM_STATUS_WRITE_DATA:
                    int []writtenLen = new int[1];

                    if (txApdu.length <= 0) {
                        break;
                    }
                    if (txApdu.length > CtCL.CM_MAX_TX_DATA_SIZE) {
                        txApdu = Arrays.copyOf(txApdu, CtCL.CM_MAX_TX_DATA_SIZE);
                    }

                    rc = mCL.cmWriteApduCommand(CtCL.CM_OPTION_DEFAULT, txApdu, writtenLen);
                    if (rc != 0) {
                        break;
                    }

                    start_time = System.currentTimeMillis();
                    break;

                default:
                    break;
            }

            status_temp = status[0];
        }

        /* End card emulation */
        rc = mCL.cmStopEmulation();
    }
};

public byte[] processCommandApdu(byte[] commandApdu) {
    /* Send card response here */
    return new byte[] {(byte)0x6D, (byte)0x00};
}

public void onDeactivated(int reason) {

}
```

Castles Technology

1.1.9 Public class CtMSR

The CtMSR class provides the functionalities to accomplish capture of MSR.

Constructor Summary	
public	CtMSR() Constructs a binder connected with service.

Method Summary	
int	getTk1Len() Get length of the track 1.
int	getTk2Len() Get length of the track 2.
int	getTk3Len() Get length of the track 3.
byte	getTk1Err() Get error code of the track 1.
byte	getTk2Err() Get error code of the track 2.
byte	getTk3Err() Get error code of the track 3.
int	read (byte[] Tk1Buf, byte[] Tk2Buf, byte[] Tk3Buf) Read magnetic stripe card.
int	getLastErr() Get last error code of the all tracks.
int	dualRead(byte[] Tk1Buf, byte[] Tk2Buf, byte[] Tk3Buf, byte[] Tk4Buf, byte[] Tk5Buf, byte[] Tk6Buf) Read dual-sided magnetic stripe card. Return. Then user can call getTkXLen() and dualGetLastErr() to get data length and last error code for each track.
int	dualGetLastErr() Get last error code of the all tracks. Return. The user can call getTkXErr() to get last error code for each track.

Constructor Detail
public CtMSR() Create a CtMSR object and connect with binder to invoke service.

Method Detail
public int getTk1Len() **Description :** Get Tk1Buf of read() parameter. **Parameters** none **Returns:** The number of Tk1Buf length
public int getTk2Len() **Description :** Get Tk2Buf of read() parameter. **Parameters** none **Returns:** The number of Tk2Buf length
public int getTk3Len() **Description :** Get Tk3Buf of read() parameter. **Parameters** none **Returns:** The number of Tk3Buf length
public byte getTk1Err()

Castles Technology

Description :
Get error code of the track 1.

Parameters
none

Returns:
The number of error code of the track 1. (GetTk1Err() Field Detail)

public byte getTk2Err()

Description :
Get error code of the track 2.

Parameters
none

Returns:
The number of error code of the track 2. (GetTk1Err() Field Detail)

public byte getTk3Err()

Description :
Get error code of the track 3.

Parameters
none

Returns:
The number of error code of the track 3. (GetTk1Err() Field Detail)

public int read(byte[] Tk1Buf, byte[] Tk2Buf, byte[] Tk3Buf)

Description :
Read magnetic stripe card.
Return. Then user can call getTkXLen() and getLastErr() to get data length and last error code for each track.

Parameters
[OUT] Tk1Buf
Buffer storing data of track 1. Buffer size needs 256 bytes.
[OUT] Tk2Buf
Buffer storing data of track 2. Buffer size needs 256 bytes.
[OUT] Tk3Buf
Buffer storing data of track 3. Buffer size needs 256 bytes.

Return Value

Constants	**Value**
Success	0
Fail	0x12xxh (Read() Field Detail)

public int getLastErr()

Description:
Get last error code of the all tracks.
Return. The user can call getTkXErr()to get last error code for each track.

Parameters
none

Return Value

Constants	**Value**
Success	0
Fail	!= 0

public int dualRead (byte[] Tk1Buf, byte[] Tk2Buf, byte[] Tk3Buf, byte[] Tk4Buf, byte[] Tk5Buf, byte[] Tk6Buf)

Description:
Only for S1L2.
Read dual-sided magnetic stripe card.
Return. Then user can call getTkXLen() and dualGetLastErr () to get data length and last error code for each track.

Parameters
[OUT]
Tk1Buf
Buffer storing data of track 1. Buffer size needs 256 bytes.
[OUT]
Tk2Buf
Buffer storing data of track 2. Buffer size needs 256 bytes.
[OUT]

Castles Technology

Tk3Buf
Buffer storing data of track 3. Buffer size needs 256 bytes.
[OUT]
Tk4Buf
Buffer storing data of track 4. Buffer size needs 256 bytes.
[OUT]
Tk5Buf
Buffer storing data of track 5. Buffer size needs 256 bytes.
[OUT]
Tk6Buf
Buffer storing data of track 6. Buffer size needs 256 bytes.

Return Value

Constants	Value
Success	0
d_MSR_NO_SWIPE	0x1201
d_MSR_TRACK_ERROR	0x1202

Note
Use function of the following to get buffer length.

public int getTk1Len()
public int getTk2Len()
public int getTk3Len()
public int getTk4Len()
public int getTk5Len()
public int getTk6Len()

Example

```
/*
Sample code
*/
import CTOS.CtMSR;
CtMSR ctmsr = new CtMSR();

int trBufLen = 256, ret = 0, tr1Len = 0, tr2Len = 0, tr3Len = 0, tr4Len = 0, tr5Len = 0, tr6Len = 0;
byte[] tk1Buf = new byte[trBufLen], tk2Buf = new byte[trBufLen], tk3Buf = new byte[trBufLen];
byte[] tk4Buf = new byte[trBufLen], tk5Buf = new byte[trBufLen], tk6Buf = new byte[trBufLen];

while(true)
{
            ret = ctmsr.dualRead(tk1Buf, tk2Buf, tk3Buf, tk4Buf, tk5Buf, tk6Buf);
     if(ret == 0)
     {
          tr1Len = ctmsr.getTk1Len();
          tr2Len = ctmsr.getTk2Len();
          tr3Len = ctmsr.getTk3Len();
          tr4Len = ctmsr.getTk4Len();
          tr5Len = ctmsr.getTk5Len();
          tr6Len = ctmsr.getTk6Len();

          Log.d(TAG, String.format("[ret:0x%02x] tr1Len:%d, tr2Len:%d tr3Len:%d", ret, tr1Len, tr2Len, tr3Len, tr4Len, tr5Len,
tr6Len));
          StringBuffer sTr1Buf = new StringBuffer();
          for (int i = 0; i < tr1Len; i++)
          {
               sTr1Buf.append((char)tk1Buf[i]);
          }
          Log.d(TAG, "TR1 :"+sTr1Buf);
     }
    else
         Log.d(TAG, String.format("[ret:0x%02x]", ret));

    if (ctmsr.dualGetLastErrEx() == 0)
    {
         Log.d(TAG, String.format("[Error code]tr1Err:0x%02x, tr2Err:0x%02x, tr3Err:0x%02x, tr4Err:0x%02x, tr5Err:0x%02x,
tr6Err:0x%02x\n", ctmsr.getTk1Err(), ctmsr.getTk2Err(), ctmsr.getTk3Err(), ctmsr.getTk4Err(), ctmsr.getTk5Err(),
ctmsr.getTk6Err()));
    }

     try{
          Thread.sleep(50);
     }catch (InterruptedException e){
          e.printStackTrace();
     }
}
public int dualGetLastErr
```

Castles Technology

Description:
Only for S1L2.
Get last error code of the all tracks.
Return. The user can call getTkXErr() to get last error code for each track.

Parameters
none

Return Value

Constants	Value
Success	0
Fail	!= 0

Note
Use function of the following to get error code of the track.

```
public byte getTk1Err ()
public byte getTk2Err ()
public byte getTk3Err ()
public byte getTk4Err ()
public byte getTk5Err ()
public byte getTk6Err ()
```

Example

```
/*
Sample code
*/
import CTOS.CtMSR;
CtMSR ctmsr = new CtMSR();

int trBufLen = 256, ret = 0, tr1Len = 0, tr2Len = 0, tr3Len = 0, tr4Len = 0, tr5Len = 0, tr6Len = 0;
byte[] tk1Buf = new byte[trBufLen], tk2Buf = new byte[trBufLen], tk3Buf = new byte[trBufLen];
byte[] tk4Buf = new byte[trBufLen], tk5Buf = new byte[trBufLen], tk6Buf = new byte[trBufLen];

while(true)
{
            ret = ctmsr.dualRead(tk1Buf, tk2Buf, tk3Buf, tk4Buf, tk5Buf, tk6Buf);
     if(ret == 0)
     {
          tr1Len = ctmsr.getTk1Len();
          tr2Len = ctmsr.getTk2Len();
          tr3Len = ctmsr.getTk3Len();
          tr4Len = ctmsr.getTk4Len();
          tr5Len = ctmsr.getTk5Len();
          tr6Len = ctmsr.getTk6Len();

          Log.d(TAG, String.format("[ret:0x%02x] tr1Len:%d, tr2Len:%d tr3Len:%d", ret, tr1Len, tr2Len, tr3Len, tr4Len, tr5Len,
tr6Len));
          StringBuffer sTr1Buf = new StringBuffer();
          for (int i = 0; i < tr1Len; i++)
          {
               sTr1Buf.append((char)tk1Buf[i]);
          }
          Log.d(TAG, "TR1 :"+sTr1Buf);
     }
    else
        Log.d(TAG, String.format("[ret:0x%02x]", ret));

    if (ctmsr.dualGetLastErrEx() == 0)
    {
         Log.d(TAG, String.format("[Error code]tr1Err:0x%02x, tr2Err:0x%02x, tr3Err:0x%02x, tr4Err:0x%02x, tr5Err:0x%02x,
tr6Err:0x%02x\n", ctmsr.getTk1Err(), ctmsr.getTk2Err(), ctmsr.getTk3Err(), ctmsr.getTk4Err(), ctmsr.getTk5Err(),
ctmsr.getTk6Err()));
    }

     try{
          Thread.sleep(50);
     }catch (InterruptedException e){
          e.printStackTrace();
     }
}
```

Read() Field Detail

Castles Technology

Constant Field	Value
d_MSR_NO_SWIPE	0x1201
d_MSR_TRACK_ERROR	0x1202

GetTkXErr() Field Detail

Constant Field	Value
d_MSR_SUCCESS	0x00
d_MSR_STX_NOT_FOUND	0x81
d_MSR_NO_DATA	0x82
d_MSR_BUF_OVERFLOW	0x83
d_MSR_ETX_NOT_FOUND	0x84
d_MSR_LRC_NOT_FOUND	0x85
d_MSR_LRC_ERR	0x86
d_MSR_UNKNOWN_CHAR	0x87
d_MSR_NONE_SWIPE	0x88

Example

```
/*
Sample code
*/
import CTOS.CtMSR;

CtMSR ctmsr = new CtMSR();
int trBufLen = 256, ret = 0, tr1Len = 0, tr2Len = 0, tr3Len = 0;
byte[] tk1Buf = new byte[trBufLen], tk2Buf = new byte[trBufLen], tk3Buf = new byte[trBufLen];
while(true) {
    ret = ctmsr.read(tk1Buf, tk2Buf, tk3Buf);
    if (ret == 0) {
        tr1Len = ctmsr.getTk1Len();
        tr2Len = ctmsr.getTk2Len();
        tr3Len = ctmsr.getTk3Len();
        Log.d(TAG1, String.format("[ret:0x%02x] tr1Len:%d, tr2Len:%d tr3Len:%d", ret, tr1Len, tr2Len, tr3Len));
        StringBuffer sTr1Buf = new StringBuffer ();
        for (int i = 0; i < tr1Len; i++) {
            sTr1Buf.append((char)tk1Buf[i]);
        }
        Log.d(TAG1, "TR1 :"+sTr1Buf);
    } else
         Log.d(TAG1, String.format("[ret:0x%02x]", ret));
    if (ctmsr.getLastErr() == 0) {
        Log.d(TAG1, String.format("[Error code]tr1Err:0x%02x, tr2Err:0x%02x, tr3Err:0x%02x\n", ctmsr.getTk1Err(),
ctmsr.getTk2Err(), ctmsr.getTk3Err()));
    }
    try {
        Thread.sleep(50);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

1.1.10 Public class CtSettings

The CtSettings class is used to provide System and Other settings.

Constructor Summary	
public	CtSettings() Constructs a binder connected with service.

Method Summary	
int	setApn(int simID,Apn apn) Set Apn Content
String	setApn(String carrierName, String Apn, String user, String password, String server, String authtype, String type, String mcc, String mnc) Set Apn detail(if mcc+mnc don't exist -> insert apn, mcc+mnc exist -> update apn).
String	getApn(String mcc, Stirng mnc) Get Apn detail.
List<Apn>	getApnList(int simID) Get Device Apn List
void	openBluetooth() Open BlueTooth
void	closeBluetooth() Close BlueTooth
String	bondBtDevice(String blueDeviceAddress) Pair BlueTooth
String	connectBtDevice(String uuid, String blueDeviceAddress) Connect BlueTooth
String	disConnectBtDevice() DisConnect BlueTooth
String	getSystemTime() Get System Time
String	setSystemTime(int year, int month, int date, int hour, int min, int sec) Set System Time
String	getSystemZone() Get System Time Zone
String	setSystemZone(String zone) Set System Time Zone
String	set24HourFormat(boolean is24Hour) Set 24 Hour Format
String	setAutoRebootTime(int hour, int min) Set Auto Reboot Time
int	getBrightness() Get Brightness Volume
String	setBrightness(int light) Set Brightness Volume
int	getScreenTimeout() Get Screen Off Timeout Volume
String	setScreenTimeOut(int time) Set Screen Off Timeout Volume
String	setNavigation(boolean Home, boolean Back, boolean Search) Set Navigation Bar hide or open
int	getMediaVolume() Get Media Sound Volume
void	setMediaVolume(int volume) Set Media Sound Volume
int	getRingVolume() Get Ring Sound Volume
void	setRingVolume(int volume) Set Ring Sound Volume
int	getAlarmVolume() Get Alarm Sound Volume
void	setAlarmVolume(int volume) Set Alarm Sound Volume
int	getNotificationVolume() Get Notification Sound Volume
void	setNotificationVolume(int volume) Set Notification Sound Volume
void	openWifi() Open Wifi
void	closeWifi() Close Wifi
String	setDhcpWifi(String ssid, String password, int type) Set Wifi DHCP

Castles Technology

String	setDhcpWifi(String ssid, String bssid, String password, int type) Set Wifi DHCP
String	setStaticWifi(String SSID, String Password, int Type, String ipAddress, String netmask, String gateway, String dns1, String dns2) Set Wifi Static
String	setStaticWifi(String SSID, String bssid, String Password, int Type, String ipAddress, String netmask, String gateway, String dns1, String dns2) Set Wifi Static
Map	getWifiConfig() Get Wifi Config
String	setEth(int enable) Set Ethernet State
int	setDHCPEth(String proxy) Set Ethernet DHCP
int	setStaticEth(String proxy, String ip, String netmask, String gateway, String dns1, String dns2) Set Ethernet Static
Map	getEthConfig() Get Ethernet Config
boolean	getDebugModeState() Get Debug Mode State
String	setDebugModeState(boolean enable) Set Debug Mode State
int	getBatteryThreshold() Get Battery Threshold Volume
String	setBatteryThreshold(int threshold) Set Battery Threshold Volume
boolean	setLoginPassword(String oldPassword1, String oldPassword2, String newPassword1, String newPassword2) Set Login Password
boolean	setUserPassword(String oldPassword, String newPassword) Set User Password
boolean	setFactoryPassword(String oldPassword, String newPassword) Set Factory Password
String	setDefaultApp(String packageName) Set Default App Package Name
String	getDefaultApp() Get Default App Package Name
String	factoryReset() Factory Reset
String	factoryReset(int type) Factory Reset
String	virtualKeyboard(int enable) Set Virtual Keyboard When Use Hard Keyboard
boolean	setLanguage(String language,String country) Set System Language
String	screenLock(boolean lock) Screen Lock
String	openBatteryPercentage(boolean open) Open Battery Percentage
boolean	getUserPasswordState() Get User Password State
String	setUserPasswordState(boolean enable) Set User Password State
int	setNTPSetting(String server,String port,boolean autoUpdate, int time) Set NTP Server
int	setNTPServer(String server,String port) Set NTP Server & port
int	setNTPIntervalTime(int time) Set NTP Interval Time
int	setNTPAutoUpdateTime(boolean auto) Set NTP AutoUpdateTime
Apn	getApn(int simID,String name) Get Apn By Carrier Name
Apn	getCurrentApn(int simID) Get Sim Current Apn
boolean	selectApn(int simID,String name) Select Sim Apn
void	setNetworkSelectionMode(int simID,boolean auto,String operatorNumeric) Set Operator Network & Select Network Mode
boolean	setAirplaneMode(boolean enable) Set Airplane Mode
String	setLockPattern(int type,String password) Set Screen Lock Type
int	getCurrentLockPattern() throws RemoteException Get Screen Lock Type
List<OperationInfo>	getAvailableNetworkList(int simID)

Castles Technology

	Get Operation Network Info List
void	setBluetoothBle(boolean enable) This API uses for setting BLE support or not
void	setBluetoothJustwork(boolean enable) This API uses for setting JUSTWORK pairing mode support or not.
boolean	SetMobileDataEnabled Set Mobile data enable or disable.
void	enableEnhancedLteMode Set EnhancedLte mode open or close.
boolean	boolean isEnhancedLteEnabled(int slotIndex) Check the status of EnhancedLTE.
boolean	boolean setTalkBack(String ComponentName) Set talkback function.

Constructor Detail

public CtSettings()
The CtSettings class is used to provide System and Other settings.

Method Detail

public int setApn(int simID,Apn apn)

Description
Set Apn Content

Parameters
simID SIM card slot (sim 1 ->0 ,sim 2 -> 1)
Apn Apn object

Returns
error code 0 -> UPDATE_APN_SUCCESS
1 -> INSERT_APN_SUCCESS
-1 -> SET_APN_MCCMNC_FAIL
-2 -> SQL_EXCEPTION

public String setApn(String carrierName, String apn, String user, String password, String server, String authtype, String type, String mcc, String mnc)

Description
Set Apn detail(if mcc+mnc don't exist -> insert apn, mcc+mnc exist -> update apn).

Parameters
carrierName Entry name
Apn APN name
user APN username
password APN password
server Server address
authtype Authentication type
type Comma-delimited list of APN types
mcc Mobile country codes
mnc Mobile network codes

Returns
Success message

public String getApn(String mcc, String mnc)

Description
Get Apn detail.

Parameters
mcc Mobile country codes
mnc Mobile network codes

Returns
Apn detail, if mcc+mnc don't exist, return "The APN doesn't exist".

public List<Apn> getApnList(int simID)

Description
Get Device Apn List Content

Parameters
simID SIM card slot (sim 1 ->0 ,sim 2 -> 1)

Return
List<Apn> Apn object list by sim card slot

Castles Technology

public void openBluetooth()

Description
Open BlueTooth

Parameters
none

Returns
none

public void closeBluetooth()

Description
Close BlueTooth

Parameters
none

Returns
none

public String bondBtDevice(String blueDeviceAddress)

Description
Pair BlueTooth

Parameters
address Pair bluetooth device address

Returns
Success message

public String connectBtDevice(String uuid, String blueDeviceAddress)

Description
Connect BlueTooth

Parameters
uuid Device uuid
address Device address
Returns
Success message

public String disConnectBtDevice()

Description
DisConnect BlueTooth

Parameters
none

Returns
Success message

public String getSystemTime()

Description
Get System Time

Parameters
none

Returns
system time Device system time

public String setSystemTime(int year, int month, int date, int hour, int min, int sec)

Description
Set System Time

Parameters
year Set A.D. year value -1900
month Set month value
date Set date value
hour Set hour value
min Set min value
sec Set sec value

Returns
Success message

public String getSystemZone()

Castles Technology

Description
Get System Time Zone

Parameters
none

Returns
time zone Device use time zone

public String setSystemZone(String zone)

Description
Set System Time Zone

Parameters
time zone Set time zone value

Returns
Success message

public String set24HourFormat(boolean is24Hour)

Description
Set 24 Hour Format

Parameters
state Use 24 hour format

Returns
Success message

public String setAutoRebootTime(int hour, int min)

Description
Set Auto Reboot Time

Parameters
hour Set auto reboot hour
min Set auto reboot min

Returns
Success message

public int getBrightness()

Description
Get Brightness Volume

Parameters
none

Returns
value Brightness volume

public String setBrightness(int light)

Description
Set Brightness Volume

Parameters
value Brightness volume

Returns
Success message

public int getScreenTimeout()

Description
Get Screen Off Timeout Volume

Parameters
none

Returns
value Timeout volume

public String setScreenTimeOut(int time)

Description
Set Screen Off Timeout Volume

Parameters
value Screen timeout volume(million second)

Castles Technology

Returns
Success message

public String setNavigation(boolean Home, boolean Back, boolean Search)

Description
Set Navigation Bar hide or open

Parameters
home Set navigation home button state
back Set navigation back button state
search Set navigation search button state

Returns
Success message

public int getMediaVolume()

Description
Get Media Sound Volume

Parameters
none

Returns
value Media volume

public void setMediaVolume(int volume)

Description
Set Media Sound Volume

Parameters
value Media volume

Returns
Success message

public int getRingVolume()

Description
Get Ring Sound Volume

Parameters
none

Returns
value Ring volume

public void setRingVolume(int volume)

Description
Set Ring Sound Volume

Parameters
value Ring volume

Returns
Success message

public int getAlarmVolume()

Description
Get Alarm Sound Volume

Parameters
none

Returns
value Alarm volume

public void setAlarmVolume(int volume)

Description
Set Alarm Sound Volume

Parameters
value Alarm volume

Returns
Success message

public int getNotificationVolume()

Description

Castles Technology

Get Notification Sound Volume

Parameters
none

Returns
value Notification volume

public void setNotificationVolume(int volume)

Description
Set Notification Sound Volume

Parameters
value Notification volume

Returns
Success message

public void openWifi()

Description
Open Wifi

Parameters
none

Returns
none

public void closeWifi()

Description
Close Wifi

Parameters
none

Returns
none

public String setDhcpWifi(String ssid, String password, int type)

Description
Set Wifi DHCP

Parameters
ssid Set wifi ssid
password Set wifi password
type Set wifi type(1->NOPASS,2->WEP,3->WPA)

Returns
Success message

public String setDhcpWifi(String ssid,String bssid, String password, int type)

Description
Set Wifi DHCP

Parameters
ssid Set wifi ssid
bssid Set wifi bssid
password Set wifi password
type Set wifi type(1->NOPASS,2->WEP,3->WPA)

Returns
Success message

Example
ctSettings.setDhcpWifi("Test","14:cf:92:37:75:3a","1234567890",3)

public String setStaticWifi(String SSID, String Password, int Type, String ipAddress,
String netmask, String gateway, String dns1, String dns2)

Description
Set Wifi Static

Parameters
ssid Set wifi ssid
password Set wifi password
type Set wifi type(1->NOPASS,2->WEP,3->WPA)
ipAddress Set wifi static ip address
netmask Set wifi static net mask

Castles Technology

gateway Set wifi static gateway dns1 Set wifi static dns1 dns2 Set wifi static dns 2 **Returns** Success message
public String setStaticWifi(String SSID,String bssid, String Password, int Type, String ipAddress, String netmask, String gateway, String dns1, String dns2) **Description** Set Wifi Static **Parameters** ssid Set wifi ssid bssid Set wifi bssid password Set wifi password type Set wifi type(1->NOPASS,2->WEP,3->WPA) ipAddress Set wifi static ip address netmask Set wifi static net mask gateway Set wifi static gateway dns1 Set wifi static dns1 dns2 Set wifi static dns 2 **Returns** Success message **Example** ctSettings.setStaticWifi("Test","14:cf:92:37:75:3a","1234567890",3,"192.168.98.20","255.255.255.0","192.168.98.20","8.8.8.8","8.8.8.8")
public Map getWifiConfig() **Description** Get Wifi Config **Parameters** none **Returns** wifi config All wifi config
public String setEth(int enable) **Description** Set Ethernet State **Parameters** 1 disable Ethernet 2 enable Ethernet **Returns** Success message
public int setDHCPEth(String proxy) **Description** Set Ethernet DHCP **Parameters** proxy Ethernet proxy **Returns** Success message
public int setStaticEth(String proxy, String ip, String netmask, String gateway, String dns1, String dns2) **Description** Set Ethernet Static **Parameters** proxy Set ethernet proxy ip Set ethernet ip address netmask Set ethernet netmask gateway Set ethernet gateway dns1 Set ethernet dns 1 dns2 Set ethernet dns 2 **Returns** Success message
public Map getEthConfig()

Castles Technology

Description
Get Ethernet Config

Parameters
none

Returns
ethernet config All ethernet config

public boolean getDebugModeState()

Description
Get Debug Mode State

Parameters
none

Returns
state Debug mode state

public String setDebugModeState(boolean enable)

Description
Set Debug Mode State

Parameters
state Set debug mode state

Returns
Success message

public int getBatteryThreshold()

Description
Get Battery Threshold Volume

Parameters
none

Returns
value Battery threshold volume

public String setBatteryThreshold(int threshold)

Description
Set Battery Threshold Volume

Parameters
value Battery threshold volume

Returns
Success message

public boolean setLoginPassword(String oldPassword1, String oldPassword2, String newPa
ssword1, String newPassword2)

Description
Set Login Password

Parameters
old Password1 Old login Password 1
old Password2 Old login Password 2
new Password1 Change login Password 1
new Password2 Change login Password 2

Returns
state Change state

public boolean setUserPassword(String oldPassword, String newPassword)

Description
Set User Password

Parameters
old Password Old user password
new Passwrod Change user paswword

Returns
state Change state

public boolean setFactoryPassword(String oldPassword, String newPassword)

Description

Castles Technology

Set Factory Password

Parameters
old Password Old factory password
new Password New factory password

Returns
state Change state

public String getDefaultApp()

Description
Get Default App Package Name

Parameters
none

Returns
name Default package name

public String setDefaultApp(String packageName)

Description
Set Default App Package Name

Parameters
name Default package name

Returns
Success message

public String factoryReset()

Description
Factory Reset

Parameters
none

Returns
none

public String factoryReset(int type)

Description
Factory Reset

Parameters
type -> 0 : Only Android factory reset
1 : Android + secure module factory reset

Returns
state -> success -> success string
fail -> exception string
error parameter -> Error Parameter string

Example

```
try{
    CtSetting ctSettings = new CtSettings();
    ctSettings.factoryReset(0);
}catch(Exception e){
    Log.d(TAG,e.toString());
}
```

public String virtualKeyboard(int enable)

Description
Set Virtual Keyboard When Use Hard Keyboard

Parameters
state Virtual keyboard state
“adb shell settings get secure show_ime_with_hard_keyboard” to check

Returns
Success message

public boolean setLanguage(String language,String country)

Description
Set System Language

Parameters
language Set system language

Castles Technology

country Set system country

Returns
state Change language state

public String screenLock(boolean lock)

Description
Screen Lock Pattern

Parameters
pattern Set lock pattern(true -> swipe,false -> unlock)

Returns
Success message

public String openBatteryPercentage(boolean open)

Description
Open Battery Percentage

Parameters
state Battery percentage state

Returns
Success message

public boolean getUserPasswordState()

Description
Get User Password State

Parameters
none

Returns
state User password state

public String setUserPasswordState(boolean enable)

Description :
Set User Password State

Parameters :
state User password state

Returns:
Success message

public int setNTPSetting (String server,String port,boolean autoUpdate, int time)

Description :
Set NTP All Setting

Parameters :
server Set NTP server
port Set NTP port
autoUpdate Set auto update state
time Set update time(min)

Returns:
state NTP state(success- > 0 ,fail-> -1 ,port fail -> 513)

public int setNTPServer(String server,String port)

Description :
Set NTP Server & port

Parameters :
server Set NTP server
port Set NTP port

Returns:
state NTP state(success- > 0 ,fail-> -1 ,port fail -> 513)

public int setNTPIntervalTime(int time)

Description :
Set NTP Interval Time

Parameters :
time Set update time(min)

Returns:

Castles Technology

state NTP state(success- > 0 ,fail-> -1 ,port fail -> 513)

public int setNTPAutoUpdateTime(boolean auto)

Description :
Set NTP AutoUpdateTime

Parameters :
autoUpdate Set auto update state

Returns:
state NTP state(success- > 0 ,fail-> -1 ,port fail -> 513)

public Apn getApn(int simID,String name)

Description :
Get Apn By Carrier Name

Parameters :
simID SIM card slot (sim 1 ->0,sim 2 -> 1)
name Apn carrier name

Returns:
Apn Apn object

Example:
CtSettings ctSettings = new CtSettings();
ctSettings.getApn(0,"CHT");

public Apn getCurrentApn(int simID)

Description :
Get Sim Current Apn

Parameters :
simID SIM card slot (sim 1 ->0,sim 2 -> 1)

Returns:
Apn Current Apn object

Example:
CtSettings ctSettings = new CtSettings();
ctSettings.getCurrentApn(0);

public boolean selectApn(int simID,String name)

Description :
Select Sim Apn

Parameters :
simID SIM card slot (sim 1 ->0,sim 2 -> 1)
name Apn carrier name

Returns:
state true -> select success
false -> select fail (apn not found)

Example:
CtSettings ctSettings = new CtSettings();
ctSettings.selectApn(0,"CHT");

public void setNetworkSelectionMode(int simID,boolean auto,String operatorNumeric)

Description:
Set Operator Network & Select Network Mode

Parameters:
simID -> SIM card slot (sim 1 ->0,sim 2 -> 1)
auto -> Auto/Manual Mode
operatorNumeric -> If choose Manual Mode,Need to set numeric

Return Value:
None
Exception:
operator Numeric can't select -> Operator Numeric fail

Example:
CtSettings ctSettings = new CtSettings();
ctSettings.setNetworkSelectionMode(0,false,"46692+3");

public boolean setAirplaneMode(boolean enable)

Description:

Castles Technology

Set Airplane Mode

Parameters:
on/off true -> enable airplane mode
false -> disable airplane mode

Return Value:
state true -> set success

Example:
CtSettings ctSettings = new CtSettings();
ctSettings.setAirplaneMode(true);

public String setLockPattern(int type,String password)

Description:
Set Screen Lock Type

Parameters:
type -> NONE : 0
SWIPE : 1
PATTERN : 2
PIN : 3
PASSWORD : 4

password Setting lock password type :

none,swipe : no password required,set password null

pattern : password must match the pattern lock rule,password string can refer to the picture
1.password can't repeat
2.password only allow numeric 1-9
3.password can't cross center number(example : 13,28),if the center number is selected,this rule allow (example : 213,258)
4.password must not be null and at least of length 4

pin : password only allow numeric,if set password contain alpha numeric,the lock change to type password
1.password must not be null and at least of length 4

password : password allow alpha numeric & alpha numeric + numeric,if set password only numeric,the lock change to type pin
1.password must not be null and at least of length 4

Return Value:
success -> empty String
fail -> exception

Example:
CtSettings ctSettings = new CtSettings();
ctSettings.setLockPattern(3,"1234")



public int getCurrentLockPattern()

Description
Get Screen Lock Type

Castles Technology

Parameters
none

Returns
type -> NONE : 0
SWIPE : 1
PATTERN : 2
PIN : 3
PASSWORD : 4

Example
CtSettings ctSettings = new CtSettings();
ctSettings.getCurrentLockPattern();

public List<OperationInfo> getAvailableNetworkList(int simID)

Description:
Get Operation Network Info List

Parameters:
simID SIM card slot (sim 1 ->0,sim 2 -> 1)

Return Value:
Operation info -> operatorAlphaLong(Operator complete name)
operatonAlohaShort(Operator abridge name)
operatorNumeric(Operamor numeric)
state(CURRENT,AVAILABLE,UNKNOW)

Example:
CtSettings ctSettings = new CtSettings();
ctSettings.getAvailableNetworkList(0);

public void setBluetoothBle(boolean enable)

Description:
This API uses for setting BLE support or not.

Parameters:
None

Return Value:
None

Note
The default setting is FALSE because the PCI regulation does not support BLE

public void setBluetoothJustwork(boolean enable)

Description:
This API uses for setting JUSTWORK pairing mode support or not

Parameters:
None

Return Value:
None

Note
The default setting is FALSE because the PCI regulation does not support JUSTWORK pairing mode.

boolean setMobileDataEnabled(boolean enabled)

Description:
Set Mobile data enable or disable.

Parameters:
on/off true -> enable Mobile data
false -> disable Mobile data

Return Value:
state true -> set success

Note
None

Example
import CTOS.CtSettings;
import android.util.Log;

Castles Technology

```
CtSettings mCtSettings = new CtSettings();

boolean ret = false;
try {
      ret = mCtSettings.setMobileDataEnabled(true);
      Log.d(TAG, String.format("CtSettings$$setMobileDataEnabled(), result=%b ..... ", ret));
} catch (Exception e) {
      e.printStackTrace();
}
```

void enableEnhancedLteMode(int slotIndex, boolean enable)

Description:
Set EnhancedLte mode open or close.

Parameters:
slotIndex sim card slot number (From 0)
on/off true -> open EnhanceLte mode
false -> close EnhanceLte mode

Return Value:
state true -> set success

Note
None

Example

```
import CTOS.CtSettings;
import android.util.Log;

--------------------

CtSettings mCtSettings = new CtSettings();

try {
      mCtSettings.enableEnhancedLteMode(0, true);
      Log.d(TAG, String.format("CtSystemTest$enableEnhancedLteMode(0, true) ..... "));
} catch (Exception e) {
      e.printStackTrace();
}
```

boolean isEnhancedLteEnabled(int slotIndex)

Description:
Check the status of EnhancedLTE.

Parameters:
slotIndex sim card slot number (From 0)

Return Value:
state true -> open

Note
None

Example

```
import CTOS.CtSettings;
import android.util.Log;

--------------------

CtSettings mCtSettings = new CtSettings();

boolean ret = false;
try {
      ret = mCtSettings.isEnhancedLteEnabled(0);
      Log.d(TAG, String.format("CtSystemTest$isEnhancedLteEnabled(0), result=%b ..... ", ret));
} catch (Exception e) {
      Log.e(TAG, e.getMessage());
}
```

boolean setTalkBack(String ComponentName)

Description:
Set talkback function.

Parameters:
ComponentName

Castles Technology

Return Value:
state true -> success

Note
None

Example
```
import CTOS.CtSettings;
import android.util.Log;

-------------------

CtSettings mCtSettings = new CtSettings();

boolean ret = false;
try {
	String ComponentName = "com.google.android.marvin.talkback/com.google.android.marvin.talkback.TalkBackService";
	ret = mCtSettings.setTalkBack(ComponentName);
	Log.d(TAG, String.format("CtSystemTest$setTalkBack(%s), result=%b ..... ", ComponentName, ret));
} catch (Exception e) {
	e.printStackTrace();
}
```

Example

```
public class MainActivity extends AppCompatActivity {

    private CtSettings ctSettings;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        //init
        ctSettings = new CtSettings();


        try {


            ctSettings.getApn("466", "01");

            ctSettings.setApn("遠傳電信(Far EasTone) (MMS)",
                    "fetnet01",
                    "",
                    "",
                    "",
                    "-1",
                    "mms",
                    "466",
                    "01");

            ctSettings.openBluetooth();

            ctSettings.closeBluetooth();

            ctSettings.bondBtDevice("address");

            ctSettings.connectBtDevice("", "address");

            ctSettings.disConnectBtDevice();

            ctSettings.getSystemTime();

            ctSettings.setSystemTime(2020, 8, 20, 0, 0, 0);

            ctSettings.getSystemZone();

            ctSettings.setSystemZone("GMT");

            ctSettings.set24HourFormat(true);

            ctSettings.setAutoRebootTime(14, 0);

            ctSettings.getBrightness();
```

Castles Technology

```
ctSettings.setBrightness(0);

ctSettings.getScreenTimeout();

ctSettings.setScreenTimeOut(300 * 1000);

ctSettings.setNavigation(true, false, true);

ctSettings.getMediaVolume();

ctSettings.setMediaVolume(15);

ctSettings.getAlarmVolume();

ctSettings.setAlarmVolume(7);

ctSettings.getRingVolume();

ctSettings.setRingVolume(7);

ctSettings.getNotificationVolume();

ctSettings.setNotificationVolume(7);

ctSettings.openWifi();

ctSettings.closeWifi();

ctSettings.setDhcpWifi("SSID","00000000",3);

ctSettings.setStaticWifi("SSID",
        "00000000",
        3,
        "192.120.1.1",
        "192.120.1.1",
        "192.120.1.1",
        "8.8.8.8",
        "8.8.8.8");

HashMap map = (HashMap) ctSettings.getWifiConfig();

Log.d(this.toString(), (String) map.get("ssid"));
Log.d(this.toString(), (String) map.get("dhcp"));
Log.d(this.toString(), (String) map.get("ipAddress"));
Log.d(this.toString(), (String) map.get("netMask"));
Log.d(this.toString(), (String) map.get("networkType"));
Log.d(this.toString(), (String) map.get("gateWay"));
Log.d(this.toString(), (String) map.get("dns1"));
Log.d(this.toString(), (String) map.get("dns2"));

ctSettings.setEth(2);

ctSettings.setDHCPEth("proxy");

ctSettings.setStaticEth("proxy"
        ,"192.120.1.1"
        ,"192.120.1.1"
        ,"192.120.1.1"
        ,"8.8.8.8"
        ,"8.8.8.8");

HashMap map = (HashMap) ctSettings.getEthConfig();

Log.d(this.toString(),(String)map.get("proxy"));
Log.d(this.toString(), (String) map.get("dhcp"));
Log.d(this.toString(),(String)map.get("ipAddress"));
Log.d(this.toString(),(String)map.get("netMask"));
Log.d(this.toString(),(String)map.get("gateWay"));
Log.d(this.toString()(String),map.get("dns1"));
Log.d(this.toString(),(String) map.get("dns2"));

ctSettings.getDebugModeState();

ctSettings.setDebugModeState(true);

ctSettings.getBatteryThreshold();
```

Castles Technology

```
                ctSettings.setBatteryThreshold(90);

                ctSettings.getDefaultApp();

                ctSettings.setDefaultApp("com.example.package");

                ctSettings.setUserPassword("00000000","11111111");

                ctSettings.setLoginPassword("00000000"
                        ,"00000000"
                        ,"11111111"
                        ,"11111111");

                ctSettings.setFactoryPassword("00000000","11111111");

                ctSettings.factoryReset();

                ctSettings.virtualKeyboard(0);

                ctSettings.setLanguage("zh","TW");

                ctSettings.screenLock(false);

                ctSettings.openBatteryPercentage(true);

                ctSettings.setNTPSetting("tock.stdtime.gov.tw","123",true,1);

                ctSettings.getUserPasswordState();

                ctSettings.setUserPasswordState(true);

                ctSettings.setNTPServer("tock.stdtime.gov.tw","123");

                ctSettings.setNTPIntervalTime(1);

                ctSettings.setNTPAutoUpdateTime(true);

                Apn apn    = ctSettings.getApn(0, "apnTest");
                Log.d(this.toString(),apn .getApn());
                Log.d(this.toString(),apn.getName());
                Log.d(this.toString(),apn.getMcc());
                Log.d(this.toString(),apn.getMnc());
                Log.d(this.toString(),""+apn.getAuthtype());
                Log.d(this.toString(),apn.getPassword());
                Log.d(this.toString(),apn.getServer());
                Log.d(this.toString(),apn.getType());
                Log.d(this.toString(),apn.getUser());

                Apn apn = new Apn();
                apn.setName("apnTest");
                apn.setAuthtype(0);
                apn.setApn("aaa");
                apn.setMcc("466");
                apn.setMnc("55");
                Log.d(this.toString(),""+ctSettings.setApn(0,apn));

                List<Apn> apnName = ctSettings.getApnList(0);
                for (Apn apn : apnName){
                    Log.d(this.toString(),apn.getName());
                    Log.d(this.toString(),apn.getType());
                    Log.d(this.toString(),""+apn.getAuthtype());
                    Log.d(this.toString(),apn.getPassword());
                    Log.d(this.toString(),apn.getMcc());
                    Log.d(this.toString(),apn.getMnc());
                }

            } catch (RemoteException e) {
                e.printStackTrace();
            }
        }
}
```

Castles Technology

1.1.11 Public class CtCradle

The **CtCradle** class is used to provide the cradle functions.

Constructor Summary	
public	CtCradle() Constructs a binder connected with service.

Method Summary	
boolean	connect(int connectType) Connect the cradle
EthernetConfiguration	getEthernetInfo() Get cradle ethernet info
boolean	setEthernetInfo(EthernetConfiguration config) Set cardle ethernet info
boolean	close() Close the cradle
boolean	hideCradleWifi(boolean hiddenNetwork) Hide Cradle Wifi SSID

Constructor Detail

public CtCradle()
Create a CtCradle object and connect with a binder to invoke service.

Method Detail

public boolean connect(int connectType)

Description :
Connect the cradle

Parameters :
connectType 0 -> Connect cradle with wifi
1 -> Connect cradle with usb

Returns:
connect state true -> Connect success
false -> Connect fail

public EthernetConfiguration getEthernetInfo()

Description :
Get Cradle Ethernet Info

Parameters :
none

Returns:
Ethernet obj Cradle ethernet obj

public boolean setEthernetInfo(EthernetConfiguration config)

Description :
Set Cradle Ethernet Info

Parameters :
Ethernet obj Cradle ethernet obj

Returns:
set state true -> Set success
false -> Set fail

public boolean close()

Description :
Close Cradle Connect

Parameters :

Castles Technology

none

Returns:
close state true -> Close success
false -> Close fail

public boolean hideCradleWifi(boolean hiddenNetwork)

Description :
Hide Cradle Wifi SSID.

Parameters :
IN] hiddenNetwork = true (hide Cradle Wifi SSID)
[IN] hiddenNetwork = false (show Cradle Wifi SSID)

Returns:

Constants	Value
Success	true
Fail	false

Example:
boolean test = server.hideCradleWifi(hide);

Example

```
public class MainActivity extends AppCompatActivity {
    private CtCradle ctCradle;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        ctCradle = new CtCradle();
    }

    public void b(View view) {
        new Thread(new Runnable() {
            @Override
            public void run() {
                try {
                    //0 -> wifi ,1 -> usb
                    if(ctCradle.connect(0)) {
                        EthernetConfiguration cradleConfig = ctCradle.getEthernetInfo();
                        Log.e(this.toString(), "" + cradleConfig.isDhcp());
                        Log.e(this.toString(), cradleConfig.getIp());
                        Log.e(this.toString(), cradleConfig.getGateWay());
                        Log.e(this.toString(), cradleConfig.getNetMask());
                        Log.e(this.toString(), cradleConfig.getDns());
                        Log.e(this.toString(), cradleConfig.getDns2());
                        Log.e(this.toString(), cradleConfig.getVersion());

                        EthernetConfiguration cradleConfig1 = new EthernetConfiguration();
                        cradleConfig1.setDhcp(false);
                        Log.e(this.toString(), "isDhcp " + cradleConfig1.isDhcp());
                        cradleConfig1.setIp("192.168.1.100");
                        cradleConfig1.setGateWay("192.168.1.1");
                        cradleConfig1.setNetMask("255.255.255.0");
                        cradleConfig1.setDns("8.8.8.8");
                        cradleConfig1.setDns2("8.8.8.8");
                        Log.e(this.toString(), "static " + ctCradle.setEthernetInfo(cradleConfig1));

                    }

                    ctCradle.close();
                } catch (Exception e) {
                    ctCradle.close();
                    e.printStackTrace();
                }
            }
        }).start();
    }
}
```

CastlesTechnology

1.1.12 Public class CtDEX

The CtDEX class is used to provide a feature to interact with VMC(Vending Machine Controller).

Constructor Summary	
public	CtDEX() Create a CtDEX object.

Method Summary	
int	open(int Comport) The function is used to open the designated COM port.
int	close() The function is used to close the COM port.
int	send(char[] Data; int Len) The function is used to send Data(such as commands) to VMC/VMD via designated COM Port.
int	getRawData(char[] Data, int[] Len) This function is used to get the raw data from VMC/VMD via designated COM Port.
int	getFieldData(char[] Tag, int Field, Boolean Reload, char[] Data, int[] Len) This function is used to get the raw data from VMC/VMD via designated COM Port.
int	getData(char[] Tag; Boolean Reload, char[] Data, int[] Len) The function is used to get the designated data from VMC/CMD via COM Port.

Constructor Detail

public CtDEX()
Create a CtDEX object and connect with a binder to invoke service.

Method Detail

public int open(int Comport)

Description :
This function is used to open the designated COM port.

Parameters :m
[IN] Comport: The Com Port // Dex_COM1

Returns:

OK	-- OK
DEX_INVALID_PARA	-- Invalid parameter
DEX_OPEN_FAILED	-- Open the designated COM Port failed.

public int close();

Description :
The function is used to close the COM Port.

Parameters :
Void

Returns:

OK	-- OK
DEX_CLOSE_FAILED	-- Close the designated COM Port failed.

public int send(char[] Data, int Len)

Description :
The function is used to send the data(such as commands) to VMC/VMD via designated COM Port.

Parameters :
[IN] Data: The data sent to VMC/VMD. it must follow the JSON format.
Eg:
[{ “Tag”:”tag1”, “Filed1”:”100”},
{ “Tag”:”tag2”, “Filed1”:”500”, “Filed2”:”20”},
{ “Tag”:”tag3”, “Filed1”:”BC”,“Filed3”:”0”, “Filed4”:ABC”}]
[IN] Len: The data size need to send to VMC/VMD.

Castles Technology

Returns:

OK	- - OK.
DEX_INVALID_PARA	- - Invalid parameter.
DEX_FORMAT_ERROR	- - The format of the data sent to VMC/VMD was wrong.
DEX_CONNECT_FAILED	- - Can't connect with VMC/VMD
DEX_TIMEOUT	- - Get DEX data timeout.

public int getRawData(char[] Data, int Len)

Description :

The function is used to send the raw data from VMC/VMD via designated COM Port.

Parameters :

[OUT] Data: The raw data getting from VMC/VMD.

[IN/OUT] Len:

[IN]: The buffer size which need to receive.

[OUT]: The size of raw data which get from VMC/VMD.

Returns:

OK	- - OK.
DEX_INVALID_PARA	- - Invalid parameter.
DEX_CONNECT_FAILED	- - Can't connect with VMC/VMD.
DEX_TIMEOUT	- - Get DEX data timeout.
DEX_MEMORY_NOT_ENOUGH	- - The Buffer is too small to store the DEX raw data.

public int getFieldData(char[] Tag, int TagIndex, int Field, Boolean Reload, char[] Data, int[] Len)

Description :

The function is used to send the raw data from VMC/VMD via designated COM Port.

Parameters :

[IN] Tag :The Tag name.

[IN] TagIndex: The index of Tag.

[IN] Field: The field of Tag.

[IN] Reload: The flag of reloading DEX data.

false: - - It will return the data stored in memory if the data exists.

true: - - It will get the DEX data from VMC/VMD again.

[OUT] Data: The data of designated Tag and Field.

[IN/OUT] Len:

[IN] :The data size which need to receive.

[OUT] : The size of receiving data.

Returns:

OK	- - OK.
DEX_INVALID_PARA	- - Invalid parameter.
DEX_NO_DATA	- - No data exists if the boReload is FALSE.
DEX_NO_TAG	- - No Tag exists in DEX data.
DEX_NO_TAG_INDEX	- - No Tag index.
DEX_NO_FIELD	- - No Field exists with designated Tag.
DEX_CONNECT_FAILED	- - Can't connect with VMC/VMD.
DEX_TIMEOUT	- - Get DEX data timeout.
DEX_MEMORY_NOT_ENOUGH	- - The Buffer is too small to store the DEX raw data.

public int getData(char[] Tag, boolean Reload, char[] Data, int[] Len)

Description :

The function is used to get the designated data from VMC/VMD via COM Port.

Parameters :

[IN] Tag: The Tags User designated, it must follow the JSON format.

Eg:

[{ "Tag":"tag1", "TagIndex":1, "Filed1":2},

{ "Tag":"tag2", "TagIndex":2, "Filed1":2, "Filed2":3},

{ "Tag":"tag3", "TagIndex":2, "Filed1":2, "Filed2":3, "Filed3":5, "Filed4":6}]

For tag2, field 2 and field 3 will be shown if they exist.

[IN] Reload : The flag of reloading DEX data.

false: - - It will return the data stored in memory if the data exists.

true: - - It will get the DEX data from VMC/VMD again.

Castles Technology

[OUT] Data: The data getting from VMC/VMD.
Eg:
[{ “Tag”:”tag1”, “TagIndex”:1, “Filed1”:”100”},
{ “Tag”:”tag2”, “TagIndex”:2, “Filed1”:”500”, “Filed2”:”0”},
{ “Tag”:”tag3”,“TagIndex”:2,“Filed1”:”BC”,“Filed2”:”0”, “Filed3”:”UNLOCK”, “Filed4”:”0”}]
[IN/OUT] Len
[IN] The data size which need to receive.
[OUT] The size of receiving data..

Returns:

OK	- - OK.
DEX_INVALID_PARA	- - Invalid parameter.
DEX_FORMAT_ERROR	- - The format of the data was wrong.
DEX_NO_DATA	- - No data exists if the boReload is FALSE.
DEX_NO_TAG	- - No Tag exists in DEX data.
DEX_NO_TAG_INDEX	- - No Tag index.
DEX_NO_FIELD	- - No Field exists with designated Tag.
DEX_CONNECT_FAILED	- - Can’t connect with VMC/VMD.
DEX_TIMEOUT	- - Get DEX data timeout.
DEX_MEMORY_NOT_ENOUGH	- - The Buffer is too small to store the DEX raw data.

Example

```
package com.castles.dexdemo;

import CTOS.CtDEX;
import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;
import android.util.Log;

public class MainActivity extends AppCompatActivity {

    final String TAG    = "dexDemo";
    private static void printArray(String message, char[] array) {
        System.out.println(message + ": [length = " + array.length + "]");
        for (int i = 0; i < array.length; i++) {
            if (i < 0x7f) {
                //System.out.print(", ");
                System.out.print(array[i]);
             }
        }
        System.out.println();
    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        //setContentView(R.layout.activity_main);
        new Thread(new Runnable() {
            @Override
            public void run() {
                int nRet = 0;

                char[] RawData = new char[5120];
                int pnLen[] = new int[1];

                CtDEX dex = new CtDEX();

                nRet = dex.open(dex.DEX_COM1);
                Log.i(TAG,"openComport return code is: " + nRet);

                String str;
                String sendCommand ="[{\"Tag\":\"PC1\",\"Field1\":\"1\",\"Field2\":700,\"Field3\":\"Food1\"},
                                    {\"Tag\":\"SD1\",\"Field5\":\"SAVE\"}]";
                nRet = dex.send(sendCommand.toCharArray(), sendCommand.length());
```

Castles Technology

```
                Log.i(TAG,"send return code is: " + nRet);


                pnLen[0] = RawData.length;
                nRet = dex.getRawData(RawData, pnLen);
                Log.i(TAG,"getRawData return code is: " + nRet);
                str = String.valueOf(RawData, 0, pnLen[0]);
                Log.i(TAG, "getRawData is : " + str);
                Log.i(TAG,"getRawData len is: " + pnLen[0]);

                String tag = "PA2";
                char[] TagData = new char[512];
                pnLen[0] = TagData.length;
                nRet = dex.getFieldData(tag.toCharArray(), 3,1, false, TagData, pnLen);
                Log.i(TAG,"getFieldData return code is: " + nRet);
                str = String.valueOf(TagData, 0, pnLen[0]);
                Log.i(TAG, "getFieldData is : " + str);
                Log.i(TAG,"getFieldData len is: " + pnLen[0]);

                pnLen[0] = TagData.length;
                String req = "[{\"Tag\":\"ID1\",\"TagIndex\":1,\"Field1\":2},
                                {\"Tag\":\"VA1\",\"TagIndex\":1,\"Field1\":1,\"Field2\":2},
                                {\"Tag\":\"ID4\",\"TagIndex\":1,\"Field1\":1,\"Field2\":2,\"Field3\":3},
                                {\"Tag\":\"PA2\",\"TagIndex\":4,\"Field1\":1,\"Field2\":4,\"Field3\":3,\"Field4\":2}]";

                nRet = dex.getData(req.toCharArray(), false, TagData, pnLen);
                Log.i(TAG,"getData return code is: " + nRet);
                str = String.valueOf(TagData, 0, pnLen[0]);
                Log.i(TAG, "getData is : " + str);
                Log.i(TAG,"getData len is: " + pnLen[0]);

                nRet = dex.close();
                Log.i(TAG,"closeComport return code is: " + nRet);
            }
        }).start();
    }
}
```

Castles Technology

1.1.13 Public class CtCertificate

public class CtCertificate
The CtCertificate class provides the functionalities to accomplish capture of CA certificate.

Constructor Summary	
public	CtCertificate() Constructs a binder connected with service.

Method Summary	
List<String>	getUserCaAliases() Get aliases list from trusted user CAs.
X509Certificate	getEncodedCaCertificate(String alias) Get X509Certificate by alias.
boolean	deleteCaCertificate(String alias) Deletes the given certificate from trusted user CAs, if present.
boolean	installCaCert (byte[] certBuffer) Installs the given certificate as a user CA.

Constructor Detail

public CtCertificate()
Create a CtCertificate object and connect with binder to invoke service.

Method Detail

public List<String> getUserCaAliases ()

Description :
Get aliases list from trusted user CAs.

Parameters
None

Returns:
The List<String> of CA aliases.

---

public X509Certificate getEncodedCaCertificate(String alias)

Description :
Get X509Certificate by alias.

Parameters
[IN] alias
CA alias. Get it by calling getUserCaAliases().

Returns:
The X509Certificate of Ca Certificate with alias.

---

public boolean deleteCaCertificate(String alias)

Description :
Deletes the given certificate from trusted user CAs, if present.

Parameters
[IN] alias
CA alias. Get it by calling getUserCaAliases().

Returns:

Constants	Value
Success	true
Fail	false

---

public boolean installCaCert(byte[] certBuffer)

Description :
Installs the given certificate as a user CA.

Parameters
[IN] certBuffer
byte: encoded form of the certificate to install.

Castles Technology

Returns:

Constants	Value
Success	true
Fail	false

Example

```
/*
Sample code
*/
import CTOS.CtCertificate;

private CtCertificate ctCertificate = new CtCertificate();

try {
		InputStream inputStream = getAssets().open("COMPANY_KEY.crt");
		try {
			byte keychain[] = new byte[(int) inputStream.available()];
			inputStream.read(keychain);
			Log.d(TAG, "installCaCert ret : "+ctCertificate.installCaCert(keychain));
		} catch (IOException e) {
			e.printStackTrace();
		}
} catch (FileNotFoundException e) {
		e.printStackTrace();
} catch (IOException e) {
		e.printStackTrace();
}

List<String> aliasStringList = ctCertificate.getUserCaAliases();
Log.d(TAG, "**Alias num : "+aliasStringList.size());
for (int i = 0; i < aliasStringList.size(); i++) {
		X509Certificate x509Certificate = ctCertificate.getEncodedCaCertificate(aliasStringList.get(i));
		showCertInfo(aliasStringList.get(i), x509Certificate);
		Log.d(TAG, "**** ctCertificate.deleteCaCertificate ret : "+ctCertificate.deleteCaCertificate(aliasStringList.get(i)));
}

private void showCertInfo(String keyName, X509Certificate certificate)
{
		Log.d(TAG, "   ====showCertInfo====");
		Log.i(TAG, "   CertificateName: " + keyName);
		Log.i(TAG, "   Version: " + certificate.getVersion() + "");
		Log.i(TAG, "   SerialNumber: " + certificate.getSerialNumber().toString(16));
		Log.i(TAG, "   SigAlgName: " + certificate.getSigAlgName());
		Log.i(TAG, "   Issuer: " + certificate.getIssuerX500Principal().toString());
		Log.i(TAG, "   NotBefore: " + certificate.getNotBefore().toString());
		Log.i(TAG, "   NotAfter: " + certificate.getNotAfter().toString());
		Log.i(TAG, "   Subject: " + certificate.getSubjectX500Principal().toString());
		Log.i(TAG, "   Signature: " +   ByteArray2HexString(certificate.getSignature()));
		Log.i(TAG, "   PublicKeyAlgorithm: " + certificate.getPublicKey().getAlgorithm());
		Log.i(TAG, "   PublicKey: " + certificate.getPublicKey().toString());

		try{
			Log.i(TAG, "   ExtendedKeyUsage: " + certificate.getExtendedKeyUsage() + "");
		}catch (CertificateParsingException e){
```

```
		}

		try{
			Log.i(TAG, "   IssuerAlternativeNames: " + certificate.getIssuerAlternativeNames() + "");
		}catch (CertificateParsingException e){

		}
}

private static String ByteArray2HexString(byte[] b)
{
		String str = "";
		for (byte aB : b) {
			str += Integer.toString((aB & 0xff) + 0x100, 16).substring(1);
		}
		return str;
}
```

Castles Technology

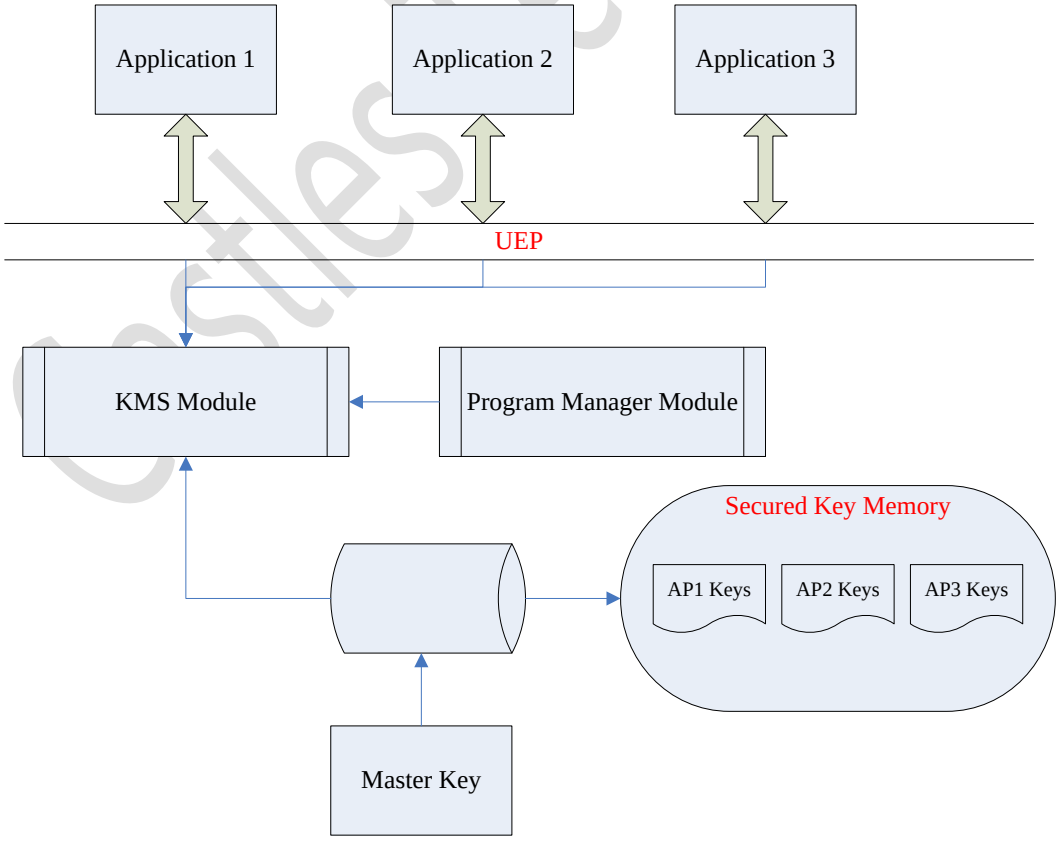
2. KMS

The following describes all the necessary information for application developers to understand how to use the Castles Android Key Management System (CAKMS) to store their application keys and perform cryptographic operations with them.

The KMS is designed to securely store sensitive data such as keys and provide cryptographic operation, which includes:

- KEK (Highest Level Master Key) maintenance.
- Key integrity checking.
- Hardware tamper for key protection.
- User keys maintenance in key storage (Injection, deletion, etc.).
- Corresponding cryptographic operations with key for user usage (PIN Cipher, Data Encryption, etc.).

2.1 KMS Framework



Castles Technology

The key storage is provided for storing sensitive data. All data in key storage are encrypted by RMK. RMK is Root Master Key that is automatically random generated during first boot.

Each application has its own keys. The key belonged to one application cannot be accessed by others. Accessing application keys is always allowed via “KMS Module”. When an application tries to use their keys by the APIs provided in KMS, the KMS Module will provide its cryptographic operations only if the application is the key owner. When an application is removed, the keys belonged to this application will be removed as well. The “Program Manager Module” is responsible for this key deletion.

2.2 Key Management Specifications

2.2.1 Methods of Key Management

Methods of key management are supported by CAKMS as below:

- Master Keys / Transaction Keys: This method uses a hierarchy of Key Encrypting Keys and Transaction Keys. The highest level of Key Encrypting Key is known as a Master Key. Master Keys are distributed and stored in CAKMS key storage before Transaction Keys are distributed. Transaction Keys are distributed and replaced encrypted under a Key Encrypting Key. This method is defined in ANS X9.24.

- Fixed Transaction Keys: This method uses keys for transaction processing directly. It is defined in ANS X9.24.

- DUKPT (Derived Unique Key per Transaction): DUKPT is a key management method which uses a unique key for each transaction, and prevents the disclosure of any past key. This method is defined in ANS X9.24.

2.2.2 Methods of Key Injection

CAKMS provides two secure methods for key injection. These methods protect the secrecy and integrity of the key block, and also make sure that the key is used only for a single purpose. The methods for key injection are:

- Key Block Binding Method Using Key Derivation: this method is the technique that defined in ANSI X9.143. The method uses a Key Block Protection key that was previously exchanged between the two communicating parties. The Key

Castles Technology

Block Protection key is only used to derive encryption key and MAC key to make sure the secrecy and integrity of the key block.

- Castles Key Block Binding Method: this method is proprietary technique that defined by Castles Technology. The method uses a CTOS Key Block Protection key that was previously exchanged between the two communicating parties. The CTOS Key Block Protection key is only used to derive encryption key and MAC key to make sure the secrecy and integrity of the key block.

2.2.3 Key Identification and Attributes

The keys are identified in CAKMS by Key Set and Key Index. Key Set and Key Index can be specified during performing key injection.

Key Set is an integer that ranges between 0x0001 and 0xBFFF; other value is reserved for future use. Key Index is an integer that ranges between 0x0000 and 0xFFFF.

The keys in the same key set (specified with the same KeySet) shall be used with the same algorithm/management method (TDES, AES, or DUKPT), and shall be owned by the same owner.

Each key has its own attributes, incl. Key Owner, Key Usage, Key Algorithm, and Mode of Use to depict the ability in CAKMS. The attributes are described in the following section.

2.2.3.1 Key Owner

The key owner is an attribute of the key for CAKMS to know who owns the key. When the app uses the key injection method provide by CAKMS, CAKMS will set the app name as the key owner automatically. It cannot be changed once the key is injected.

To comply with PCI PTS, CAKMS only allow the key to be accessed by its owner.

Castles Technology

2.2.3.2 Key Usage

The key usage is an attribute of the key for CAKMS to identify the type of the key, whether it is used to encrypt data, calculate a MAC, etc. The key usage is specified inside key block; and cannot be changed once the key is injected.

The following table shows the key usages provided by CAKMS.

Method: Master Keys / Transaction Keys		
'KB'	0x4B, 0x42	Master Key.
Method: Fixed Transaction Keys		
'P0'	0x50, 0x30	PIN Encryption Key.
'D0'	0x44, 0x30	Data Encryption Key.
'M0'	0x4D, 0x30	MAC Key.
Method: Derived Unique Key per Transaction		
'B1'	0x42, 0x31	DUKPT Initial Key, also known as IPEK.
Method: Key Block Binding Method Using Key Derivation		
'K1'	0x4B, 0x31	ANSI X9.143 Key Block Protection Key
Method: Castles Key Block Binding Method		
'KA'	0x4B, 0x41	Castles Proprietary Key Block Protection Key

2.2.3.3 Algorithm

The algorithm is an attribute of the key for CAKMS to know which algorithm is used with the key. It is specified inside key block; and cannot be changed once the key is injected.

The following table shows the algorithms provided by CAKMS.

Algorithm		
'T'	0x54	Triple DES
'A'	0x41	AES

2.2.3.4 Mode of Use

The mode of use is an attribute of the key for CAKMS to know what kind of the operation modes are allowed. It is specified inside key block; and cannot be changed once the key is injected.

The following table shows the mode of uses provided by CAKMS.

Mode of Use		
'B'	0x42	Both Encrypt & Decrypt
'D'	0x44	Decrypt only
'E'	0x45	Encrypt only
'G'	0x46	Generate only
'X'	0x58	Key used to derive other keys

Castles Technology

2.2.4 Single Purpose and Unique Value

CAKMS guarantees the key that only has single purpose. It is achieved by the design of the secure key injection methods. During injection, the key value will be checked to ensure its value is unique in the key storage.

The key injection methods are described in the following sections.

2.3 Key Injection Methods

The methods for key injection provided by CAKMS support following properties and make sure they are protected appropriately in the process:

- Key confidentiality
- Key integrity
- Key purpose
- Algorithm used for the key
- Key length

The available methods for the app are shown as below:

- Method: Key Block Binding Method Using Key Derivation
- Method: Castles Key Block Binding Method

2.3.1 Notation

The following notations are used to indicate field encoding in the key block:

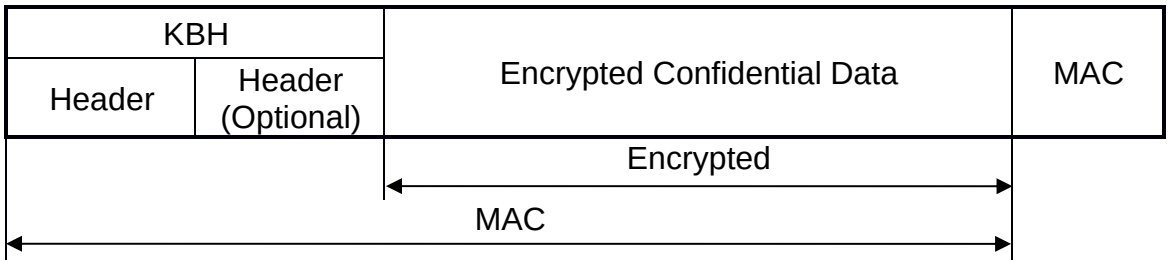
nA - n-digits of Alphabetic ('A'-'Z', 'a'-'z'), e.g., 6A means exactly 6 alphabetic characters in ASCII
nAN - Alphanumeric ('A'-'Z', 'a'-'z', '0'-'9'), e.g., 6AN means exactly 6 alphanumeric characters in ASCII
nH - Hex-ASCII ('0'-'9', 'A'-'F'), e.g., 6H means exactly 6 hex-ASCII characters
nN - Numeric-ASCII ('0'-'9'), e.g., 6N means exactly 6 decimal characters in ASCII
nB – Binary bytes (0x00 to 0xFF), e.g., 6B means exactly 6 bytes of binary data
nPA – n Printable ASCII characters. Printable ASCII characters are those with hex values in the range 20-7E, inclusive. 6PA means exactly 6 printable ASCII characters.
|| - used for concatenation of bit strings, e.g., hex 30A4F || 0C1 is equal to hex 30A4F0C1

2.3.2 Key Block Binding Method Using Key Derivation

Castles Technology

This section introduces the key injection method using **Method: Key Block Binding Method Using Key Derivation** which is following X9.143

The key block defined in X9.143 contains 3 parts show as below,



The 1st part is Key Block Header (KBH), which is not encrypted and contains attribute information about protected key. This part is composed of a 16 bytes fixed format Header section and an optional Header section. The format and usage for KBH is described in following section: ***2.3.2.1 Key Block Header*.**

The 2nd part is Encrypted Confidential Data, which contains the information of the key length, the key, and any required padding. The format of this is described in following section: ***2.3.2.2 Encrypted Confidential Data***.

The 3rd part is MAC, which is 16 bytes in Hex-ASCII. The format of this is described in following section: ***2.3.2.3 MAC***.

2.3.2.1 Key Block Header (KBH)

The Key Block Header is a plain-text section which contains attribute information about the key. The data in this section uses uppercase ASCII printable characters for better supportability (i.e. human readability).

The below table shows the format of the Key Block Header for this method:

Byte #	Field Name	Description	Encoding
0	Key Block Version ID	Identifies the version of the key block, this field should be: ◆ **'B' (0x42) – Key Block Protected using the Key Derivation Binding Method 2010**	1AN
1-4	Key Block Length	ASCII numeric digits providing key block length after encoding. The length includes KBH, encrypted confidential data, and MAC in decimal. e.g., a 112 bytes key block would contain, '0' in byte #1, '1' in byte #2, '1' in byte #3, and '2' in byte #4.	4N
5-6	Key Usage	Provides information about the intended function of the protected key. The available values are defined as below, ◆ **'B1' (0x42, 0x31) – DUKPT Initial Key** ◆ **'D0' (0x44, 0x30) – Data Encryption Key** ◆ **'P0' (0x50, 0x30) – PIN Encryption Key**	2AN

Castles Technology

		◆ **'K1' (0x4B, 0x31) – ANSI X9.143 Key Block Protection Key** ◆ **'M0' (0x4D, 0x30) – MAC Key**	
7	Algorithm	The algorithm for the protected key may be used. The available values are defined as below, ◆ **'T' (0x54) – Triple DES**	1AN
8	Mode of Use	Defines the operation the protected key can perform. For **DUKPT Initial Key**, the available options as below**,** ◆ **'X' (0x58) – Key used to derive other keys** For **Data Encryption Key**, the available options as below, ◆ **'B' (0x42) – Encrypt and Decrypt** For **PIN Encryption Key**, the available options as below, ◆ **'E' (0x45) – Encrypt Only** For **Key Block Encryption Key**, the available options as below, ◆ **'D' (0x44) – Decrypt Only** For **ANSI X9.143 Key Block Protection Key**, the available options as below, ◆ **'D' (0x44) – Decrypt Only** For **Master/Session Key**, the available options as below, ◆ **'D' (0x44) – Decrypt Only** ◆ **'E' (0x45) – Encrypt Only** For **MAC Key**, the available options as below, ◆ **'G' (0x47) – Generate Only**	1AN
9-10	Key Version Number	Two-digit ASCII character version number, optionally used to prevent re-injection of the old keys.	2AN
11	Exportability	This field should be filled with below value, ◆ **'N' (0x4E) – Non-exportable**	1AN
12-13	NoOB	Defines the number of Optional Blocks included in the key block. For example, 5 Optional Blocks would be represented by '0' (0x30) in byte 12 and '5' (0x35) in byte 13.	2N
14-15	RFU	This field is reserved for future used and is filled with ASCII zero (0x30) character.	2N
Note. The remaining fields are only present if bytes 12-13 contain a value other then '00'			
16-17	First Optional Block ID	ID field for the First Optional Block. The possible ID values are defined as below: ◆ **'KS' (0x4B, 0x53) - Key Set Identifier** ◆ **'PB' (0x50, 0x42) – A variable-length padding field used as the last Optional Block.**	2AN
18-19	Optional Block 1 Length	The length of that First Optional Block in bytes, including the field's ID, Length, and Data. Byte 18 contains the high order byte of the length and byte 19 contains the low-order byte, both in hex-ASCII form. E.g., for a 24 byte Optional Block (18 in hexadecimal notation), byte 18 would contain a '1' (0x31) and byte 19 would contain an '8' (0x38). (Note: Optional Block Data may be omitted, in which case the Optional Block Length will be '04')	2H
20-n	Optional Block 1 Data	The Data for that Optional Block. It will contain only printable ASCII characters.	PA
n-m	Additional Optional Blocks, if present	A variable number of Optional Blocks can follow the first Optional Block, as indicated by the count in bytes 12-13. Each Optional Block contains a 2-byte ID, 2-byte length, and variable-length data field following the same format described above for Optional Block 1.	

Castles Technology

If the Key Usage in KBH is specified as **'B1' (0x42, 0x31) – DUKPT Initial Key**, the First Optional Block is required for Key Set Identifier. A sample for Optional Block contains a Key Set Identifier shows as below:

Identifier: 'KS'
Length: '18'
Data: 'FFFF9876543210E00000'
Note: The length value equals 2+2+20 (24). The length value is encoded in Hex-ASCII; 18 hex equals 24 decimal.

The total length of all optional blocks in the key block shall be a multiple of the encryption block size. This may require padding, and if padding is needed it is included in a special final optional block that is filled with an appropriate number of padding characters. For example, if the total length of all optional blocks before padding is 90, six additional bytes are needed in order to bring the total to the next multiple of the encryption block size, 96. A padding block would be added as the last optional block, containing the two-byte identifier 'PB' a two-byte length field '06', and two characters of padding that filled with readable ASCII characters.

2.3.2.2 Encrypted Confidential Data

The below table shows the format of the encrypted confidential data:

Byte #	Field Name	Description	Encoding
var.	Encrypted Confidential Data	The encrypted confidential data.	nH

The encrypted confidential data contains the information of the key length, the key, and the padding if any. This part is CBC-encrypted with TDES, and using MAC (MAC is described in next section) as IV. The resultant of encryption is required to be converted to Hex-ASCII only constructing the complete key block. The derivation method of the encryption key is described in X9.143

The binary key data in plain-text is constructing as below table:

Byte #	Field Name	Description	Encoding
0-1	Key Length	The bit length of the Key. E.g., 128 bits, byte 0 would contain a 0x00 and byte 1 would contain a 0x80.	2B
var.	Key	The key value field.	nB
var.	Padding	The padding data field. This field is padded on the right with the minimum number of random pad bytes in the range 0x00 through 0xFF, inclusive until binary key data is a multiple of 8 bytes.	nB

Castles Technology

A sample for binary key data contains 128 bits key show as below:

The length of the key: 128 bits
The key value: A970BC5E82B91F9528D15FC2E14B5FD1
The padding: D14F41BB4334
Thus, the binary key data to be encrypted consists of the following 24 bytes, shown in hex:
0080 A970BC5E82B91F9528D15FC2E14B5FD1 D14F41BB4334.

2.3.2.3 MAC

The below table shows the format of MAC:

Byte #	Field Name	Description	Encoding
0-15	MAC	MAC.	16H

MAC is calculated across the KBH and binary key data. This part is using CMAC as a pseudorandom function to produce 8 bytes MAC. The resultant of CMAC is required to be converted to Hex-ASCII only constructing the complete key block. The derivation method of the MAC key and calculation method are described in X9.143

2.3.3 Castles Proprietary Key Binding Method

This section introduces the key injection method: Castles proprietary key binding method. This method includes the cryptographic binding of the key-usage information to the key value, and ensures the key in the bundle is secret and has not been altered in an unauthorized manner.

The bundle is consist of several blocks: header, key data block, and hash block.

Header (plain-text)	Key Data Block (cipher-text)	Hash Block (cipher-text)

Except for header, the key data block and hash block are cipher-text, so no plain-text data can be retrieved without unbinding.

2.3.3.1 Header

The header is a plain-text data block which contains the information to describe the capability of the key. As mentioned before, the data in the header uses uppercase ASCII printable characters. The header is consist of below fields.

Castles Technology

Byte #	Len	Field Name	Description
0	1	Key Block Version	This field identifies the version of the key block, and shall be: ◆ **'0' (0x30) – key block is protected by the Castles proprietary key block protection key.**
1-2	2	Key Usage	This field provides the key usage of the key, and the available options are: ◆ **'B1' (0x42, 0x31) – DUKPT Initial Key** ◆ **'D0' (0x44, 0x30) – Data Encryption Key** ◆ **'K1' (0x4B, 0x31) – X9.143 Key Block Protection Key** ◆ **'KA' (0x4B, 0x41) – Castles Proprietary Key Block Protection Key** ◆ **'KB' (0x4B, 0x42) – Master Key** ◆ **'M0' (0x4D, 0x30) – MAC Key** ◆ **'P0' (0x50, 0x30) – PIN Encryption Key**
3	1	Algorithm	This field provides the algorithm of the key, and the available options are: ◆ **'T' (0x54) – Triple DES** ◆ **'A' (0x41) – AES**
4	1	Mode of Use	This field provides the mode of use of the key, and available options are depend on the key usage and listed as below: For **DUKPT Initial Key,** the available options as below, ◆ **'X' (0x58) – Key used to derive other keys** For **Data Encryption Key**, the available options as below, ◆ **'B' (0x42) – Encrypt and Decrypt** ◆ **'D' (0x44) – Decrypt only** ◆ **'E' (0x45) – Encrypt only** For **Castles Proprietary Key Block Protection Key,** the available options as below, ◆ **'D' (0x44) – Decrypt only** For **X9.143 Key Block Protection Key,** the available options as below, ◆ **'D' (0x44) – Decrypt only** For **Master Key,** the available options as below, ◆ **'D' (0x44) – Decrypt only** ◆ **'E' (0x45) – Encrypt only** For **MAC Key,** the available options as below, ◆ **'G' (0x47) – Generate only** For **PIN Encryption Key,** the available options as below, ◆ **'E' (0x45) – Encrypt only**
5-6	2	Key Version Number	Two-digit HEX ASCII ('0' - '9', 'A' - 'F') character version number, used to prevent re-injection of the old keys. E.g., version 160 (0xA0) would contain 'A' in byte #5, and '0' in byte #6
7	1	Exportability	This field should be filled with below value, ◆ **'N' (0x4E) – Non-exportable**
8-15	8	RFU	RFU, fixed to '0' (0x30)

If the key usage is specified as **'B1' (0x42, 0x31) – DUKPT Initial Key**, an extend header block shall be connected to the end:

Castles Technology

Byte #	Len	Field Name	Description
0-1	2	Tag	The tag shall be fixed to 'KS' (0x4B53)
2-3	2	Length	It should be '18' (0x3138) or '1C' (0x3143), it means the overall length of the field "Tag", "Length", and "Value" is 24 or 26 bytes data.
4-Var	20 / 24	Value	KSN for IPEK in HEX ASCII format.

2.3.3.2 Key Data Block

The key data block is a cipher-text data block which contains the protected key length and key data.

The plain-text of this block is consisted of below fields:

Byte #	Len	Field Name	Description
0-1	2	Key Length	This field uses to indicate how many bytes are availale for next field. i.e. 0x00 0x10 indicates 16 bytes are available in next field.
2-Var	Var	Key Data	The plaintext key data. It shall be 16 or 24 bytes data for TDES key; and shall be 16, 24, or 32 bytes data for AES key.
Var	Var	Padding	Padding random data until this block is multiple of 16 bytes.

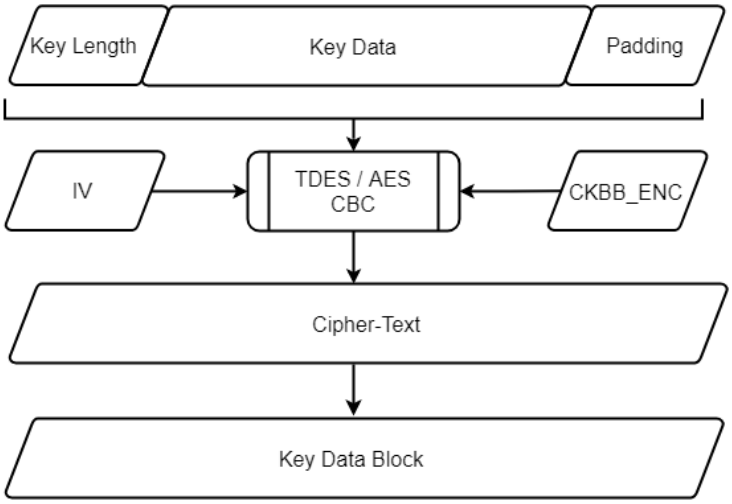
It is mandatory that the Castles Proprietary Key Block Protection Key shall be equal or greater strength than the key that to be injected by this method. For purposes of this requirement, the following algorithms and keys sizes by row are considered equivalent.

Algorithm	DES	RSA	AES
Minimum key size in number of bits:	112	-	-
	168	2048	-
	-	2048	128
	-	7680	192
	-	15360	256

For example,
(1) DES key 112 bits can only be used to protect DES key 112 bits.
(2) DES key 168 bits can be used to protect DES key 112, 168 bits, and RSA 2048 bits.
(3) AES key 128 bits can be used to protect DES key 112, 168 bits, RSA 2048 bits, and AES key-128 bits.

Castles Technology

The plain-text data of key data block is protected by CKBB_ENC key. The IV (initialization vector) is the first eight/sixteen (TDES/AES) Cipher-Hash. The Cipher-Hash is defined in the Hash Block.The used algorithm is TDES/AES in CBC mode that is depend on the algorithm of selected Castles Proprietary Key Block Protection Key. The cipher-text that encrypted by CKBB_ENC key.



The derivation method for CKBB_ENC key is described in following section.

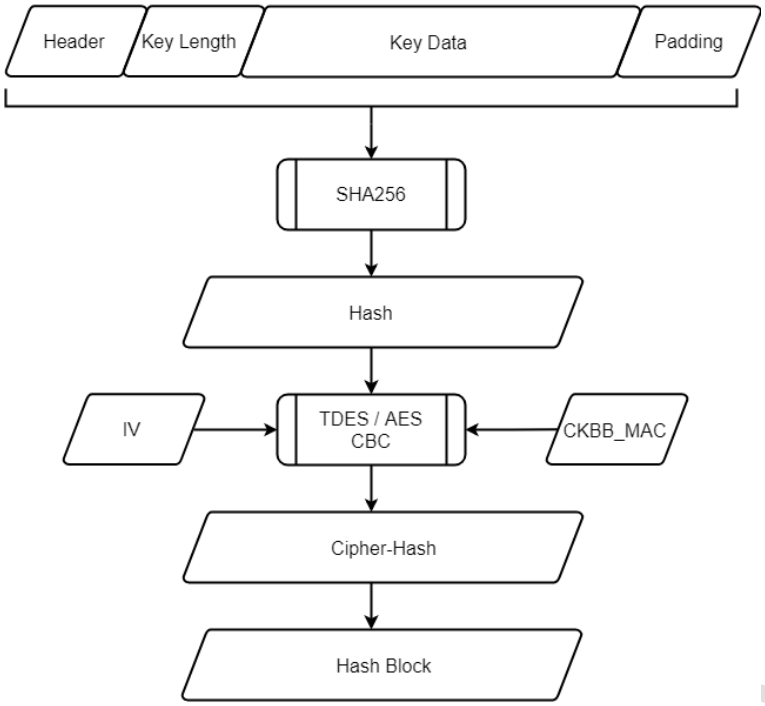
2.3.3.3 Hash Block

The hash block contains an encrypted hash which is protected by CKBB_MAC key.

Byte #	Len	Field Name	Description
0	32	Encrypted Hash	Above data will be calculated to 32 Bytes hash with SHA256 for checking key integrity.

The SHA256 is used to generate hash which covers header, key length, key data, and padding. The IV of the MAC calculation is fixed to zero(0x00). The algorithm is TDES/AES in CBC mode that is depend on the algorithm of selected Castles Proprietary Key Block Protection Key.

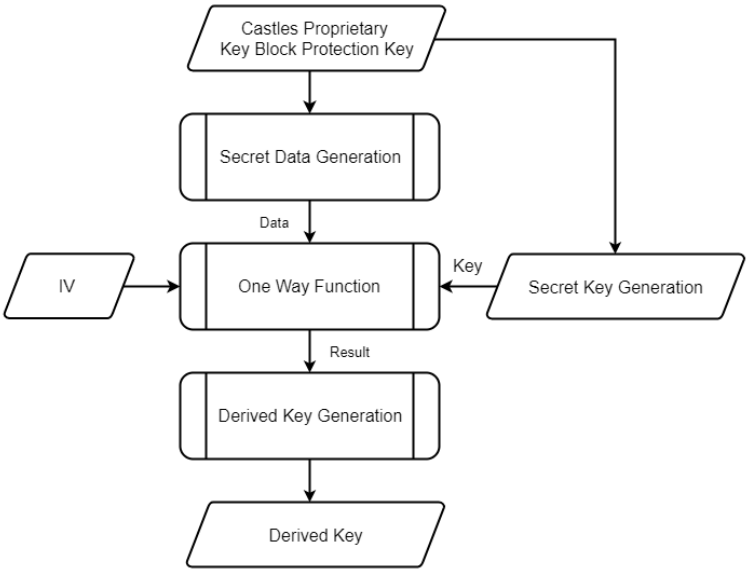
Castles Technology



The derivation method for CKBB_MAC key is described in following section.

2.3.3.4 CKBB_ENC and CKBB_MAC Key Derivation Method

This section introduces how CKBB_ENC and CKBB_MAC are derived from Castles Proprietary Key Block Protection Key. The derivation method is shown as below:



Secret Key Generation, Secret Data Generation, One Way Function and Derived Key Generation are described in following sections.

Technology

2.3.3.4.1 Constant for Key Derivation

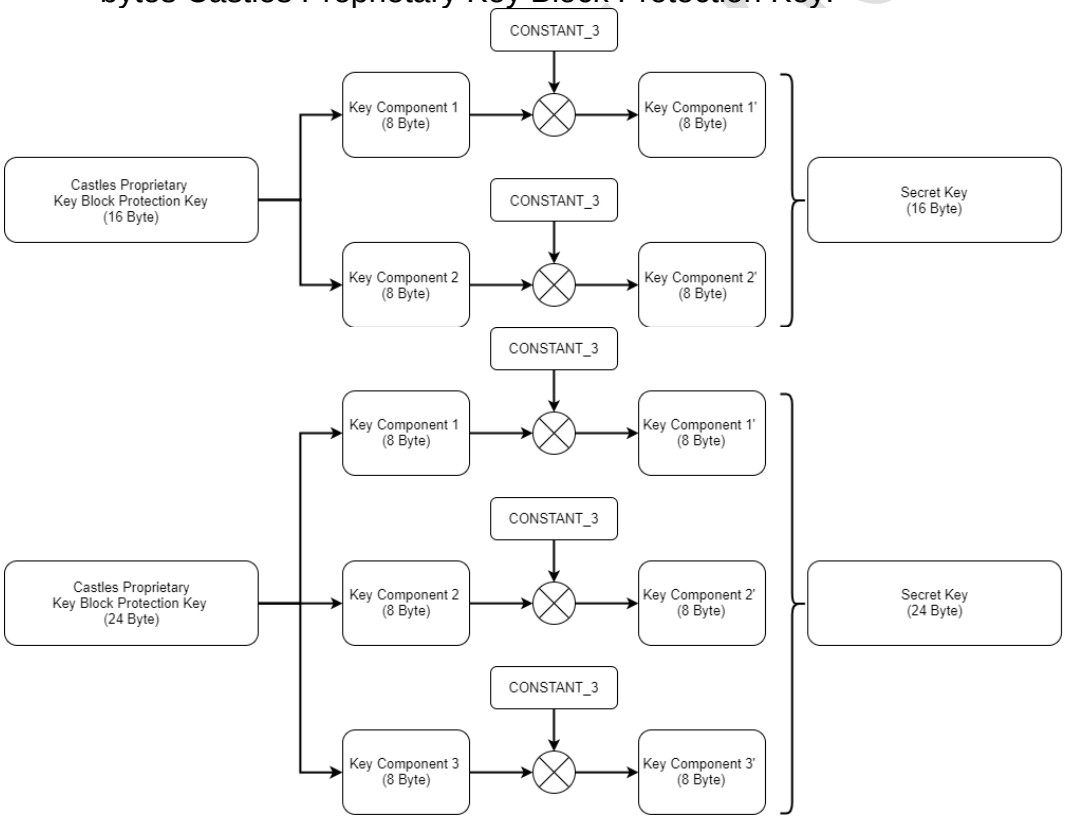
This section describes the constant value (CONSTANT_1, CONSTANT_2, and CONSTANT_3) used in the Secret Key Generation, Secret Data Generation, and Derived Key Generation.

	CONSTANT_1	CONSTANT_2	CONSTANT_3
CKBB_ENC	00 00 00 00 00 00 00 FF	00 00 00 00 00 00 FF 00	00 00 00 00 00 FF 00 00
CKBB_MAC	00 00 00 FF 00 00 00 00	00 00 FF 00 00 00 00 00	00 FF 00 00 00 00 00 00

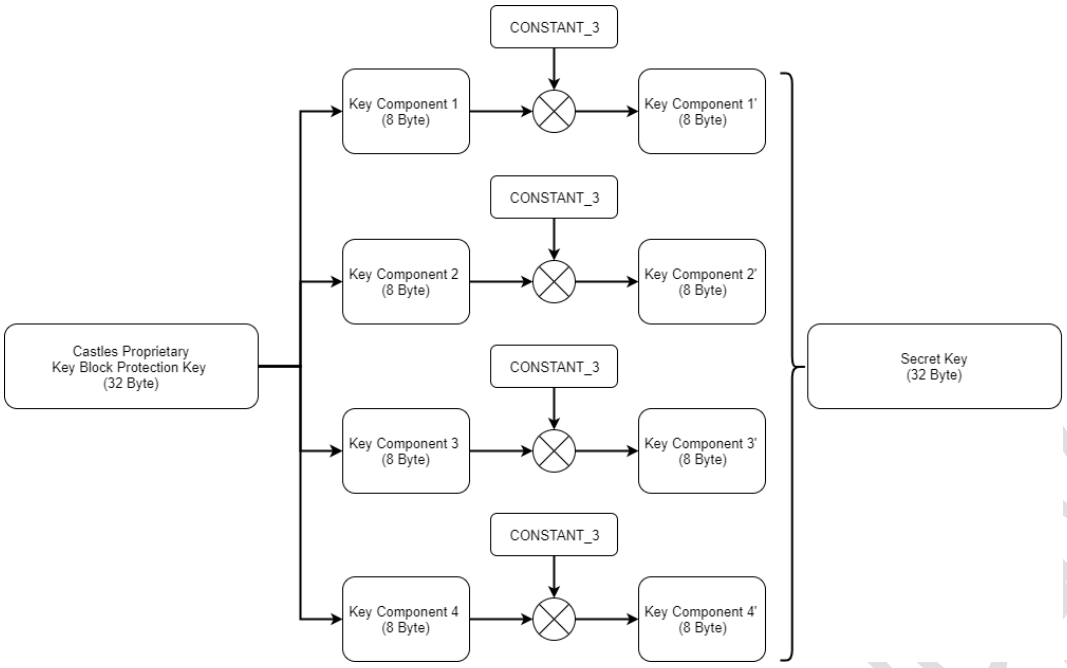
2.3.3.4.2 Secret Key Generation

To generate the secret key for OWF (one way function), Castles Proprietary Key Block Protection Key must be divided into several 8 bytes length key blocks. Depending on the length of output of Castles Proprietary Key Block Protection Key, it might have 2, 3, or 4 key blocks. Each key block needs to be put into XOR (Exclusive-OR) operation with CONSTANT_3 independently. The secret key is made up of the output of XOR operations one by one.

The following pictures show how to generate a secret key from 16, 24, and 32 bytes Castles Proprietary Key Block Protection Key.



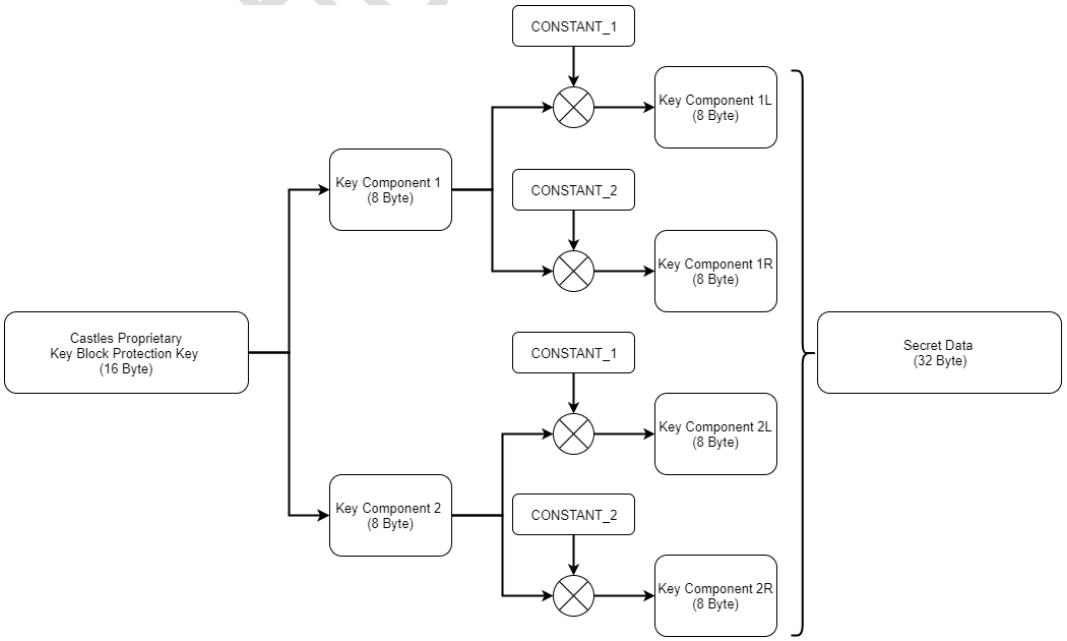
Confidential

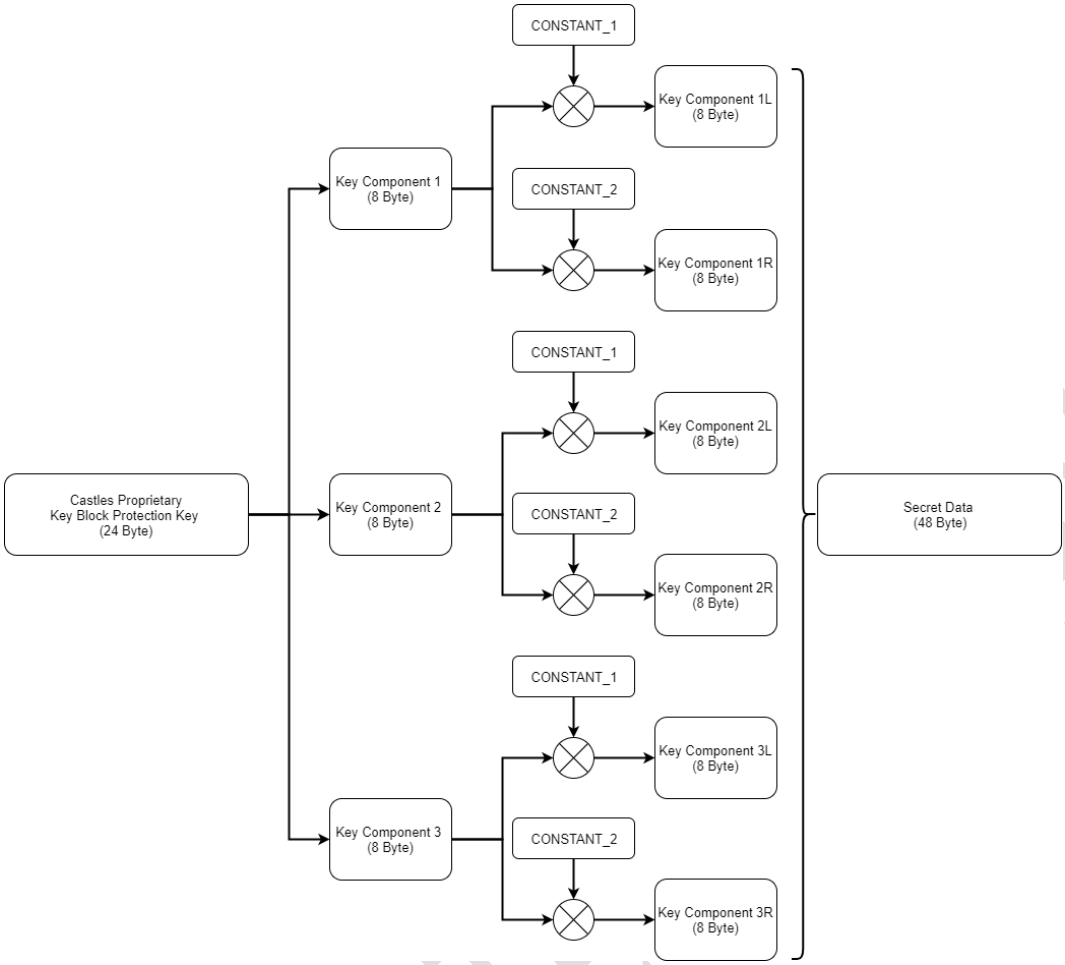


2.3.3.4.3 Secret Data Generation

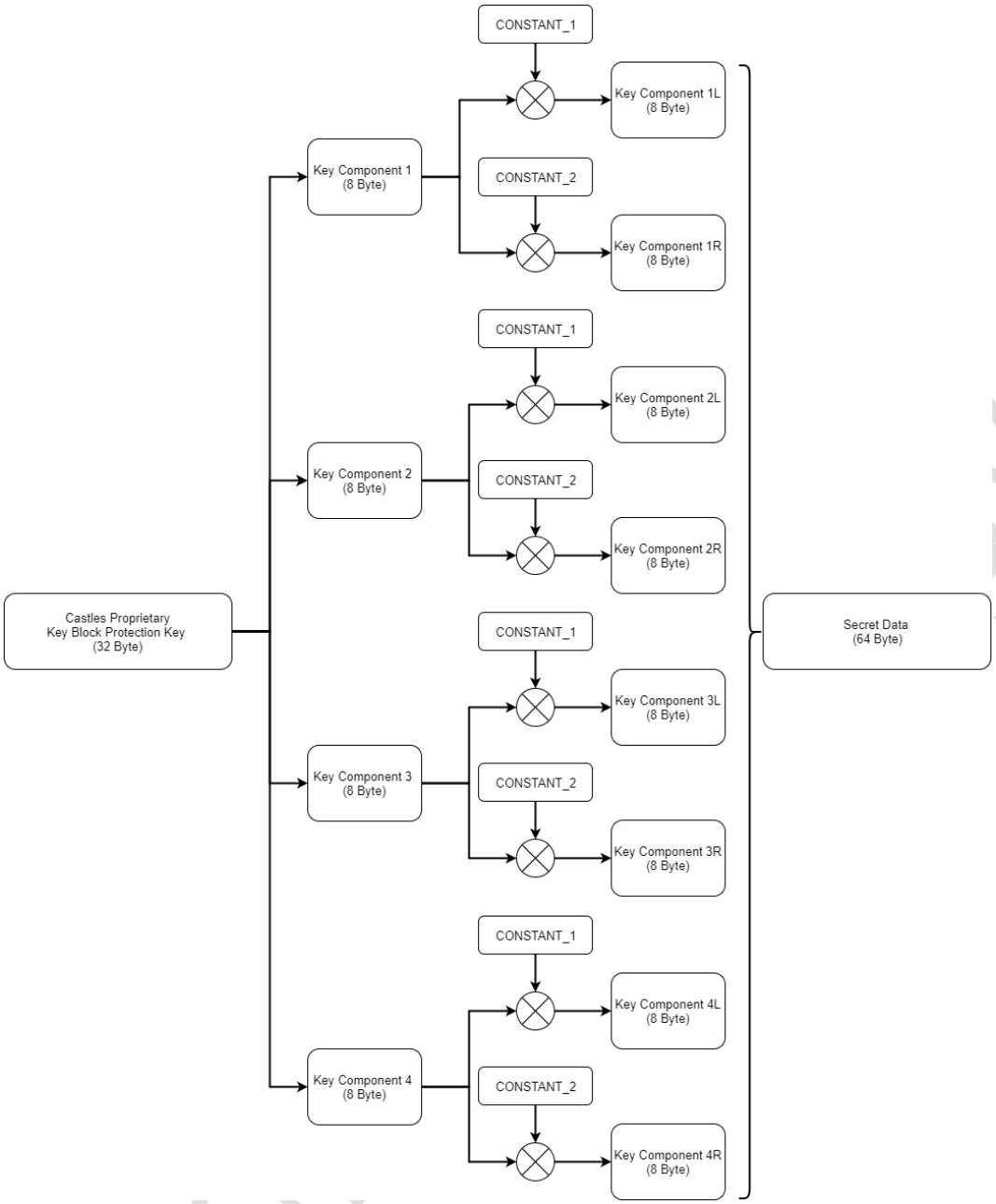
To generate the secret key for OWF (one way function), Castles Proprietary Key Block Protection Key must be divided into several 8 bytes length data blocks. Depending on the length of output of Castles Proprietary Key Block Protection Key, it might have 2, 3, or 4 data blocks. Each data block needs to be put into XOR (Exclusive-OR) operation twice to make a double length data block, one with CONSTANT_1, and one with CONSTANT_2. The secret data is made up of the output of XOR operations one by one.

The following pictures show how to generate a secret data from 16, 24, and 32 bytes Castles Proprietary Key Block Protection Key.





Castles Technology



2.3.3.4.4 One Way Function

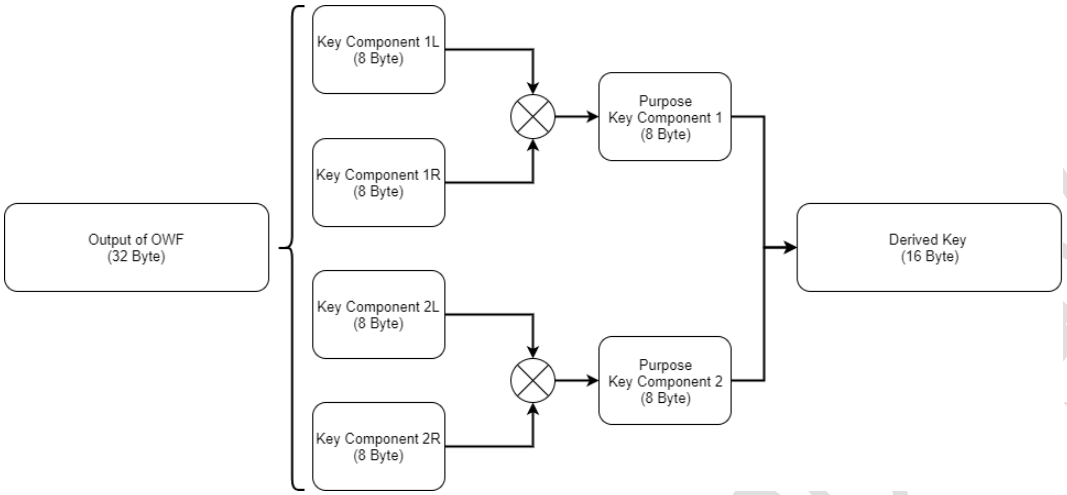
OWF is chosen either TDES in CBC mode or AES in CBC mode. It is depended on the selected Castles Proprietary Key Block Protection Key. If the algorithm of selected Castles Proprietary Key Block Protection Key is TDES, then OWF will be TDES in CBC mode; if the algorithm of selected Castles Proprietary Key Block Protection Key is AES, then OWF will be AES in CBC mode. The IV of the one way function is fixed to zero.

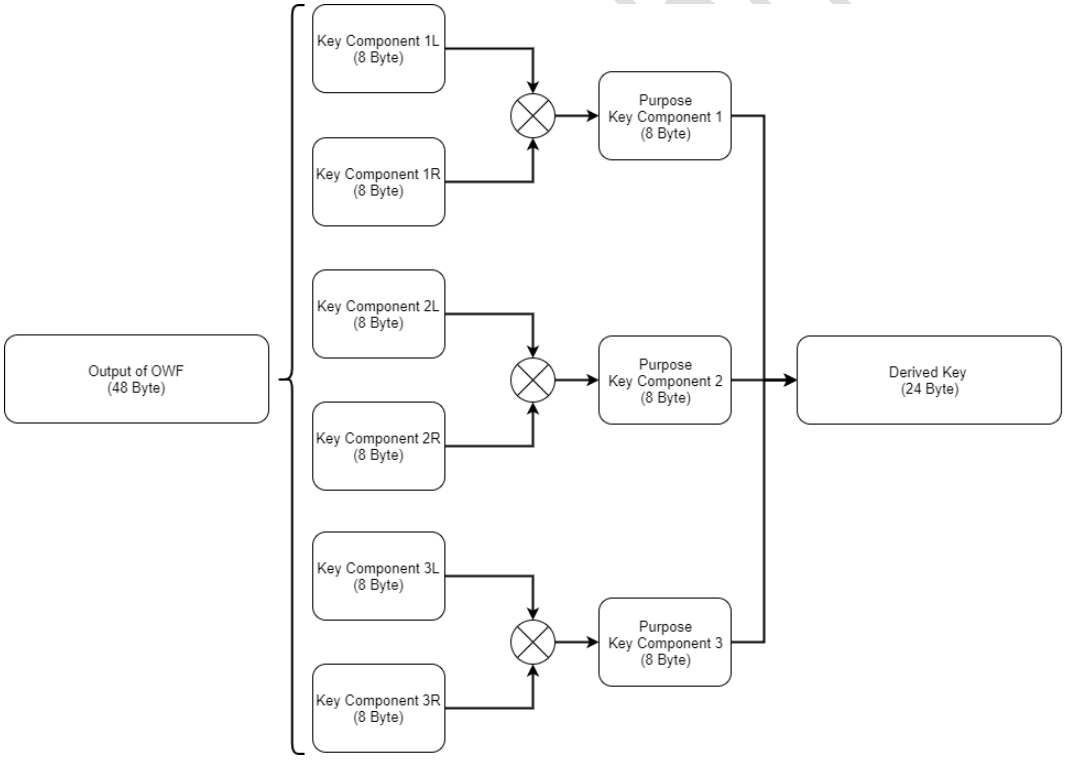
2.3.3.4.5 Derived Key Generation

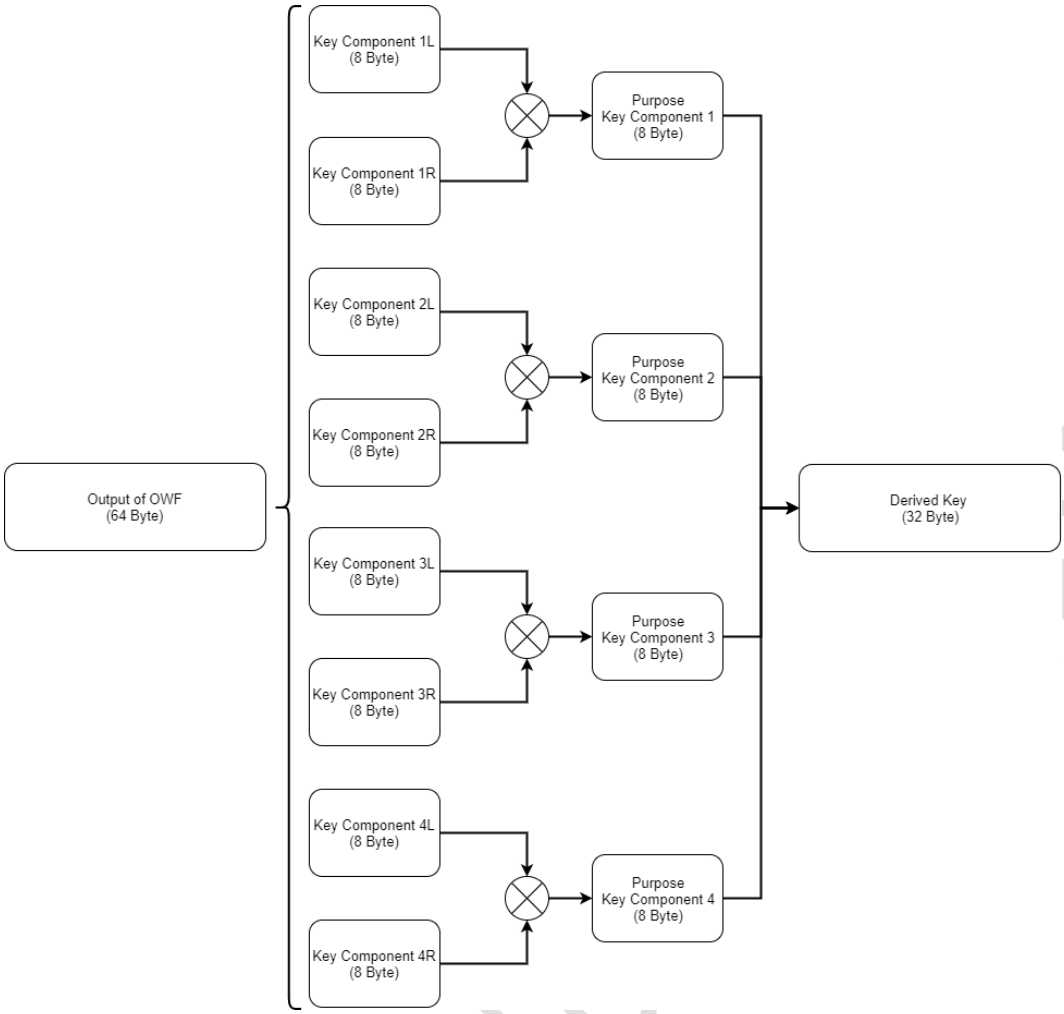
The output of OWF is the input of Derived Key Generation. It needs to be divided into several 16 bytes length data blocks. Depending on the length of output of OWF, it might have 2, 3, or 4 data block. Each 16 bytes data needs to be divided into two parts: first half for left part, and second half for right part.

Then, put the left part and right part into XOR (Exclusive-OR) operation. The last step to generate the derived key (CKBB_ENC or CKBB_MAC) is made up of the output of XOR operations one by one.

The following pictures show how to get the derived key from 16, 24, and 32 bytes output of OWF.







Castles Technology

2.4 SBKMS Application Programming Interfaces

Package CTOS
Contains all of the classes for Castles user API.

Public Class Summary	
CtKMS2System	This class provides the functionality of management of SBKMS.
CtKMS2UserData	This class provides the functionality of additional storage in SBKMS.
CtKMS2Key	This class provides the functionality of obtaining key information.
CtKMS2TR31	This class provides the functionality of public regulations key injection, X9.143
CtKMS2CKBB	This class provides the functionality of one kind of key injection supported by Castles.
CtKMS2FixedKey	This class provides the functionality of fixed key cryptography.
CtKMS2MKSK	This class provides the functionality of master key/session key cryptography.
CtKMS2Dukpt	This class provides the functionality of TDES-DUKPT cryptography.
CtKMS2Callback	This class provides the functionality of setting callback function.

Exception Summary	
CtKMS2Exception	SBKMS error collector.

Castles Technology

2.4.1 Class: CtKMS2System

public class **CtKMS2System**
The **CtKMS2System** class is used to provide the management functionality of SBKMS.

Constructor Summary	
public	CtKMS2System() Constructs a binder connected with service.

Method Summary	
String	getKMS2LibVersion() Get version number of library SBKMS2.
void	init() Initialize the KMS-II.
void	checkAllKey() Check all key belong to caller.
byte[]	getAllKeyResult() Get result of caller's all key.
void	deleteAllKey() Delete all key belong to caller.
void	checkKey(int keyset, int keyindex) Check if the specified key exists or not.
void	deleteKey(int keyset, int keyindex) Delete the specified key.
void	swapKey(int src1KeySet, int src1KeyIndex, int src2KeySet, int src2KeyIndex) Swap two key.
void	setPINSound(boolean bool) Set button sound during get PIN function.
String	getPINSound() Get value of parameter PINSound which is set by setPINSound(boolean bool)
void	setPINPADMoving(boolean bool) Set randomly moving PINPAD during get PIN function.
String	getPINPADMoving() Get value of parameter PINPADMoving which is set by setPINPADMoving(boolean bool)
void	setPINScrambling(boolean bool) Set scramble digital button during get PIN function.
String	getPINScrambling() Get value of parameter PINScrambling which is set by setPINScrambling(boolean bool)

Constructor Detail
public CtKMS2System() Create a CtKMS2System object and connect with binder to invoke service.

Method Detail
public final String getKMS2LibVersion() This method is used to get the version number of library SBKMS2. **Returns:** The version number of library SBKMS2.
public final void init() throws CtKMS2Exception This method initialize the Key Management System II and it ***shall*** be called at first before any other KMS-II method. **Throws:** CtKMS2Exception- if the object obtains the Castles Key Management System II error code. **Note:** This function does not erase the keys in key storage.
public final void checkAllKey() throws CtKMS2Exception This method check integrity and authenticity of all the key which are belonged to the (caller) application. Throws: CtKMS2Exception- if the object obtains the Castles Key Management System II error code.
public final byte[] getAllKeyResult() throws CtKMS2Exception This method has to call function checkAllKey() first. And it will return result of all key which belongs caller. The result includes 2 bytes keyset, 2 bytes keyindex and 2bytes valid value in sequence. There are two valid value which are 0x0000 and 0x0001. Value 0x0000 and 0x0001 separately means key is valid and key integrity error.

Castles Technology

Throws:
CtKMS2Exception- if the object obtains the Castles Key Management System II error code.

public final void deleteAllKey()
throws CtKMS2Exception
This method delete all key belong to the (caller) application.
Throws:
CtKMS2Exception- if the object obtains the Castles Key Management System II error code.

public final void checkKey(int keyset, int keyindex)
throws CtKMS2Exception
This method check integrity and authenticity of the specified key exists or not.
Parameters:
keyset – indicate which key set application belong to.
keyindex - specify its index in the key set.

Throws:
CtKMS2Exception- if the object obtains the Castles Key Management System II error code.

public final void deleteKey(int keyset, int keyindex)
throws CtKMS2Exception
This method is used to delete the specific key.
Parameters:
keyset – indicate which key set application belong to.
keyindex - specify its index in the key set.

Throws:
CtKMS2Exception- if the object obtains the Castles Key Management System II error code.

public final void swapKey(int src1KeySet, int src1KeyIndex, int src2KeySet, int src2KeyIndex)
throws CtKMS2Exception
This method is used to specify two keys which are belonged to the same owner (application), and exchange the key slot directly.
Parameters:
src1KeySet– specify the 1st key set.
src1KeyIndex- specify the 1st key Index.
src2KeySet– specify the 2nd key set.
src2KeyIndex- specify the 2nd key Index.

Throws:
CtKMS2Exception- if the object obtains the Castles Key Management System II error code.

Note:
The S1F2 does not support for swapping the keys in different KeyOwnerType. For example, it cannot swap the common key (0xC000, 0x0001) and normal key (0xAA00, 0x0001).

public final void setPINSound(boolean bool)
throws CtKMS2Exception
This method is used to turn the PINSound on or off.
Parameters:
bool – indicate the PINSound status in the application.

Throws:
CtKMS2Exception- if the object obtains the JSONException error code.

public static final String getPINSound()

This method is used to get the PINSound status in the application.

Returns:
The PINSound status in the application.

Note:
If PINSound does not be set before using this function, it will return the default value, **TRUE**.

public final void setPINPADMoving(boolean bool)
throws CtKMS2Exception
This method is used to turn the PINPADMoving on or off.
Parameters:
bool – indicate the PINPADMoving status in this application

Throws:
CtKMS2Exception- if the object obtains the JSONException error code.

public static final String getPINPADMoving()

This method is used to get the PINPADMoving status in the application.

Returns:
The PINPADMoving status in the application.

Castles Technology

Note:
If PINPADMoving does not be set before using this function, it will return the default value, **FALSE**.

public final void setPINScrambling(boolean bool)
throws CtKMS2Exception

This method is used to turn the PINScrambling on or off.

Parameters:
bool – indicate the PINScrambling status in this application

Throws:
CtKMS2Exception- if the object obtains the JSONException error code.

public static final String getPINScrambling()

This method is used to get the PINScrambling status in the application.

Returns:
The PINScrambling status in the application.

Note:
If PINScrambling does not be set before using this function, it will return the default value, **TRUE**.

Example

```
try {
	CtKMS2System system = new CtKMS2System();
	system.init();
	String version = system.getKMS2LibVersion();
	system.checkAllKey();
	byte[] keyResult = system.getAllKeyResult();

	int KeySet = 0xC602;
	int KeyIndex = 0x0001;
	system.checkKey(KeySet, KeyIndex);
	system.deleteKey(KeySet, KeyIndex);

	int KeySetA = 0xC001;
	int KeyIndexA = 0x0001;
	int KeySetB = 0xC002;
	int KeyIndexB = 0x0001;
	system.swapKey(KeySetA, KeyIndexA, KeySetB, KeyIndexB);

	system.deleteAllKey();

	boolean PINSound = true;
	system.setPINSound(PINSound);

	boolean PINPADMoving = true;
	system.setPINPADMoving(PINPADMoving);

	boolean PINScrambling = true;
	system.setPINScrambling(PINScrambling);
	system.getPINScrambling();
} catch(CtKMS2Exception e){
		//exception handle
}
```

Castles Technology

2.4.2 Class: CtKMS2UserData

public class **CtKMS2UserData**
The **CtKMS2UserData** class provides the additional storage for user.

Constructor Summary	
public	CtKMS2UserData() Constructs a binder connected with service.

Method Summary	
void	write(boolean isCommon, int offset, byte [] inBuf) Write the additional data into KMS-II data storage.
byte []	read(boolean isCommon, int offset, int length) Read the additional data from KMS-II data storage.

Constructor Detail

public CtKMS2UserData()
Create a CtKMS2UserData object and connect with binder to invoke service.

Method Detail

public final void write(boolean isCommon, int offset, byte [] inBuf)
throws CtKMS2Exception

Write the additional data into KMS-II data storage; A secure free-usage memory space provided for user applications to store their sensitive data by themselves.

Parameters:

isCommon – indicate to access common additional storage or private user data area.
offset – specify the offset of the additional storage to write the data, it must >= 0. If 0, it means the start of the storage.
inBuf – the data buffer that be writhed into additional storage, and its size must <= 2048, please make sure the size of byte array is the expected input data length, otherwise, it will have some unpredictable result. If the inBuf is **null** or length = 0, it will throws CtKMS2Exception.

Throws:

CtKMS2Exception - if the object obtains the Castles Key Management System II error code.

Note:

The space of common user data is 64K, while the space of each application's user data(private) is 16K. Because of the memory limit, the sum of the size cannot exceed the 72K in S1F2.

public final byte [] read(boolean isCommon, int offset, int length)
throws CtKMS2Exception

Read the additional data from secure free-usage memory KMS-II data storage.

Parameters:

isCommon – indicate to access common additional storage or private user data area.
offset – specify the offset of the additional storage to read, it must >= 0. If 0, it means the start of the storage.
length – the length of read data, must <= 2048 and > 0.

Returns:

The data that store in KMS-II.

Throws:

CtKMS2Exception - if the object obtains the Castles Key Management System II error code.

Example

```
try {
    CtKMS2UserData userData = new CtKMS2UserData();
    boolean isCommon = false;
    int Offset = 0;
    byte[] InputData = {(byte) 0x11, (byte) 0x12, (byte) 0x13, (byte) 0x14, (byte) 0x15, (byte) 0x16, (byte) 0x17, (byte) 0x18};

    userData.write(isCommon, Offset, InputData);

    int Offset = 0;
    int Length = 16;

    byte[] data = userData.read(isCommon, Offset, Length);
    } catch(CtKMS2Exception e){
        //exception handle
}
```

Castles Technology

2.4.3 Class: CtKMS2Key

public class **CtKMS2Key**

The **CtKMS2Key** class is used to obtain the key related information; there are a great variety of classes extending it.

Constructor Summary	
public	CtKMS2Key() Constructs a binder connected with service.
public	CtKMS2Key(int keyset, int keyindex) Constructs a binder connected with service and select the key with the argument.

Method Summary	
void	selectKey(int keyset, int keyindex) Select one key in KMS-II and obtain the specified key information.
void	setInputData(byte[] InputData, int from, int length) Set input data
void	setInputData(byte[] InputData) Set input data
byte []	getOutputData() Get out put encrypted data
void	getKeyInfo() Get key information.
int	getKeySet() Get the set of selected key.
int	getKeyIndex() Get the index of selected key.
byte	getKeyVersion() Get the version of selected key.
byte	getKeyType() Get the type of selected key.
int	getKeyAttribute() Get the attribute of selected key.
int	getKeyLength() Get the length of selected key.
int	getKeyExponentLength() Get the length of key exponent.
byte []	getCV(int length) Get the CV byte array of selected key.

Constructor Detail
public CtKMS2Key() Create a CtKMS2Key object and connect with binder to invoke service.
public CtKMS2Key(int ketset, int keyindex) throws CtKMS2Exception Create a CtKMS2Key object, connect with binder to invoke service, select the key and then get the key information.

Method Detail
public void selectKey(int keyset, int keyindex) throws CtKMS2Exception This method let object select key that store in KMS-II storage and get related information. **Parameters:** keyset – the set of key. keyindex - the index of key. **Throws:** CtKMS2Exception- if the object obtains the Castles Key Management System II error code. **Note:** The range of keyset is 0x0001~0xBFFF, and the range of keyindex is 0x0000~0xFFFF.
public final void setInputData(byte[] InputData, int from, int length) This method can set input data. **Parameters:** InputData – the byte array of input data. from – the start address of byte array of input data. length – the length of input data. **Note:** The input data size must <= 2048
public final void setInputData(byte[] InputData)

Castles Technology

This method can set input data. **Parameters:** InputData – the byte array of input data.
public final byte[] getOutputData() This method can get output encrypted data after cryptographic calculations. **Returns:** The encrypted data which is the result of doing cryptography service.
public final void getKeyInfo() throws CtKMS2Exception After select key, this method can get key type, version, attribute, length and cv.
public int getKeySet() After select key, this method can get key set. **Returns:** The set of key.
public int getKeyIndex() After select key, this method can get key index. **Returns:** The index of key.
public byte getKeyVersion() After select key, this method can get key version. **Returns:** The version of key.
public byte getKeyType() After select key, this method can get key type. **Returns:** The type of key.
public int getKeyAttribute() After select key, this method can get attribute. **Returns:** The attribute of key.
public int getKeyLength() After select key, this method can get key length. **Returns:** The length of key.
public int getKeyExponentLength() throws CtKMS2Exception After select key, this method can get key expnent length. **Returns:** The length of key exponent.
public byte [] getCV(int length) After select key, this method can get CV. **Parameters:** length – the length of return CV size, len must <= 3 and > 0, no matter what length is, CAKMS return at most 3 bytes of check value(CV). **Returns:** The byte array of CV.

Example

```
try {
        //select key before get key information, make sure the key is existence
        int KeySet = 0xC602;
        int KeyIndex = 0x0001;
        CtKMS2Key key = new CtKMS2Key();
        key.selectKey(KeySet, KeyIndex);
        key.getKeyInfo();
        byte keyVersion = key.getKeyVersion();
        int keySet = key.getKeySet();
        int keyIndex = key.getKeyIndex();
        byte keyType = key.getKeyType();
        int keyAttribute = key.getKeyAttribute();
        int keyLength = key.getKeyLength();
        int KeyExponentLength = key.getKeyExponentLength();
        byte[] cv = key.getCV(3);
} catch(CtKMS2Exception e){
        //exception handle
}
```

Castles Technology

2.4.4 Class: CtKMS2InjectionKey

abstract class CtKMS2InjectionKey extends CtKMS2Key
The CtKMS2InjectionKey is an abstract class extends CtKMS2Key

2.4.4.1 Inherited Methods

From class CtKMS2Key

2.4.4.2 Public Methods

Method Detail
public final void setKeyLocation(int keyset, int keyindex) This method can set the new/update key location. **Parameters**: ➢ keyset – specify the key set the new/update key belong to. ➢ keyindex – specify the key index the new/update key belong to.
public final void setKeyBlock(byte[] src) This method can set Key Block. **Parameters**: ➢ src – the key block source.
public final void writeKey() **throws** CtKMS2Exception After set related parameters, this method will write key that protected by CKBB key. **Throws**: CtKMS2Exception– if the object obtains the Castles Key Management System II error code.

2.4.5 Class: CtKMS2TR31

public class **CtKMS2TR31**
The **CtKMS2TR31** class is used to write or update a key into CAKMS by X9.143 All the keys written by this class will take the caller application as their key owner.

Constructor Summary	
public	CtKMS2TR31() Constructs a binder connected with service.
public	CtKMS2TR31(int keyset, int keyindex) Constructs a binder connected with service and select the key.

Methods inherited from class CTOS.CtKMS2Key
selectKey, setInputData, getOutputData, getKeyInfo, getKeySet, getKeyIndex, getKeyVersion, getKeyType, getKeyAttribute, getKeyLength, getKeyExponentLength, getCV

Constructor Detail
public CtKMS2TR31() Create a CtKMS2TR31 object and connect with binder to invoke service.
public CtKMS2TR31(int keyset, int keyindex) throws CtKMS2Exception Create a CtKMS2TR31 object and connect with binder to invoke service and select the key.

Example

try {
int KeySet = 0xC101;
int KeyIndex = 0x0008;
int KeySetLocation = 0x0001;
int KeyIndexLocation = 0x0013;

Castles Technology

```
        byte[] baTR31KeyBlock = {0x42, 0x30, 0x30, 0x38, 0x30, 0x50, 0x30, 0x54,
                0x45, 0x30, 0x31, 0x4E, 0x30, 0x30, 0x30, 0x30,
                0x41, 0x36, 0x36, 0x42, 0x37, 0x35, 0x39, 0x34,
                0x43, 0x35, 0x43, 0x30, 0x42, 0x32, 0x33, 0x43,
                0x46, 0x37, 0x36, 0x46, 0x31, 0x41, 0x30, 0x41,
                0x43, 0x38, 0x41, 0x41, 0x32, 0x44, 0x38, 0x32,
                0x38, 0x32, 0x39, 0x37, 0x36, 0x39, 0x45, 0x33,
                0x33, 0x45, 0x32, 0x35, 0x42, 0x33, 0x34, 0x30,
                0x37, 0x37, 0x35, 0x46, 0x36, 0x30, 0x44, 0x30,
                0x37, 0x41, 0x31, 0x31, 0x31, 0x42, 0x34, 0x43};

                CtKMS2TR31 tr31 = new CtKMS2TR31();
                tr31.selectKey(KeySet, KeyIndex);
                tr31.setKeyLocation(KeySetLocation, KeyIndexLocation);
                tr31.setKeyBlock(baTR31KeyBlock);
                tr31.writeKey();
} catch(CtKMS2Exception e){
                 //exception handle
}
```

Castles Technology

2.4.6 Class: CtKMS2CKBB

public class **CtKMS2CKBB**
The **CtKMS2CKBB** class is used to write or update a key into CAKMS by CKBB(Castles Key Block Binding). CKBB is a format of Castles. All the keys written by this class will take the caller application as their key owner.

Constructor Summary	
public	CtKMS2CKBB() Constructs a binder connected with service.
public	CtKMS2CKBB(int keyset, int keyindex) Constructs a binder connected with service and select the key.

Methods inherited from class CTOS.CtKMS2Key
selectKey, setInputData, getOutputData, getKeyInfo, getKeySet, getKeyIndex, getKeyVersion, getKeyType, getKeyAttribute, getKeyLength, getKeyExponentLength, getCV

Constructor Detail
public CtKMS2CKBB() Create a CtKMS2CKBB object and connect with binder to invoke service.
public CtKMS2CKBB(int keyset, int keyindex) throws CtKMS2Exception Create a CtKMS2CKBB object and connect with binder to invoke service and select the key.

Example

```
try {
      int KeySet = 0x0101;
      int KeyIndex = 0x0009;
      int KeySetLocation = 0x2101;
      int KeyIndexLocation = 0x0001;
      byte[] baCKBBKeyBlock = {0x30, 0x50, 0x30, 0x54, 0x45, 0x30, 0x31, 0x4E,
            0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30,
            0x44, 0x32, 0x41, 0x45, 0x46, 0x43, 0x44, 0x37,
            0x46, 0x46, 0x39, 0x33, 0x41, 0x41, 0x41, 0x36,
            0x41, 0x35, 0x43, 0x33, 0x38, 0x41, 0x37, 0x43,
            0x46, 0x44, 0x32, 0x38, 0x36, 0x42, 0x35, 0x30,
            0x37, 0x42, 0x46, 0x31, 0x42, 0x46, 0x43, 0x42,
            0x39, 0x37, 0x37, 0x38, 0x37, 0x37, 0x46, 0x38,
            0x33, 0x38, 0x45, 0x44, 0x31, 0x33, 0x46, 0x39,
            0x31, 0x46, 0x43, 0x45, 0x37, 0x37, 0x37, 0x31,
            0x46, 0x37, 0x37, 0x36, 0x37, 0x45, 0x30, 0x38,
            0x44, 0x46, 0x37, 0x41, 0x41, 0x39, 0x46, 0x46,
            0x45, 0x36, 0x37, 0x36, 0x41, 0x30, 0x37, 0x36,
            0x42, 0x32, 0x45, 0x31, 0x35, 0x44, 0x31, 0x33,
            0x44, 0x36, 0x31, 0x30, 0x39, 0x35, 0x36, 0x44,
            0x41, 0x39, 0x44, 0x30, 0x35, 0x31, 0x41, 0x38,
            0x32, 0x30, 0x36, 0x45, 0x38, 0x36, 0x38, 0x34,
            0x34, 0x34, 0x43, 0x34, 0x41, 0x38, 0x42, 0x34};

            CtKMS2CKBB ckbb = new CtKMS2CKBB();
            ckbb.selectKey(KeySet, KeyIndex);
            ckbb.setKeyLocation(KeySetLocation, KeyIndexLocation);
            ckbb.setKeyBlock(baCKBBKeyBlock);
            ckbb.writeKey();
} catch(CtKMS2Exception e){
              //exception handle
}
```

Castles Technology

2.4.7 Class: CtKMS2SymmetricKey

The CtKMS2SymmetricKey class provides the parameter of symmetry cryptography.

Modifier and Type	Constant Field	Description	Value
public static final byte	DATA_ENCRYPT_METHOD_ECB	ECB mode data encrypt	0x00
public static final byte	DATA_ENCRYPT_METHOD_CBC	CBC mode data encrypt	0x01
public static final byte	MAC_METHOD_CBC	Normal CBC Mac.	0x00
public static final byte	MAC_METHOD_X9_19	Retail Mac, done in one call.	0x01
public static final byte	MAC_METHOD_X9_19_START	Start for Retail Mac. Retail Mac is done in three or more calls.	0x02
public static final byte	MAC_METHOD_X9_19_UPDATE	Continue for Retail Mac. Retail Mac is done in three or more calls.	0x03
public static final byte	MAC_METHOD_X9_19_FINAL	End for Retail Mac. Retail Mac is done in three or more calls.	0x04
public static final byte	MAC_METHOD_CMAC	CMAC mode	0x05
public static final byte	PIN_BLOCKTYPE_ANSI_X9_8_ISO_0	ANSI X9.8 PIN; ISO Format 0	0x00
public static final byte	PIN_BLOCKTYPE_ANSI_X9_8_ISO_4	ANSI X9.8 PIN; ISO Format 4	0x01
public static final byte	PIN_BLOCKTYPE_ANSI_X9_8_ISO_1	ANSI X9.8 PIN; ISO Format 1	0x02
public static final byte	PIN_BLOCKTYPE_ANSI_X9_8_ISO_3	ANSI X9.8 PIN; ISO Format 3	0x04
public static final byte	PIN_CIPHER_METHOD_ECB	ECB mode PIN cipher	0x00
public static final byte	PIN_CIPHER_METHOD_CBC	CBC mode PIN cipher	0x01

2.4.8 Class: CtKMS2SymmetryKey.Inner

abstract class Inner extends CtKMS2Key
The CtKMS2SymmetricKey.Inner is an abstract class extends CtKMS2Key.

2.4.8.1 Inherited Fields

From class CtKMS2SymmetryKey

2.4.8.2 Inherited Methods

From class CtKMS2Key

2.4.8.3 Public Methods

Constructor Detail
public final void setCipherMethod(byte CipherMethod) This method can set the cipher method which according to what cryptography want to do. **For data cipher:** ➢ DATA_ENCRYPT_METHOD_ECB ➢ DATA_ENCRYPT_METHOD_CBC **For mac:** ➢ MAC_METHOD_CBC ➢ MAC_METHOD_X9_19 ➢ MAC_METHOD_X9_19_START ➢ MAC_METHOD_X9_19_UPDATE ➢ MAC_METHOD_X9_19_FINAL

Castles Technology

For PIN cipher:
- PIN_CIPHER_METHOD_ECB
- PIN_CIPHER_METHOD_CBC

Parameters:
CipherMethod – the cryptography service method.

---

public final void setICV(byte[] icv)
This method can set ICV for CBC mode.
Parameters:
- icv – the byte array of ICV, and the ICV size must = 8

---

public final void setCallback(CtKMS2Callback callback) **throws** CtKMS2Exception
This method is only used for startVirtualPin(Activity activity, int layoutID) method, user can override the callback method by implementing CtKMS2Callback class, when calling this method, it will register a callback service from callback server.
Parameters:
- callback – KMS defines some callback function. User can override it to create custom callback functions.

Throws: CtKMS2Exception– if the object obtains the Castles Key Management System II error code.

---

public final void setPAN(byte[] pan)
Parameters:
- pan – the buffer of PAN

This method can set the PAN.

---

public final void setPinInfo(byte blockType, byte pinDigitMaxLength, byte pinDigitMinLength)
This method can set the information about PIN
Parameters:
- blockType
  - PIN_BLOCKTYPE_ANSI_X9_8_ISO_0
  - PIN_BLOCKTYPE_ANSI_X9_8_ISO_4
  - PIN_BLOCKTYPE_ANSI_X9_8_ISO_1
  - PIN_BLOCKTYPE_ANSI_X9_8_ISO_3
- pinDigitMaxLength– must be 4~12 and >= Min Digit Length

pinDigitMinLength– must be 4~12 and <= Max Digit Length

---

public final void setPinControl(byte nullPIN, int timeout, int firstKeyEntryTimeout)
This method can set the control of PIN
Parameters:
- nullPIN– whether allow Null PIN or not
- timeout– timeout
- firstKeyEntryTimeout– timeout before first key entry.

---

public final void setPinControl(byte nullPIN, int timeout, int firstKeyEntryTimeout, int nonOperationTimeout)
This method can set the control of PIN
Parameters:
- nullPIN– whether allow Null PIN or not
- timeout– timeout
- firstKeyEntryTimeout– timeout before first key entry.

nonOperationTimeout – timeout between two operations

---

public final void setPinControl(byte nullPIN, byte clearPIN, int timeout, int firstKeyEntryTimeout, int nonOperationTimeout)
This method can set the control of PIN
Parameters:
- nullPIN– whether allow Null PIN or not
- clearPIN – the behavior when user click “clear” button
  - PIN_CLEAR_METHOD_ONE_PIN – **default value**
  - PIN_CLEAR_METHOD_ALL_PIN
- timeout– timeout
- firstKeyEntryTimeout– timeout before first key entry.

nonOperationTimeout – timeout between two operations

---

public final void setTapbehavior(byte method)

Castles Technology

When finger touch the screen, this setting will decide whether the location is selected. There are two selections, touch: when finger touches the screen, the location is selected; release: after finger leaves the screen, the location is selected.
Parameters:
- method – select touch or relase
  - PIN_TAP_BEHAVIOR_TOUCH – **default value**
  - PIN_TAP_BEHAVIOR_RELEASE

Returns: The encrypted result.

public byte[] dataEncrypt() **throws** CtKMS2Exception
After setting related parameters, this method can encrypt input data.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Returns: The encrypted result.

public byte[] dataDecrypt() **throws** CtKMS2Exception
After setting related parameters, this method can decrypt input data.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Returns: The decrypted result.

public byte[] generateMAC() **throws** CtKMS2Exception
After setting related parameters, this method can generate MAC.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Returns: The MAC.

public final byte[] getPinFromKbd() **throws** CtKMS2Exception
This method can get PIN from keyboard.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Returns: Encrypted PINBlock.
Notes: Only for terminals with physical keyboard.

public final byte[] startVirtualPin(CtKMS2VirtualPINPad pinPad) **throws** CtKMS2Exception
This method can start get PIN process with the selected CtKMS2VirtualPINPad.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Parameters:
- pinPad – selected CtKMS2VirtualPINPad

Returns: Encrypted PINBlock.
Notes: Only for terminals without physical keyboard.

public final byte[] startVirtualPin(Activity activity, RelativeLayout inlayout) **throws** CtKMS2Exception
This method can start get PIN process with the selected CtKMS2VirtualPINPad.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Parameters:
- activity – selected Activity
- inlayout – selected layout

Returns: Encrypted PINBlock.
Notes: Only for terminals without physical keyboard.

public final byte[] startVirtualPin(RelativeLayout inlayout) **throws** CtKMS2Exception
This method can start get PIN process with the selected CtKMS2VirtualPINPad.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Parameters:
- inlayout – selected layout

Returns: Encrypted PINBlock.
Notes: Only for terminals without physical keyboard.

public final byte[] startVirtualPin() **throws** CtKMS2Exception
This method can start get PIN process with the selected CtKMS2VirtualPINPad.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Returns: Encrypted PINBlock.
Notes: Only for terminals without physical keyboard.

Castles Technology

public final byte[] startVirtualPin(Activity activity, CtKMS2CustomPINPadbyImage customPINPadbyImage) **throws** CtKMS2Exception
This method can start get PIN process with the selected Activity and CtKMS2CustomPINPadbyImage.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Parameters:
- activity–selected Activity
- customPINPadbyImage – custom CtKMS2CustomPINPadbyImage

Returns: Encrypted PINBlock.
Notes: Only for terminals without physical keyboard.

public final byte[] startVirtualPin(CtKMS2CustomPINPadbyImage customPINPadbyImage) **throws** CtKMS2Exception
This method can start get PIN process with the selected Activity and CtKMS2CustomPINPadbyImage.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Parameters:
- customPINPadbyImage – custom CtKMS2CustomPINPadbyImage

Returns: Encrypted PINBlock.
Notes: Only for terminals without physical keyboard.

public final byte[] startSwipePin(final CtKMS2SwipePINPad pinPad) **throws** CtKMS2Exception
This method can start get PIN process with Swipe PINpad which is used for people who is visually impaired.
Throws: CtKMS2Exception– if the object obtains the Castles Key Management System II error code.
Parameters:
- pinPad – CtKMS2SwipePINPad

Returns: Encrypted PINBlock.

2.4.9 Class: CtKMS2FixedKey

public class **CtKMS2FixedKey**
The **CtKMS2FixedKey** class is a base of cryptographic key; there are DUKPT key and master key/session key extends it.

Constructor Summary	
public	CtKMS2FixedKey() Constructs a binder connected with service.
public	CtKMS2FixedKey(int keyset, int keyindex) Constructs a binder connected with service and select the key.

Methods inherited from class CTOS.CtKMS2Key
selectKey, setInputData, getOutputData, getKeyInfo, getKeySet, getKeyIndex, getKeyVersion, getKeyType, getKeyAttribute, getKeyLength, getKeyExponentLength, getCV

Constructor Detail
public CtKMS2FixedKey() Create a CtKMS2FixedKey object and connect with binder to invoke service
public CtKMS2FixedKey(int keyset, int keyindex) throws CtKMS2Exception Create a CtKMS2FixedKey object and connect with binder to invoke service and select the key.

Example

/* Default : there is a PIN key in KMS-II storage */
private TextView editText;
CtKMS2FixedKey fixedKey = new CtKMS2FixedKey();
int keySet =0xC001;
int keyIndex = 0x001A;
byte Null_PIN = 0;
int Timeout = 60;
int First_timeout = 0;

Castles Technology

```
int MaxDigit = 12, MinDigit = 4;
byte outlen = 8;
byte baOutData[] = null;

CtKMS2Callbacck callback1 = new CtKMS2Callback()
{
	@Override
	public int testCancel()
	{
		return 0;
	}
	@Override
	public void onGetDigit(byte Digit)
	{
		Log.d("", String.format("NoDigits = %d, OnGetPINDigit.", Digit));
		final StringBuffer sb = new StringBuffer();
		sb.setLength(0);
		for (int i = 0; i < Digit; i++)
		{
			sb.append("*");
		}
		activity.runOnUiThread(new Runnable()
		{
			@Override
			public void run()
			{
				PinDigitET.setText(sb);
			}
		});
	}
	@Override
	public void onGetFunctionKey(byte FunctionKey)
	{
		Log.d("", String.format("FunctionKey = %d, OnGetPINOtherKeys.", FunctionKey));
	}
};

try {
	/* Set related parameters */
	fixedKey.selectKey(keyset, keyindex);
	fixedKey.setCipherMethod(fixedKey.PIN_CIPHER_METHOD_ECB);
	fixedKey.setPinInfo(fixedKey.PIN_BLOCKTYPE_ANSI_X9_8_ISO_0, MaxDigit, MinDigit, outlen);
	fixedKey.setPinControl(Null_PIN, Timeout, First_timeout);
	//user can choose one of the below two setCallback functions
	fixedKey.setCallback(callback1);
	fixedKey. startVirtualPin(activity, layoutID);
} catch(CtKMS2Exception e){
	//exception handle
}
baOutData = fixedKey.getOutpuData();
```

Castles Technology

2.4.10 Class:CtKMS2MKSK

public class **CtKMS2MKSK**
The **CtKMS2MKSK** class provides the Master Key/Session Key Crypto functionality of KMS-II.

Constructor Summary	
public	CtKMS2MKSK() Constructs a binder connected with service.
public	CtKMS2MKSK(int keyset, int keyindex) Constructs a binder connected with service, and select the key.

Method Summary	
void	setSK(byte[] SK) Set the cipher method.
void	setSK(byte[] SK, int from, int length) Set session key.
Methods inherited from class CTOS.CtKMS2Key	
selectKey, setInputData, getOutputData, getKeyInfo, getKeySet, getKeyIndex, getKeyVersion, getKeyType, getKeyAttribute, getKeyLength, getKeyExponentLength, getCV	

Constructor Detail
public CtKMS2MKSK() Create a CtKMS2MKSK object and connect with binder to invoke service.
public CtKMS2MKSK(int keyset, int keyindex) Create a CtKMS2MKSK object and connect with binder to invoke service and select the key.

Method Detail
public final void setSK(byte[] SK) This method can set session key. **Parameters:** SK – the byte array of SK. **Note:** SK size must = 16 or 24
public final void (byte[] SK, int from, int length) This method can set session key. **Parameters:** SK – the byte array of SK. from – the start location of SK byte array. length – the length of SK.

Example

```
/* MKSK – Data Encrypt */
try {
        /* Set related parameters */
        int KeySet = 0xC101;
        int KeyIndex = 0x0005;
        byte CipherMethod = DATA_ENCRYPT_METHOD_CBC;
        byte[] SK = {(byte) 0xA1, (byte) 0xA2, (byte) 0xA3, (byte) 0xA4, (byte) 0xA5, (byte) 0xA6, (byte) 0xA7, (byte) 0xA8,
        (byte) 0xB1, (byte) 0xB2, (byte) 0xB3, (byte) 0xB4, (byte) 0xB5, (byte) 0xB6, (byte) 0xB7, (byte) 0xB8};

        byte[] ICV = {(byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11};
        byte[] InputData = {(byte) 0x01, (byte) 0x02, (byte) 0x03, (byte) 0x04, (byte) 0x05, (byte) 0x06, (byte) 0x07, (byte)
0x08};

        CtKMS2MKSK mksk = new CtKMS2MKSK();
        mksk.selectKey(KeySet, KeyIndex);
        mksk.setCipherMethod(CipherMethod);
        mksk.setICV(ICV, 0, ICV.length);
        mksk.setSK(SK);
        mksk.setInputData(InputData, 0, InputData.length);
        byte[] output = mksk.dataEncrypt();
} catch(CtKMS2Exception e){
        //exception handle
}
```

Castles Technology

2.4.11 CtKMS2Dukpt

public class **CtKMS2Dukpt**
The **CtKMS2Dukpt** class provide the ANSI X9.24(2004) DUKPT Crypto functionality of KMS-II.

Constructor Summary	
public	CtKMS2Dukpt() Constructs a binder connected with service.
public	CtKMS2Dukpt(int keyset, int keyindex) Constructs a binder connected with service, and select the key.

Method Summary	
void	useCurrentKey(boolean isCurrentKey) User current derive key or not.
void	choooseResponseKey(Boolean isResponseKey) Response key or not
byte[]	getKSN() Get current KSN of the specified DUKPT key.
boolean	isEnable() Check the specified DUKPT key exceeds generating limit or not.
Methods inherited from class CTOS.CtKMS2Key	
selectKey, setInputData, getOutputData, getKeyInfo, getKeySet, getKeyIndex, getKeyVersion, getKeyType, getKeyAttribute, getKeyLength, getKeyExponentLength, getCV	

Constructor Detail
public CtKMS2Dukpt() Create a CtKMS2Dukpt object and connect with binder to invoke service.
public CtKMS2Dukpt(int keyset, int keyindex) Create a CtKMS2Dukpt object and connect with binder to invoke service, and select the key.

Method Detail
public final void useCurrentKey(boolean isCurrentKey) Once DUKPT key crypto service, it will generate derived key, therefore, if you want to use the last time transaction key (the current stored key), set the parameter. **Parameters:** isUse - use current key or not.
public final void chooseResponseKey(boolean isResponseKey) DUKPT will generate some kind of the derived key (request or response), therefore, if you want to use the response key, set the parameter. **Parameters:** isResponseKey - use response key or not.
public final byte[] getKSN() After crypto service, this method can get immediately KSN. **Note:** At least one IPKE for DUKPT shall be injected first before execute this method.
public final boolean isEnable() This method can check the specified DUKPT exceeds generating limit or not. **Returns:** Ture: the key can be utilized. False: the key exceeds the times of generating limit.

Example

```
/* Dukpt – MAC */
try {
        /* Set related parameters */
        int KeySet = 0xC602;
        int KeyIndex = 0x0001;
        byte CipherMethod = MAC_METHOD_CBC;
        boolean UseCurrentKey = false;
        byte[] ICV = {(byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11};
        byte[] InputData = {(byte) 0x01, (byte) 0x02, (byte) 0x03, (byte) 0x04, (byte) 0x05, (byte) 0x06};

        CtKMS2Dukpt dukpt = new CtKMS2Dukpt();
        dukpt.selectKey(KeySet, KeyIndex);
        dukpt.setCipherMethod(CipherMethod);
```

Castles Technology

```
            dukpt.useCurrentKey(UseCurrentKey);
            dukpt.setICV(ICV, 0, ICV.length);
            dukpt.setInputData(InputData, 0, InputData.length);
            byte[] output = dukpt.generateMAC();
            byte[] ksn = dukpt.getKSN();

} catch(CtKMS2Exception e){
            //exception handle
}
```

2.4.12 Class: CtKMS2AESDukpt

public class **CtKMS2AESDukpt** extends CtKMS2Dukpt
The **CtKMS2AESDukpt** class provide the ANSI X9.24(2017) AES DUKPT Crypto functionality of KMS.

2.4.12.1 Inherited Methods

From class CtKMS2SymmetryKey

2.4.12.2 Inherited Methods

From class CtKMS2Key
From class CtKMS2SymmetryKey.Inner
From class CtKMS2Dukpt

2.4.12.3 Fields

Modifier and Type	Constant Field	Description	Value
public static final int	DerivedKeyType_2TDEA	Derive key with 2TDEA.	0x00
public static final int	DerivedKeyType_3TDEA	Derive key with 3TDEA.	0x01
public static final int	DerivedKeyType_AES128	Derive key with AES128.	0x02
public static final int	DerivedKeyType_AES192	Derive key with AES192.	0x03
public static final int	DerivedKeyType_AES256	Derive key with AES256.	0x04
public static final int	DerivedKeyUsage_PINEncryption	Derive key for PINEncryption.	0x01
public static final int	DerivedKeyUsage_MessageAuthenticationGeneration	Derive key for MessageAuthenticationGeneration.	0x02
public static final int	DerivedKeyUsage_MessageAuthenticationVerification	Derive key for MessageAuthenticationVerification.	0x03
public static final int	DerivedKeyUsage_MessageAuthenticationBothWays	Derive key for MessageAuthenticationBothWays.	0x04
public static final int	DerivedKeyUsage_DataEncryptionEncrypt	Derive key for DataEncryptionEncrypt.	0x05
public static final int	DerivedKeyUsage_DataEncryptionDecrypt	Derive key for DataEncryptionDecrypt.	0x06
public static final int	DerivedKeyUsage_DataEncryptionBothWays	Derive key for DataEncryptionBothWays.	0x07

2.4.12.4 Public Constructors

CtKMS2AESDukpt
public CtKMS2AESDukpt() Create a CtKMS2AESDukpt object and connect with binder to invoke service.
CtKMS2AESDukpt
public CtKMS2AESDukpt(int keyset, int keyindex) Create a CtKMS2AESDukpt object and connect with binder to invoke service, and select the key.

Castles Technology

2.4.12.5 Public Methods

setDerivedKey

public final void setDerivedKey(int derivedKeyType, int derivedKeyUsage)
This method can set derive key and utilize the derive key to execute specified function.
Parameters:

- derivedKeyType – must be above–mentioned key type.
  - DerivedKeyType_2TDEA
  - DerivedKeyType_3TDEA
  - DerivedKeyType_AES128
  - DerivedKeyType_AES192
  - DerivedKeyType_AES256
- derivedKeyUsage – must be above–mentioned key usage.
  - DerivedKeyUsage_PINEncryption
  - DerivedKeyUsage_MessageAuthenticationGeneration
  - DerivedKeyUsage_MessageAuthenticationVerification
  - DerivedKeyUsage_MessageAuthenticationBothWays
  - DerivedKeyUsage_DataEncryptionEncrypt
  - DerivedKeyUsage_DataEncryptionDecrypt
  - DerivedKeyUsage_DataEncryptionBothWays

Note: Terminal will reject deriving strong key from weak key. For example, if user inject AES 128 DUKPT key, the key can not derive AES192 and AES256 key. Derived key usage must correspond with the specify function.

setDerivedKeyType

public final void setDerivedKeyType(int derivedKeyType)
This method can set derive key type and utilize the derive key to execute specified function.
Parameters:

- derivedKeyType – must be above–mentioned key type.
  - DerivedKeyType_2TDEA
  - DerivedKeyType_3TDEA
  - DerivedKeyType_AES128
  - DerivedKeyType_AES192
  - DerivedKeyType_AES256

Note: Terminal will reject deriving strong key from weak key. For example, if user inject AES 128 DUKPT key, the key cannot derive AES192 and AES256 key. Derived key usage must correspond with the specify function.

setDerivedKeyUsage

public final void setDerivedKeyUsage(int derivedKeyUsage)
This method can set derive key usage and utilize the derive key to execute specified function.
Parameters:

- derivedKeyUsage – must be above–mentioned key usage.
  - DerivedKeyUsage_PINEncryption
  - DerivedKeyUsage_MessageAuthenticationGeneration
  - DerivedKeyUsage_MessageAuthenticationVerification
  - DerivedKeyUsage_MessageAuthenticationBothWays
  - DerivedKeyUsage_DataEncryptionEncrypt

Castles Technology

- ✧ DerivedKeyUsage_DataEncryptionDecrypt
- ✧ DerivedKeyUsage_DataEncryptionBothWays

Note: Terminal will reject deriving strong key from weak key. For example, if user inject AES 128 DUKPT key, the key can not derive AES192 and AES256 key. Derived key usage must correspond with the specify function.

2.4.12.6 Example

aesDukpt

```
/* Dukpt – MAC */
try {
        /* Set related parameters */
        int KeySet = 0xC607;
    int KeyIndex = 0x0002;
    byte CipherMethod = MAC_METHOD_CBC;
    int DerivedKeyType = DerivedKeyType_AES128;
    int DerivedKeyUsage = DerivedKeyUsage_MessageAuthenticationGeneration;
    boolean UseCurrentKey = true;
    byte[] ICV = {(byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11, (byte) 0x11};
    byte[] InputData = {(byte) 0x01, (byte) 0x02, (byte) 0x03, (byte) 0x04, (byte) 0x05, (byte) 0x06, (byte) 0x07, (byte) 0x08, (byte) 0x11, (byte) 0x12, (byte) 0x13, (byte) 0x14, (byte) 0x15, (byte) 0x16};

    CtKMS2AESDukpt aesDukpt = new CtKMS2AESDukpt();
    aesDukpt.selectKey(KeySet, KeyIndex);
    aesDukpt.setCipherMethod(CipherMethod);
    aesDukpt.useCurrentKey(UseCurrentKey);
    aesDukpt.setDerivedKey(DerivedKeyType, DerivedKeyUsage);
    aesDukpt.setICV(ICV, 0, ICV.length);
    aesDukpt.setInputData(InputData, 0, InputData.length);
    byte[] output = aesDukpt.generateMAC();
    byte[] ksn = aesDukpt.getKSN();
} catch(CtKMS2Exception e){
        //exception handle
}
```

2.4.13 Class: CtKMS2AsymmetricKey

abstract class **CtKMS2AsymmetricKey** extends CtKMS2Key
The **CtKMS2AsymmetricKey** is an abstract class extends CtKMS2Key

2.4.13.1 Inherited Methods

From class CtKMS2Key

2.4.13.2 Inherited Methods

selectKey

public void selectKey(int keyset, int keyIndex) **throws** CtKMS2Exception
This method let object select key that store in KMS–II storage and get related information.
Parameters:

- ➢ keyset – the set of key.
- ➢ keyIndex – the index of key.

Throws: CtKMS2Exception– if the object obtains the Castles Key Management System II error code.
Note: The range of keyset is 0x0001~0xBFFF, and the range of keyindex is 0x0000~0xFFFF.

setKeyVersion

public final void setKeyVersion(byte keyVersion)
This method can set the new/update key version.
Parameters:

➢ keyVersion – specify the key version to the new/update key.
setKeyAttribute
public final void setKeyAttribute(int keyAttribute) This method can set the new/update key attribute. **Parameters**: ➢ keyAttribute– specify the key attribute to the new/update key.
setKeyLocation
public final void setKeyLocation(int keyset, int keyindex) This method can set the new/update key location. **Parameters**: ➢ keyset – specify the key set the new/update key belong to. ➢ keyindex – specify the key index the new/update key belong to.
encryptByPublicKey
public byte[] encryptByPublicKey() **throws** CtKMS2Exception This method can get public key data after selectKey. **Returns**: The public key from the selected key. **Throws:** CtKMS2Exception– if the object obtains the Castles Key Management System II error code.
decryptByPrivateKey
public byte[] decryptByPrivateKey() **throws** CtKMS2Exception This method can get output decrypted data by private key. **Returns**: The decrypted data which is the result of decrypted input data by private key. **Throws:** CtKMS2Exception– if the object obtains the Castles Key Management System II error code.
generateKeyPair
public byte[] generateKeyPair() **throws** CtKMS2Exception This method can generate RSA key pair and get the public key. **Returns**: Modulus of the public key. **Throws:** CtKMS2Exception– if the object obtains the Castles Key Management System II error code.

2.4.14 Class: CtKMS2RSAKey

public class **CtKMS2RSAKey** extends CtKMS2AsymmetricKey
The **CtKMS2RSAKey** class provides RSA Crypto functionality of KMS–II.

2.4.14.1 Inherited Methods

From class CtKMS2Key
From class CtKMS2AsymmetricKey

2.4.14.2 Fields

Modifier and Type	Constant Field	Description	Value
public static final int	ATTR_RSA_PUBLICKEYENCRYPT	Attribute public key encrypt for method setKeyAttribute	0x0100
public static final int	ATTR_RSA_PRIVATEKEYENCRYPT	Attribute private key encrypt for method setKeyAttribute	0x0200

2.4.14.3 Public Constructors

CtKMS2RSAKey

Castles Technology

public CtKMS2RSAKey() Create a CtKMS2RSAKey object and connect with binder to invoke service.
CtKMS2RSAKey
public CtKMS2RSAKey(int keyset, int keyindex) **throws** CtKMS2Exception Create a CtKMS2RSAKey object and connect with binder to invoke service and select the key. **Throws:** CtKMS2Exception– if the object obtains the Castles Key Management System II error code.

2.4.14.4 Public Methods

setExponent
public final void setExponent(int exponent) This method can set exponent data by integer. **Parameters**: ➢ exponent– the integer value of exponent data.
setExponent
public final void setExponent(byte[] exponent, int from, int inputLength) This method can set exponent data by byte array. **Parameters**: ➢ exponent– the byte array of exponent data. ➢ from – the start address of byte array of input data. ➢ inputLength– the length of exponent data.
setBits
public final void setBits(int bits) This method can set bits length of RSA keys. **Parameters:** ➢ bits – the bits of length of RSA key, **default value is 2048**
setMethod
public final void setMethod(int method) This method can set SN padding method in common name (CN) of CSR. **Parameters:** ➢ method – set the padding method ✧ METHOD_RSA_CSR_CN_USE_SN_PADDING – **default value** ✧ METHOD_RSA_CSR_CN_USE_SN_NON_PADDING
getExponent
public final byte[] getExponent() **throws** CtKMS2Exception This method can get exponent data after selectKey. **Returns**: The exponent data from the selected key. **Throws:** CtKMS2Exception– if the object obtains the Castles Key Management System II error code.
getModulus
public final byte[] getModulus() **throws** CtKMS2Exception This method can get modulus data after selectKey. **Returns**: The modulus data from the selected key. **Throws:** CtKMS2Exception– if the object obtains the Castles Key Management System II error code.
getPublicKey
public final byte[] getPublicKey() throws CtKMS2Exception This method can get public key data after selectKey. **Returns**: The public key from the selected key. **Throws:** CtKMS2Exception– if the object obtains the Castles Key Management System II error code.
setCSRInfo
public void setCSRInfo(String csrCountry, String csrState, String csrLocality, String csrOrganization, String csrOrganizationUnit, String csrCommonName) This method can set information into CSR.

Castles Technology

Parameters:

- csrCountry – set Country into CSR
- csrState – set State into CSR
- csrLocality – set Locality into CSR
- csrOrganization – set Organization into CSR
- csrOrganizationUnit – set Organization Unit into CSR
- csrCommonName – set Common Name into CSR

setCSRCountry

public void setCSRCountry(String csrCountry)
This method can set information into CSR.
Parameters:

- csrCountry – set Country into CSR

setCSRState

public void setCSRState(String csrState)
This method can set information into CSR.
Parameters:

- csrState – set State into CSR

setCSRLocality

public void setCSRLocality(String csrLocality)
This method can set information into CSR.
Parameters:

- csrLocality – set Locality into CSR

setCSROrganization

public void setCSROrganization(String csrOrganization)
This method can set information into CSR.
Parameters:

- csrOrganization – set Organization into CSR

setCSROrganizationUnit

public void setCSROrganizationUnit(String csrOrganizationUnit)
This method can set information into CSR.
Parameters:

- csrOrganizationUnit – set Organization Unit into CSR

setCSRCommonName

public void setCSRCommonName(String csrCommonName)
This method can set information into CSR.
Parameters:

- csrCommonName – set Common Name into CSR

getCSR

public final byte[] getCSR() **throws** CtKMS2Exception
This method can get csr data after selectKey.
Returns: The csr data from the selected key.
Throws: CtKMS2Exception– if the object obtains the Castles Key Management System II error code.

generateKeyPair

public final byte[] generateKeyPair(int bits) **throws** CtKMS2Exception
This method can generate RSA key pair and get the public key.
Parameters:

- bits – key length of the RSA key pair. Please make sure the size smaller than 2048, otherwise, it will have some unpredictable result. If the bit is null or length = 0, it will throws CtKMS2Exception.

Returns: Modulus of the public key.
Throws: CtKMS2Exception– if the object obtains the Castles Key Management System II error code.

Castles Technology

2.4.14.5 Example

rsa

```
//generate key pairs
try {
	int KeyAttribute = ATTR_RSA_PRIVATEKEYENCRYPT | ATTR_RSA_PUBLICKEYENCRYPT;
	int KeySetLocation = 0xC704;
	int KeyIndexLocation = 0x0001;
	byte KeyVersion = (byte) 0x01;
	int Exponent = 65537;
	int bits = 1024;

	CtKMS2RSAKey rsa = new CtKMS2RSAKey();
	rsa.setKeyVersion(KeyVersion);
	rsa.setExponent(Exponent);
	rsa.setKeyAttribute(KeyAttribute);
	rsa.setKeyLocation(KeySetLocation, KeyIndexLocation);
	byte[] output = rsa.generateKeyPair(bits);
}catch(CtKMS2Exception e){
	//exception handle
}
//get public key info
try {
int KeySet = 0xC704;
	int KeyIndex = 0x0001;

	CtKMS2RSAKey rsa = new CtKMS2RSAKey();
	rsa.selectKey(KeySet, KeyIndex);

	byte[] output = rsa.getExponent();
byte[] output = rsa.getModulus();
byte[] output = rsa.getPublicKey();
byte[] output = rsa.getCSR();
} catch(CtKMS2Exception e){
	//exception handle
}
//encrypt
// data length has to corresponding with RSA key length
try {
	int KeySet = 0xC704;
	int KeyIndex = 0x0001;
	byte[] InputData = {(byte) 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};

	CtKMS2RSAKey rsa = new CtKMS2RSAKey();
	rsa.selectKey(KeySet, KeyIndex);
	rsa.setInputData(InputData, 0, InputData.length);

	byte[] output = rsa.encryptByPublicKey();
} catch(CtKMS2Exception e){
	//exception handle
}

//decrypt
// data length has to corresponding with RSA key length
try {
	int KeySet = 0xC704;
	int KeyIndex = 0x0001;
```

```
    byte[] InputData = {(byte) 0xA7, (byte) 0x49, (byte) 0x67, (byte) 0x0E, (byte) 0xEA, (byte) 0xE2, (byte) 0xD7, (byte) 0x13,
        (byte) 0x55, (byte) 0x06, (byte) 0xCF, (byte) 0xFF, (byte) 0x02, (byte) 0x75, (byte) 0xFF, (byte) 0xB4,
        (byte) 0x3D, (byte) 0x55, (byte) 0xDB, (byte) 0xB6, (byte) 0x69, (byte) 0xA2, (byte) 0xF3, (byte) 0x69,
        (byte) 0xF1, (byte) 0x01, (byte) 0xF7, (byte) 0xD2, (byte) 0x02, (byte) 0x52, (byte) 0x65, (byte) 0xCE,
        (byte) 0xD1, (byte) 0xEB, (byte) 0x99, (byte) 0x03, (byte) 0xDE, (byte) 0x8D, (byte) 0x2F, (byte) 0x11,
        (byte) 0x62, (byte) 0x8E, (byte) 0x26, (byte) 0x0F, (byte) 0xC3, (byte) 0x9B, (byte) 0xD4, (byte) 0x05,
        (byte) 0xDB, (byte) 0xD8, (byte) 0xC6, (byte) 0xBF, (byte) 0x8C, (byte) 0x5A, (byte) 0xA7, (byte) 0x88,
        (byte) 0x4E, (byte) 0xB4, (byte) 0x51, (byte) 0xF7, (byte) 0x4B, (byte) 0x37, (byte) 0xC6, (byte) 0xFC,
        (byte) 0x83, (byte) 0x42, (byte) 0xB1, (byte) 0xEA, (byte) 0x1B, (byte) 0x81, (byte) 0xA6, (byte) 0x1F,
        (byte) 0x5E, (byte) 0xEB, (byte) 0xEE, (byte) 0xFD, (byte) 0xE3, (byte) 0x12, (byte) 0xDD, (byte) 0xED,
        (byte) 0xE7, (byte) 0xEE, (byte) 0x31, (byte) 0xD1, (byte) 0xD2, (byte) 0x53, (byte) 0x86, (byte) 0x80,
        (byte) 0xB8, (byte) 0x0C, (byte) 0x2E, (byte) 0x50, (byte) 0x24, (byte) 0xD8, (byte) 0x70, (byte) 0xD1,
        (byte) 0xE9, (byte) 0xCC, (byte) 0x90, (byte) 0x06, (byte) 0xC8, (byte) 0xBB, (byte) 0x31, (byte) 0x34,
        (byte) 0x85, (byte) 0xB9, (byte) 0xA3, (byte) 0xB2, (byte) 0xA2, (byte) 0x07, (byte) 0xC7, (byte) 0x51,
        (byte) 0xE2, (byte) 0x9E, (byte) 0x95, (byte) 0x94, (byte) 0x9C, (byte) 0xC1, (byte) 0x88, (byte) 0x96,
        (byte) 0x93, (byte) 0xAF, (byte) 0x83, (byte) 0x58, (byte) 0x87, (byte) 0x4B, (byte) 0x9A, (byte) 0x4E};

    CtKMS2RSAKey rsa = new CtKMS2RSAKey();
    rsa.selectKey(KeySet, KeyIndex);
    rsa.setInputData(InputData, 0, InputData.length);

    byte[] output = rsa.decryptByPrivateKey();
} catch(CtKMS2Exception e){
    //exception handle
}
```

2.4.15 Class: CtKMS2ECKey

public class CtKMS2ECKey extends CtKMS2AsymmetricKey
The CtKMS2ECKey class provides EC Crypto functionality of KMS–II.

2.4.15.1 Inherited Methods

From class CtKMS2Key
From class CtKMS2AsymmetricKey

2.4.15.2 Fields

Modifier and Type	Constant Field	Description	Value
public static final int	CURVE_ID_SECP192_K1	Curve ID: SECP192 K1 for method setCurve	0
public static final int	CURVE_ID_SECP192_R1	Curve ID: SECP192 R1 for method setCurve	1
public static final int	CURVE_ID_SECP224_R1	Curve ID: SECP224 R1 for method setCurve	3
public static final int	CURVE_ID_SECP256_K1	Curve ID: SECP256 K1 for method setCurve	4
public static final int	CURVE_ID_SECP256_R1	Curve ID: SECP256 R1 for method setCurve	5
public static final int	CURVE_ID_SECP384_R1	Curve ID: SECP384 R1 for method setCurve	6
public static final int	CURVE_ID_SECP521_R1	Curve ID: SECP521 R1 for method setCurve	7
public static final int	ATTR_EC_DH	Attribute ECDH for method setKeyAttribute	0x1000
public static final int	ATTR_EC_DSA_SIGN	Attribute ECDSA sign for method setKeyAttribute	0x2000

Castles Technology

public static final int	ATTR_EC_DSA_VERIFY	Attribute ECDSA verify for method setKeyAttribute	0x4000
public static final int	ECDSA_HASHTYPE_NULL	Set Null in hash alg of ECDSA for method setHashType.	0
public static final int	ECDSA_HASHTYPE_SHA1	Set SHA1 in hash alg of ECDSA for method setHashType.	1
public static final int	ECDSA_HASHTYPE_SHA256	Set SHA256 in hash alg of ECDSA for method setHashType.	2

2.4.15.3 Public Constructors

CtKMS2ECKey
public CtKMS2ECKey() Create a CtKMS2ECKey object and connect with binder to invoke service.
CtKMS2ECKey
public CtKMS2ECKey(int keyset, int keyindex) throws CtKMS2Exception Create a CtKMS2ECKey object and connect with binder to invoke service and select the key. Throws: CtKMS2Exception– if the object obtains the Castles Key Management System II error code.

2.4.15.4 Public Methods

getCurve
public final int getCurve() throws CtKMS2Exception This method can get current curve id. **Parameters**: ➢ exponent– the integer value of exponent data. ➢ **Throws:** CtKMS2Exception– if the object obtains the Castles Key Management System II error code
getPublicKey
public final byte[] getPublicKey() throws CtKMS2Exception This method can get public data after key selection. **Parameters**: ➢ exponent– the byte array of exponent data. ➢ **Throws:** CtKMS2Exception– if the object obtains the Castles Key Management System II error code
generateKeyPair
public final byte[] generateKeyPair(int curveID) throws CtKMS2Exception This method can generate EC key pair by curve ID. **Parameters:** ➢ curveID – the curve id selection. **Returns**: Modulus of the public key. ➢ **Throws:** CtKMS2Exception– if the object obtains the Castles Key Management System II error code
setHashType
public final void setHashType(byte hashType) This method can set hash algorithm for method ECDSA. ✧ **Returns**: The exponent data from the selected key.
setVerifyRS
public final void setVerifyRS(byte[] input_r, byte[] input_s) This method can set the signature to verify.

Castles Technology

A signature S = (r, s) on Message consisting of a pair of integers r and s.
r = x_R mod n. s = k^{-1} (e + rd_U) mod n. They are defined in *SEC 1: Elliptic Curve Cryptography*.

generateSharedSecret

public final byte[] generateSharedSecret(byte[] publicKey) throws CtKMS2Exception
This method can compute shared secret key by ECDH.
Parameters:

- publicKey – the peer's public key

Returns: The shared secret key computed from ECDH.
Throws: CtKMS2Exception– if the object obtains the Castles Key Management System II error code.

2.4.15.5 Example

ec

```
public void start(View view) {

        final int curveID = CURVE_ID_SECP256_R1;
        //CURVE_ID_SECP192_K1 CURVE_ID_SECP192_R1 CURVE_ID_SECP224_R1 CURVE_ID_SECP256_K1 CURVE_ID_SECP256_R1
CURVE_ID_SECP384_R1 CURVE_ID_SECP521_R1
        final int hash_type = ECDSA_HASHTYPE_SHA256;
        //ECDSA_HASHTYPE_SHA1 ECDSA_HASHTYPE_SHA256

        int keyset = 0xC001;
        int keyindex = 0x0001;

        int Extkeyset = 0xC002 ;
        int ExtKeyindex = 0x0002;

        int keyset_sign = 0xC003;
        int keyindex_sign = 0x0003;

        int iCVLength = 3;

        String strData = "TEST";
        byte[] byteArray = strData.getBytes();

        try {

            CtKMS2ECKey KMS2ECKey = new CtKMS2ECKey();
            KMS2ECKey.setKeyLocation(keyset,keyindex);
            KMS2ECKey.setKeyAttribute(ATTR_EC_DH);
            KMS2ECKey.setKeyVersion((byte) 0x00);
            byte[] output = KMS2ECKey.generateKeyPair(curveID);
            Log.d(TAG, "output :" +    bytesToHex(output));

            KMS2ECKey.selectKey(keyset,keyindex);
            byte [] Publickey = KMS2ECKey.getPublicKey();
            Log.d(TAG, "Publickey :" +    bytesToHex(Publickey));

            CtKMS2ECKey ExtKMS2ECKey = new CtKMS2ECKey();
            ExtKMS2ECKey.setKeyLocation(Extkeyset,ExtKeyindex);
            ExtKMS2ECKey.setKeyAttribute(ATTR_EC_DH);
            ExtKMS2ECKey.setKeyVersion((byte) 0x00);
            ExtKMS2ECKey.generateKeyPair(curveID);

            ExtKMS2ECKey.selectKey(Extkeyset,ExtKeyindex);
            byte [] ExtPublickey = ExtKMS2ECKey.getPublicKey();
            Log.d(TAG, "ExtPublickey :" +    bytesToHex(ExtPublickey));


            CtKMS2ECKey ECDH1 = new CtKMS2ECKey();
            ECDH1.selectKey(keyset, keyindex);
            byte[] ECDH1_KEY = ECDH1.generateSharedSecret(ExtPublickey);
            Log.d(TAG, "ECDH1_KEY :" +    bytesToHex(ECDH1_KEY));

            CtKMS2ECKey ECDH2 = new CtKMS2ECKey();
            ECDH2.selectKey(Extkeyset,ExtKeyindex);
            byte[] ECDH2_KEY = ECDH2.generateSharedSecret(Publickey);
```

Castles Technology

```
                Log.d(TAG, "ECDH2_KEY :" +    bytesToHex(ECDH2_KEY));

                boolean isSame = Arrays.equals(ECDH1_KEY, ECDH2_KEY);
                Log.d(TAG, "2 ECDH KEY is same :" +    isSame);




                CtKMS2ECKey KMS2ECKey_sign = new CtKMS2ECKey();
                KMS2ECKey_sign.setKeyLocation(keyset_sign,keyindex_sign);
                KMS2ECKey_sign.setKeyAttribute(ATTR_EC_DSA_SIGN | ATTR_EC_DSA_VERIFY);
                KMS2ECKey_sign.setKeyVersion((byte) 0x00);
                KMS2ECKey_sign.generateKeyPair(curveID);

                KMS2ECKey_sign.selectKey(keyset_sign,keyindex_sign);
                KMS2ECKey_sign.setInputData(byteArray);
                KMS2ECKey_sign.setHashType((byte)hash_type);
                byte [] signature = KMS2ECKey_sign.sign();
                Log.d(TAG, "signature :" +    bytesToHex(signature));

// output :
304502202518655e1827e1f8dc18abed8bdee6e8130a72030d6b6bf45869b539842ba136022100b70c1e153f0a85ce7769fd2e8f3375c101
db7590bf38546282311a2876cb858a

//                byte [] R =
{(byte)0x25,(byte)0x18,(byte)0x65,(byte)0x5e,(byte)0x18,(byte)0x27,(byte)0xe1,(byte)0xf8,(byte)0xdc,(byte)0x18,(byte)0xab,(byte)0xed,
(byte)0x8b,(byte)0xde,(byte)0xe6,(byte)0xe8,(byte)0x13,(byte)0x0a,(byte)0x72,(byte)0x03,(byte)0x0d,(byte)0x6b,(byte)0x6b,(byte)0xf4,
(byte)0x58,(byte)0x69,(byte)0xb5,(byte)0x39,(byte)0x84,(byte)0x2b,(byte)0xa1,(byte)0x36};
//                byte [] S =
{(byte)0xb7,(byte)0x0c,(byte)0x1e,(byte)0x15,(byte)0x3f,(byte)0x0a,(byte)0x85,(byte)0xce,(byte)0x77,(byte)0x69,(byte)0xfd,(byte)0x2e,(
byte)0x8f,(byte)0x33,(byte)0x75,(byte)0xc1,(byte)0x01,(byte)0xdb,(byte)0x75,(byte)0x90,(byte)0xbf,(byte)0x38,(byte)0x54,(byte)0x62,(
byte)0x82,(byte)0x31,(byte)0x1a,(byte)0x28,(byte)0x76,(byte)0xcb,(byte)0x85,(byte)0x8a};
//
//                KMS2ECKey_sign.selectKey(keyset_sign,keyindex_sign);
//                KMS2ECKey_sign.setInputData(byteArray);
//                KMS2ECKey_sign.setHashType((byte)hash_type);
//                KMS2ECKey_sign.setVerifyRS(R,S);
//                boolean bo_verify = KMS2ECKey_sign.verify();
//                Log.d(TAG, "bo_verify :" + bo_verify);


                CtKMS2Key key = new CtKMS2Key();
                key.selectKey(keyset, keyindex);
                key.getKeyInfo();
                byte keyType = key.getKeyType();
                Log.d(TAG, "keyType :" +    keyType);
                int KeyAttribute = key.getKeyAttribute();
                Log.d(TAG, "KeyAttribute :" +    KeyAttribute);
                byte[] KCV = key.getCV(iCVLength);
                Log.d(TAG, "CV :" +    bytesToHex(KCV));

                key.selectKey(Extkeyset,ExtKeyindex);
                key.getKeyInfo();
                keyType = key.getKeyType();
                Log.d(TAG, "keyType :" +    keyType);
                KeyAttribute = key.getKeyAttribute();
                Log.d(TAG, "KeyAttribute :" +    KeyAttribute);
                KCV = key.getCV(iCVLength);
                Log.d(TAG, "CV :" +    bytesToHex(KCV));

                key.selectKey(keyset_sign,keyindex_sign);
                key.getKeyInfo();
                keyType = key.getKeyType();
                Log.d(TAG, "keyType :" +    keyType);
                KeyAttribute = key.getKeyAttribute();
                Log.d(TAG, "KeyAttribute :" +    KeyAttribute);
                KCV = key.getCV(iCVLength);
                Log.d(TAG, "CV :" +    bytesToHex(KCV));

                CtKMS2System system = new CtKMS2System();
                system.checkKey(keyset, keyindex);
                Log.d(TAG, "checkKey OK ");
```

Castles Technology

```
                system.deleteKey(keyset,keyindex);
                Log.d(TAG, "deleteKey ok");

            }catch (CtKMS2Exception e){
                e.printStackTrace();
                Log.d(TAG, "" + e.getMessage());
            }
        }
```

2.4.16 Class: CtKMS2CustomPINPadbyImage

public **CtKMS2CustomPINPadbyImage**
The **CtKMS2CustomPINPadbyImage** class is used to set PIN Pad screen. User can set image and location of all buttons.

2.4.16.1 Public Constructors

CtKMS2CustomPINPadbyImage
public CtKMS2CustomPINPadbyImage() Create a CtKMS2CustomPINPadbyImage object. User must set all x, y, width, height and filepath. EX_function keys are option. Please assign"0xFF" to EX_FUNCTIONKEY.value, if user does not need function key.

2.4.16.2 Public Methods

setMovingRange
public void setMovingRange(iint topX, int topY, int bottomX, int bottomY, int offsetX, int offsetY) This method can set moving range when user enable moving PINPad. PIN Pad will random move in range (topX, topY) to (bottomX, bottomY) and the weight of random is offsetX and offsetY. **Parameters**: ➢ topX – X–coordinate of the upper left corner point of moving range. ➢ topY – Y–coordinate of the upper left corner point of moving range. ➢ bottom – X–coordinate of the bottom right corner point of moving range. ➢ bottom – Y–coordinate of the bottom right corner point of moving range. ➢ offsetX – offset of horizontal displacement. ➢ offset – offset of vertical displacement. *Note: Origin is at the upper left corner on the screen. If it has status bar, the origin is at bottom left of status bar. That means bottom must be smaller than (Y–resolution – height of status bar.)* *The X–coordinate is (RNG % ((bottomX – topX– PINPad_width) / offsetX)) * offsetX + topX.* *The Y–coordinate is (RNG % ((bottomY – topY– PINPad_height) / offsetY)) * offsetY+ topY*

2.4.16.3 Example

setvirtualpinpad_para

try{
CtKMS2CustomPINPadbyImage setvirtualpinpad_para = **new** CtKMS2CustomPINPadbyImage();
setvirtualpinpad_para.VKBD_0.x = 50;
setvirtualpinpad_para.VKBD_0.y = 545;
setvirtualpinpad_para.VKBD_0.width = 140;
setvirtualpinpad_para.VKBD_0.height = 140;
setvirtualpinpad_para.VKBD_0.filepath = **new** File("/sdcard/0.bmp");
setvirtualpinpad_para.VKBD_1.x = 210;
setvirtualpinpad_para.VKBD_1.y = 545;
setvirtualpinpad_para.VKBD_1.width = 140;
setvirtualpinpad_para.VKBD_1.height = 140;

Castles Technology

```
setvirtualpinpad_para.VKBD_1.filepath = new File("/sdcard/1.bmp");
setvirtualpinpad_para.VKBD_2.x = 370;
setvirtualpinpad_para.VKBD_2.y = 545;
setvirtualpinpad_para.VKBD_2.width = 140;
setvirtualpinpad_para.VKBD_2.height = 140;
setvirtualpinpad_para.VKBD_2.filepath = new File("/sdcard/2.bmp");
setvirtualpinpad_para.VKBD_3.x = 50;
setvirtualpinpad_para.VKBD_3.y = 695;
setvirtualpinpad_para.VKBD_3.width = 140;
setvirtualpinpad_para.VKBD_3.height = 140;
setvirtualpinpad_para.VKBD_3.filepath = new File("/sdcard/3.bmp");
setvirtualpinpad_para.VKBD_4.x = 210;
setvirtualpinpad_para.VKBD_4.y = 695;
setvirtualpinpad_para.VKBD_4.width = 140;
setvirtualpinpad_para.VKBD_4.height = 140;
setvirtualpinpad_para.VKBD_4.filepath = new File("/sdcard/4.bmp");
setvirtualpinpad_para.VKBD_5.x = 370;
setvirtualpinpad_para.VKBD_5.y = 695;
setvirtualpinpad_para.VKBD_5.width = 140;
setvirtualpinpad_para.VKBD_5.height = 140;
setvirtualpinpad_para.VKBD_5.filepath = new File("/sdcard/5.bmp");
setvirtualpinpad_para.VKBD_6.x = 50;
setvirtualpinpad_para.VKBD_6.y = 845;
setvirtualpinpad_para.VKBD_6.width = 140;
setvirtualpinpad_para.VKBD_6.height = 140;
setvirtualpinpad_para.VKBD_6.filepath = new File("/sdcard/6.bmp");
setvirtualpinpad_para.VKBD_7.x = 210;
setvirtualpinpad_para.VKBD_7.y = 845;
setvirtualpinpad_para.VKBD_7.width = 140;
setvirtualpinpad_para.VKBD_7.height = 140;
setvirtualpinpad_para.VKBD_7.filepath = new File("/sdcard/7.bmp");
setvirtualpinpad_para.VKBD_8.x = 370;
setvirtualpinpad_para.VKBD_8.y = 845;
setvirtualpinpad_para.VKBD_8.width = 140;
setvirtualpinpad_para.VKBD_8.height = 140;
setvirtualpinpad_para.VKBD_8.filepath = new File("/sdcard/8.bmp");
setvirtualpinpad_para.VKBD_9.x = 210;
setvirtualpinpad_para.VKBD_9.y = 995;
setvirtualpinpad_para.VKBD_9.width = 140;
setvirtualpinpad_para.VKBD_9.height = 140;
setvirtualpinpad_para.VKBD_9.filepath = new File("/sdcard/9.bmp");
setvirtualpinpad_para.VKBD_CLEAR.x = 530;
setvirtualpinpad_para.VKBD_CLEAR.y = 695;
setvirtualpinpad_para.VKBD_CLEAR.width = 140;
setvirtualpinpad_para.VKBD_CLEAR.height = 140;
setvirtualpinpad_para.VKBD_CLEAR.filepath = new File("/sdcard/backspace.bmp");
setvirtualpinpad_para.VKBD_CANCEL.x = 530;
setvirtualpinpad_para.VKBD_CANCEL.y = 845;
setvirtualpinpad_para.VKBD_CANCEL.width = 140;
setvirtualpinpad_para.VKBD_CANCEL.height = 140;
setvirtualpinpad_para.VKBD_CANCEL.filepath = new File("/sdcard/cancel.bmp");
setvirtualpinpad_para.VKBD_ENTER.x = 530;
setvirtualpinpad_para.VKBD_ENTER.y = 545;
setvirtualpinpad_para.VKBD_ENTER.width = 140;
setvirtualpinpad_para.VKBD_ENTER.height = 140;
setvirtualpinpad_para.VKBD_ENTER.filepath = new File("/sdcard/enter.bmp");
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY1.x = 50;
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY1.y = 445;
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY1.width = 140;
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY1.height = 90;
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY1.value = 'X';
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY1.filepath = new File("/sdcard/f1.bmp");
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY2.x = 210;
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY2.y = 445;
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY2.width = 140;
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY2.height = 90;
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY2.value = 'Y';
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY2.filepath = new File("/sdcard/f2.bmp");
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY3.x = 370;
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY3.y = 445;
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY3.width = 140;
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY3.height = 90;
setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY3.value = 'I';
```

```
        setvirtualpinpad_para.VKBD_EX_FUNCTIONKEY3.filepath = new File("/sdcard/f3.bmp");
        setvirtualpinpad_para.setMovingRange(0,0,720,1200,20,20);
        CtKMS2Callback callback = new CtKMS2Callback(){
            StringBuffer sb = new StringBuffer();
            byte recv;
            @Override
            public int testCancel(){
                return 0;
            }
            @Override
            public void onGetDigit(byte Digit) {
                Log.d(TAG, String.format("NoDigits = %d, OnGetPINDigit.", Digit));
                recv = Digit;
                sb.setLength(0);
                for(int i = 0;i < recv;i ++)
                    sb.append("*");
                activity.runOnUiThread(new Runnable() {
                    @Override
                    public void run() {
                        ((EditText) activity.findViewById(R.id.rlPINDigit).findViewById(R.id.etPinDigit)).setText(sb);
                    }
                });
            }
            @Override
            public void onGetFunctionKey(byte FunctionKey) {
                Log.d(TAG, String.format("FunctionKey = %d, OnGetPINOtherKeys.", FunctionKey));
                recv = FunctionKey;
                activity.runOnUiThread(new Runnable() {
                    @Override
                    public void run() {
                        //FunctionKeyET.setText(String.format("FunctionKey = %d",recv));
                    }
                });
            }};
            byte outblocklen = 8;
            byte[] PAN_number = {0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31,0x31};
            int keySet = 0xC101;
            int keyIndex = 0x0001;
            int PAN_offset = 0;
            int PAN_length = 17;
            byte Null_PIN = 0;
            int Timeout = 64;
            int First_timeout = 0;
            byte PINBlockType = PIN_BLOCKTYPE_ANSI_X9_8_ISO_0;
            byte PINMaxLen = 12;
            byte PINMinLen = 4;
            byte PINBlockLen = 8;

            CtKMS2System system = new CtKMS2System();
            system.setPINPADMoving(false);
            system.setPINScrambling(true);
            system.setPINSound(true);
            CtKMS2FixedKey fixedKey = new CtKMS2FixedKey();
            fixedKey.selectKey(keySet, keyIndex);
            fixedKey.setCipherMethod(MAC_METHOD_CBC);
            fixedKey.setInputData(PAN_number, PAN_offset, PAN_length);
            fixedKey.setPinInfo(PINBlockType, PINMaxLen, PINMinLen, PINBlockLen);
            fixedKey.setPinControl(Null_PIN, Timeout, First_timeout);
            fixedKey.setCallback(callback);
            activity.runOnUiThread(new Runnable() {
                @Override
                public void run() {
                    activity.findViewById(R.id.rlPINDigit).setVisibility(View.VISIBLE);
                }
            });
            fixedKey.startVirtualPin(setvirtualpinpad_para);
    } catch (CTOS.CtKMS2Exception e){
            //exception handle
    }
```

2.4.17 Class: CtKMS2OfflinePIN

public class **CtKMS2OfflinePIN**
The **CtKMS2OfflinePIN** class is used to generate encrypted offline PIN block and store in KMS. The encrypted PIN block is used for EMV library.

2.4.17.1 Public Constructors

CtKMS2OfflinePIN
public CtKMS2OfflinePIN() Create a CtKMS2CustomPINPadbyImage object.

2.4.17.2 Public Methods

setMovingRange
public final void setPinInfo(byte pinDigitMaxLength, byte pinDigitMinLength) This method can set the number of PIN. **Parameters**: ➢ pinDigitMaxLength– must be 4~12 and >= Min Digit Length ➢ pinDigitMinLength– must be 4~12 and <= Max Digit Length
setPinControl
public final void setPinControl(byte nullPIN, int timeout, int firstKeyEntryTimeout) This method can set the control of PIN **Parameters**: ➢ nullPIN– whether allow Null PIN or not ➢ timeout– timeout ➢ firstKeyEntryTimeout– timeout before first key entry.
setPinControl
public final void setPinControl(byte nullPIN, int timeout, int firstKeyEntryTimeout, int nonOperationTimeout) This method can set the control of PIN **Parameters**: ➢ nullPIN– whether allow Null PIN or not ➢ timeout– timeout ➢ firstKeyEntryTimeout– timeout before first key entry. ➢ nonOperationTimeout – timeout between two operations
setPinControl
public final void setPinControl(byte nullPIN, byte clearPIN, int timeout, int firstKeyEntryTimeout, int nonOperationTimeout) This method can set the control of PIN **Parameters**: ➢ nullPIN– whether allow Null PIN or not ➢ clearPIN – the behavior when user click “clear” button ✧ PIN_CLEAR_METHOD_ONE_PIN – **default value** ✧ PIN_CLEAR_METHOD_ALL_PIN ➢ timeout– timeout ➢ firstKeyEntryTimeout– timeout before first key entry. ➢ nonOperationTimeout – timeout between two operations
setTapbehavior

Castles Technology

public final void setTapbehavior(byte method)
When finger touch the screen, this setting will decide whether the location is selected. There are two selections, touch: when finger touches the screen, the location is selected; release: after finger leaves the screen, the location is selected.

Parameters:

- method – select touch or relase
  - ✧ PIN_TAP_BEHAVIOR_TOUCH – **default value**
  - ✧ PIN_TAP_BEHAVIOR_RELEASE

Returns: The encrypted result.

setCallback

public final void setCallback(CtKMS2Callback callback) **throws** CtKMS2Exception
This method is only used for startVirtualPin(Activity activity, int layoutID) method, user can override the callback method by implementing CtKMS2Callback class, when calling this method, it will register a callback service from callback server.

Parameters:

- callback – KMS defines some callback function. User can override it to create custom callback functions.

Throws: CtKMS2Exception– if the object obtains the Castles Key Management System II error code.

startSwipePin

public final byte[] startSwipePin(final CtKMS2SwipePINPad pinPad) **throws** CtKMS2Exception
This method can start get PIN process with Swipe PINpad which is used for people who is visually impaired.
Throws: CtKMS2Exception– if the object obtains the Castles Key Management System II error code.
Parameters:

- pinPad – CtKMS2SwipePINPad

Returns: Encrypted PINBlock.

startVirtualPin

public final byte[] startVirtualPin(CtKMS2VirtualPINPad pinPad) **throws** CtKMS2Exception
This method can start get PIN process with the selected CtKMS2VirtualPINPad.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Parameters:

- pinPad – selected CtKMS2VirtualPINPad

Returns: Encrypted PINBlock.
Notes: Only for terminals without physical keyboard.

startVirtualPin

public final byte[] startVirtualPin(Activity activity, RelativeLayout inlayout) **throws** CtKMS2Exception
This method can start get PIN process with the selected CtKMS2VirtualPINPad.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Parameters:

- activity – selected Activity
- inlayout – selected layout

Returns: Encrypted PINBlock.
Notes: Only for terminals without physical keyboard.

startVirtualPin

public final byte[] startVirtualPin(RelativeLayout inlayout) **throws** CtKMS2Exception
This method can start get PIN process with the selected CtKMS2VirtualPINPad.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Parameters:

- inlayout – selected layout

Returns: Encrypted PINBlock.
Notes: Only for terminals without physical keyboard.

startVirtualPin

public final byte[] startVirtualPin() **throws** CtKMS2Exception
This method can start get PIN process with the selected CtKMS2VirtualPINPad.

Castles Technology

Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Returns: Encrypted PINBlock.
Notes: Only for terminals without physical keyboard.

getPinFromKbd

public final byte[] getPinFromKbd() **throws** CtKMS2Exception
This method can get PIN from keyboard.
Throws: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.
Returns: Encrypted PINBlock.
Notes: Only for terminals with physical keyboard.

2.4.17.3 Example

setvirtualpinpad_para

```
private CtKMS2Callback callback = new CtKMS2Callback() {

  @Override
  public int testCancel() {
    return 0;
  }
  @Override
  public void onGetDigit(byte Digit) {
    Log.d(TAG, String.format(Locale.US, "NoDigits = %d, OnGetPINDigit.", Digit));
    switch(Digit){
      case 1:{
        sound.stop(playID);
        playID = sound.play(8, 1, 1, 0, 0, 1);
        break;
      }
      case 2:{
        sound.stop(playID);
        playID = sound.play(9, 1, 1, 0, 0, 1);
        break;
      }
      case 3:{
        sound.stop(playID);
        playID = sound.play(10, 1, 1, 0, 0, 1);
        break;
      }
      case 4:{
        sound.stop(playID);
        playID = sound.play(11, 1, 1, 0, 0, 1);
        break;
      }
      case 5:{
        sound.stop(playID);
        playID = sound.play(12, 1, 1, 0, 0, 1);
        break;
      }

      case 0:{
        sound.stop(playID);
        playID = sound.play(13, 1, 1, 0, 0, 1);
        break;
      }
    }
    final StringBuffer sb = new StringBuffer();
    sb.setLength(0);
    for (int i = 0; i < Digit; i++) {
      sb.append("*");
    }
    activity.runOnUiThread(new Runnable() {
      @Override
      public void run() {
        ((TextView) activity.findViewById(R.id.tvMsg)).setText("");
        ((EditText)
activity.findViewById(R.id.rlPINDigit).findViewById(R.id.etPinDigit)).setText(sb);
      }
    });
  }
  @Override
  public void onGetFunctionKey(byte FunctionKey) {
```

```
    Log.d(TAG, String.format(Locale.US, "FunctionKey = %d, OnGetPINOtherKeys.",
FunctionKey));
  }
  @Override
  public void onGetBlindModeStatus(byte status) {
    Log.d(TAG, String.format(Locale.US, "Status = %d, onGetBlindModeStatus.", status));
    activity.runOnUiThread(new Runnable() {
      @Override
      public void run() {
        ((TextView) activity.findViewById(R.id.tvMsg)).setText("");
        activity.findViewById(R.id.tvLeft).setVisibility(View.INVISIBLE);
        activity.findViewById(R.id.tvRight).setVisibility(View.INVISIBLE);
        activity.findViewById(R.id.tvTop).setVisibility(View.INVISIBLE);
        activity.findViewById(R.id.tvBottom).setVisibility(View.INVISIBLE);
        switch (status) {
          case SHIELD_OK: {
            if (shieldEnter == 0) {
              ((EditText)
activity.findViewById(R.id.rlPINDigit).findViewById(R.id.etPinDigit)).setText("");
              toneG.startTone(ToneGenerator.TONE_CDMA_MED_L, 100);
              shieldEnter = 1;
            }

activity.findViewById(R.id.rlPINDigit).findViewById(R.id.etPinDigit).setBackground(ContextCo
mpat.getDrawable(activity, R.color.Bright_Green));
            break;
          }
          case SHIELD_MISS: {

activity.findViewById(R.id.rlPINDigit).findViewById(R.id.etPinDigit).setBackground(ContextCo
mpat.getDrawable(activity, R.color.Bright_Red));
            break;
          }
          case SWIPE_CENTER_TOO_LEFT: {
            //activity.findViewById(R.id.tvLeft).setVisibility(View.VISIBLE);

            sound.stop(playID);
            playID = sound.play(4, 1, 1, 0, 0, 1);
            break;
          }
          case SWIPE_CENTER_TOO_RIGHT: {
            //activity.findViewById(R.id.tvRight).setVisibility(View.VISIBLE);

            sound.stop(playID);
            playID = sound.play(3, 1, 1, 0, 0, 1);
            break;
          }
          case SWIPE_CENTER_TOO_HIGH: {
            //activity.findViewById(R.id.tvTop).setVisibility(View.VISIBLE);

            sound.stop(playID);
            playID = sound.play(2, 1, 1, 0, 0, 1);
            break;
          }
          case SWIPE CENTER TOO LOW: {
            //activity.findViewById(R.id.tvBottom).setVisibility(View.VISIBLE);

            //sound.stop(playID);
            //playID = sound.play(1, 1, 1, 0, -1, 1);
            break;
          }
          case SWIPE_MULTI_TOUCH: {
            ((TextView) activity.findViewById(R.id.tvMsg)).setText("MULTITOUCH");
            sound.stop(playID);
            playID = sound.play(7, 1, 1, 0, 0, 1);
            break;
          }
          case SWIPE_TOUCH_UP: {
            ((TextView) activity.findViewById(R.id.tvMsg)).setText("Touch UP");
            //sound.stop(playID);
            //swipePINPad.removePINPad();
            break;
          }
          case SWIPE_KEY_SWITCH: {
            ((TextView) activity.findViewById(R.id.tvMsg)).setText("Key");
            sound.stop(playID);
            toneG.startTone(ToneGenerator.TONE_DTMF_S, 100);
            break;
          }
```

```
        case SWIPE_KEY_CANCEL: {
          ((TextView) activity.findViewById(R.id.tvMsg)).setText("Cancel");
          sound.stop(playID);
          sound.play(5, 1, 1, 0, 0, 1);
          break;
        }
        case SWIPE_KEY_OK: {
          ((TextView) activity.findViewById(R.id.tvMsg)).setText("OK");
          sound.stop(playID);
          sound.play(6, 1, 1, 0, 0, 1);
          break;
        }
        case SWIPE_PINPAD_LEFT: {
          ((TextView) activity.findViewById(R.id.tvMsg)).setText("SWIPE LEFT");
          sound.stop(playID);
          playID = sound.play(3, 1, 1, 0, 0, 1);
          break;
        }
        case SWIPE_PINPAD_RIGHT: {
          ((TextView) activity.findViewById(R.id.tvMsg)).setText("SWIPE RIGHT");
          sound.stop(playID);
          playID = sound.play(4, 1, 1, 0, 0, 1);
          break;
        }
        case SWIPE_PINPAD_UP: {
          ((TextView) activity.findViewById(R.id.tvMsg)).setText("SWIPE UP");
          sound.stop(playID);
          playID = sound.play(1, 1, 1, 0, 0, 1);
          break;
        }
        case SWIPE_PINPAD_DOWN: {
          ((TextView) activity.findViewById(R.id.tvMsg)).setText("SWIPE DOWN");
          sound.stop(playID);
          playID = sound.play(2, 1, 1, 0, 0, 1);
          break;
        }
      }
    }
  });
}
@Override
public void onSwipePINPad(int x, int y) {
  activity.runOnUiThread(new Runnable() {
    @Override
    public void run() {
      ((TextView) activity.findViewById(R.id.tvMsg)).setText("");
      activity.findViewById(R.id.tvLeft).setVisibility(View.INVISIBLE);
      activity.findViewById(R.id.tvRight).setVisibility(View.INVISIBLE);
      activity.findViewById(R.id.tvTop).setVisibility(View.INVISIBLE);
      activity.findViewById(R.id.tvBottom).setVisibility(View.INVISIBLE);
    }
  });

  sound.stop(playID);
  toneG.startTone(ToneGenerator.TONE DTMF S, 100);
  swipePINPad.showPINPad(x, y);
}
@Override
public void onNonOperation(byte Digit) {
  Log.d(TAG, String.format(Locale.US, "onNonOperationTime = %d.", Digit));
  activity.runOnUiThread(new Runnable() {
    @Override
    public void run() {
      switch(Digit)
      {
        case 1:{
          sound.stop(playID);
          playID = sound.play(17, 1, 1, 0, 0, 1);
          break;
        }
        case 2:{
          sound.stop(playID);
          playID = sound.play(18, 1, 1, 0, 0, 1);
          break;
        }
        case 3:{
          sound.stop(playID);
          playID = sound.play(19, 1, 1, 0, 0, 1);
          break;
```

```
          }
          case 4:{
            sound.stop(playID);
            playID = sound.play(20, 1, 1, 0, 0, 1);
            break;
          }
          case 5:{
            sound.stop(playID);
            playID = sound.play(21, 1, 1, 0, 0, 1);
            break;
          }
          case 0:{
            sound.stop(playID);
            playID = sound.play(22, 1, 1, 0, 0, 1);
            break;
          }
        }
      }
    });
  }
};

protected void onCreate(Bundle savedInstanceState) {
  byte PINMaxLen = 12;
  byte PINMinLen = 4;
  byte NullPIN = (byte) 0x00;
  int Timeout = 60;
  int nonOPTimeout = 10;
  int FirstKeyEntryTimeout = 0;
  CtKMS2OfflinePIN offlinepin = new CtKMS2OfflinePIN();
  offlinepin.setPinInfo(PINMaxLen, PINMinLen);
  offlinepin.setCallback(callback);
  offlinepin.setPinControl(NullPIN, Timeout, FirstKeyEntryTimeout,nonOPTimeout);
  activity.runOnUiThread(new Runnable() {
    @Override
    public void run() {
      activity.findViewById(R.id.rlPINDigit).setVisibility(View.VISIBLE);
      toneG = new ToneGenerator(AudioManager.STREAM_NOTIFICATION, 80);
      ((TextView) activity.findViewById(R.id.tvMsg)).setText("");
    }
  });
  swipePINPad = new CtKMS2SwipePINPad();
  swipePINPad.setMode(SWIPE_MODE_STATIC);
  swipePINPad.setPINPad(keyWidth, keyHeight);
  swipePINPad.setControl(clickWait, clickDelay, doubleClickTime);
  swipePINPad.setTypeface(tfArial);
  offlinepin.startSwipePin(swipePINPad);
  sound.stop(playID);
  playID = sound.play(14, 1, 1, 0, 0, 1);
  Log.d(TAG,"getOfflinepin successful!!!!!!!!");
}
```

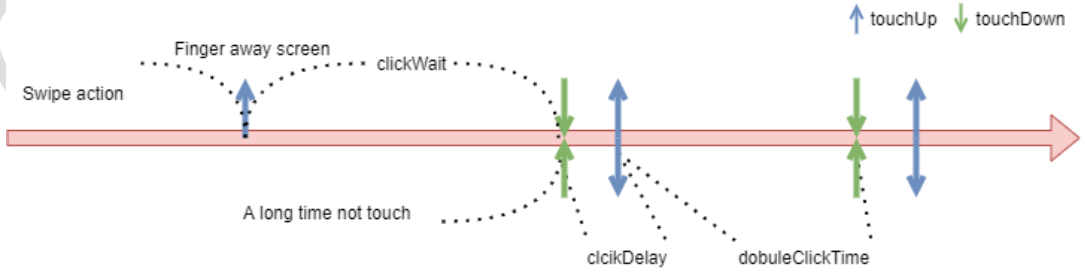
2.4.18 Class: CtKMS2SwipePINPad

public class **CtKMS2SwipePINPad**
The **CtKMS2SwipePINPad** class is used to set PIN Pad screen for visually impaired. User can set image and location of all buttons.

2.4.18.1 Public Constructors

CtKMS2CustomPINPadbyImage
public CtKMS2SwipePINPad() **throws** CtKMS2Exception Create a CtKMS2SwipePINPad object. **Throws**: CtKMS2Exception – if the object obtains the Castles Key Management System II error code.

2.4.18.2 Public Methods

setMode
public void setMode(int mode) The swipe PIN is used for visually impaired. **Parameters**: ➢ mode ✧ SWIPE_MODE_STATIC – generate a static PINPad. User swipe finger to know their finger position. ✧ SWIPE_MODE_FLOATING – Generate a PINPad from user first touching point.
setPINPad
public void setPINPad(int keyWidth, int keyHeight) This method can set key button size. **Parameters**: ➢ keyWidth – the default value is the one third of the width of screen ➢ keyHeight –the default value is the one third of the width of screen
setPINPad
public void setPINPad(int keyWidth, int keyHeight, int Show_X, int Show_Y) This method can set key button size. *Note. This function is applicable from KMS JAR v3.1.4.* **Parameters**: ➢ keyWidth – the default value is the one third of the width of screen ➢ keyHeight –the default value is the one third of the width of screen ➢ Show_X – the X-coordinate of upper-left corner of keyboard ➢ Show_Y – the Y-coordinate of upper-left corner of keyboard
setControl
public void setControl(int clickWait, int clickDelay, int doubleClickTime) This method can set timer for click option. The more detail, please refer to the KMS description.    **Parameters**: ➢ clickWait – the time between touch up and next time touch down. **Default value is 1500** ➢ clickDelay – the time of click. If over this time, system judges that click is valid. **Default value is 250** ➢ doubleClickTime – the time between two clicks. **Default value is 400**
setTypeface
public void setTypeface(Typeface tf, int style) This method can set the typeface for key button.

Castles Technology

Parameters:

- tf – the name of typeface in JAVA, define font. **Default value is Typeface.SANS_SERIF**
- style –the style of typeface in JAVA, define font. **Default value is Typeface.BOLD**

setTypeface

public void setTypeface(Typeface tf)
This method can set the typeface for key button.
Parameters:

- tf – the name of typeface in JAVA, define font. **Default value is Typeface.SANS_SERIF**

showPINPad

public void showPINPad(final int x, final int y)
The swipe PIN is used for visually impaired.
Parameters:

- x – the x-coordinate
- y – the y-coordinate

removePINPad

public void removePINPad()
This method is used for remove PINPad.

2.4.18.3 Example

CtKMS2SwipePINPad

```
private CtKMS2Callback callback = new CtKMS2Callback() {

    @Override
    public int testCancel() {
        return 0;
    }
    @Override
    public void onGetDigit(byte Digit) {
        Log.d(TAG, String.format(Locale.US, "NoDigits = %d, OnGetPINDigit.", Digit));

        final StringBuffer sb = new StringBuffer();
        sb.setLength(0);
        for (int i = 0; i < Digit; i++) {
            sb.append("*");
        }
        activity.runOnUiThread(new Runnable() {
            @Override
            public void run() {
                ((TextView) activity.findViewById(R.id.tvMsg)).setText("");
                ((EditText) activity.findViewById(R.id.rlPINDigit).findViewById(R.id.etPinDigit)).setText(sb);
            }
        });
    }
    @Override
    public void onGetFunctionKey(byte FunctionKey) {
        Log.d(TAG, String.format(Locale.US, "FunctionKey = %d, OnGetPINOtherKeys.", FunctionKey));
    }
    @Override
    public void onGetBlindModeStatus(byte status) {
        Log.d(TAG, String.format(Locale.US, "Status = %d, onGetBlindModeStatus.", status));
        activity.runOnUiThread(new Runnable() {
            @Override
            public void run() {
                ((TextView) activity.findViewById(R.id.tvMsg)).setText("");
                activity.findViewById(R.id.tvLeft).setVisibility(View.INVISIBLE);
                activity.findViewById(R.id.tvRight).setVisibility(View.INVISIBLE);
                activity.findViewById(R.id.tvTop).setVisibility(View.INVISIBLE);
                activity.findViewById(R.id.tvBottom).setVisibility(View.INVISIBLE);
                switch (status) {
                    case SHIELD_OK: {
                        if (shieldEnter == 0) {
```

```
                    ((EditText) activity.findViewById(R.id.rlPINDigit).findViewById(R.id.etPinDigit)).setText("");
                    toneG.startTone(ToneGenerator.TONE_CDMA_MED_L, 100);
                    shieldEnter = 1;
                  }
                  activity.findViewById(R.id.rlPINDigit).findViewById(R.id.etPinDigit).setBackground(ContextCompat.getDrawable(activity,
R.color.Bright_Green));
                  break;
                }
                case SHIELD_MISS: {
                  activity.findViewById(R.id.rlPINDigit).findViewById(R.id.etPinDigit).setBackground(ContextCompat.getDrawable(activity,
R.color.Bright_Red));
                  break;
                }
                case SWIPE_CENTER_TOO_LEFT: {
                  activity.findViewById(R.id.tvLeft).setVisibility(View.VISIBLE);
                  sound.stop(playID);
                  playID = sound.play(4, 1, 1, 0, -1, 1);
                  break;
                }
                case SWIPE_CENTER_TOO_RIGHT: {
                  activity.findViewById(R.id.tvRight).setVisibility(View.VISIBLE);
                  sound.stop(playID);
                  playID = sound.play(3, 1, 1, 0, -1, 1);
                  break;
                }
                case SWIPE_CENTER_TOO_HIGH: {
                  activity.findViewById(R.id.tvTop).setVisibility(View.VISIBLE);
                  sound.stop(playID);
                  playID = sound.play(2, 1, 1, 0, -1, 1);
                  break;
                }
                case SWIPE_CENTER_TOO_LOW: {
                  activity.findViewById(R.id.tvBottom).setVisibility(View.VISIBLE);
                  sound.stop(playID);
                  playID = sound.play(1, 1, 1, 0, -1, 1);
                  break;
                }
                case SWIPE_MULTI_TOUCH: {
                  ((TextView) activity.findViewById(R.id.tvMsg)).setText("MULTITOUCH");
                  sound.stop(playID);
                  playID = sound.play(7, 1, 1, 0, -1, 1);
                  break;
                }
                case SWIPE_TOUCH_UP: {
                  ((TextView) activity.findViewById(R.id.tvMsg)).setText("Touch UP");
                  sound.stop(playID);
                  swipePINPad.removePINPad();
                  break;
                }
                case SWIPE_KEY_SWITCH: {
                  ((TextView) activity.findViewById(R.id.tvMsg)).setText("Key");
                  sound.stop(playID);
                  toneG.startTone(ToneGenerator.TONE_DTMF_S, 100);
                  break;
                }
                case SWIPE_KEY_CANCEL: {
                  ((TextView) activity.findViewById(R.id.tvMsg)).setText("Cancel");
                  sound.stop(playID);
                  sound.play(5, 1, 1, 0, 0, 1);
                  break;
                }
                case SWIPE_KEY_OK: {
                  ((TextView) activity.findViewById(R.id.tvMsg)).setText("OK");
                  sound.stop(playID);
                  sound.play(6, 1, 1, 0, 0, 1);
                  break;
                }
                case SWIPE_PINPAD_LEFT: {
                  ((TextView) activity.findViewById(R.id.tvMsg)).setText("SWIPE LEFT");
                  sound.stop(playID);
                  playID = sound.play(3, 1, 1, 0, -1, 1);
                  break;
                }
```

```
                case SWIPE_PINPAD_RIGHT: {
                    ((TextView) activity.findViewById(R.id.tvMsg)).setText("SWIPE RIGHT");
                    sound.stop(playID);
                    playID = sound.play(4, 1, 1, 0, -1, 1);
                    break;
                }
                case SWIPE_PINPAD_UP: {
                    ((TextView) activity.findViewById(R.id.tvMsg)).setText("SWIPE UP");
                    sound.stop(playID);
                    playID = sound.play(1, 1, 1, 0, -1, 1);
                    break;
                }
                case SWIPE_PINPAD_DOWN: {
                    ((TextView) activity.findViewById(R.id.tvMsg)).setText("SWIPE DOWN");
                    sound.stop(playID);
                    playID = sound.play(2, 1, 1, 0, -1, 1);
                    break;
                }
            }
        }
    });
  }
  @Override
  public void onSwipePINPad(int x, int y) {
    Log.d(TAG,"onSwipePINPad" + x + y);
    activity.runOnUiThread(new Runnable() {
      @Override
      public void run() {
        ((TextView) activity.findViewById(R.id.tvMsg)).setText("");
        activity.findViewById(R.id.tvLeft).setVisibility(View.INVISIBLE);
        activity.findViewById(R.id.tvRight).setVisibility(View.INVISIBLE);
        activity.findViewById(R.id.tvTop).setVisibility(View.INVISIBLE);
        activity.findViewById(R.id.tvBottom).setVisibility(View.INVISIBLE);
      }
    });
    sound.stop(playID);
    toneG.startTone(ToneGenerator.TONE_DTMF_S, 100);
    swipePINPad.showPINPad(x, y);
  }
};

protected void onCreate(Bundle savedInstanceState) {
  int KeySet = 0xCC06;
  int KeyIndex = 0x0001;
  byte CipherMethod = PIN_CIPHER_METHOD_ECB;
  byte PINBlockType = PIN_BLOCKTYPE_ANSI_X9_8_ISO_0;
  byte PINMaxLen = 12;
  byte PINMinLen = 4;
  byte NullPIN = (byte) 0x00;
  int Timeout = 200;
  byte[] PAN = {(byte) 0x31, (byte) 0x32, (byte) 0x33, (byte) 0x34, (byte) 0x35, (byte) 0x36, (byte) 0x37, (byte) 0x38, (byte) 0x39, (byte)
0x31, (byte) 0x32, (byte) 0x33, (byte) 0x34};
  int FirstKeyEntryTimeout = 0;
  CtKMS2FixedKey fixedKey = new CtKMS2FixedKey();
  fixedKey.selectKey(KeySet, KeyIndex);
  fixedKey.setCipherMethod(CipherMethod);
  fixedKey.setPinInfo(PINBlockType, PINMaxLen, PINMinLen);
  fixedKey.setCallback(callback2);
  fixedKey.setPinControl(NullPIN, Timeout, FirstKeyEntryTimeout);
  fixedKey.setPAN(PAN);
  activity.runOnUiThread(new Runnable() {
    @Override
    public void run() {
      activity.findViewById(R.id.rlPINDigit).setVisibility(View.VISIBLE);
      toneG = new ToneGenerator(AudioManager.STREAM_NOTIFICATION, 80);
      ((TextView) activity.findViewById(R.id.tvMsg)).setText("");
    }
  });
  swipePINPad = new CtKMS2SwipePINPad();
  swipePINPad.setMode(SWIPE_MODE_FLOATING);
  swipePINPad.setPINPad(keyWidth, keyHeight);
  swipePINPad.setControl(clickWait, clickDelay, doubleClickTime);
  swipePINPad.setTypeface(tfArial);
```

```
    fixedKey.startSwipePin(swipePINPad);
    byte[] output = fixedKey.getOutpuData();
    msg = "PINBlock: " + getHexString(output);
    Log.d(TAG, msg);
}
```

2.4.19 Class: CtKMS2Callback

public class CtKMS2Callback
The CtKMS2Callback class provides the callback interface functionality of get PIN of KMS2. When user uses the class to set callback, terminal will use default PIN sound, PIN scrambling and PIN moving by setting setPINSound(boolean bool), setPINPADMoving(boolean bool) and setPINScrambling(boolean bool) on CtKMS2System.

Method Summary	
int	testCancel() The user initiative method to cancel get pin process.
Void	onGetDigit(byte digit); On get digit length changing event, it will be triggered.
Void	onGetFunctionKey(byte functionkey) On get function key push event, it will be triggered.

Method Detail
int testCancel() Once this function return non-zero, the get pin process will cancel, therefore, the user can control it according to how it designs.
void onGetDigit(byte digit) If user pushes digit or backspace, it will call this method and use the current length of input digit as parameter. **Parameters:** digit – the current length of input digit.
void onGetFunctionKey(byte functionkey) If user push function button, it will call this method and use the value of function key as parameter. **Parameters:** functionkey – the function key that be push.

Example

```
/* Callback function */
private CtKMS2Callback callback = new CtKMS2Callback() {
        @Override
        public int testCancel() {
            return 0;
        }

        @Override
        public void onGetDigit(byte Digit) {
            Log.d(TAG, "CtKMS2Callback onGetDigit");
            Log.d(TAG, String.format(Locale.US, "NoDigits = %d, OnGetPINDigit.", Digit));

            final StringBuffer sb = new StringBuffer();
            sb.setLength(0);
            for (int i = 0; i < Digit; i++) {
                sb.append("*");
            }

            activity.runOnUiThread(new Runnable() {
                @Override
                public void run() {

                    ((EditText) activity.findViewById(R.id.rlPINDigit).findViewById(R.id.etPinDigit)).setText(sb);

                }
```

Castles Technology

```
                });

            }

            @Override
            public void onGetFunctionKey(byte FunctionKey) {
                Log.d(TAG, String.format(Locale.US, "FunctionKey = %d, OnGetPINOtherKeys.", FunctionKey));
            }
};
/* fixedkey startVirtualPin*/
try {
/* Set related parameters */
int KeySet = 0xC101;
int KeyIndex = 0x0001;
byte CipherMethod = PIN_CIPHER_METHOD_ECB;
byte PINBlockType = PIN_BLOCKTYPE_ANSI_X9_8_ISO_0;
byte PINMaxLen = 12;
byte PINMinLen = 4;
byte PINBlockLen = 8;
byte NullPIN = (byte) 0x00;
int Timeout = 10;
byte[] PAN = {(byte) 0x31, (byte) 0x32, (byte) 0x33, (byte) 0x34, (byte) 0x35, (byte) 0x36, (byte) 0x37, (byte) 0x38, (byte) 0x39, (byte)
0x31, (byte) 0x32, (byte) 0x33, (byte) 0x34};
int FirstKeyEntryTimeout = 0;

CtKMS2System system = new CtKMS2System();
system.setPINSound(true);
CtKMS2FixedKey fixedKey = new CtKMS2FixedKey();
fixedKey.selectKey(KeySet, KeyIndex);
fixedKey.setCipherMethod(CipherMethod);
fixedKey.setPinInfo(PINBlockType, PINMaxLen, PINMinLen, PINBlockLen);
fixedKey.setCallback(callback);
fixedKey.setPinControl(NullPIN, Timeout, FirstKeyEntryTimeout);
fixedKey.setInputData(PAN, 0, PAN.length);
activity.runOnUiThread(new Runnable() {
@Override
   public void run() {
        activity.findViewById(R.id.rlPINDigit).setVisibility(View.VISIBLE);
   }
});
            byte[] output = fixedKey.startVirtualPin((RelativeLayout)activity.findViewById(R.id.rlPINPAD));
} catch(CtKMS2Exception e){
                //exception handle
}
```

Castles

2.4.20 Class: CtKMS2Exception

public class **CtKMS2Exception**
The **CtKMS2Exception** class provides the functionality of get error message when an exception occurs.

Field Summary			
Modifier and Type	Constant Field	Description	Value
public static final int	OK	OK	0x0000
public static final int	INVALID_PARA	The parameter is invalid	0x2901
public static final int	FAIL	General Failure	0x2902
public static final int	SYSTEM_ERROR	System Error	0x2903
public static final int	NOT_OWNER	The key does not belong to this application	0x2904
public static final int	KEY_NOT_EXIST	The key does not exist	0x2905
public static final int	KEYTYPE_INCORRECT	The key type is incorrect	0x2906
public static final int	KEY_NOT_ALLOWED	The key attribute is not allowed to use this operation	0x2907
public static final int	KEY_VERIFY_INCORRECT	The verification code is incorrect	0x2908
public static final int	NOT_SUPPORTED	The function (with the input argument) is not supported	0x2909
public static final int	INSUFFICIENT_BUFFER	The buffer is insufficient	0x290D
public static final int	DUKPT_KEY_NOT_GENERATED	The DUKPT key has not yet been generated.	0x290E
public static final int	GET_PIN_ABORT	User presses CANCEL key during getting PIN	0x290F
public static final int	GET_PIN_TIMEOUT	Timeout occurs during getting PIN	0x2910
public static final int	GET_PIN_NULL_PIN	User enters empty PIN.	0x2911
public static final int	KEY_VALUE_NOT_UNIQUE	Key value is not unique.	0x2913
public static final int	KEY_TYPE_NOT_MATCH	The key type between source and destination are different.	0x2914
public static final int	DUKPT_KEY_EXPIRED	The KSN reaches the maximum value.	0x2915
public static final int	PURPOSE_NOT_UNIQUE	The purpose of key attribute is not unique.	0x2916
public static final int	PINPAD_WIDTH_HEIGHT_NULL	The parameter of custom virtual PIN Pad is invalid	0x2917
public static final int	JAVAKMS_INVALID	KMS jar is incorrect	0x29B0
public static final int	GETIKMS2API_FAIL	KMS jar is incorrect	0x29B1
public static final int	GETIKMS2REMOTE_FAIL	KMS jar is incorrect	0x29B2
public static final int	JAR_ERROR	KMS jar is incorrect	0x29B3
public static final int	PERMISSION_DENIED	If device active “Access Control” feature, the limited method will throw this exception.	0x29B4

Method Summary	
int	getErrorCode() Get error code
int	getErrorBlockID() Get erro block ID
String	getMessage() Get error message
String	toString() Get error message

Method Detail
public int getErrorCode() This method can get the error code after CtKMS2 function fail.
public int getErrorBlockId() This method can get the error block ID after CtKMS2 function fail.
public String getMessage () This method can get error message which shows detail of error after CtKMS2 function fail.
public String toString() This method can get error message which shows detail of error after CtKMS2 function fail and append TAG to the error message.

Example

try {
int KeySet = 0xC101;
int KeyIndex = 0x0008;

Castles Technology

```
            int KeySetLocation = 0x0001;
            int KeyIndexLocation = 0x0013;
            byte[] baTR31KeyBlock = {0x42, 0x30, 0x30, 0x38, 0x30, 0x50, 0x30, 0x54,
                  0x45, 0x30, 0x31, 0x4E, 0x30, 0x30, 0x30, 0x30,
                  0x41, 0x36, 0x36, 0x42, 0x37, 0x35, 0x39, 0x34,
                  0x43, 0x35, 0x43, 0x30, 0x42, 0x32, 0x33, 0x43,
                  0x46, 0x37, 0x36, 0x46, 0x31, 0x41, 0x30, 0x41,
                  0x43, 0x38, 0x41, 0x41, 0x32, 0x44, 0x38, 0x32,
                  0x38, 0x32, 0x39, 0x37, 0x36, 0x39, 0x45, 0x33,
                  0x33, 0x45, 0x32, 0x35, 0x42, 0x33, 0x34, 0x30,
                  0x37, 0x37, 0x35, 0x46, 0x36, 0x30, 0x44, 0x30,
                  0x37, 0x41, 0x31, 0x31, 0x31, 0x42, 0x34, 0x43};

                  CtKMS2TR31 tr31 = new CtKMS2TR31();
                  tr31.selectKey(KeySet, KeyIndex);
                  tr31.setKeyLocation(KeySetLocation, KeyIndexLocation);
                  tr31.setKeyBlock(baTR31KeyBlock);
                  tr31.writeKey();
} catch(CtKMS2Exception e){
            int rtn = e.getErrorCode();
            int id = e.getBlockId();
            String msg = e.getMessage();
            String log = e.toString();
}
```

Castles Technology

2.5 CRYPTO Application Programming Interfaces

Package CTOS
Contains all of the classes for Castles user api.

Class Summary	
CtCrypto	This class provides the functionality of management of CRYPTO

Exception Summary	
CtCryptoException	CRYPTO error collector.

2.5.1 Class: CtCrypto

public class **CtCrypto**
The **CtCrypto** class is used to provide the management functionality of Castles Android Crypto.

Constructor Summary	
public	CtCrypto() Constructs a binder connected with service.

Field Summary			
Modifier and Type	Constant Field	Description	Value
public static final byte	ENCRYPT_MODE	Data encrypt	0x00
public static final byte	DECRYPT_METHOD_CBC	Data decrypt	0x01

Method Summary	
byte[]	rng(int size) generate Random number.
byte[]	des_ecb(byte mode, byte[] key, byte[] data) Execute des ecb.
byte[]	des_cbc(byte mode, byte[] key, byte[] data, byte[] icv) Execute des_cbc.
byte[]	aes_ecb(byte mode, byte[] key, byte[] data) Execute aes ecb.
byte[]	aes_cbc(byte mode, byte[] key, byte[] data, byte[] icv) Execute aes_cbc.

Constructor Detail
public CtCrypto() Create a CtCrypto object and connect with binder to invoke service.

Method Detail
public final byte[] rng(int size) throws CtCryptoException This method using to generate random number. **Parameters:** size – the size of generating number, must > 0 & <= 2048 **throws:** CtCryptoException - if the object obtains the Castles Crypto error code. **Note:** This function does not erase the keys in key storage.
public final byte[] des_ecb(byte mode, byte[] key, byte[] data) throws CtCryptoException This method is used to encrypt/decrypt data by DES ECB. **Parameters:** mode –ENCRYPT_MODE or DECRYPT_MODE key – the des key, length must be 8, 16 or 24 data – the input data, must be 8 multiple and <= 2048

Castles Technology

Throws:
CtCryptoException - if the object obtains the Castles Crypto error code.

public final byte[] des_cbc(byte mode, byte[] key, byte[] data, byte[] icv)
throws CtCryptoException
This method is used to encrypt/decrypt data by DES CBC.
Parameters:
mode –ENCRYPT_MODE or DECRYPT_MODE
key – the des key, length must be 8, 16 or 24
data – the input data, must be 8 multiple and <= 2048
icv – the initial vector, length be 8

Throws:
CtCryptoException - if the object obtains the Castles Crypto error code.

public final byte[] aes_ecb(byte mode, byte[] key, byte[] data)
throws CtCryptoException
This method is used to encrypt/decrypt data by AES ECB.
Parameters:
mode –ENCRYPT_MODE or DECRYPT_MODE
key – the des key, length must be 16, 24 or 32
data – the input data, must be 8 multiple and <= 2048

Throws:
CtCryptoException - if the object obtains the Castles Crypto error code.

public final byte[] aes_cbc(byte mode, byte[] key, byte[] data, byte[] icv)
throws CtCryptoException
This method is used to encrypt/decrypt data by AES CBC.
Parameters:
mode –ENCRYPT_MODE or DECRYPT_MODE
key – the des key, length must be 16, 24 or 32
data – the input data, must be 8 multiple and <= 2048
icv – the initial vector, length be 16

Throws:
CtCryptoException - if the object obtains the Castles Crypto error code.

Example

```
CtCrypto crypto = new CtCrypto();
try {
        byte[] random = crypto.rng(8);
       /**
        * assign some key , data and icv.
        */
        byte[] des_ecb = crypto.des_ecb(CtCrypto.ENCRYPT_MODE, key, data);
        byte[] des_cbc = crypto.des_cbc(CtCrypto.ENCRYPT_MODE, key, data, icv);
        byte[] aes_ecb = crypto.aes_ecb(CtCrypto.ENCRYPT_MODE, key, data);
        byte[] aes_cbc = crypto.aes_cbc(CtCrypto.ENCRYPT_MODE, key, data, icv);

} catch(CtCryptoException e){
        //exception handle
}
```

2.5.2 Class: CtCryptoException

public class **CtCryptoException**

The **CtCryptoException** class is used for CtCrypto error collector.

Field Summary

Modifier and Type	Constant Field	Description	Value
public static final int	OK	OK	0x0000
public static final int	Crypto_INVALID_PARA	The parameter is invalid	0x2901
public static final int	Crypto_HALT_FAULT	Halt fault.	0x2902

Constructor Summary

public	CtCryptoException() Constructs a binder connected with service.

Castles Technology

Method Summary	
int	getError() Get error status

Constructor Detail

public CtCryptoException(int ret)
Create a CtCryptoException object and to collect error information.

Method Detail

public int getError()
This method is used to get error status.

Example

```
CtCrypto crypto = new CtCrypto();
try {
                byte[] random = crypto.rng(8);
              /**
               * assign some key , data and icv.
               */
               byte[] des_ecb = crypto.des_ecb(CtCrypto.ENCRYPT_MODE, key, data);
               byte[] des_cbc = crypto.des_cbc(CtCrypto.ENCRYPT_MODE, key, data, icv);
               byte[] aes_ecb = crypto.aes_ecb(CtCrypto.ENCRYPT_MODE, key, data);
               byte[] aes_cbc = crypto.aes_cbc(CtCrypto.ENCRYPT_MODE, key, data, icv);

} catch(CtCryptoException e){
            Log.e(TAG, Integer.toHexString(e.getError()));
}
```

Castles Technology

3. EMV

3.1 CAEMV Application Programming Interfaces

Package CTOS
Contains all of the classes for Castles EMV user API.

Class Summary	
CtEMV	This class provides the functionality to accomplish EMV contact transaction.
CtEMVMSR	This class provides the functionality to accomplish capture of encrypted MSR.
CtEMVManualEntry	This class provides the functionality to accomplish manual entry.

Package CTOS.emv
Contains all of the classes, which are used in methods of CtEMV.

Class Summary	
EMVEvent	This class consists of a set of callback functions used by CAEMV.
EMVAppInfo	This class consists of a set of variables and is used in method txnAppSelect of CtEMV.
EMVOnlineResponseData	This class consists of a set of variables and is used in method txnCompletion of CtEMV.
EMVTxnData	This class consists of a set of variables and is used in callback event onTxnDataGet of EMVEvent.
EMVAppListExData	This class consists of a set of variables and is used in callback event onAppListEx of EMVEvent.
EMVGetPINFuncPara	This class consists of a set of variables and is used in callback event onGetPINNotify of EMVEvent.
EMVVirtualPINPara	This class consists of a set of variables and is used in callback event IEventShowVirtualPIN.
EMVCandidateList	This class consists of a set of variables and is used in method txnCandidateList of CtEMV.
EMVCandidateData	This class consists of a set of variables of candidate data.
EMVTxnPSERsp	This class consists of a set of variables and is used in method txnPSERsp of CtEMV.
EMVCandidateDirEntry	This class consists of a set of variables and is used in method txnCandidateDirEntry of CtEMV.
EMVSecureDataInfo	This class consists of a set of variables and is used in method secureDataEncryptInfoSet of CtEMV.
TlvData	This class consists of a set of variables of TLV data.
EMVDataGetExPara	This class consists of a set of variables and is used in method dataGetEx of CtEMV.
EMVOnlinePinData	This class consists of a set of variables and is used in interface IeventOnlinePinBlockGet of CtEMV.
EMVApplicationPara	This class consists of a set of variables and is used in method applicationListSet of CtEMV
CAPublicKey	This class consists of a set of variables and is used in method capkSet of CtEMV
EMVByteBufData	This class consists of byte array and length.
EMVCardAcquisitionData	This class consists of a set of variables and is used in method txnCardAcquisitionAppDataGet of CtEMV.
EMVConfigData	This class consists of a set of variables and is used in method setConfigurationEx of CtEMV
CAPublicKeyEx	This class consists of a set of variables and is used in method capkSetEx of CtEMV

Package CTOS.emv
Contains all of the classes, which are used in methods of CtEMVMSR.

Class Summary	
EMVMSRTracksLen	This class consists of a set of variables and is used in method getTracksLen of CtEMVMSR.
EMVMSREncryptedTracks	This class consists of a set of variables and is used in method getEncrptedTracks of CtEMVMSR.
EMVMSRMaskedTracks	This class consists of a set of variables and is used in method getMaskedTracks of CtEMVMSR.

Castles Technology

Package CTOS.emv

Contains all of the classes, which are used in methods of CtEMVManualEntry.

Class Summary	
EMVManualEntryEvent	This class consists of a set of callback functions used by CAEMVManualEntry.
EMVManualEntryEncryptedData	This class consists of a set of variables and is used in method getEncryptedManualEntry of CtEMVManualEntry.

Note: For the EMV example, please refer to the example which contains to the SDK.
The file path is "C:\Program Files (x86)\Castles\SATURN1000\examples\EMVL2_SampleAP".

Castles Technology

3.1.1 Class: CtEMV

public class **CtEMV**

The **CtEMV** class provides the functionalities to accomplish EMV contact transaction.

Constructor Summary	
public	CtEMV() Constructs a binder connected with service.

Fields Summary	
static byte	d_EVENTID_FIRSTGENAC_TERMDECISION Special Event Id used in method specialEventRegister.
static byte	d_EVENTID_TXN_FORCED_ONLINE Special Event Id used in method specialEventRegister.
static byte	d_EVENTID_SHOW_VIRTUAL_PIN Special Event Id used in method specialEventRegister.
static byte	d_EVENTID_GET_PIN_DONE Special Event Id used in method specialEventRegister.
static byte	d_EVENTID_SHOW_VIRTUAL_PIN_EX Special Event Id used in method specialEventRegister.
static byte	d_EVENTID_ONLINE_PINBLOCK_GET Special Event Id used in method specialEventRegister.
static byte	d_EVENTID_OFFLINE_PIN_VERIFY_RESULT Special Event Id used in method specialEventRegister.
static byte	d_EVENTID_PIN_BYPASS Special Event Id used in method specialEventRegister.
static byte	d_EVENTID_TXN_FORCED_ACCEPTANCE Special Event Id used in method specialEventRegister.
static byte	d_EVENTID_APP_LIST_ALWAYS Special Event Id used in method specialEventRegister.

Method Summary	
int	initialize(EMVEvent event) Initalize.
int	setConfiguration(InputStream inStream) Set Configuration. The tags and CAPKs …etc. in the configuration data will be loaded to EMV kernel.
int	setConfiguration (String filename, InputStream inStream) The configuration file is sent from android app to the EMV kernel in the secure module. The tags and CAPKs …etc. in the configuration file will be loaded to EMV kernel.
int	setConfiguration (String filename) Inform the secure module to load configuration with the input file name
int	setConfigurationFile (String filename, InputStream inStream) The configuration file is sent from android app to the EMV kernel in the secure module. The configuration will NOT be loaded to the EMV kernel.
int	setConfigurationEx (EMVConfigData configData) The configuration file is sent from android app to the EMV kernel in the secure module. There are conditions to select specific configuration and load it to EMV kernel.
int	delConfigurationFileAll () Delete all configuration files stored in the EMV kernel.
int	delConfigurationFile (String filename) Detete the specific configuration file.
int	txnAppSelect (EMVAppInfo appInfo) The first step of performing transaction is Select Application after card inserted.
int	txnPerform () Actions based EMV specification, such as Initiate Application Processing, Read Application Data, Offline Authentication, Processing restrictions, Cardholder Verification, Terminal Risk Management, First Generate AC.
int	txnCompletion (EMVOnlineResponseData onlRspData) Actions after Online Processing, such as External Authentication, Second Generate AC, Issuer Script Processing.
int	dataGet(TlvData tlvData) Get specific tag data (terminal data or card data) in the current transaction from EMV kernel.
int	txnDataSet(TlvData tlvData) Set transaction-related data to EMV kernel during transaction.
int	txnDataSetEx(byte[] tlvBuf, int tlvLen) Set multiple transaction-related data to kernel during transaction.
int	txnPreDataSet (byte[] tlvBuf, int tlvLen) Pre-set tags prior to transaction. The pre-setting tags will be added to kernel before GPO.
int	cdaModeSet(byte mode) Set CDA Mode.

Castles Technology

int	subsequentPINBypassSupport(boolean isSupported) Subsequent PIN bypass is supported or not.
int	specialEventRegister(byte eventId, IEMVSpecialEvent spEvent) Register special event (callback function) to EMVkernel, provided by the user application using.
int	txnCandidateList(EMVCandidateList candidateList) Build candidate list
int	txnFinalSelection(int appIdx) Select application, Initiate Application Processing and Read Application Data.
int	txnPSERsp(EMVTxnPSERsp rsp) Get PSE response.
int	txnCandidateDirEntry(EMVCandidateDirEntry candidateDirEntry) Get entry data of candidate.
int	secureDataEncryptInfoSet(EMVSecureDataInfo info) Set encryption method of security data.
int	secureDataWhitelistSet (byte action, byte[] listData, int listDataLen) Set whitelist of security data.
int	setMaskChar(byte maskChar) Set mask character of security data.
int	maskPANDigits1To6(byte maskDigits) Digits 1 to 6 of PAN are masked or not.
int	maskPANDigits13To16(byte maskDigits) Digits 13 to 16 of PAN are masked or not.
int	maskEXPDigits1To4(byte maskDigits) Digits 1 to 4 of expiration date of track 2 data are masked or not.
int	maskCSCDigits1To3 (byte maskDigits) Digits 1 to 3 of card service code of track 2 data are masked or not.
int	dataGetEx(EMVDataGetExPara reqData) Get multiple tag data (terminal data or card data) in the current transaction from EMV kernel.
int	applicationListSet (byte aidNum, EMVApplicationPara[] aidPara) Set AIDs
int	capkSet (byte[] rid, CAPublicKey capk) Set CAPK
int	capkSetEx (int capkNum, CAPublicKeyEx[] capks) Set multiple CAPKs at once
int	capkDelete (byte[] rid, byte capkIndex) Delete specified CAPK
int	capkDeleteAll () Delete all of CAPKs
int	terminalConfigSet (byte tlvDataNum, TlvData[] tlvData) Set terminal tags
int	terminalConfigDelete (byte tagNum, int[] tags) Delete specified terminal tags
int	terminalConfigDeleteAll () Delete all of terminal tags.
int	appConfigSet (byte[] aid, byte aidLen, byte tlvDataNum, TlvData[] tlvData) Set tags that are belong to the specified AID
int	appConfigDelete (byte[] aid, byte aidLen, byte tagNum, int[] tags) Delete tags that are belong to the specified AID
int	appConfigDeleteAll () Delete all of tags that are belong to the specified AID
int	partialTransactionSet (int type) Set type of partial transaction After reading card data, requesting AAC at the 1st Generate AC to complete transaction.
int	partialTransactionGet () Get current type of partial transaction
int	getLastError () Get detail error code
int	enableNullPIN () For offline PIN only. PIN entry is able to be null.
int	disableNullPIN () For offline pin only. PIN entry is NOT able to be null.
int	enablePINSound () For offline PIN only. Enable PIN sound. Sound when a PIN is pressed.
int	disablePINSound () For offline PIN only. Disable PIN sound.
void	setPINPADMoving (Boolean bool) For offline PIN & eventShowVirtualPIN() only. Set randomly moving PINPAD during get PIN function.
void	setPINScrambling (Boolean bool) For offline PIN & eventShowVirtualPIN() only. Set scramble digital button during get PIN function.
int	preferredOrderSet (EMVByteBufData preferredOrder) Set preferred order of card application (AID). In application selection phase, the candidate list is rearranged by the preferred order.
int	preferredOrderGet (EMVByteBufData preferredOrder) Get preferred order data.
int	appFilteringParaSet (EMVByteBufData para) Candidate list filtering. Disable specific card applications before application selection.

Castles Technology

int	appFilteringParaGet (EMVByteBufData para) Get Application Filtering Parameter.
int	txnCardAcquisitionRead () The EMV kernel performs card acquisition. Each AID in the card, collecting card tags from response of SELECT AID command to response of READ RECORD command. For response of SELECT PSE command, it will be returned to txnPSERsp()
int	txnCardAcquisitionAppNumGet () Get the number of card acquisition data (the same as AID number in the card). Call this after txnCardAcquisitionRead().
int	txnCardAcquisitionAppDataGet (EMVCardAcquisitionData[] acq) Input buffer to get card acquisition data. Call this after txnCardAcquisitionRead().
int	cancelVirtualPIN () For offline PIN only. Cancel PIN entry.
int	changeWDBPath () The EMV kernel default set the working database path to the Android Application ID and store the files such as the xml configuration files in it. Each Android App has its configurations. This method is used to change the working database path If the developer would like to share the configurations between multiple Android App.
int	getWDBPath () Get the current working database path. If the developer does not change WDB path, the default working database path is the string of Android Application ID.
int	resetWDBPath () Reset the working database path to default.
String	getRtnString () This method is used after specific methods to get String variable.
int	verifyCommandSW12Get () For Offline PIN verification. Obtain the status and SW12 of Verify Command.

Interface Summary

IEMVSpecialEvent	Tag interface for special event.
IEventFirstGenACTermDecision	Special event : terminal decision in First GenAC.
IEventTxnForcedOnline	Special event : transaction forced online
IEventShowVirtualPIN	Special event : show virtual PIN pad on the screen.
IEventGetPINDone	Special event : get information of completing PIN entry.
IEventShowVirtualPINEx	Special event : show virtual PIN pad on the screen, return keyboard information.
IEventOnlinePinBlockGet	Special event : online PIN processing.
IEventOfflinePINVerifyResult	Special event : get the result of verifying offline PIN
IEventPINBypass	Special event : action of PIN Bypass
IEventForcedAcceptance	Special event : action to determine if the transaction is forced acceptance
IEventAppListEx	Special event : list candidate for application selection

Constructor Detail

public CtEMV()
Create a CtEMV object and connect with binder to invoke service.

Method Detail

public int initialize(EMVEvent event)

This method initializes the EMV environment and sets callback events.
Parameters:
[IN] event – Class EMVEvent that implements mandatory callback events.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int setConfiguration(InputStream inStream)

This method is used to set configuration, such like AIDs, Tags, CAPK.
Parameters:
[IN] inStream – Stream of configuration file.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int setConfiguration(String filename, InputStream inStream)

The configuration file is sent from android app to the EMV kernel in the secure module. The tags and CAPKs …etc. in the configuration file will be loaded to EMV kernel.
Parameters:
[IN] filename – File name of configuration file.

Castles Technology

[IN] inStream – Stream of configuration file.

Returns:

d_EMVAPLIB_OK – Success
Other – Error code

public int setConfiguration(String filename)

Inform the secure module to load configuration with the input file name.

Parameters:

[IN] filename – File name of configuration file.

Returns:

d_EMVAPLIB_OK – Success
Other – Error code

public int setConfigurationFile(String filename, InputStream inStream)

The configuration file is sent from android app to the EMV kernel in the secure module. The configuration will NOT be loaded to the EMV kernel.

Parameters:

[IN] filename – File name of configuration file.
[IN] inStream – Stream of configuration file.

Returns:

d_EMVAPLIB_OK – Success
Other – Error code

public int setConfigurationEx(EMVConfigData configData)

The configuration file is sent from android app to the EMV kernel in the secure module. There are conditions to select specific configuration and load it to EMV kernel.

Parameters:

[IN] configData – Class EMVConfigData that consists of input variables to set configuration.

Returns:

d_EMVAPLIB_OK – Success
Other – Error code

public int delConfigurationFileAll()

Delete all configuration files stored in the EMV kernel.

Parameters:

None

Returns:

d_EMVAPLIB_OK – Success
Other – Error code

public int delConfigurationFile(String filename)

Detete the specific configuration file.

Parameters:

[IN] filename – File name of configuration file.

Returns:

d_EMVAPLIB_OK – Success
Other – Error code

public int txnAppSelect(EMVAppInfo appInfo)

The first step of performing transaction is Select Application after card inserted. If the card returns to indicate it has multiple applications, the event function onAppListEx( ) will be triggered to provide those Application Labels for the application. The application then is required to show on the screen for user selecting. If the card has only one application, the event function onAppSelectedConfirm() will be triggered with indicating if the card requires the user to do the confirmation.

Parameters:

[OUT] appInfo – Class EMVAppInfo that consists of output variables to get selected application information.

Returns:

d_EMVAPLIB_OK – Success
Other – Error code

public int txnPerform()

This method shall be called to perform transaction after txnAppSelect() finished. This method will perform the actions based EMV specification, such as Initiate Application Processing, Read Application Data, Offline Authentication, Processing restrictions, Cardholder Verification, Terminal Risk Management, First Generate AC. During the period of this method processing, required event functions will be triggered for the application to provide the necessary data or do the required actions, such as the retrieving for PIN block, CAPK, Terminal Data, or blacklist, or connecting with the host, etc.

Parameters:

None

Returns:

d_EMVAPLIB_OK – Success. Transaction is approved or declined offline. Check onTxnResult() in class EMVEvent to get transaction outcome.
d_EMVAPLIB_ERR_EVENT_ONLINE – The transaction should be transmitted online, perform online processing and call txnCompletion() to complete transaction.
Other – Error code

public int txnCompletion(EMVOnlineResponseData onlRspData)

Castles Technology

Actions after Online Processing, such as External Authentication, Second Generate AC, Issuer Script Processing.
Parameters:
[IN] onlRspData – Class EMVOnlineResponseData that consists of input variables to bring host response (ARC, ARPC) to EMV kernel.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int dataGet(TlvData tlvData)

Get specific tag data (terminal data or card data) in the current transaction from EMV kernel.
Parameters:
[IN][OUT] tlvData – Class TlvData consists of TLV data. Input Tag and get its length and value.
[IN] tag – Requested Tag.
[IN] len – Size of buffer to receive data.
[IN] value – Buffer to receive data.
[OUT] len – Actual length of the value.
[OUT] value – Terminal data or card data.
Returns:
d_EMVAPLIB_OK – Success
d_EMVAPLIB_ERR_DATA_BUFFER_ERROR – Buffer is not enough for receive data.
d_EMVAPLIB_ERR_NO_DATA – Tag is not found.
Other – Error code

public int txnDataSet(TlvData tlvData)

Set transaction-related data to EMV kernel during transaction.
Parameters:
[IN] tlvData – Class TlvData consists of TLV data. Input Tag, length and value.
[IN] tag – Tag.
[IN] len – Actual length of the value.
[IN] value – Transaction-related data.

Returns:
d_EMVAPLIB_OK – Success
d_EMVAPLIB_ERR_DATA_BUF_OVERFLOW – Tag buffer is full.
d_EMVAPLIB_ERR_TXN_NOT_PROCESSING – Operation allowed only during transaction.
Other – Error code

public final int txnDataSetEx(byte[] tlvBuf, int tlvLen)

Set multiple transaction-related data to kernel during transaction.
Parameters:
[IN] tlvBuf – Input TLV data, format is TLV1 + TLV2 + ... + TLVn.
[IN] tlvLen – Length of Input data.
Returns:
d_EMVAPLIB_OK – Success
d_EMVAPLIB_ERR_DATA_BUF_OVERFLOW – Tag buffer is full.
d_EMVAPLIB_ERR_TXN_NOT_PROCESSING – Operation allowed only during transaction.
d_EMVAPLIB_ERR_INVALID_PARA – Invalid input data.
Other – Error code

public final int txnPreDataSet(byte[] tlvBuf, int tlvLen)

Pre-set tags prior to transaction. The pre-setting tags will be added to kernel before GPO.
Parameters:
[IN] tlvBuf – Input TLV data, format is TLV1 + TLV2 + ... + TLVn.
[IN] tlvLen – Length of Input data.
Returns:
d_EMVAPLIB_OK – Success
d_EMVAPLIB_ERR_DATA_BUFFER_EXCEEDED – Input data length exceeds the buffer size.
d_EMVAPLIB_ERR_TXN_NOT_PROCESSING – Operation allowed only during transaction.
d_EMVAPLIB_ERR_INVALID_PARA – Invalid input data.
Other – Error code

public int cdaModeSet(byte mode)

Set CDA Mode.
Parameters:
[IN] mode – mode 1 (0x01) or mode 3 (0x03).
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int subsequentPINBypassSupport(boolean isSupported)

Please refer to EMV Book 4 v4.3, chapter 6.3.4.3 PIN Entry Bypass.
Subsequent PIN Bypass: When PIN entry has been bypassed for one PIN-related CVM, it may be considered bypassed for

Castles Technology

any subsequent PIN-related CVM during the current transaction.
Ex : In one transaction, the order of CVM is Online PIN -->(if Verify PIN Fails)-->Offline Plaintext PIN -->(if Verify PIN Fails) --> Offline Enciphered PIN
Subsequent PIN Bypass Supported : Online PIN -->(PIN Bypass)--> End of CVM ,the result is PIN Bypass
Subsequent PIN Bypass NOT Supported : Online PIN -->(PIN Bypass)--> Offline Plaintext PIN -->(PIN Bypass)--> Offline Enciphered PIN -->(PIN Bypass)--> End of CVM , the result is PIN Bypass
The EMV kernel does NOT support Subsequent PIN Bypass by default.
If developer wants to support Subsequent PIN Bypass, please call this method at startup of User Application. EMV kernel will keep this status unless developer calls this function again to change the status.
Parameters:
[IN]isSupported – support (true) or not (false).
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int specialEventRegister(byte eventId, IEMVSpecialEvent spEvent)

Register special event (callback function) to EMVkernel, provided by the user application using.
Parameters:
[IN] eventId – Identifier for registering special event.
[IN] spEvent – Register event (callback function).
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int txnCandidateList(EMVCandidateList candidateList)

Build candidate list.
Parameters:
[IN][OUT] candidateList – Input specific buffer and get candidate list.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int txnFinalSelection(int appIdx)

Select application, Initiate Application Processing and Read Application Data.
Parameters:
[IN] appIdx – Index of the selected application (from 0 and n), as the sequence returned by txnCandidateList.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int txnPSERsp(EMVTxnPSERsp rsp)

Call this method after txnCandidateList to get PSE response.
Parameters:
[IN][OUT] rsp – Input specific buffer and get PSE response
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int txnCandidateDirEntry(EMVCandidateDirEntry candidateDirEntry)

Call this method after txnCandidateList to get entry data of candidate.
Parameters:
[IN][OUT] candidateDirEntry –Input index of the selected application and specific buffer then get entry data.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int secureDataEncryptInfoSet(EMVSecureDataInfo info)

Determine encryption method of security data.
As security reason, EMV Kernel protect the secured tags like Tag 5A and 57, that are not allowed retrieved in plaintext under Android environment. Instead of Tag 5A and 57, developer can retrieve below tags to get information like BIN, expiration date, service code

Tag DF30 : Encrypt PAN
Value Format : EncryptedDataLen (1B) + EncryptedData(var B) + RealLen (1B) + KSN Len(1B) + KSN(var B)

Tag DF31 :PAN Check Sum (HASH or MAC)
Value Format : CheckSumLen (1B) + CheckSum(var B) + RealLen (1B) + KSN Len(1B) + KSN(var B)

Tag DF32 : Mask PAN

Tag DF33 : Encrypt Track2ED
Value Format : EncryptedDataLen (1B) + EncryptedData(var B) + RealLen (1B) + KSN Len(1B) + KSN(var B)

Tag DF34 : Track2ED Check Sum(HASH or MAC)

Castles Technology

Value Format : CheckSumLen (1B) + CheckSum(var B) + RealLen (1B) + KSN Len(1B) + KSN(var B)

Tag DF35 : Mask Track2ED

Parameters:
[IN] info – encryption method.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int secureDataWhitelistSet(byte action, byte[] listData, int listDataLen)

Set whitelist of security data. The feature is to allow the secured tags to be returned in the plaintext. The developer can add BIN or partial digits of PAN or a full PAN to the white list. When EMV kernel read the PAN from a card, it compares PAN to the white list. When the result of the comparison is matched, the secured tags are returned in plaintext.
Parameters:
[IN] action – New (0x00) or Delete all (0x01).
[IN] listData – The whitelist data.
Format : entry num (4 Byte) + entry1 len(1 Byte) + entry1 data (var Byte) + ….. + entryN len (1 Byte) + entryN data (var Byte).
Max entry num is 512.
Example : 3 entries need to be added into whiltelist PAN1 : 4567, PAN2 : 123, PAN3 : 56
listData need to be filled with 0x00 0x00 0x00 0x03 0x04 0x34 0x35 0x36 0x37 0x03 0x31 0x32 0x33 0x02 0x35 0x36.
0x00 0x00 0x00 0x03 -> entry num
0x04 0x34 0x35 0x36 0x37 -> entry1 [Length : 4 + '4567' in ASCII]
0x03 0x31 0x32 0x33 -> entry2 [Length : 3 + '123'in ASCII]
0x02 0x35 0x36. -> entry 3 [Length : 2 +'56'in ASCII]
[IN] listDataLen – The length of input whitelist data.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public final int setMaskChar(byte maskChar)

Set mask character of security data.
Parameters:
[IN] maskChar – Maskchar like ' * ', ' 0 '….
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public final int maskPANDigits1To6(byte maskDigits)

Set masked status of Digits 1 to 6 of PAN. (D1D2D3D4 D5D6XX XXXX XXXX)
Parameters:
[IN] maskDigits – Masked status of digits. One byte (bit 8 bit 7 bit6 …..bit 2 bit 1).
bit 8 : 1 – Digit 1 is maked.
bit 7 : 1 – Digit 2 is maked.
bit 6 : 1 – Digit 3 is maked.
bit 5 : 1 – Digit 4 is maked.
bit 4 : 1 – Digit 5 is maked.
bit 3 : 1 – Digit 6 is maked.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public final int maskPANDigits13To16(byte maskDigits)

Set masked status of Digits 13 to 16 of PAN. (XXXX XXXX XXXX D13D14D15D16)
Parameters:
[IN] maskDigits – Masked status of digits. One byte (bit 8 bit 7 bit6 …..bit 2 bit 1).
bit 8 : 1 – Digit 13 is maked.
bit 7 : 1 – Digit 14 is maked.
bit 6 : 1 – Digit 15 is maked.
bit 5 : 1 – Digit 16 is maked.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public final int maskEXPDigits1To4(byte maskDigits)

Set masked status of Digits 1 to 4 of expiration date of track 2 data. (MMYY : D1D2D3D4)
Parameters:
[IN] maskDigits – Masked status of digits. One byte (bit 8 bit 7 bit6 …..bit 2 bit 1).
bit 8 : 1 – Digit 1(M) is maked.
bit 7 : 1 – Digit 2(M) is maked.
bit 6 : 1 – Digit 3(Y) is maked.
bit 5 : 1 – Digit 4(Y) is maked.
Returns:

Castles Technology

d_EMVAPLIB_OK – Success
Other – Error code

public final int maskCSCDigits1To3(byte maskDigits)

Set masked status of Digits 1 to 3 of card service code. (D1D2D3)
Parameters:
[IN] maskDigits – Masked status of digits. One byte (bit 8 bit 7 bit6 …..bit 2 bit 1).
bit 8 : 1 – Digit 1 is maked.
bit 7 : 1 – Digit 2 is maked.
bit 6 : 1 – Digit 3 is maked.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public final int dataGetEx(EMVDataGetExPara reqData)

Get multiple tag data (terminal data or card data) in the current transaction from EMV kernel.
Parameters:
[IN][OUT] reqData – Requesting list of tags (Tag1+Tag2+..+Tagn) and receiving data of tags as TLV format (TLV1+TLV2+…+TLVn).
Returns:
d_EMVAPLIB_OK – Success
d_EMVAPLIB_ERR_INVALID_PARA – Invalid input data
d_EMVAPLIB_ERR_DATA_BUFFER_ERROR – Buffer is not enough for receive response.
Other – Error code

public int applicationListSet(byte aidNum, EMVApplicationPara[] aidPara)

Establish AID Table. When one AID is added to AID Table, it means EMV kernel supports this AID. The supporting AIDs in AID Table will participate in the selection process with card to establish a candidate list of transaction.
Parameters:
[IN] aidNum – Number of AID.
[IN] aidPara – Class EMVApplicationPara consists of AID para. Input AID, ASI
Returns:
d_EMVAPLIB_OK – Success.
d_EMVAPLIB_ERR_SET_APPLIST_ERROR – Set AID table fail

public int capkSet(byte[] rid, CAPublicKey capk)

Set CAPK
Parameters:
[IN] rid – RID must be 5 bytes
[IN] capk – Class CAPublicKey consists of CAPK para. Input CAPK Index, Modulus, Exponent, Hash.
Returns:
d_EMVAPLIB_OK – Success.
d_EMVAPLIB_ERR_SET_CAPK_ERROR – Set CAPK fail

public int capkSetEx(int capkNum, CAPublicKeyEx[] capks)

Set multiple CAPKs at once
Parameters:
[IN] capkNum – Number of CAPKs
[IN] capks – Buffer to set multiple CAPKs.
Returns:
d_EMVAPLIB_OK – Success.
d_EMVAPLIB_ERR_SET_CAPK_ERROR – Set CAPK fail

public int capkDelete(byte[] rid, byte capkIndex)

Delete the specified CAPK
Parameters:
[IN] rid – RID must be 5 bytes.
[IN] capkIndex – CAPK Index.
Returns:
d_EMVAPLIB_OK – Success.
d_EMVAPLIB_ERR_SET_CAPK_ERROR – Set CAPK fail

public int capkDeleteAll()

Detele all of CAPKs.
Returns:
d_EMVAPLIB_OK – Success.
d_EMVAPLIB_ERR_SET_CAPK_ERROR – Set CAPK fail

public int terminalConfigSet(byte tlvDataNum, TlvData[] tlvData)

Set terminal tags.
Parameters:
[IN] tlvDataNum – Number of tgas.
[IN] tlvData – Class TlvData consists of TLV data. Array of TLV data. Input Tags, lengths and values.

Castles Technology

Returns:
d_EMVAPLIB_OK – Success.
d_EMVAPLIB_ERR_SET_TERMINAL_DATA_ERROR – Set terminal tags fail
d_EMVAPLIB_ERR_TXN_PROCESSING – Not allow to change tags because transaction is ongoing.

public int terminalConfigDelete(byte tagNum, int[] tags)

Delete the specified terminal tags
Parameters:
[IN] tagNum – Number of tags.
[IN] tags – Array of tag.
Returns:
d_EMVAPLIB_OK – Success.
d_EMVAPLIB_ERR_SET_TERMINAL_DATA_ERROR – Set terminal tags fail
d_EMVAPLIB_ERR_TXN_PROCESSING – Not allow to change tags because transaction is ongoing.

public int terminalConfigDeleteAll()

Delete all of terminal tags
Returns:
d_EMVAPLIB_OK – Success.
d_EMVAPLIB_ERR_SET_TERMINAL_DATA_ERROR – Set terminal tags fail
d_EMVAPLIB_ERR_TXN_PROCESSING – Not allow to change tags because transaction is ongoing.

public int appConfigSet(byte[] aid, byte aidLen, byte tlvDataNum, TlvData[] tlvData)

Set tags that are belong to the specified AID
Parameters:
[IN] aid – AID.
[IN] aidLen – AID length.
[IN] tlvDataNum – Number of tgas.
[IN] tlvData – Class TlvData consists of TLV data. Array of TLV data. Input Tags, lengths and values.
Returns:
d_EMVAPLIB_OK – Success.
d_EMVAPLIB_ERR_SET_APP_TAG_DATA_ERROR – Set app tags fail
d_EMVAPLIB_ERR_TXN_PROCESSING – Not allow to change tags because transaction is ongoing.

public int appConfigDelete(byte[] aid, byte aidLen, byte tagNum, int[] tags)

Delete tags that are belong to the specified AID
Parameters:
[IN] aid – AID.
[IN] aidLen – AID length.
[IN] tagNum – Number of tags.
[IN] tags – Array of tag.
Returns:
d_EMVAPLIB_OK – Success.
d_EMVAPLIB_ERR_SET_APP_TAG_DATA_ERROR – Set app tags fail
d_EMVAPLIB_ERR_TXN_PROCESSING – Not allow to change tags because transaction is ongoing.

public int appConfigDeleteAll()

Delete all of tags that are belong to the specified AID
Parameters:
None
Returns:
d_EMVAPLIB_OK – Success.
d_EMVAPLIB_ERR_SET_APP_TAG_DATA_ERROR – Set app tags fail
d_EMVAPLIB_ERR_TXN_PROCESSING – Not allow to change tags because transaction is ongoing.

public final int partialTransactionSet(int type)

Set type of partial transaction

Full transaction : Perform all of the processes defined in EMV Book

Partial transaction : After reading card data, ignore processes of verification such as oda, cvm, directly requesting AAC at the 1st Generate AC to complete the transaction
Parameters:
[IN] type – Processing type of the transaction
0x00 – Full/normal transaction
0x01 – Partial transaction.
Returns:
0 – Success
Other – Error code

public final int partialTransactionGet()

Get current type of partial transaction
Parameters:

Castles Technology

None
Returns:
0x00 – Full/normal transaction
0x01 – Partial transaction
Other – Error code

public final int getLastError()

Get detail error code
Parameters:
None
Returns:
d_EMVAPLIB_OK – No last error information

Last error codes are returned when calling getLastError() after below scenarios

(1) The return code of txnAppSelect() is d_EMVAPLIB_ERR_APPLICATION_NOT_ALLOW
(2) The return code of txnFinalSelection() is d_EMVAPLIB_ERR_SELECTION_FAIL
d_EMVAPLIB_ERR_FINAL_SELECTION_SW_6283 – the SW12 of final selection is 6283
d_EMVAPLIB_ERR_FINAL_SELECTION_SW_NON_9000_NON_6283 – the SW12 of final selection is neither 9000 nor 6283
d_EMVAPLIB_ERR_FINAL_SELECTION_RSP_FORMAT_ERROR – the response of final selection is format error
d_EMVAPLIB_ERR_GPO_SW_6985 – the SW12 of GPO is 6985

(3) The return code of txnCompletion() is d_EMVAPLIB_OK and the transaction result is declined
d_EMVAPLIB_ERR_CAS_GAC2_WRONG_CID – incorrect CID in GenAC2

(4) The return code of txnPerform() is a general error 0x1003
d_EMVAPLIB_ERR_OFFLINE_PIN_TIMEOUT – timeout occurred during offline pin entry
d_EMVAPLIB_ERR_OFFLINE_PIN_CANCEL – user cancel transaction during offline pin entry
d_EMVAPLIB_ERR_OFFLINE_PIN_CARD_REMOVED – card has removed during offline pin entry

public int enableNullPIN()

For offline PIN only. PIN entry is able to be null
Parameters:
None
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int disableNullPIN()

For offline pin only. PIN entry is NOT able to be null.
Parameters:
None
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int enablePINSound()

For offline PIN only. Enable PIN sound. Sound when a PIN is pressed.
Parameters:
None
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int disablePINSound()

For offline PIN only. Disable PIN sound.
Parameters:
None
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public void setPINPADMoving(boolean bool)

For offline PIN & eventShowVirtualPIN() only. Set randomly moving PINPAD during get PIN function.
Parameters:
[IN] bool – Indicate the PINPAD is moving or not.
Returns:
None
Notes: Only for terminals without physical keyboard.

public void setPINScrambling(boolean bool)

For offline PIN & eventShowVirtualPIN() only. Set scramble digital button during get PIN function.
Parameters:

Castles Technology

[IN] bool – indicate the key on PINPAD is random or not.

Returns:

None

Notes: Only for terminals without physical keyboard.

public int preferredOrderSet(EMVByteBufData preferredOrder)

Set preferred order of card application (AID). In application selection phase, the candidate list is rearranged by the preferred order.

Parameters:

[IN] preferredOrder – buffer to set preferred order
[IN] dataBuf – AID1 Len (1 byte) + AID1 + AID2 Len(1 byte) + AID2 + … + AIDn Len(1 byte) + AIDn
[IN] detaLen – Data length
Ex : Set two AIDs A0000000031010 and A0000000132020
preferredOrder.dataBuf is 07A0000000031010 07A0000000132020 (HEX)
preferredOrder.detaLen is 0x10

Returns:

d_EMVAPLIB_OK – Success
Other – Error code

public int preferredOrderGet(EMVByteBufData preferredOrder)

Get preferred order data.

Parameters:

[OUT] preferredOrder – buffer to get preferred order
[OUT] dataBuf – AID1 Len (1 byte) + AID1 + AID2 Len(1 byte) + AID2 + … + AIDn Len(1 byte) + AIDn
[OUT] detaLen – Data length.

Returns:

d_EMVAPLIB_OK – Success
Other – Error code

public int appFilteringParaSet(EMVByteBufData para)

Candidate list filtering. Disable specific card applications before application selection

Parameters:

[IN] para – buffer to set filtering parameter.
[IN] dataBuf –version(1 byte) + [type(1 byte) + dataLen(1 byte) + data(var bytes)] * n
Version (1 byte) : 0x01
type (1 byte) :
0x00 – AID
0x01 – Label
Other – RFU
[IN] detaLen – Data length.
Ex : Filtering parameter are set to AID "A0000000031010" and "VISA CREDIT"
AID "A0000000031010" --> card application with this AID will not be added to candidate list
Label "VISA CREDIT" --> card application with this label will not be added to candidate list

para.dataBuf is 010007A000000031010010B5649534120435245444954 (HEX)
para.dataLen is 0x17

01	version
00	type : AID
07	data len : AID length
A0000000031010	data : AID
01	type : Label
0B	data len : Label length
5649534120435245444954	data : Label

Returns:

d_EMVAPLIB_OK – Success
Other – Error code

public int appFilteringParaGet(EMVByteBufData para)

Get Application Filtering Parameter.

Parameters:

[OUT] para – buffer to get filtering parameter.
[OUT] dataBuf – Filtering parameter
[OUT] detaLen – Data length.

Returns:

d_EMVAPLIB_OK – Success
Other – Error code

public int txnCardAcquisitionRead()

The EMV kernel performs card acquisition. Each AID in the card, collecting card tags from response of SELECT AID command to response of READ RECORD command. For response of SELECT PSE command, it will be returned to txnPSERsp()
After calling txnCardAcquisitionRead(), please call
txnCardAcquisitionAppNumGet() and txnCardAcquisitionAppDataGet() to get result of card acquisition.
txnPSERsp() to get select pse response.

Castles Technology

Parameters:
None
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int txnCardAcquisitionAppNumGet()

Get the number of card acquisition data (the same as AID number in the card). Call this after txnCardAcquisitionRead().
Parameters:
None
Returns:
value – Number of card acquisition data

public int txnCardAcquisitionAppDataGet(EMVCardAcquisitionData[] acq)

Input buffer to get card acquisition data. Call this after txnCardAcquisitionRead().
Parameters:
[OUT] acq – buffer to get card acquisition data.
[OUT] acq.tlv.dataBuf – All tags. Format is tlv + tlv + tlv + ..
[OUT] acq.tlv.dataLen – Length of tlv data.
[OUT] acq.aid.dataBuf – AID
[OUT] acq.aid.dataLen – Length of AID.
[OUT] acq.pan.dataBuf – PAN
[OUT] acq.pan.dataLen – Length of PAN.
[OUT] acq.issuerCountryCode.dataBuf – Issuer country code.
[OUT] acq.issuerCountryCode.dataLen – Length of issuerCountryCode
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int cancelVirtualPIN()

For offline PIN only. During getting offline PIN processing, call this method to cancel PIN entry.
Parameters:
None
Returns:
d_EMVAPLIB_OK – Success
d_EMVAPLIB_ERR_VIRTUAL_PIN_NOT_PROCESSING – The current procedure is not in offline PIN processing.
Notes: Only for terminals without physical keyboard.

public int changeWDBPath(String path)

The EMV kernel default set the working database path to the Android Application ID and store the files such as the xml configuration files in it. Each Android App has its configurations.
This method is used to change the working database path If the developer would like to share the configurations between multiple Android App.
Parameters:
[IN] path – Working database path.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int getWDBPath()

Get the current working database path. If the developer does not change WDB path, the default working database path is the string of Android Application ID. After calling getWDBPath(), please call getRtnString() to get the working database path.
Parameters:
None.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public int resetWDBPath()

Reset the working database path to default.
Parameters:
None.
Returns:
d_EMVAPLIB_OK – Success
Other – Error code

public String getRtnString()

This method is used after specific methods to get String variable.
Parameters:
None
Returns:
value – Get a String variable.

public int verifyCommandSW12Get(EMVByteBufData cmdStatus, EMVByteBufData sw12)

Castles Technology

For Offline PIN verification. Obtain the status and SW12 of Verify Command.
Calling this method in callback eventOfflinePINVerifyResult() to get SW12 of Verify command.

Parameters:

[OUT] cmdStatus – Status of Verify Command.
- d_VERIFY_CMD_PERFORMED
- d_VERIFY_CMD_NOT_PERFORMED
- d_VERIFY_CMD_APDU_FAIL

[OUT] sw12 – SW12 of Verify comand.

Returns:

d_EMVAPLIB_OK – Success
Other – Error code

Fields Detail

Event Id used in method specialEventRegister (public static final byte)

Constant Field	Value	Additional Description
d_EVENTID_FIRSTGENAC_TERMDECISION	0x02	
d_EVENTID_TXN_FORCED_ONLINE	0x09	
d_EVENTID_SHOW_VIRTUAL_PIN	0x0C	
d_EVENTID_GET_PIN_DONE	0x0D	
d_EVENTID_SHOW_VIRTUAL_PIN_EX	0x0E	
d_EVENTID_ONLINE_PINBLOCK_GET	0x10	
d_EVENTID_OFFLINE_PIN_VERIFY_RESULT	0xC0	
d_EVENTID_PIN_BYPASS	0xC2	
d_EVENTID_TXN_FORCED_ACCEPTANCE	0x14	
d_EVENTID_APP_LIST_ALWAYS	0x1F	

Offline Pin Verify Result

Constant Field	Value	Additional Description
d_PIN_RESULT_OK	0x01	
d_PIN_RESULT_FAIL	0x02	
d_PIN_RESULT_BLOCKED	0x03	
d_PIN_RESULT_FAILBLOCKED	0x04	

Verify Command Status

Constant Field	Value	Additional Description
d_VERIFY_CMD_PERFORMED	0x00	
d_VERIFY_CMD_NOT_PERFORMED	0x01	
d_VERIFY_CMD_APDU_FAIL	0x02	

EMVSecureDataInfo keyType

Constant Field	Value	Additional Description
KeyType_3DES	0x01	
KeyType_3DES_DUKPT	0x02	
KeyType_AES	0x03	
KeyType_AES_DUKPT	0x07	

EMVSecureDataInfo cipherMethod

Constant Field	Value	Additional Description

Castles Technology

DATA_ENCRYPT_METHOD_ECB	0x00	
DATA_ENCRYPT_METHOD_CBC	0x01	
EMVSecureDataInfo macCipherMethod		
Constant Field	Value	Additional Description
MAC_METHOD_CBC	0x00	
MAC_METHOD_X9_19	0x01	
MAC_METHOD_X9_19_START	0x02	
MAC_METHOD_X9_19_UPDATE	0x03	
MAC_METHOD_X9_19_FINAL	0x04	
EMVSecureDataInfo checksumType		
Constant Field	Value	Additional Description
ChecksumType_SHA1	0x00	
ChecksumType_MAC	0x01	
ChecksumType_SHA256	0x02	
ChecksumType_NONE	0xFF	
EMVSecureDataInfo paddingMethod		
Constant Field	Value	Additional Description
PaddingMethod_00	0x00	
PaddingMethod_FF	0x01	
PaddingMethod_PKCS7	0x02	
EMVSecureDataInfo derivedKeyType & macDerivedKeyType		
Constant Field	Value	Additional Description
DerivedKeyType_2TDEA	0x00	
DerivedKeyType_3TDEA	0x01	
DerivedKeyType_AES128	0x02	
DerivedKeyType_AES192	0x03	
DerivedKeyType_AES256	0x04	
EMVSecureDataInfo derivedKeyUsage		
Constant Field	Value	Additional Description
DerivedKeyUsage_DataEncryptionEncrypt	0x05	
DerivedKeyUsage_DataEncryptionDecrypt	0x06	
DerivedKeyUsage_DataEncryptionBothWays	0x07	
EMVSecureDataInfo macDerivedKeyUsage		
Constant Field	Value	Additional Description
DerivedKeyUsage_MessageAuthenticationGeneration	0x02	
DerivedKeyUsage_MessageAuthenticationVerification	0x03	
DerivedKeyUsage_MessageAuthenticationBothWays	0x04	
Return Code (public static final int)		
Constant Field	Value	Additional Description

Castles Technology

d_EMVAPLIB_OK	0x0000	
d_EMVAPLIB_ERR_SC_NOT_PRESENT	0x1401	
d_SC_NOT_ACTIVATED	0x1402	
d_SC_COMM_ERROR	0x1403	
d_SC_MUTE	0x1404	
d_SC_PARITY_ERROR	0x1405	
d_SC_EDC_ERROR	0x1406	
d_SC_BUFFER_OVERRUN	0x1407	
d_SC_HW_ERROR	0x1408	
d_SC_DEACTIVATED_PROTOCOL	0x1409	
d_SC_BAD_TS	0x140A	
d_SC_ATR_TOO_LONG	0x140B	
d_SC_ATR_INVALID	0x140C	
d_SC_TA1_NOT_SUPPORTED	0x140D	
d_SC_PTS_RESPONSE_ERROR	0x140E	
d_SC_PROCEDURE_BYTE_CONFLICT	0x140F	
d_SC_POWER_FAILED	0x1410	
d_SC_INVALID_PARAMETER	0x1411	
d_SC_INSUFFICIENT_BUF	0x1420	
d_SC_SOCKET_NOT_EXIST	0x1430	
d_SC_BUF_TOO_SMALL	0X1431	
d_EMVAPLIB_ERR_DATA_BUFFER_EXCEEDED	0x0001	
d_EMVAPLIB_ERR_INVALID_PARA	0x0002	
d_EMVAPLIB_ERR_CRITICAL_MISTAKES	0x0003	
d_EMVAPLIB_ERR_SELECTION_FAIL	0x0004	
d_EMVAPLIB_ERR_PIN_BY_PASS	0x0005	
d_EMVAPLIB_ERR_TERM_DATA_MISSING	0x0006	
d_EMVAPLIB_ERR_DATA_NOT_FOUND	0x0007	
d_EMVAPLIB_ERR_GAC1_6985_FALLBACK	0x0008	
d_EMVAPLIB_ERR_ONLY_1_AP_NO_FALLBACK	0x0009	
d_EMVAPLIB_ERR_IAP_9481_FALLBACK	0x000A	
d_EMVAPLIB_ERR_FORCE_ACCEPTANCE	0x000B	
d_EMVAPLIB_ERR_FUNCTION_NOT_SUPPORTED	0x000C	
d_EMVAPLIB_ERR_ENTER_KMS_OFFLINEPIN	0x000D	
d_EMVAPLIB_ERR_TXN_PROCESSING	0x000E	
d_EMVAPLIB_ERR_TXN_NOT_PROCESSING	0x000F	
d_EMVAPLIB_ERR_SECURE_DATA	0x0010	
d_EMVAPLIB_ERR_APP_CONFIRM_FAIL	0x0031	
d_EMVAPLIB_ERR_ON_TXN_DATA_GET_FAIL	0x0032	

Castles Technology

d_EMVAPLIB_ERR_ON_APP_LIST_FAIL	0x0033	
d_MEVAPLIB_ERR_LASTERR_MISSING_TERMINAL_DATA	0x0075	
d_EMVAPLIB_ERR_MCCS_SELECT_PREFER_AID_FAIL	0x007A	
d_EMVAPLIB_ERR_AMOUNT_FORMAT_ERROR	0x0101	
d_EMVAPLIB_ERR_NO_AP_FOUND	0x0102	
d_EMVAPLIB_ERR_ABORT_TX	0x0103	
d_EMVAPLIB_ERR_USER_ABORT	0x0104	
d_EMVAPLIB_ERR_CARD_PROBLEM	0x0105	
d_EMVAPLIB_ERR_SETTING_ERROR	0x0106	
d_EMVAPLIB_ERR_NO_CAPK	0x0107	
d_EMVAPLIB_ERR_NO_DATA	0x0108	
d_EMVAPLIB_ERR_DATA_BUFFER_ERROR	0x0109	
d_EMVAPLIB_ERR_DATA_BUF_OVERFLOW	0x010A	
d_EMVAPLIB_ERR_SET_CDA_MODE_BUFFER_ERROR	0x010B	
d_EMVAPLIB_ERR_SET_CDA_MODE_FAIL	0x010C	
d_EMVAPLIB_ERR_SUBSEQUENT_PIN_BYPASS_BUFFER_ERRO R	0x010D	
d_EMVAPLIB_ERR_SUBSEQUENT_PIN_BYPASS_FAIL	0x010E	
d_EMVAPLIB_ERR_OFFLINE_PIN_TIMEOUT	0x010F	
d_EMVAPLIB_ERR_OFFLINE_PIN_CANCEL	0X0110	
d_EMVAPLIB_ERR_VIRTUAL_PIN_NOT_PROCESSING	0X0111	
d_EMVAPLIB_ERR_OFFLINE_PIN_CARD_REMOVED	0x0117	
d_EMVAPLIB_ERR_CONFIG_READ_FAIL	0x0201	
d_EMVAPLIB_ERR_NO_ACTIVE_INDEX	0x0202	
d_EMVAPLIB_ERR_SET_TERMINAL_DATA_ERROR	0x0203	
d_EMVAPLIB_ERR_SET_APPLIST_ERROR	0x0204	
d_EMVAPLIB_ERR_SET_APP_TAG_DATA_ERROR	0x0205	
d_EMVAPLIB_ERR_SET_CAPK_ERROR	0x0206	
d_EMVAPLIB_ERR_SET_DEFAULT_DATA_ERROR	0x0207	
d_EMVAPLIB_ERR_DLL_LOAD_ERROR	0x0208	
d_EMVAPLIB_ERR_CAPK_HASH_ERROR	0x0209	
d_EMVAPLIB_ERR_INVALID_INPUT_PARA	0x020A	
d_EMVAPLIB_ERR_SET_PARAMETER_ERROR	0x020B	
d_EMVAPLIB_ERR_GET_PARAMETER_ERROR	0x020C	
d_EMVAPLIB_ERR_STORAGE_ERROR	0x020D	
d_EMVAPLIB_ERR_EVENT_CONFIRMED	0x0301	
d_EMVAPLIB_ERR_EVENT_SELECTED	0x0302	
d_EMVAPLIB_ERR_EVENT_GET_TXNDATA	0x0303	
d_EMVAPLIB_ERR_EVENT_ONLINE	0x0304	
d_EMVAPLIB_ERR_EVENT_ISSUER_RESULT	0x0305	

Castles Technology

d_EMVAPLIB_ERR_EVENT_TXN_RESULT	0x0306	
d_EMVAPLIB_ERR_EVENT_BUFFER_DEFINE	0x0307	
d_EMVAPLIB_ERR_EVENT_VERSION	0x0308	
d_EMVAPLIB_ERR_EVENT_GETPIN_NOTIFY	0x0309	
d_EMVAPLIB_ERR_EVENT_SETAPPDATA	0x030A	
d_EMVAPLIB_ERR_EVENT_NON_EMV_CARD	0x030B	
d_EMVAPLIB_ERR_CANDIDATE_LIST_FULL	0x1002	
d_EMVAPLIB_ERR_SEND_APDU_CMD_FAIL	0x1004	
d_EMVAPLIB_ERR_APPLICATION_NOT_ALLOW	0x1007	
d_EMVAPLIB_ERR_IAP_AIP_AFL_ERROR	0x1008	
d_EMVAPLIB_ERR_IAP_UNKNOW_SW12	0x1009	
d_EMVAPLIB_ERR_IAP_MISS_PAN	0x100A	
d_EMVAPLIB_ERR_IAP_MISS_CDOL1	0x100B	
d_EMVAPLIB_ERR_IAP_MISS_CDOL2	0x100C	
d_EMVAPLIB_ERR_IAP_APPLICATION_NOT_ALLOW	0x100D	
d_EMVAPLIB_ERR_ERROR_9F4A_RULE	0x100E	
d_EMVAPLIB_ERR_KEY_NO_FOUND	0x1010	
d_EMVAPLIB_ERR_PAN_NOT_SAME	0x1011	
d_EMVAPLIB_ERR_DDOL_MISS	0x1012	
d_EMVAPLIB_ERR_INTERNAL_AUTHENTICATE_FAIL	0x1013	
d_EMVAPLIB_ERR_SDA_LENGTH_NOT_MATCH	0x1014	
d_EMVAPLIB_ERR_NO_OFFLINE_DATA_AUTH_MATCH	0x1015	
d_EMVAPLIB_ERR_READ_DATA_TAG_NOT_70	0x1016	
d_EMVAPLIB_ERR_MISS_APP_EXPIRATION_DATE	0x1017	
d_EMVAPLIB_ERR_CARDHOLDER_VER_NOT_SUPP	0x1018	
d_EMVAPLIB_ERR_ERR_CVM_LIST_MISSING	0x1019	
d_EMVAPLIB_ERR_GET_DATA_CMD_ERROR	0x1021	
d_EMVAPLIB_ERR_CAS_GAC1_FAIL	0x1023	
d_EMVAPLIB_ERR_CAS_MERCHANT_FORCE_APPROVED	0x1024	
d_EMVAPLIB_ERR_ONLY_1_AP_BLOCKED	0x1026	
d_EMVAPLIB_ERR_CAS_NEED_ONLINE	0x1027	
d_EMVAPLIB_ERR_CAS_CARD_INIT_REFERRAL	0x1028	
d_EMVAPLIB_ERR_CAS_DDAAC_FAIL	0x1029	
d_EMVAPLIB_ERR_CAS_ONLINE_FAIL_APPROVAL	0x102A	
d_EMVAPLIB_ERR_CAS_GAC1_WRONG_CID	0x102C	
d_EMVAPLIB_ERR_CAS_GAC2_FAIL	0x102E	
d_EMVAPLIB_ERR_DIS_ADF_UNKNOW_TAG	0x102F	
d_EMVAPLIB_ERR_DIS_ADF_DATA_MISSING	0x1030	
d_EMVAPLIB_ERR_DIS_FCI_DATA_ERROR	0x1031	

Castles Technology

d_EMVAPLIB_ERR_DIS_TLV_TAG_ZERO	0x1032	
d_EMVAPLIB_ERR_DIS_TLV_FAIL	0x1033	
d_EMVAPLIB_ERR_DIS_TLV_EXCEED_MAX_LEN	0x1035	
d_EMVAPLIB_ERR_CAS_GAC1_FORMAT_1_PADDING	0x1036	
d_EMVAPLIB_ERR_MISS_AIP_AFL	0x1037	
d_EMVAPLIB_ERR_DDOL_NOT_HAVE_9F37	0x1038	
d_EMVAPLIB_ERR_SDA_DATA_ERROR	0x103C	
d_EMVAPLIB_ERR_SDA_ALGORITHM_NOT_SUPPORT	0x103D	
d_EMVAPLIB_ERR_DDA_DATA_ERROR	0x103E	
d_EMVAPLIB_ERR_DDA_ALGORITHM_NOT_SUPPORT	0x103F	
d_EMVAPLIB_ERR_DDAAC_DATA_ERROR	0x1040	
d_EMVAPLIB_ERR_DDAAC_ALGORITHM_NOT_SUPPORT	0x1041	
d_EMVAPLIB_ERR_KEY_LENGTH_ERROR	0x1043	
d_EMVAPLIB_ERR_DDAAC_HASH1_ERROR	0x1045	
d_EMVAPLIB_ERR_DDAAC_HASH2_ERROR	0x1046	
d_EMVAPLIB_ERR_ISSUER_CERT_NOT_EXIST	0x1047	
d_EMVAPLIB_ERR_ISSUER_CERT_FORMAT_ERROR	0x1048	
d_EMVAPLIB_ERR_ISSUER_CERT_IIN_PAN_NOT_SAME	0x1049	
d_EMVAPLIB_ERR_ISSUER_CERT_REVOCATION_FOUND	0x104A	
d_EMVAPLIB_ERR_ISSUER_CERT_ALGORITHM_NOT_SUPPORT	0x104B	
d_EMVAPLIB_ERR_ISSUER_CERT_LENGTH_ERROR	0x104C	
d_EMVAPLIB_ERR_ISSUER_CERT_EXPIRATION_DATE	0x104D	
d_EMVAPLIB_ERR_ISSUER_CERT_HASH_NOT_MATCH	0x104E	
d_EMVAPLIB_ERR_ISSUER_CERT_EXPONENT_NOT_EXIST	0x104F	
d_EMVAPLIB_ERR_ISSUER_CERT_REMAINDER_MISSING	0x1050	
d_EMVAPLIB_ERR_ICC_CERT_NOT_EXIST	0x1051	
d_EMVAPLIB_ERR_ICC_CERT_FORMAT_ERROR	0x1052	
d_EMVAPLIB_ERR_ICC_CERT_ALGORITHM_NOT_SUPPORT	0x1053	
d_EMVAPLIB_ERR_ICC_CERT_LENGTH_ERROR	0x1054	
d_EMVAPLIB_ERR_ICC_CERT_HASH_NOT_MATCH	0x1055	
d_EMVAPLIB_ERR_ICC_CERT_EXPIRATION_DATE	0x1056	
d_EMVAPLIB_ERR_ICC_CERT_EXPONENT_NOT_EXIST	0x1057	
d_EMVAPLIB_ERR_ICC_ISSUER_PK_NOT_EXIST	0x1058	
d_EMVAPLIB_ERR_ICC_CERT_REMAINDER_MISSING	0x1059	
d_EMVAPLIB_ERR_PIN_CERT_NOT_EXIST	0x105A	
d_EMVAPLIB_ERR_PIN_CERT_LENGTH_ERROR	0x105B	
d_EMVAPLIB_ERR_PIN_CERT_FORMAT_ERROR	0x105C	
d_EMVAPLIB_ERR_PIN_CERT_ALGORITHM_NOT_SUPPORT	0x105D	
d_EMVAPLIB_ERR_PIN_CERT_HASH_NOT_MATCH	0x105E	

Castles Technology

d_EMVAPLIB_ERR_PIN_CERT_EXPIRATION_DATE	0x105F	
d_EMVAPLIB_ERR_PIN_CERT_KEY_LENGTH_ERROR	0x1060	
d_EMVAPLIB_ERR_READ_PIN_TRY_COUNT_FAIL	0x1061	
d_EMVAPLIB_ERR_CVM_PLAIN_TEXT_PIN_NOT_KEYIN	0x1062	
d_EMVAPLIB_ERR_CVM_PLAIN_TEXT_PIN_TRY_LIMIT_EXCEEDED	0x1063	
d_EMVAPLIB_ERR_CVM_PLAIN_TEXT_PIN_WRONG	0x1066	
d_EMVAPLIB_ERR_CVM_PLAIN_TEXT_UNKNOW_SW12	0x1067	
d_EMVAPLIB_ERR_CVM_TERMINAL_NOT_SUPPORT_SPECIFY_CVM	0x1068	
d_EMVAPLIB_ERR_CVM_ENC_PIN_ONLINE_PIN_NOT_KEYIN	0x1069	
d_EMVAPLIB_ERR_CVM_ENCIPHERED_PIN_NOT_KEYIN	0x106C	
d_EMVAPLIB_ERR_CVM_ENCIPHERED_PIN_TRY_LIMIT_EXCEEDED	0x106D	
d_EMVAPLIB_ERR_CVM_ENCIPHERED_PIN_UNKNOW_SW12	0x106E	
d_EMVAPLIB_ERR_CVM_ENCIPHERED_PIN_WORNG	0x1070	
d_EMVAPLIB_ERR_CVM_ENCIPHERED_PIN_GET_RN_UNKNOW_SW12	0x1071	
d_EMVAPLIB_ERR_DIS_GAC_UNKNOW_TAG	0x1072	
d_EMVAPLIB_ERR_MISSING_TERMINAL_DATA	0x1075	
d_EMVAPLIB_ERR_CARD_DATA_MULTIPLE	0x1076	
d_EMVAPLIB_ERR_CARD_DATA_BUF_OVERFLOW	0x1077	
d_EMVAPLIB_ERR_CAS_ONLINE_FAIL_DECLINED	0x1078	
d_EMVAPLIB_ERR_ISSUER_CERT_CAPKI_NOT_EXIST	0x1079	
d_EMVAPLIB_ERR_CAS_DDAAC_FAIL_NEED_GAC2	0x107B	
d_EMVAPLIB_ERR_SFI_TEMPLATE_RULE_WRONG	0x107C	
d_EMVAPLIB_ERR_CVM_PLAIN_TEXT_PIN_BLOCK	0x107D	
d_EMVAPLIB_ERR_CVM_ENCIPHERED_PIN_BLOCK	0x107E	
d_EMVAPLIB_ERR_CAS_GAC1_6985_FALLBACK	0x107F	
d_EMVAPLIB_ERR_IAP_BAD_PADDING	0x1081	
d_EMVAPLIB_ERR_CAS_GAC2_WRONG_CID	0x1082	
d_EMVAPLIB_ERR_DIS_GAC_ATC_LEN_WRONG	0x1083	
d_EMVAPLIB_ERR_DIS_GAC_DATA_MISSING	0x1084	
d_EMVAPLIB_ERR_PIN_CERT_EXP_NOT_EXIST	0x1085	
d_EMVAPLIB_ERR_DIS_GAC_AC_LEN_WRONG	0x1086	
d_EMVAPLIB_ERR_DIS_GAC_CID_LEN_WRONG	0x1087	
d_EMVAPLIB_ERR_IAP_FALLBACK_9481	0x1088	
d_EMVAPLIB_ERR_UP_APPLICATION_UNKNOW_SW12	0x1228	
d_EMVAPLIB_ERR_FINAL_SELECTION_SW_6283	0x1384	
d_EMVAPLIB_ERR_FINAL_SELECTION_SW_NON_9000_NON_6283	0x1385	
d_EMVAPLIB_ERR_FINAL_SELECTION_RSP_FORMAT_ERROR	0x1386	
d_EMVAPLIB_ERR_GPO_SW_6985	0x1387	
d_EMVAPLIB_ERR_CRITICAL_ERROR	0x7FFF	

Castles Technology

3.1.1.1.1 Interface: IEMVSpecialEvent

public interface **IEMVSpecialEvent**

The **IEMVSpecialEvent** interface is tag interface. It has no methods or fields and is used as primary datatype for entire Special Event.

Fields Summary
None

Method Summary	
None	

Castles Technology

3.1.1.1.2 Interface: IEventFirstGenACTermDecision

public interface **IEventFirstGenACTermDecision**
extends IEMVSpecialEvent
The **IEventFirstGenACTermDecision** interface extends IEMVSpecialEvent interface with method that user can implement in the application to make terminal decision of first GenAC.

Method Summary	
byte	eventFirstGenACTermDecision (byte currentTermDecision) Make terminal decision in First GenAC.

Method Detail

byte eventFirstGenACTermDecision(byte currentTermDecision);

This event allowes user to make terminal decision in first GenAC as their proprietary verification.

Parameters:

[IN] currentTermDecision – Current terminal decision made by EMV kernel according to the EMV specification.
0x01 – Approval
0x02 – Decline
0x03 – Go Online

Returns:

Terminal decision made by User.
0x01 – Approval
0x02 – Decline
0x03 – Go Online

Castles Technology

3.1.1.1.3 Interface: IEventTxnForcedOnline

public interface **IEventTxnForcedOnline**
extends IEMVSpecialEvent
The **IEventTxnForcedOnline** interface extends IEMVSpecialEvent interface with method that user can implement in the application to make transaction forced online.

Method Summary	
byte	eventTxnForcedOnline () Transaction forced online.

Method Detail
byte eventTxnForcedOnline(); This event allowes user to make transaction forced online. **Parameters:** None **Returns:** 0x01 – Forced online Other – Normal process

Castles Technology

3.1.1.1.4 Interface: IEventShowVirtualPIN

public interface **IEventShowVirtualPIN**
extends IEMVSpecialEvent
The **IEventShowVirtualPIN** interface extends IEMVSpecialEvent interface with method that user can implement in the application to show virtual PIN pad on the screen.

Method Summary	
EMVVirtual PINPara	eventShowVirtualPIN () Show virtual PIN pad on the screen.

Method Detail
EMVVirtualPINPara eventShowVirtualPIN(); This event will be triggered when EMV kerenl require PIN entry for offline PIN. Please return Activity and RelativeLayout back, EMV kernel will show the PIN pad for PIN entry. **Parameters:** None **Returns:** EMVVirtualPINPara – Activity and RelativeLayout. **Notes:** Only for terminals without physical keyboard.

Castles Technology

3.1.1.1.5 Interface: IEventGetPINDone

public interface **IEventGetPINDone**
extends IEMVSpecialEvent
The **IEventGetPINDone** interface extends IEMVSpecialEvent interface with method that user can implement in the application to get information of completing PIN entry.

Method Summary	
int	eventGetPINDone () Get information of completing PIN entry..

Method Detail
int eventGetPINDone(); This event will be triggered when Offline PIN entry is completed. **Parameters:** None **Returns:** 0x00 – Success

Castles Technology

3.1.1.1.6 Interface: IEventShowVirtualPINEx

public interface **IEventShowVirtualPINEx**
extends **IEMVSpecialEvent**
The **IEventShowVirtualPINEx** interface extends IEMVSpecialEvent interface with method that user can implement in the application to show the customized virtual PIN pad on the screen.

Method Summary	
int[][]	eventShowVirtualPINEx () Show virtual PIN pad on the screen then return customized keyboard information.

Method Detail

int[][] eventShowVirtualPINEx();

This event will be triggered when EMV kernel require PIN entry. Please customize the keyboard layout and return keyboard information (x, y, x+width, y+height, key value) back, EMV kernel will use these information to get PIN entry.

Parameters:

None

Returns:

int [][] – Customized keyboard layout. Keys with 5 parameters.
int[i][j] – i : key index ; j : parameters
[][0] – Position x.
[][1] – Position y.
[][2] – Position x + Width.
[][3] – Position y + Height.
[][4] – Key value.
Ex : Implement 16 keys, please declare buffer as int[16][5]

Notes: Only for terminals without physical keyboard.

Castles Technology

3.1.1.1.7 Interface: IEventOnlinePinBlockGet

public interface **IEventOnlinePinBlockGet**
extends **IEMVSpecialEvent**
The **IEventOnlinePinBlockGet** interface extends IEMVSpecialEvent interface with method that user can implement online PIN processing.

Method Summary	
int	eventOnlinePinBlockGet (EMVOnlinePinData onlinePinData) Implement online PIN processing.

Method Detail
int eventOnlinePinBlockGet(EMVOnlinePinData onlinePinData); This event will be triggered when EMV kerenl require Online PIN entry. **Parameters:** [OUT] onlinePinData – Online PIN data. **Returns:** d_EMVAPLIB_OK – Success. d_EMVAPLIB_ERR_PIN_BY_PASS – PIN bypass d_EMVAPLIB_ERR_CRITICAL_MISTAKES – Critical error

Castles Technology

3.1.1.1.8 Interface: IEventOfflinePINVerifyResult

public interface **IEventOfflinePINVertifyResult**
extends **IEMVSpecialEvent**
The **IEventOfflinePINVerifyResult** interface extends IEMVSpecialEvent interface with method that user can get the results of verification of Offline PIN.

Method Summary	
int	eventOfflinePINVerifyResult (int result) Get the result of verifying offline PIN.

Method Detail
int eventOfflinePINVerifyResult(int result); This event will be triggered after the verification of offline pin to inform the result of verification. **Parameters:** [IN] result – Result of verification of offline PIN d_PIN_RESULT_OK d_PIN_RESULT_FAIL d_PIN_RESULT_BLOCKED d_PIN_RESULT_FAILBLOCKED **Returns:** 0 – RFU

Castles Technology

3.1.1.1.9 Interface: IEventPINBypass

public interface **IEventPINBypass**
extends **IEMVSpecialEvent**
The **IEventPINBypass** interface extends IEMVSpecialEvent interface with method that user can give an action when PIN Bypass key is pressed

Method Summary	
byte	eventPINBypass () Action of PIN Bypass.

Method Detail
byte eventPINBypass(); This event will be triggered when PIN Bypass key is pressed in an offline pin process. The message of confirmation of PIN Bypass can be shown to the cardholder for confirming if they would like to PIN Bypass. The following actions can be brought back to the kernel to bypass PIN or restart a process of getting offline PIN. **Parameters:** None **Returns:** 0 - Action : Bypass PIN 1 - Action : Don't bypass PIN, back to get PIN process 2 - Action : Don't bypass PIN, trigger eventShowVirtualPINEx() again then back to get PIN process

Castles Technology

3.1.1.1.10 Interface: IEventForcedAcceptance

public interface **IEventForcedAcceptance**
extends **IEMVSpecialEvent**
The **IEventForcedAcceptance** interface extends IEMVSpecialEvent interface with method that user can determine if the transaction is forced acceptance.

Method Summary	
byte	eventForcedAcceptance () Action of forced acceptance.

Method Detail
byte eventForcedAcceptance(); This event allowes user to make transaction forced acceptance. **Parameters:** None **Returns:** 0x01 – Forced acceptance Other – Normal process

Castles Technology

3.1.1.1.11 Interface: IEventAppListEx

public interface **IEventAppListEx**
extends **IEMVSpecialEvent**
The **IEventAppListEx** interface extends IEMVSpecialEvent interface with method. The interface is used for event APP LIST ALWAYS.

Method Summary	
int	eventAppListEx (EMVAppListExData appListExData) Interface for event APP LIST ALWAYS (d_EVENTID_APP_LIST_ALWAYS)

Method Detail
int eventAppListEx(EMVAppListExData appListExData); When the card has multiple applications, this event function will be triggered to provide Application Labels for the application to show on the screen to let user do the selection. **If the interface is registered for APP LIST ALWAYS, the callback will always be triggered every transaction regardless of the candidate is 1 or more than 1. **Parameters:** [IN][OUT] appListExData – Class EMVAppListExData that consists of output variables to show application list for selection. **Returns:** d_EMVAPLIB_OK – Success. d_EMVAPLIB_ERR_ON_APP_LIST_FAIL –Transaction terminated as process failed inside eventAppListEx.

Castles Technology

3.1.1.2 Class: EMVEvent

public abstract class **EMVEvent**

The **EMVEvent** class consists of a set of callback functions which shall be implemented.

Constructor Summary
None

Method Summary	
int	onTxnDataGet(EMVTxnData txnData) After application selection, this event function will be triggered for the application to provide the transaction data, such as transaction amount, date, time, etc.
int	onAppListEx(EMVAppListExData appListExData) When the card has multiple applications, this event function will be triggered to provide Application Labels for the application to show on the screen to let user do the selection.
int	onAppSelectedConfirm(boolean isRequiredByCard, byte[] label, byte labelLen) When the card has only one application, this event function will be triggered with card requiring confirmation indicator for the application to show on the screen to let user do the confirmation.
int	onGetPINNotify(byte type, int remainingCounter, EMVGetPINFuncPara para) This event function will be triggered when the card requires the user to enter the online or offline PIN.
void	onShowPINDigit(byte digitsNum) This event function will be triggered when PINs are entered to show PIN digits.
Void	onShowPINBypass() This event function will be triggered when cardholder bypass the PIN entry.
void	onTxnResult(byte result, boolean isSignatureRequired) This event function will be triggered to inform transaction outcome.

Constructor Detail
None

Method Detail
public int onTxnDataGet(EMVTxnData txnData)

Castles Technology

After application selection, this event function will be triggered for the application to provide the transaction data, such as transaction amount, date, time, etc. **Parameters:** [OUT] txnData – Class EMVTxnData that consists of input variables to provide transaction data. **Returns:** d_EMVAPLIB_OK – Success. d_EMVAPLIB_ERR_ON_TXN_DATA_GET_FAIL – Transaction terminated as process failed inside onTxnDataGet.
public int onAppListEx(EMVAppListExData appListExData) When the card has multiple applications, this event function will be triggered to provide Application Labels for the application to show on the screen to let user do the selection. **Parameters:** [IN][OUT] appListExData – Class EMVAppListExData that consists of output variables to show application list for selection. **Returns:** d_EMVAPLIB_OK – Success. d_EMVAPLIB_ERR_ON_APP_LIST_FAIL –Transaction terminated as process failed inside onAppListEx.
public int onAppSelectedConfirm(boolean isRequiredByCard, byte[] label, byte labelLen) When the card has only one application, this event function will be triggered with card requiring confirmation indicator for the application to show on the screen to let user do the confirmation. **Parameters:** [IN] isRequiredByCard – Indicate if the confirmation action is required by card. [IN] label – Confirmed application label. [IN] labelLen – Length of the application label. **Returns:** d_EMVAPLIB_OK – Success. d_EMVAPLIB_ERR_APP_CONFIRM_FAIL –Transaction terminated as process failed inside onAppSelectedConfirm.
public int onGetPINNotify(byte type, int remainingCounter, EMVGetPINFuncPara para) This event function will be triggered when the card requires the user to enter the online or offline PIN. **Parameters:** [IN] type – Online or Offline PIN.

Castles Technology

0x00 – Online PIN
0x01 – Offline PIN
[IN] remainingCounter – Remaining PIN try counter if PIN type is offline PIN.
[OUT] para – Class EMVGetPINFuncPara that consists of input variables.

Returns:

d_EMVAPLIB_OK – Success.

public void onShowPINDigit(byte digitsNum)

This event function will be triggered when PINs are entered to show PIN digits.

Parameters:

[IN] digitsNum – Number of PIN digits.

Returns:

None

public void onShowPINBypass()

This event function will be triggered when cardholder bypass the PIN entry.

Parameters:

None

Returns:

None

public void onTxnResult(byte result, boolean isSignatureRequired)

This event function will be triggered to inform transaction outcome.

Parameters:

[IN] result – Transaction result.
0x01 – Approval
0x02 – Decline
0x03 – Go Online
[IN] isSignatureRequired – Signature required flag.
true – Signature is required
false – Signature is not required

Returns:

None

Castles Technology

3.1.1.3 Class: EMVAppInfo

public class **EMVAppInfo**

Fields Detail	
int	[OUT] version Version. Fixed to 0x01
byte[]	[OUT] appLabel = new byte[33] Application label.
byte	[OUT] appLabelLen Length of application label.
byte[]	[OUT] aid = new byte[16] Application identification.
byte	[OUT] aidLen Length of application identification.

Castles Technology

3.1.1.4 Class: EMVOnlineResponseData

public class **EMVOnlineResponseData**

Fields Detail	
int	[IN] version Version. Fixed to 0x01
Byte	[IN] action Action to info the EMV kernel the online result. 0x01 – Online approval 0x02 – Online decline 0x03 – Unable go online 0x04 – Issuer referral approval 0x05 – Issuer referral approval 0x06 – Unable go online with specific ARC
byte[]	[IN] authorizationCode = new byte[2] Response code from the host.
byte[]	[IN] issuerAuthenticationData Issuer Authentication Data from the issuer host. Input value of Tag 91 in HEX format
int	[IN] issuerAuthenticationDataLen Length of the Issuer Authentication Data..
byte[]	[IN] issuerScript Issuer Script from the issuer host. Input data is T(71/72)LV + T(71/72)LV + T(71/72)LV + … in HEX format
int	[IN] issuerScriptLen Length of the Issuer Script.

Castles Technology

3.1.1.5 Class: EMVTxnData

public class **EMVTxnData**

Fields Detail	
int	[OUT] version Version. Fixed to 0x03
byte[]	[OUT] amount = new byte[6] Amount. The value of Tag 9F02
byte[]	[OUT] amountOther = new byte[6] Amount Other. The value of Tag 9F03
byte	[OUT] posEntryMode Point-of-Service Entry Mode.
byte	[OUT] txnType Transaction type.
byte[]	[OUT] txnDate = new byte[6] Transaction Date, format is “YYMMDD”.
byte[]	[OUT] txnTime = new byte[6] Transaction Time, format is “HHMMSS”

Castles Technology

3.1.1.6 Class: EMVAppListExData

public class **EMVAppListExData**

Fields Detail	
int	[IN] version Version. Fixed to 0x00
short	[IN] appNum Number of application.
EMVAppInfo[]	[IN] appInfo Array of application info.
short	[OUT] appSelectedIndex Output selected application index, range of value is 0 to AppNum-1.

Castles Technology

3.1.1.7 Class: EMVGetPINFuncPara

public class **EMVGetPINFuncPara**

Fields Detail	
int	[OUT] version Version. Fixed to 0x01.
byte	[OUT] isInternalPINPad Indicate if the EMV Kernel used internal PINPad. Please set isInternalPINPad to 1 for offline PIN Please set isInternalPINPad to 0 for online PIN
Int	[OUT] timeout Waiting Time in seconds for PIN entry (0means infinite)
byte	[OUT] maxPINDigitLength Maximum number of PIN digits
byte	[OUT] minPINDigitLength Minimum number of PIN digits.
int	[OUT] onlinePINCipherKeySet Key set value of a KMS key for encrypting PIN
int	[OUT] onlinePINCipherKeyIndex Key index value of a KMS key for encrypting PIN

Castles Technology

3.1.1.8 Class: EMVVirtualPINPara
public class **EMVVirtualPINPara**

Fields Detail	
Activity	[OUT] activity Android Activity.
RelativeLayout	[OUT] layout Android RelativeLayout.

3.1.1.9 Class: EMVCandidateList
public class **EMVCandidateList**

Fields Detail	
int	[IN] version Version. Fixed to 0x01.
byte	[IN] newSel When newSel is TRUE, reload the candidate list from card, otherwise only remove the failed selection application and refresh the candidate list content.
int	[OUT] candidateNum Number of candidates.
EMVCandidateData[]	[OUT] candidateBuf Buffer for receiving candidates.
int	[IN] maxCandidateNum Maximum candidate buffer.
boolean	[OUT] isCardholderConfirmation Apply to only one candidate returned; check if cardholder confirmation is required.

Castles Technology

3.1.1.10 Class: EMVCandidateData

public class **EMVCandidateData**

Fields Detail	
int	[OUT] version Version. Fixed to 0x00.
byte[]	[OUT] appLabel = new byte[33] Application label.
byte	[OUT] appLabelLen Length of application label.
byte[]	[OUT] aid = new byte[16] AID.
byte	[OUT] aidLen Length of AID.

Castles Technology

3.1.1.11 Class: EMVTxnPSERsp

public class **EMVTxnPSERsp**

Fields Detail	
int	[IN] version Version. Fixed to 0x01.
byte[]	[OUT] rspBuf Buffer for receiving PSE response data.
int	[OUT] rspBufLen Length of PSE response data.
int	[IN] maxRspBufSize Maximum buffer size.

Castles Technology

3.1.1.12 Class: EMVCandidateDirEntry

public class **EMVCandidateDirEntry**

Fields Detail	
int	[IN] version Version. Fixed to 0x01.
int	[IN] appIdx Index of the selected application (from 0 and n), as the sequence returned by txnCandidateList.
byte[]	[OUT] dirEntry Buffer for receiving entry data.
int	[OUT] dirEntryLen Length of entry.
Int	[IN] maxDirEntrySize Maximum buffer size.

Castles Technology

3.1.1.13 Class: EMVSecureDataInfo

public class **EMVSecureDataInfo**

Fields Detail	
int	[IN] version Version. Set to 0x04 which is the latest supported version.
byte	[IN] keyType Key type. 0x01 – 3DES 0x02 – 3DES_DUKPT 0x03 – AES 0x07 – AES_DUKPT
int	[IN] cipherKeySet Key set.
int	[IN] cipherKeyIndex Key index.
byte	[IN] cipherMethod Cipher method. 0x00 – ECB 0x01 – CBC
byte[]	[IN] ICV ICV. Used when cipher method is CBC.
byte	[IN]ICVLen Length of ICV. 8 – when key type is 3DES or 3DES_DUKPT 16 – when key type is AES or AES_DUKPT
byte	[IN] checksumType Checksum type. 0x00 – SHA1 0x01 – MAC 0x02 – SHA256 0xFF – NONE
byte	[IN] paddingMethod Padding method. 0x00 – Padding 0x00. 0x01 – Padding 0xFF. 0x02 – Padding PKCS#7. 0x03 – Padding ISO/IEC 7816-4. Note: The paddingMethod is applied for version sets to 4 and above. Padding 0xFF is the default method when the version is set to under 4.
byte	[IN] LRCIncluded

Castles Technology

	Applicable to track data only. LRC is included in track data or not. 0x01 – Include LRC.
byte	[IN] SS_ESIncluded Applicable to track data only. SS (Start sentinel) and ES (End sentinel) are included in track data or not. 0x01 – Include SS ES.
byte	[IN] isKSNFixed Indicate whether to increase the KSN and generate the session key or not. If this field is TRUE, the KSN won't be increased and session key won't be re-generated, but it is required the session key already generated before.
int	[IN] derivedKeyType Derived key type for AES DUKPT. 0x00 – 2TDEA 0x01 – 3TDEA 0x02 – AES128 0x03 – AES192 0x04 – AES256
int	[IN] derivedKeyUsage Derived key usage for AES DUKPT. 0x05 – DataEncryptionEncrypt 0x06 – DataEncryptionDecrypt 0x07 – DataEncryptionBothWays
byte	[IN] macPara Use parameters macXXX for generating MAC. This item is applicable if checksum type is set to MAC. 0x00 – Generate MAC by using above parameters 0x01 – Generate MAC by using below macXXX paramaters
int	[IN] macCipherKeySet Key set used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01.
int	[IN] macCipherKeyIndex Key index used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01.
byte	[IN] macCipherMethod Cipher method used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01. 0x00 – MAC_METHOD_CBC 0x01 – MAC_METHOD_X9_19

Castles Technology

	0x02 – MAC_METHOD_X9_19_START 0x03 – MAC_METHOD_X9_19_UPDATE 0x04 – MAC_METHOD_X9_19_FINAL
byte[]	[IN] macICV ICV used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01.
byte	[IN] macICVLen Length of ICV used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01. 8 – when key type is 3DES or 3DES_DUKPT 16 – when key type is AES or AES_DUKPT
byte	[IN] macIsKSNFixed Indicate whether to increase the KSN used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01.
int	[IN] macDerivedKeyTyp Derived key type used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01. 0x00 – 2TDEA 0x01 – 3TDEA 0x02 – AES128 0x03 – AES192 0x04 – AES256
int	[IN] macDerivedKeyUsage Derived key usage used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01. 0x02 – MessageAuthenticationGeneration 0x03 – MessageAuthenticationVerification 0x04 – MessageAuthenticationBothWays

Castles Technology

3.1.1.14 Class: TlvData

public class **TlvData**

Fields Detail	
int	tag Tag.
int	len Length.
byte[]	value Value.

Castles Technology

3.1.1.15 Class: EMVDataGetExPara

public class **EMVDataGetExPara**

Fields Detail	
int	[IN] version Version. Fixed to 0x00.
byte[]	[IN] tagList Request tags (Tag1+Tag2+…+Tagn).
int	[IN] tagListLen Length of tagList.
byte[]	[OUT] tlvBuf Receive tag data (TLV1 + TLV2 + … + TLVn).
int	[OUT] tlvLen Length of tag data.
int	[IN] maxTlvBufSize Maximum buffer size.

Castles Technology

3.1.1.16 Class: EMVOnlinePinData

public class **EMVOnlinePinData**

Fields Detail	
int	[OUT] version Version. Fixed to 0x01.
boolean	[OUT] isOnlinePinRquired Online PIN required flag.
byte[]	[OUT] pin Online PIN block.
byte	[OUT] pinLen Length of online PIN block.

Castles Technology

3.1.1.17 Class: EMVApplicationPara

public class **EMVApplicationPara**

Fields Detail	
byte[]	[IN] aid = new byte[16] AID
byte	[IN] aidLen Length of AID.
byte	[IN] asi Application Selection Indicator. 0x00 – Partial matching 0x01 – Full matching

Castles Technology

3.1.1.18 Class: CAPublicKey

public class **CAPublicKey**

Fields Detail	
byte	[IN] index CAPK Index
int	[IN] modulusLen Length of modulus.
byte[]	[IN] modulus = new byte[248] Modulus.
int	[IN] exponentLen Length of exponent.
byte[]	[IN] exponent = new byte[3] Exponent.
byte[]	[IN] hash = new byte[20] Hash.

Castles Technology

3.1.1.19 Class: EMVByteBufData

public class **EMVByteBufData**

Fields Detail	
byte	[IN] index CAPK Index
int	[IN] modulusLen Data length

Castles Technology

3.1.1.20 Class: EMVCardAcquisitionData

public class **EMVCardAcquisitionData**

Fields Detail	
EMVByteBufData	[OUT]tlv All tags. Format: tlv + tlv + tlv …
EMVByteBufData	[OUT] aid AID.
EMVByteBufData	[OUT] pan PAN.
EMVByteBufData	[OUT] issuerCountryCode Issuer Country Code.

Castles Technology

3.1.1.21 Class: EMVConfigData

public class **EMVConfigData**

Fields Detail	
Int	[IN] version Version. Fixed to 0x00.
String	[IN] configFilename File name of configuration file.
byte	[IN] isSelActive Set to TRUE to Selcet Active.
byte	[IN] selIndex Select index if isSelActive is FALSE.
byte	[IN] action RFU.

Castles Technology

3.1.1.22 Class: CAPublicKeyEx

public class **CAPublicKeyEx**

Fields Detail	
Int	[IN] version Version. Fixed to 0x01.
byte	[IN] action 0x00 – Add CAPK (Fill in all data) 0x01 – Delete CAPK (Fill in RID and CAPK index)
byte[]	[IN] rid = new byte[5] RID
byte	[IN] index CAPK Index
int	[IN] modulusLen Length of modulus.
byte[]	[IN] modulus = new byte[248] Modulus.
int	[IN] exponentLen Length of exponent.
byte[]	[IN] exponent = new byte[3] Exponent.
byte[]	[IN] hash = new byte[20] Hash.

Castles Technology

3.1.2 Class: CtEMVMSR

public class **CtEMVMSR**
The **CtEMVMSR** class provides the functionalities to accomplish capture of encrypted MSR.

Constructor Summary	
Public	CtEMVMSR() Constructs a binder connected with service.

Method Summary	
int	initialize() Initialize CtEMVMSR environment.
int	enableTracks (boolean track1Enable, boolean track2Enable, boolean track3Enable) Enable capture of tracks.
int	setTracksEncryptInfo (EMVSecureDataInfo info) Set method of encrypting tracks.
int	setTracksEncrpytedSK (byte[] sessionKey, int sessionKeyLen) Set session key that participate in encrypting tracks.
int	setTracksMaskChar (byte maskChar) Set mask character for masking tracks.
int	setPANMaskChar (byte maskChar) Set mask character for masking PAN.
int	readTracks () Read magnetic stripe card.
EMVMSRTracksLen	getTracksLen () Get lengths of the actual tracks.
EMVMSREncrypted Tracks	getEncrptedTracks () Get encrypted tracks.
EMVMSRMaskedTracks	getMaskedTracks () Get masked tracks.
byte[]	getMaskedPAN () Get masked PAN.
Int	flushTracksBuffer () Flush tracks buffer.

Constructor Detail
public CtEMVMSR() Create a CtEMVMSR object and connect with binder to invoke service.

Method Detail
public final int initialize() Initialize the CtEMVMSR environment. **Parameters:** None **Returns:** d_EMVMSR_OK – Success. Other – Error code.
public final int enableTracks(boolean track1Enable, boolean track2Enable, boolean track3Enable) Enable capture of tracks. **Parameters:** [IN] track1Enable – enable capture of track 1. [IN] track2Enable – enable capture of track 2. [IN] track3Enable – enable capture of track 3. **Returns:** d_EMVMSR_OK – Success. Other – Error code.
public final int setTracksEncryptInfo(EMVSecureDataInfo info) Set method of encrypting tracks. **Parameters:** [IN] info – Encryption method. **Returns:** d_EMVMSR_OK – Success. Other – Error code.
public final int setTracksEncrpytedSK(byte[] sessionKey, int sessionKeyLen)

Castles Technology

Set session key that participate in encrypting tracks.
Parameters:
[IN] sessionKey – Session key.
[IN] sessionKeyLen – Length of session key.
Returns:
d_EMVMSR_OK – Success.
Other – Error code.

public final int setTracksMaskChar(byte maskChar)

Set mask character for masking tracks.
Parameters:
[IN] maskChar – Maskchar like ' * ', ' 0 '….
Returns:
d_EMVMSR_OK – Success.
Other – Error code.

public final int setPANMaskChar(byte maskChar)

Set mask character for masking PAN.
Parameters:
[IN] maskChar – Maskchar like ' * ', ' 0 '….
Returns:
d_EMVMSR_OK – Success.
Other – Error code.

public final int readTracks()

Read magnetic stripe card.
The following processing will be performed when magnetic stripe card has been reead successfully.
1. Encrypt tracks data by using encrypted method set in setTracksEncryptInfo() and session key (if need) set in setTracksEncrpytedSK().
2. Mask tracks by using mask character set in setTracksMaskChar().
3. Extract PAN from track and mask it by using mask character set in setPANMaskChar().
4. Flush the plaintext magnetic stripe data.
5. Return. Then user can call getTracksLen(),getEncrptedTracks() and getMaskedTracks() to get encrypted and masked tracks.
6. User can call getMaskedPAN() to get masked PAN.
Parameters:
None
Returns:
d_EMVMSR_OK – Read magnetic stripe card successfully.
Other – Error code.

public final EMVMSRTracksLen getTracksLen()

This method should be called after readTracks() to get lengths of the actual tracks.
Parameters:
None
Returns:
EMVMSRTracksLen – Actual lengths of tracks.

public final EMVMSREncryptedTracks getEncrptedTracks()

This method should be called after readTracks() to get encrypted tracks.
Parameters:
None
Parameters:
EMVMSREncryptedTracks – Encrypted tracks.

public final EMVMSRMaskedTracks getMaskedTracks()

This method should be called after readTracks() to get masked tracks.
Parameters:
None
Parameters:
EMVMSRMaskedTracks – Masked tracks.

public final byte[] getMaskedPAN()

This method should be called after readTracks() to get masked PAN.
Parameters:
None
Parameters:
byte[] – Masked PAN.

public final int flushTracksBuffer()

Flush tracks buffer.
Parameters:

Castles Technology

None

Returns:

d_EMVMSR_OK – Success.

Fields Detail

Return Code (public static final int)

Constant Field	Value	Additional Description
d_EMVMSR_OK	0x00000000	
d_EMVMSR_ERR_CRITICAL_ERR	0x61000001	
d_EMVMSR_ERR_ENCRYPTINFO_ICV_ERR_OR_MISSNG	0x61000002	
d_EMVMSR_ERR_ENCRYPTINFO_VRESION_ERR	0x61000003	
d_EMVMSR_ERR_MASK_PAN_FAIL	0x61000004	
d_EMVMSR_ERR_ENRYPT_KEY_NOT_EXIST	0x61000005	
d_EMVMSR_ERR_NO_SWIPE	0x61000007	
d_EMVMSR_ERR_TRACK_ERROR	0x61000008	
d_EMVMSR_ERR_ENCRYPTINFO_NOT_SET	0x61000009	
d_EMVMSR_ERR_DUKPT_KSN_FIXED_IN_FIRST_ENCRYPT	0x6100000A	
d_EMVMSR_ERR_INVALID_INPUT_PARA	0x6100000B	

Castles Technology

3.1.2.1 Class: EMVMSRTracksLen

public class **EMVMSRTracksLen**

Fields Detail	
Int	[OUT] version Version. Fixed to 0x00.
Boolean	[OUT] track1Enabled The capture of track 1 is enabled or not.
Int	[OUT] track1Len Length of actual track 1.
Boolean	[OUT] track2Enabled The capture of track 2 is enabled or not.
Int	[OUT] track2Len Length of actual track 2.
Boolean	[OUT] track3Enabled The capture of track 3 is enabled or not.
Int	[OUT] track3Len Length of actual track 3.

Castles Technology

3.1.2.2 Class: EMVMSREncryptedTracks

public class **EMVMSREncryptedTracks**

Fields Detail	
Int	[OUT] version Version. Fixed to 0x00.
Int	[OUT] track1EncryptedDataLen Length of encrypted track 1.
byte[]	[OUT] track1EncryptedData Encrypted track 1.
Int	[OUT] track1ChecksumLen Length of checksum of track 1.
byte[]	[OUT] track1Checksum Checksum of track 1.
Int	[OUT] track1KSNLen Length of KSN of track 1.
byte[]	[OUT] track1KSN KSN of track 1.
Int	[OUT] track2EncryptedDataLen Length of encrypted track 2.
byte[]	[OUT] track2EncryptedData Encrypted track 2.
Int	[OUT] track2ChecksumLen Length of checksum of track 2.
byte[]	[OUT] track2Checksum Checksum of track 2.
Int	[OUT] track2KSNLen Length of KSN of track 2.
byte[]	[OUT] track2KSN KSN of track 2.
Int	[OUT] track3EncryptedDataLen Length of encrypted track 3.
byte[]	[OUT] track3EncryptedData Encrypted track 3.
Int	[OUT] track3ChecksumLen Length of checksum of track 3.
byte[]	[OUT] track3Checksum Checksum of track 3.
Int	[OUT] track3KSNLen Length of KSN of track 3.
byte[]	[OUT] track3KSN KSN of track 3.

Castles Technology

3.1.2.3 Class: EMVMSRMaskedTracks

public class **EMVMSRMaskedTracks**

tail	
int	[OUT] Version Version. Fixed to 0x00.
int	[OUT] track1maskedDataLen Length of masked track 1.
byte[]	[OUT] track1maskedData Masked track 1.
int	[OUT] track2maskedDataLen Length of masked track 2.
byte[]	[OUT] track2maskedData Masked track 2.
int	[OUT] track3maskedDataLen Length of masked track 3.
byte[]	[OUT] track3maskedData Masked track 3.

Castles Technology

3.1.3 Class: CtEMVManualEntry

public class **CtEMVManualEntry**
The **CtEMVManualEntry** class provides the functionalities to accomplish manual entry, that the card number, expiration date and card security code are typed then packed as track format, encrypted and masked.

Constructor Summary	
public	CtEMVManualEntry () Constructs a binder connected with service.

mmary	
int	initialize() Initialize CtEMVManualEntry environment.
int	registerEvent(EMVManualEntryEvent event) Register callback event.
int	setEncryptInfo(EMVSecureDataInfo info) Set method of encrypting manual entry.
int	setEncrpytedSK(byte[] sessionKey, int sessionKeyLen) Set session key that participate in encrypting manual entry.
int	setMaskChar(byte maskChar) Set mask character for masking manual entry.
int	maskPANDigits1To6(byte maskDigits) Digits 1 to 6 of PAN are masked or not.
int	maskPANDigits13To16(byte maskDigits) Digits 13 to 16 of PAN are masked or not.
int	maskEXPDigits1To4(byte maskDigits) Digits 1 to 4 of expiration date are masked or not.
int	keyinPAN(int[][] kbd) Key-in card number.
int	keyinEXP(int[][] kbd) Key-in card expiration date.
int	keyinCSC(int[][] kbd) Key-in card security code.
int	keyinADR(int[][] kbd) Key-in address.
int	keyinZIP(int[][] kbd) Key-in post code.
int	keyinDone() Key-in manual entry finished. Pack manual entry as track format then encrypt and mask the data.
int	getEncrptedManualEntry(EMVManualEntryEncryptedData encData) Get encrypted manual entry.
int	getManualEntryLen() Get actual length of the manual entry.
byte[]	getMaskedManualEntry() Get masked manual entry.
byte[]	getMaskedPAN () Get masked PAN.
byte[]	getAdditionalEntry () Get additional manual entry.
int	flushBuffer() Flush buffer.
int	setEXPKeyInFormat () Set format for keying in the card expiration date.
int	setPANBackspaceMode () Set backspace mode for keying in the card number.
int	setEXPBackspaceMode () Set backspace mode for keying in the card expiration date.
int	setCSCBackspaceMode () Set backspace mode for keying in the card security code.
int	setADRBackspaceMode () Set backspace mode for keying in the address.
int	setZIPBackspaceMode () Set backspace mode for keying in the post code.

Constructor Detail
public CtEMVManualEntry() Create a CtEMVManualEntry object and connect with binder to invoke service.

Castles Technology

Method Detail

public int initialize()

Initialize the CtEMVManualEntry environment.
Parameters:
None
Returns:
d_EMVMKE_OK – Success.

public final int registerEvent(EMVManualEntryEvent event)

Register callback function of CtEMVManualEntry.
Parameters:
[IN] event – Class EMVManualEntryEvent that implements mandatorycallback event.
Returns:
d_EMVMKE_OK – Success.
Other – Error code.

public final int setEncryptInfo(EMVSecureDataInfo info)

Set method of encrypting manual entry.
Parameters:
[IN] info – Encryption method.
Returns:
d_EMVMKE_OK – Success.
Other – Error code.

public final int setEncrpytedSK(byte[] sessionKey, int sessionKeyLen)

Set session key that participate in encrypting manual entry.
Parameters:
[IN] sessionKey – Session key.
[IN] sessionKeyLen – Length of session key.
Returns:
d_EMVMKE_OK – Success.
Other – Error code.

public final int setMaskChar(byte maskChar)

Set mask character for masking manual entry.
Parameters:
[IN] maskChar – Maskchar like ' * ', ' 0 '….
Returns:
d_EMVMKE_OK – Success.
Other – Error code.

public final int maskPANDigits1To6(byte maskDigits)

Set masked status of Digits 1 to 6 of PAN. (D1D2D3D4 D5D6XX XXXX XXXX)
Parameters:
[IN] maskDigits – Masked status of digits. One byte (bit 8 bit 7 bit6 …..bit 2 bit 1).
bit 8 : 1 – Digit 1 is maked.
bit 7 : 1 – Digit 2 is maked.
bit 6 : 1 – Digit 3 is maked.
bit 5 : 1 – Digit 4 is maked.
bit 4 : 1 – Digit 5 is maked.
bit 3 : 1 – Digit 6 is maked.
Returns:
d_EMVMKE_OK – Success.
Other – Error code.

public final int maskPANDigits13To16(byte maskDigits)

Set masked status of Digits 13 to 16 of PAN. (XXXX XXXX XXXX D13D14D15D16)
Parameters:
[IN] maskDigits – Masked status of digits. One byte (bit 8 bit 7 bit6 …..bit 2 bit 1).
bit 8 : 1 – Digit 13 is maked.
bit 7 : 1 – Digit 14 is maked.
bit 6 : 1 – Digit 15 is maked.
bit 5 : 1 – Digit 16 is maked.
Returns:
d_EMVMKE_OK – Success.
Other – Error code.

public final int maskEXPDigits1To4(byte maskDigits)

Set masked status of Digits 1 to 4 of expiration date. (YYMM or MM/YY : D1D2/D3D4)
Parameters:

Castles Technology

[IN] maskDigits – Masked status of digits. One byte (bit 8 bit 7 bit6 …..bit 2 bit 1).
bit 8 : 1 – Digit 1 is maked.
bit 7 : 1 – Digit 2 is maked.
bit 6 : 1 – Digit 3 is maked.
bit 5 : 1 – Digit 4 is maked.

Returns:

d_EMVMKE_OK – Success.
Other – Error code.

public final int keyinPAN(int[][] kbd)

Key-in card number.

Parameters:

[IN] kbd – Customized keyboard layout. One key with 5 parameters.
int[i][j] – i : key index ; j : parameters
[][0] – Possition x.
[][1] – Possition y.
[][2] – Possition x + Width.
[][3] – Possition y + Height.
[][4] – Key value.
Ex : Implement 16 keys, please declare buffer as int[16][5].

Returns:

d_EMVMKE_OK – Success.
Other – Error code.

public final int keyinEXP(int[][] kbd)

Key-in card expiration date. The key-in format is YYMM by default. Calling setEXPKeyInFormat is able to set the format.

Parameters:

[IN] kbd – Customized keyboard layout. One key with 5 parameters.
int[i][j] – i : key index ; j : parameters
[][0] – Possition x.
[][1] – Possition y.
[][2] – Possition x + Width.
[][3] – Possition y + Height.
[][4] – Key value.
Ex : Implement 16 keys, please declare buffer as int[16][5].

Returns:

d_EMVMKE_OK – Success.
Other – Error code.

public final int keyinCSC(int[][] kbd)

Key-in card security code.

Parameters:

[IN] kbd – Customized keyboard layout. One key with 5 parameters.
int[i][j] – i : key index ; j : parameters
[][0] – Possition x.
[][1] – Possition y.
[][2] – Possition x + Width.
[][3] – Possition y + Height.
[][4] – Key value.
Ex : Implement 16 keys, please declare buffer as int[16][5].

Returns:

d_EMVMKE_OK – Success.
Other – Error code.

public final int keyinADR(int[][] kbd)

Key-in address.

Parameters:

[IN] kbd – Customized keyboard layout. One key with 5 parameters.
int[i][j] – i : key index ; j : parameters
[][0] – Possition x.
[][1] – Possition y.
[][2] – Possition x + Width.
[][3] – Possition y + Height.
[][4] – Key value.
Ex : Implement 16 keys, please declare buffer as int[16][5].

Returns:

d_EMVMKE_OK – Success.
Other – Error code.

public final int keyinZIP(int[][] kbd)

Key-in post code.

Parameters:

[IN] kbd – Customized keyboard layout. One key with 5 parameters.
int[i][j] – i : key index ; j : parameters
[][0] – Possition x.

Castles Technology

[][1] – Possition y.
[][2] – Possition x + Width.
[][3] – Possition y + Height.
[][4] – Key value.
Ex : Implement 16 keys, please declare buffer as int[16][5].

Returns:
d_EMVMKE_OK – Success.
Other – Error code.

---

public final int keyin keyinDone ()

Key-in manual entry finished. The following processing will be performed.
1. Pack manual entries as track format(ASCII) : ;PAN=EXP:CSC?LRC
1a. Pack additional entry : 1ADR=0ZIP=
2. Encrypt data by using encrypted method set in setEncryptInfo() and session key (if need) set in setEncrpytedSK().
3. Mask data and PAN by using mask character set in setMaskChar().
4. Flush the plaintext key-in data.
5. Return status.
Then user can call getEncrptedManualEntry (), getManualEntryLen() and getMaskedManualEntry() to get encrypted and masked data.
User can call getMaskedPAN() to get masked PAN.
User can call getAdditionalEntry() to get additional entry.

Parameters:
None

Returns:
d_EMVMKE_OK – Success.
Other – Error code.

---

public final int getEncrptedManualEntry(EMVManualEntryEncryptedData encData)

This method should be called after keyinDone() to get encrypted manual entry data.

Parameters:
[OUT] encData – Encrypted data of manual entry.

Returns:
d_EMVMKE_OK – Success.
Other – Error code.

---

public final int getManualEntryLen()

This method should be called after keyinDone() to get actual length of the manual entry.

Parameters:
None

Returns:
int – Actual length of manual entry.

---

public final byte[] getMaskedManualEntry()

This method should be called after keyinDone() to get masked data of manual entry.

Parameters:
None

Returns:
byte[] – Masked data.

---

public final byte[] getMaskedPAN()

This method should be called after keyinDone() to get masked PAN.

Parameters:
None

Returns:
byte[] – Masked PAN.

---

public final byte[] getAdditionalEntry()

This method should be called after keyinDone() to get additional manual entry (address and.or post code).

Parameters:
None

Returns:
byte[] – Additional manual entry.

---

public final int flushBuffer()

Flush working buffer.

Parameters:
None

Returns:
d_EMVMKE_OK – Success.
Other – Error code.

---

public final int setEXPKeyInFormat(int format)

Set format for keying in the card expiration date.

Parameters:

Castles Technology

[IN] format – Key-in format
0x01 – Key-in MMYY
Other – Key-in YYMM (Default)

Returns:

d_EMVMKE_OK – Success.
Other – Error code.

public final int setPANBackspaceMode(byte mode)

Set backspace mode for keying in the card number.

Parameters:

[IN] mode – behavior while pressing the backspace key
0x01 – Clear the last digit
Other – Clear all digits (Default)

Returns:

d_EMVMKE_OK – Success.
Other – Error code.

public final int setEXPBackspaceMode(byte mode)

Set backspace mode for keying in the card expiration date.

Parameters:

[IN] mode – behavior while pressing the backspace key
0x01 – Clear the last digit
Other – Clear all digits (Default)

Returns:

d_EMVMKE_OK – Success.
Other – Error code.

public final int setCSCBackspaceMode(byte mode)

Set backspace mode for keying in the card security code.

Parameters:

[IN] mode – behavior while pressing the backspace key
0x01 – Clear the last digit
Other – Clear all digits (Default)

Returns:

d_EMVMKE_OK – Success.
Other – Error code.

public final int setADRBackspaceMode(byte mode)

Set backspace mode for keying in the address.

Parameters:

[IN] mode – behavior while pressing the backspace key
0x01 – Clear the last digit
Other – Clear all digits (Default)

Returns:

d_EMVMKE_OK – Success.
Other – Error code.

public final int setZIPBackspaceMode(byte mode)

Set backspace mode for keying in the post code.

Parameters:

[IN] mode – behavior while pressing the backspace key
0x01 – Clear the last digit
Other – Clear all digits (Default)

Returns:

d_EMVMKE_OK – Success.
Other – Error code.

Castles Technology

3.1.3.1 Class: EMVManualEntryEvent

public abstract class **EMVManualEntryEvent**

The **EMVManualEntryEvent** class consists of a set of callback functions which shall be implemented.

Constructor Summary
None

Method Summary	
void	onShowPAN(byte digitChar) This event function will be triggered during key-in PAN to show PAN on screen.
Void	onShowEXP(byte digitChar) This event function will be triggered during key-in EXP to show expiration date on screen.
void	onShowCSC(byte digitChar) This event function will be triggered during key-in CSC to show card security code on the screen.
void	onShowADR(byte digitChar) This event function will be triggered during key-in ADR to show address on the screen.
void	onShowZIP(byte digitChar) This event function will be triggered during key-in ZIP to show post code on the screen.

Constructor Detail
None

Method Detail
public abstract void onShowPAN(byte digitChar) This event function will be triggered during key-in PAN to show PAN on screen. **Parameters:** [IN] digitChar – Character shown on screen or command data for special behavior. 0x08 – Command data means clearing all input. other – ASCII character shown on screen, number 0-9 or masked character. 0xA0 – Command data means clearing the last input. **Returns:** None
public abstract void onShowEXP(byte digitChar)

Castles Technology

This event function will be triggered during key-in EXP to show expiration date on screen.

Parameters:

[IN] digitChar – Character shown on screen or command data for special behavior.

0x08 – Command data means clearing all input.

other – ASCII character shown on screen, number 0-9 or masked character.

0xA0 – Command data means clearing the last input.

Returns:

None

---

public abstract void onShowCSC(byte digitChar);

This event function will be triggered during key-in CSC to show card security code on the screen.

Parameters:

[IN] digitChar – Character shown on screen or command data for special behavior.

0x08 – Command data means clearing all input.

other – ASCII character shown on screen, only masked character.

0xA0 – Command data means clearing the last input.

Returns:

None

---

public abstract void onShowADR(byte digitChar);

This event function will be triggered during key-in ADR to show address on the screen.

Parameters:

[IN] digitChar – Character shown on screen or command data for special behavior.

0x08 – Command data means clearing all input.

other – ASCII character shown on screen, only masked character.

0xA0 – Command data means clearing the last input.

Returns:

None

---

public abstract void onShowZIP(byte digitChar);

This event function will be triggered during key-in ZIP to show card security code on the screen.

Parameters:

[IN] digitChar – Character shown on screen or command data for special behavior.

Castles Technology

0x08 – Command data means clearing all input.
other – ASCII character shown on screen, only masked character.
0xA0 – Command data means clearing the last input.

Returns:

None

Fields Detail		
Return Code (public static final int)		
Constant Field	Value	Additional Description
d_EMVMKE_OK	0x00000000	
d_EMVMKE_ERR_CRITICAL_ERR	0x62000001	
d_EMVMKE_ERR_ENCRYPTINFO_ICV_ERR_OR_MISSNG	0x62000002	
d_EMVMKE_ERR_ENCRYPTINFO_VRESION_ERR	0x62000003	
d_EMVMKE_ERR_NOT_KEY_IN_DONE	0x62000004	
d_EMVMKE_ERR_INVALID_INPUT_PARA	0x62000005	
d_EMVMKE_ERR_ENTRY_ENC_ERR	0x62000006	
d_EMVMKE_ERR_ENTRY_CHECKSUM_ERR	0x62000007	
d_EMVMKE_ERR_ENTRY_GEN_LRC_ERR	0x62000008	
d_EMVMKE_ERR_ENCRYPT_DATA_NOT_READ	0x62000009	
d_EMVMKE_ERR_KEY_IN_CANCEL	0x6200000A	
d_EMVMKE_ERR_PIN_PAD_MALFUNCTION	0x6200000B	

Castles Technology

3.1.3.2 Class: EMVManualEntryEncryptedData

public class **EMVManualEntryEncryptedData**

Fields Detail	
int	[OUT] encryptedDataLen Length of encrypted manual entry.
byte[]	[OUT] encryptedData Encrypted manual entry.
int	[OUT] checksumLen Length of checksum of manual entry.
byte[]	[OUT] Checksum Checksum of manual entry.
int	[OUT] KSNLen Length of KSN of manual entry.
byte[]	[OUT] KSN KSN of manual entry.
int	[OUT] additionalEncryptedDataLen Additional Length of encrypted manual entry.
byte[]	[OUT] additionalEncryptedData Additional Encrypted manual entry.
Int	[OUT] additionalChecksumLen Additional Length of checksum of manual entry.
byte[]	[OUT] additionalChecksum Additional Checksum of manual entry.
Int	[OUT] additionalKSNLen Additional Length of KSN of manual entry.
byte[]	[OUT] additionalKSN Additional KSN of manual entry.

Castles Technology

4. EMVCL

4.1 CAEMVCL Application Programming Interfaces

Package CTOS
Contains all of the classes for Castles EMVCL user API.

Class Summary	
CtEMVCL	This class provides the functionality to accomplish EMV contactless transaction.

Package CTOS.emvcl
Contains all of the classes, which are used in methods of CtEMVCL

Class Summary	
EMVCLActData	This class consists of a set of variables and is used as transaction related data.
EMVCLRcDataEx	This class consists of a set of variables and is used for receiving transaction result.
EMVCLRcDataAnalyze	This class consists of a set of variables and is used for analyzing transaction result.
EMVCLDetectTxnData	This class consists of a set of variables and is used for detecting card application.
EMVCLGetCardData	This class consists of a set of variables and is used for requesting card data.
EMVCLAidSetTagData	This class consists of a set of variables and is used for setting tag data.
EMVCLAidGetTagData	This class consists of a set of variables and is used for getting tag data.
EMVCLAidCombinationList	This class consists of a set of variables and is used for getting AID combination list.
EMVCLCAPublicKey	This class consists of a set of variables and is used for setting CAPK.
EMVCLParameterData	This class consists of a set of variables and is used for setting Parameter.
EMVCLUIReqData	This class consists of a set of variables and is used for getting UI Request Data.
EMVCLSecureDataInfo	This class consists of a set of variables and is used in method secureDataEncryptInfoSet of CtEMV.CL
EMVCLAppInfo	This class consists of a set of variables and is used to provide card application information, AID and App label, for application selection.
EMVCLAppListV2	This class consists of a set of variables and is used to provide the candidate list for application selection.
EMVCLByteBufData	This class consists of byte array and length.
EMVCLCardAcquisitionData	This class consists of a set of variables and is used for getting card acquisition data.
EMVCLConfigData	This class consists of a set of variables and is used to select a specific configuration in the configuration file.
EMVCLCAPublicKeyEx	This class consists of a set of variables and is used for setting multiple CAPKs at once.
EMVCLAppListV3	This class consists of a set of variables and is used to provide the candidate list for application selection.
EMVCLAppListV4	This class consists of a set of variables and is used to provide the candidate list for application selection.

Note: For the EMVCL example, please refer to the example which contains to the SDK.
The file path is "C:\Program Files (x86)\Castles\SATURN1000\examples\EMVCL_APIDemo"

Castles Technology

4.1.1 Class: CtEMVCL

public class **CtEMVCL**
The **CtEMVCL** class provides the functionalities to accomplish EMV contactless transaction.

Constructor Summary	
public	CtEMVCL() Constructs a binder connected with service.

Fields Summary	
static byte	d_EMVCL_EVENTID_LED_PIC_SHOW Special Event Id used in method specialEventRegister.
static byte	d_EMVCL_EVENTID_AUDIO_INDICATION Special Event Id used in method specialEventRegister.
static byte	d_EMVCL_EVENTID_CANCEL_TRANSACTION Special Event Id used in method specialEventRegister.
static byte	d_EMVCL_EVENTID_SHOW_MESSAGE Special Event Id used in method specialEventRegister.
static byte	d_EMVCL_EVENTID_APP_LIST_V2 Special Event Id used in method specialEventRegister.
static byte	d_EMVCL_EVENTID_NON_EMV_CARD Special Event Id used in method specialEventRegister.
static byte	d_EMVCL_EVENTID_PRE_NON_EMV_CARD Special Event Id used in method specialEventRegister.
static byte	d_EMVCL_EVENTID_RMR_DATA_EXCHANGE Special Event Id used in method specialEventRegister.
static byte	d_EMVCL_EVENTID_SHOW_VIRTUAL_PIN_EX Special Event Id used in method specialEventRegister
static byte	d_EMVCL_EVENTID_APP_LIST_V4 Special Event Id used in method specialEventRegister
static byte	d_EMVCL_EVENTID_PIN_BYPASS Special Event Id used in method specialEventRegister
static byte	d_EMVCL_EVENTID_SHOW_PIN_BYPASS Special Event Id used in method specialEventRegister
static byte	EVENT_EMVCL_PIN_TRY_COUNTER Special Event Id used in method specialEventRegister
static byte	d_EMVCL_EVENTID_OFFLINE_PIN_VERIFY_RESULT Special Event Id used in method specialEventRegister
static byte	d_EMVCL_EVENTID_CANCEL_PIN_CONFIRM Special Event Id used in method specialEventRegister
static byte	d_EMVCL_EVENTID_GET_PIN_DONE Special Event Id used in method specialEventRegister
static byte	d_EMVCL_EVENTID_SHOW_PIN_DIGIT Special Event Id used in method specialEventRegister

Method Summary	
int	initialize() Initalize.
int	setConfiguration(InputStream inStream) Set Configuration. The tags and CAPKs …etc. in the configuration data will be loaded to EMVCL kernel.
int	setConfiguration (String filename, InputStream inStream) The configuration file is sent from android app to the EMVCL kernel in the secure module. The tags and CAPKs …etc. in the configuration file will be loaded to EMVCL kernel.
int	setConfiguration (String filename) Inform the secure module to load configuration with the input file name.
int	setConfigurationFile (String filename, InputStream inStream) The configuration file is sent from android app to the EMVCL kernel in the secure module. The configuration will NOT be loaded to the EMVCL kernel.
int	setConfigurationEx (EMVCLConfigData configData) The configuration file is sent from android app to the EMVCL kernel in the secure module. There are conditions to select specific configuration and load it to EMVCL kernel.
int	delConfigurationFileAll () Delete all configuration files stored in the EMVCL kernel.
int	delConfigurationFile (String fileName) Detete the specific configuration file.
Int	startTransactionEx(EMVCLActData actData, EMVCLRcDataEx rcDataEx) Start contactless transaction.
int	initTransactionEx (short tagNum, byte[] transData, int transDataLen) Init contactless transaction.
int	performTransactionEx (EMVCLRcDataEx rcDataEx)

Castles Technology

	Poll contactless card, perform contactless transaction and get transaction result.
int	analyzeTransactionEx (EMVCLRcDataEx rcDataEx, EMVCLRcDataAnalyze rcDataAnalyze) Analyze transaction result.
int	completeEx (byte action, byte[] arc, int iadLen, byte[] iad, int scriptLen, byte[] script, EMVCLRcDataEx rcDataEx) Completion / Issuer Update Processing.
int	detectTransactionEx (EMVCLDetectTxnData detectTxnData) Detect card application and return PPSE response, selected AID and select AID response.
int	getCardData (EMVCLGetCardData cardData) Request card data.
int	cancelTransaction () Cancel transaction.
int	aidSetTagData (byte action, EMVCLAidSetTagData tagData) Set tag data to specific AID.
int	aidGetTagData (EMVCLAidGetTagData tagData) Get tag data of specific AID.
int	aidGetCombinationList (EMVCLAidCombinationList aidCombiList) Get all of AID combination list {Transaction type – AID – Kernel ID}.
int	setCAPK (byte[] rid, EMVCLCAPublicKey capk) Set CA Public Key.
int	getCAPK (byte[] rid, byte index, EMVCLCAPublicKey capk) Get CA Public Key.
int	listAllCAPKId (EMVCLByteBufData capkIdList) List all CA Public Key Indexs.
int	setParameter (EMVCLParameterData paraData) Set parameter data.
int	getParameter (int index, EMVCLParameterData paraData) Get parameter data.
int	specialEventRegister (byte eventId, ISpecialEvent spEvent) Register special event (callback function) to EMVCL kernel, provided by the user application using.
int	secureDataEncryptInfoSet(EMVCLSecureDataInfo info) Set encryption method of security data.
int	secureDataWhitelistSet(byte action, byte[] listData, int listDataLen) Set whitelist of security data.
int	setMaskChar(byte maskChar) Set mask character of security data.
int	maskPANDigits1To6(byte maskDigits) Digits 1 to 6 of PAN are masked or not.
int	maskPANDigits13To16(byte maskDigits) Digits 13 to 16 of PAN are masked or not.
int	maskEXPDigits1To4(byte maskDigits) Digits 1 to 4 of expiration date of track 2 data are masked or not.
int	maskCSCDigits1To3 (byte maskDigits) Digits 1 to 3 of card service code of track 2 data are masked or not.
int	getLastError () Get detail error code.
int	detectCard () Detect whether card is presented on the CL field.
int	performCardAcquisition () Card data acquisition. For each AID in the card, collecting card tags from response of SELECT AID command to response of READ RECORD command.
int	getCardAcquisitionAppNum () Get the number of card acquisition data (the same as AID number in the card). Call this after performCardAcquisition().
int	getCardAcquisitionAppData (EMVCLCardAcquisitionData[] acq) Input buffer to get card acquisition data. Call this after performCardAcquisition()
int	changeWDBPath (String path) The EMVCL kernel default set the working database path to the Android Application ID and store the files such as the xml configuration files in it. Each Android App has its configurations. This method is used to change the working database path If the developer would like to share the configurations between multiple Android App.
int	getWDBPath () Get the current working database path. If the developer does not change WDB path, the default working database path is the string of Android Application ID.
int	resetWDBPath () Reset the working database path to default.
String	getRtnString () This method is used after specific methods to get String variable.
int	setReadyToReadBehavior (byte mode) This method is used to set an action for “Ready to Read” status of user interface of EMV Book A.

Interface Summary	
ISpecialEvent	Tag interface for special event.
IEventEMVCLLEDPicShow	Special event : implement proprietary contactless LED.
IEventEMVCLAudioIndication	Special event : implement proprietary audio indication.

Castles Technology

IEventEMVCLCancelTransaction	Special event : cancel transaction.
IEventEMVCLShowMessage	Special event : show message.
IEventEMVCLAppListV2	Special event : list candidate for application selection.
IEventEMVCLNonEMVCard	Special event : non EMV card process
IEventEMVCLPreNonEMVCard	Special event : pre non EMV card process
IEventEMVCLRMRDataExchange	Special event : data exchange process for MIR kernel
IEventEMVCLAppListV3	Special event : list candidate for application selection ver3.
IEventEMVCLAppListV4	Special event : list candidate for application selection ver4.
IEventEMVCLShowVirtualPIN	Special event : show virtual PIN pad on the screen. (used for girocard kernel only)
IEventEMVCLShowPINDigit	Special event : inform PIN digits that have been entered. (used for girocard kernel only)
IEventEMVCLGetPINDone	Special event : get information of completing PIN entry. (used for girocard kernel only)
IEventEMVCLShowPinBypass	Special event : inform cardholder bypass the PIN entry. (used for girocard kernel only)
IEventEMVCLPinBypass	Special event : action of PIN Bypass. (used for girocard kernel only)
IEventEMVCLCancelPinConfirm	Special event : action of cancel PIN entry. (used for girocard kernel only)
IEventEMVCLOfflinePinVerifyResult	Special event : get the result of verifying offline PIN. (used for girocard kernel only)
IEventEMVCLPinTryCounter	Special event : get PIN Try Counter. (used for girocard kernel only)

Constructor Detail

public CtEMVCL()
Create a CtEMVCL object and connect with binder to invoke service.

Fields Detail

public static final byte d_EMVCL_EVENTID_LED_PIC_SHOW = 0x01;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_AUDIO_INDICATION = 0x04;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_CANCEL_TRANSACTION = 0xC0;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_SHOW_MESSAGE = 0xC1;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_APP_LIST_V2 = 0x07;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_NON_EMV_CARD = 0x05;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_PRE_NON_EMV_CARD = 0x0C;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_RMR_DATA_EXCHANGE = 0x10;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_APP_LIST_V3 = 0x15;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_APP_LIST_V4 = 0x1D;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_SHOW_VIRTUAL_PIN_EX = 0xC2;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_SHOW_PIN_DIGIT = 0x16;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_GET_PIN_DONE = 0x17;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_SHOW_PIN_BYPASS = 0x18;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_PIN_BYPASS = 0x19;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_CANCEL_PIN_CONFIRM = 0x1A;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_OFFLINE_PIN_VERIFY_RESULT = 0x1B;
Special Event Id used in method specialEventRegister

public static final byte d_EMVCL_EVENTID_PIN_TRY_COUNTER = 0x1C;
Special Event Id used in method specialEventRegister

Method Detail

public int initialize()

This method initializes the EMVCL environment.
Parameters:
None
Returns:

Castles Technology

d_EMVCL_NO_ERROR – Success.
Other – Error code.

public int setConfiguration(InputStream inStream)

This method is used to set configuration, such like AIDs, Tags, CAPK.

Parameters:
[IN] inStream – Stream of configuration file.

Returns:
d_EMVCL_NO_ERROR – Success.
Other – Error code.

public final int setConfiguration(String filename, InputStream inStream)

The configuration file is sent from android app to the EMVCL kernel in the secure module. The tags and CAPKs …etc. in the configuration file will be loaded to EMVCL kernel.

Parameters:
[IN] filename – File name of configuration file.
[IN] inStream – Stream of configuration file.

Returns:
d_EMVCL_NO_ERROR – Success.
Other – Error code.

public final int setConfiguration(String filename)

Inform the secure module to load configuration with the input file name.

Parameters:
[IN] filename – File name of configuration file.

Returns:
d_EMVCL_NO_ERROR – Success.
Other – Error code.

public final int setConfigurationFile(String filename, InputStream inStream)

The configuration file is sent from android app to the EMVCL kernel in the secure module. The configuration will NOT be loaded to the EMVCL kernel.

Parameters:
[IN] filename – File name of configuration file.
[IN] inStream – Stream of configuration file.

Returns:
d_EMVCL_NO_ERROR – Success.
Other – Error code.

public final int setConfigurationEx(EMVCLConfigData configData)

The configuration file is sent from android app to the EMVCL kernel in the secure module. There are conditions to select specific configuration and load it to EMVCL kernel.

Parameters:
[IN] configData – Class EMVCLConfigData that consists of input variables to set configuration.

Returns:
d_EMVCL_NO_ERROR – Success.
Other – Error code.

public final int delConfigurationFileAll()

Delete all configuration files stored in the EMVCL kernel.

Parameters:
None

Returns:
d_EMVCL_NO_ERROR – Success.
Other – Error code.

public final int delConfigurationFile(String filename)

Detete the specific configuration file.

Parameters:
[IN] filename – File name of configuration file.

Returns:
d_EMVCL_NO_ERROR – Success.
Other – Error code.

public int startTransactionEx(EMVCLActData actData, EMVCLRcDataEx rcDataEx)

Start an EMV contactless transaction with transaction related data. Get the transaction response when the transaction is finish.

Parameters:
[IN] actData – Class EMVCLActData that consists of input variables to send transaction related data.
[OUT] rcDataEx – Class EMVCLRcDataEx that consists of output variables to get transaction result.

Returns:
d_EMVCL_RC_DATA – Indicate the transaction data received.
d_EMVCL_RC_FAILURE – General failure.

Castles Technology

d_EMVCL_RC_FALLBACK – Fall back.
d_EMVCL_TRY_AGAIN – Try again.
d_EMVCL_TX_CANCEL – Transaction is canceled.
d_EMVCL_RC_MORE_CARDS – More than 1 card.
d_EMVCL_RC_NO_CARD – Contactless card is not presented.
Other – Error code.

Note:

startTransactionEx is not supported for S1F2, please use initTransactionEx() + performTransactionEx() instead.

public int initTransactionEx(short tagNum, byte[] transData, int transDataLen)

Init an EMV contactless transaction with transaction related data. Please use method performTransactionEx to receive transaction result.

Parameters:

[IN] tagNum – Number of transaction related data.
[IN] transData – Transaction related data.
[IN] transDataLen – Length of transaction related data.

Returns:

d_EMVCL_NO_ERROR – Success.
d_EMVCL_RC_FAILURE – General failure.
Other – Error code.

public int performTransactionEx(EMVCLRcDataEx rcDataEx)

Perform transaction and receive the result of transaction strated by initTransactionEx .

Parameters:

[OUT] rcDataEx – Get transaction result. The developer can set Upload DOL Tags to request tags that are returned to Chip Data and Additional Data of the rcDataEx. Refer to “Castles EMV Contactless Setting Manual” for details.

Returns:

d_EMVCL_RC_DATA – Indicate the transaction data received.
d_EMVCL_RC_FAILURE – General failure.
d_EMVCL_RC_FALLBACK – Fall back.
d_EMVCL_TRY_AGAIN – Try again.
d_EMVCL_TX_CANCEL – Transaction is canceled.
d_EMVCL_RC_MORE_CARDS – More than 1 card.
d_EMVCL_RC_NO_CARD – Contactless card is not presented.
d_EMVCL_PENDING – Pending
d_EMVCL_RC_DEK_SIGNAL – DEK Signal.
Other – Error code.

public int analyzeTransactionEx(EMVCLRcDataEx rcDataEx, EMVCLRcDataAnalyze rcDataAnalyze)

Analyze transaction result.

Parameters:

[IN] rcDataEx – Input transaction result.
[OUT] rcDataAnalyze – Get result of analysis.

Returns:

d_EMVCL_NO_ERROR – Success.

public int completeEx(byte action, byte[] arc, int iadLen, byte[] iad, int scriptLen, byte[] script, EMVCLRcDataEx rcDataEx)

Completion / Issuer Update Processing.
For contactless transaction, completion processing (some payment is issuer update processing) is optional since startTransactionEx and initTransactionEx / performTransactionEx have performed all the process that card interact with. Completion/Issuer Update Processing is usually used to update card status without changing the transaction result from host. We recommend not call this method unless your market requires the processing on contactless interface.

Parameters:

[IN] action – Input action.
0x01 – Host responds to "APPROVED ".
0x02 – Host responds to "DECLINED".
0x03 – Unable connect to Host.
[IN] arc – Input ARC, Authorization Response Code, Tag 8A, This value must be 2 ASCII chars.
[IN] iadLen – Input length of Issuer Authentication Data (8 to 16 bytes).
[IN] iad – Input Issuer Authentication Data, Tag 91. Please only input the data of Tag 91. Only data (no TL).
[IN] scriptLen – Input length of Issuer Script Data.
[IN] script – Input Issuer script data. Pointer to the emitter scripts, composed by one or more '71' or '72' concatenated templates (complete TLV structure).
[OUT] rcDataEx – Get transaction result.

Returns:

d_EMVCL_RC_DATA – Indicate the transaction data received.
d_EMVCL_RC_FAILURE – General failure.
d_EMVCL_RC_FALLBACK – Fall back.
d_EMVCL_TRY_AGAIN – Try again.
d_EMVCL_TX_CANCEL – Transaction is canceled.
d_EMVCL_RC_MORE_CARDS – More than 1 card.
d_EMVCL_RC_NO_CARD – Contactless card is not presented.
d_EMVCL_RC_DEK_SIGNAL – DEK Signal.
Other – Error code.

public int detectTransactionEx(EMVCLDetectTxnData detectTxnData)

Castles Technology

Detect card application and return PPSE response, selected AID and select AID response.

Parameters:

[IN][OUT] detectTxnData – Input transaction related data and get detected card application.

Returns:

d_EMVCL_RC_DATA – Indicate the transaction data received.
d_EMVCL_RC_FAILURE – General failure.
d_EMVCL_TX_CANCEL – Transaction is canceled.
d_EMVCL_RC_MORE_CARDS – More than 1 card.
d_EMVCL_RC_NO_CARD – Contactless card is not presented.
Other – Error code.

public int getCardData(EMVCLGetCardData cardData)

Request card data. When calling this method, EMVCL kernel will request to present card. After card detected, the below data will be returned.

Selected AID
Select PPSE Response
Select AID Response
Requested Card Data

Parameters:

[IN][OUT] cardData – Input transaction related data and get requested data.

Returns:

d_EMVCL_RC_DATA – Indicate the transaction data received.
d_EMVCL_RC_FAILURE – General failure.
d_EMVCL_TX_CANCEL – Transaction is canceled.
d_EMVCL_RC_MORE_CARDS – More than 1 card.
d_EMVCL_RC_NO_CARD – Contactless card is not presented.
Other – Error code.

public int cancelTransaction()

Abort current transaction.

Parameters:

None.

Returns:

d_EMVCL_NO_ERROR – Success.

public int aidSetTagData(byte action, EMVCLAidSetTagData tagData)

Set AID with TagsSetting. EMVCL kernel can handle and storage more tags settings for different combination list {TransType-AID-KERNEL ID}.

Parameters:

[IN] action – Action.
0x00 – one TagSetting Addition.
0x01 – one TagSetting Deletion.
0x02 – All TagSetting Deletion.
[IN] tagData – Input tag data.

Returns:

d_EMVCL_NO_ERROR – Success.
d_EMVCL_RC_INVALID_PARAM – Invalid input data
d_EMVCL_RC_FAILURE – General failure.
Other – Error code.

public int aidGetTagData(EMVCLAidGetTagData tagData)

Get AID with TagsSetting.

Parameters:

[IN][OUT] tagData – Input TransType-AID-KERNEL ID and get its tag data.

Returns:

d_EMVCL_NO_ERROR – Success
d_EMVCL_DATA_NOT_FOUND – Data is NOT found
Other – Error code.

public int aidGetCombinationList(EMVCLAidCombinationList aidCombiList)

Get all of AID combination list {Transaction type – AID – Kernel ID}.
One AID Combination List : {aidlen[x] + aid[x][] +kernelIdLen[x] + kernelId[x] + transactionType[x]} (x = 0 ~ n).

Parameters:

[OUT] aidCombiList – Buffer to get all of AID combinations.

Returns:

d_EMVCL_NO_ERROR – Success.
d_EMVCL_RC_INVALID_DATA – Invalid input data
Other – Error code.

public int setCAPK(byte[] rid, EMVCLCAPublicKey capk)

Set CA Public Key with specified RID.

Parameters:

[IN] rid – RID must be 5 bytes.

Castles Technology

[IN] capk – Input CAPK data.

Returns:

d_EMVCL_NO_ERROR – Success.
d_EMVCL_RC_INVALID_PARAM – Invalid action
d_EMVCL_RC_INVALID_DATA – Invalid input data
d_EMVCL_RC_FAILURE – General failure.
Other – Error code.

public int setCAPKEx(int capkNum, EMVCLCAPublicKeyEx[] capks)

Set multiple CAPKs at once

Parameters:

[IN] capkNum – Number of CAPKs
[IN] capks – Buffer to set multiple CAPKs.

Returns:

d_EMVCL_NO_ERROR – Success.
d_EMVCL_RC_INVALID_PARAM – Invalid action
d_EMVCL_RC_INVALID_DATA – Invalid input data
d_EMVCL_RC_FAILURE – General failure.
Other – Error code.

public int getCAPK(byte[] rid, byte index, EMVCLCAPublicKey capk)

Get CA Public Key with specified RID.

Parameters:

[IN] rid – RID must be 5 byte.s
[IN] index – CAPK index.
[OUT] capk – Get CAPK data.
[OUT] capk.modulusLen
[OUT] capk.modulus
[OUT] capk.exponentLen
[OUT] capk.exponent
[OUT] capk.hash

Returns:

d_EMVCL_NO_ERROR – Success
d_EMVCL_DATA_NOT_FOUND – Data is NOT found
Other – Error code.

public int listAllCAPKId(EMVCLByteBufData capkIdList)

List all CA Public Key Indexs.

Parameters:

[OUT] capkIdList – Buffer to retrieve CAPK Index list
[OUT] dataBuf –The format is
Number of RID(s) – 1 byte
RID – 5 bytes
Number of Keys – 1 byte, noted as N1
Key Indexs – N1 byte(s)
RID – 5 bytes
Number of Keys – 1 byte, noted as N2
Key Indexs – N2 byte(s)
....
[OUT] dataLen –The total length of CAPK Index list

Returns:

d_EMVCL_NO_ERROR – Success
Other – Error code.

public int setParameter(EMVCLParameterData paraData)

Set parameter.

Parameter index	Len (bytes)	
0x0002	4	Sale Timeout. default value is 0x00003A98, 15000ms
0x100B	1	Visa EUR CL TIG Specification Follow. 0x00 - NOT follow TIG rule(default) 0x02 - Follow
0x3008	1	See Phone Restart Behavior 0x01 - All of the kernels use consistent behavior. The code d_EMVCL_RC_SEE_PHONE is returned from transaction API for requesting a see phone process Other - Use behavior which was implemented during kernel certification

Parameters:

[IN] paraData – Input Parameter data.

Returns:

d_EMVCL_NO_ERROR – Success.
d_EMVCL_RC_INVALID_PARAM – Invalid parameter id
d_EMVCL_RC_FAILURE – General failure.
Other – Error code.

public int getParameter(int index, EMVCLParameterData paraData)

Castles Technology

Get one parameter.
Parameters:
[IN] index – Parameter index.
[OUT] paraData – Get parameter data.
Returns:
d_EMVCL_NO_ERROR – Success.
d_EMVCL_RC_INVALID_PARAM – Invalid parameter id
d_EMVCL_RC_FAILURE – General failure.
Other – Error code.

public int specialEventRegister(byte eventId, ISpecialEvent spEvent)

Register special event (callback function) to EMVCL kernel, provided by the user application using.
Parameters:
[IN] eventId – Identifier for registering special event.
[IN] spEvent – Register event (callback function).
Returns:
d_EMVCL_NO_ERROR – Success.
d_EMVCL_RC_INVALID_PARAM – The event id is not supported
Other – Error code.

public final int secureDataEncryptInfoSet(EMVCLSecureDataInfo info)

Determine encryption method of security data.
As security reason, EMVCL Kernel protect the secured tags like Tag 5A and 57, that are not allowed retrieved in plaintext under Android environment. Instead of Tag 5A and 57, developer can retrieve below tags to get information like BIN, expiration date, service code

Tag DF30 : Encrypt PAN
Value Format : EncryptedDataLen (1B) + EncryptedData(var B) + RealLen (1B) + KSN Len(1B) + KSN(var B)

Tag DF31 :PAN Check Sum (HASH or MAC)
Value Format : CheckSumLen (1B) + CheckSum(var B) + RealLen (1B) + KSN Len(1B) + KSN(var B)

Tag DF32 : Mask PAN

Tag DF33 : Encrypt Track2
Value Format : EncryptedDataLen (1B) + EncryptedData(var B) + RealLen (1B) + KSN Len(1B) + KSN(var B)

Tag DF34 : Track2 Check Sum(HASH or MAC)
Value Format : CheckSumLen (1B) + CheckSum(var B) + RealLen (1B) + KSN Len(1B) + KSN(var B)

Tag DF35 : Mask Track2

Tag DF36 : Encrypt Track1
Value Format : EncryptedDataLen (1B) + EncryptedData(var B) + RealLen (1B) + KSN Len(1B) + KSN(var B)

Tag DF37 : Track1 Check Sum(HASH or MAC)
Value Format : CheckSumLen (1B) + CheckSum(var B) + RealLen (1B) + KSN Len(1B) + KSN(var B)

Tag DF38 : Mask Track1

Parameters:
[IN] info – encryption method.
Returns:
d_EMVCL_NO_ERROR – Success.
Other – Error code.

public final int secureDataWhitelistSet(byte action, byte[] listData, int listDataLen)

Set whitelist of security data. The feature is to allow the secured tags to be returned in the plaintext. The developer can add BIN or partial digits of PAN or a full PAN to the white list. When EMV kernel read the PAN from a card, it compares PAN to the white list. When the result of the comparison is matched, the secured tags are returned in plaintext.
Parameters:
[IN] action – New (0x00) or Delete all (0x01).
[IN] listData – The whitelist data.
Format : entry num (4 Byte) + entry1 len(1 Byte) + entry1 data (var Byte) + ….. + entryN len (1 Byte) + entryN data (var Byte).
Max entry num is 512.
Example : 3 entries need to be added into whiltelist PAN1 : 4567, PAN2 : 123, PAN3 : 56
listData need to be filled with 0x00 0x00 0x00 0x03 0x04 0x34 0x35 0x36 0x37 0x03 0x31 0x32 0x33 0x02 0x35 0x36.
0x00 0x00 0x00 0x03 -> entry num
0x04 0x34 0x35 0x36 0x37 -> entry1 [Length : 4 + '4567' in ASCII]
0x03 0x31 0x32 0x33 -> entry2 [Length : 3 + '123'in ASCII]
0x02 0x35 0x36. -> entry 3 [Length : 2 +'56'in ASCII]
[IN] listDataLen – The length of input whitelist data.

Castles Technology

Returns:
d_EMVCL_NO_ERROR – Success.
d_EMVCL_RC_INVALID_PARAM – Invalid action.
d_EMVCL_RC_FAILURE – General failure.
Other – Error code.

public final int setMaskChar(byte maskChar)

Set mask character of security data.
Parameters:
[IN] maskChar – Maskchar like ' * ', ' 0 '….
Returns:
d_EMVCL_NO_ERROR – Success.

public final int maskPANDigits1To6(byte maskDigits)

Set masked status of Digits 1 to 6 of PAN. (D1D2D3D4 D5D6XX XXXX XXXX)
Parameters:
[IN] maskDigits – Masked status of digits. One byte (bit 8 bit 7 bit6 …..bit 2 bit 1).
bit 8 : 1 – Digit 1 is maked.
bit 7 : 1 – Digit 2 is maked.
bit 6 : 1 – Digit 3 is maked.
bit 5 : 1 – Digit 4 is maked.
bit 4 : 1 – Digit 5 is maked.
bit 3 : 1 – Digit 6 is maked.
Returns:
d_EMVCL_NO_ERROR – Success.

public final int maskPANDigits13To16(byte maskDigits)

Set masked status of Digits 13 to 16 of PAN. (XXXX XXXX XXXX D13D14D15D16)
Parameters:
[IN] maskDigits – Masked status of digits. One byte (bit 8 bit 7 bit6 …..bit 2 bit 1).
bit 8 : 1 – Digit 13 is maked.
bit 7 : 1 – Digit 14 is maked.
bit 6 : 1 – Digit 15 is maked.
bit 5 : 1 – Digit 16 is maked.
Returns:
d_EMVCL_NO_ERROR – Success.

public final int maskEXPDigits1To4(byte maskDigits)

Set masked status of Digits 1 to 4 of expiration date of track 2 data. (MMYY : D1D2D3D4)
Parameters:
[IN] maskDigits – Masked status of digits. One byte (bit 8 bit 7 bit6 …..bit 2 bit 1).
bit 8 : 1 – Digit 1(M) is maked.
bit 7 : 1 – Digit 2(M) is maked.
bit 6 : 1 – Digit 3(Y) is maked.
bit 5 : 1 – Digit 4(Y) is maked.
Returns:
d_EMVCL_NO_ERROR – Success.

public final int maskCSCDigits1To3(byte maskDigits)

Set masked status of Digits 1 to 3 of card service code. (D1D2D3)
Parameters:
[IN] maskDigits – Masked status of digits. One byte (bit 8 bit 7 bit6 …..bit 2 bit 1).
bit 8 : 1 – Digit 1 is maked.
bit 7 : 1 – Digit 2 is maked.
bit 6 : 1 – Digit 3 is maked.
Returns:
d_EMVCL_NO_ERROR – Success.

public final int getLastError()

Get detail error code
Parameters:
None
Returns:
d_EMVCL_NO_ERROR – No last error information
Other – Error code

public final int detectCard()

Detect whether card is presented on the CL field.
Parameters:
None
Returns:
d_EMVCL_NO_ERROR – Card is detected.
d_EMVCL_RC_NO_CARD – No card.
d_EMVCL_RC_MORE_CARDS – Collision. Multiple cards are detected.

Castles Technology

d_EMVCL_RC_FAILURE – General failure.

public final int performCardAcquisition()

Card data acquisition. For each AID in the card, collecting card tags from response of SELECT AID command to response of READ RECORD command.
Note : initTransactionEx() should be called prior to this method.

Parameters:
None

Returns:
d_EMVCL_NO_ERROR – Success.
d_EMVCL_RC_FAILURE – General failure.
Other – Error code.

public final int getCardAcquisitionAppNum()

Get the number of card acquisition data (the same as AID number in the card). Call this after performCardAcquisition()

Parameters:
None

Returns:
value – Number of card acquisition data

public final int getCardAcquisitionAppData(EMVCLCardAcquisitionData[] acq)

Input buffer to get card acquisition data. Call this after performCardAcquisition()

Parameters:
[OUT] acq – buffer to get card acquisition data.
[OUT] acq.tlv.dataBuf – All tags. Format is tlv + tlv + tlv + ..
[OUT] acq.tlv.dataLen – Length of tlv data.
[OUT] acq.aid.dataBuf – AID
[OUT] acq.aid.dataLen – Length of AID.
[OUT] acq.pan.dataBuf – PAN
[OUT] acq.pan.dataLen – Length of PAN.
[OUT] acq.issuerCountryCode.dataBuf – Issuer country code.
[OUT] acq.issuerCountryCode.dataLen – Length of issuerCountryCode

Returns:
d_EMVCL_NO_ERROR – Success.
d_EMVCL_RC_FAILURE – General failure.
Other – Error code.

public final int changeWDBPath(String path)

The EMVCL kernel default set the working database path to the Android Application ID and store the files such as the xml configuration files in it. Each Android App has its configurations.
This method is used to change the working database path If the developer would like to share the configurations between multiple Android App.

Parameters:
[IN] path –Working database path.

Returns:
d_EMVCL_NO_ERROR – Success.
d_EMVCL_RC_INVALID_PARAM – Invalid input data.
Other – Error code.

public final int getWDBPath ()

Get the current working database path. If the developer does not change WDB path, the default working database path is the string of Android Application ID. After calling getWDBPath(), please call getRtnString() to get the working database path

Parameters:
None.

Returns:
d_EMVCL_NO_ERROR – Success.
d_EMVCL_BUF_OVERFLOW – The internal buffer is overflow.
Other – Error code.

public final int resetWDBPath()

Reset the working database path to default.

Parameters:
None.

Returns:
d_EMVCL_NO_ERROR – Success
Other – Error code

Castles Technology

public final String getRtnString()

This method is used after specific methods to get String variable.

Parameters:

None

Returns:

value – Get a String variable.

public final int setReadyToReadBehavior(byte mode)

This method is used to set an action for “Ready to Read” status of user interface of EMV Book A.

Parameters:

[IN] mode –

0x01 – Pause “Ready to Read” LED behavior. Event eventEMVCLLEDPicShow will not be triggered at “Ready to Read” status. No UI changed in the phase of finding card.

Other – Normal “Ready to Read” LED behavior.

Returns:

d_EMVCL_NO_ERROR – Success

Fields Detail

Event Id used in method specialEventRegister (public static final byte)

Constant Field	Value	Additional Description
d_EMVCL_EVENTID_LED_PIC_SHOW	0x01	
d_EMVCL_EVENTID_AUDIO_INDICATION	0x04	
d_EMVCL_EVENTID_CANCEL_TRANSACTION	0xC0	
d_EMVCL_EVENTID_SHOW_MESSAGE	0xC1	
d_EMVCL_EVENTID_APP_LIST_V2	0x07	
d_EMVCL_EVENTID_NON_EMV_CARD	0x05	
d_EMVCL_EVENTID_PRE_NON_EMV_CARD	0x0C	
d_EMVCL_EVENTID_RMR_DATA_EXCHANGE	0x10	
d_EMVCL_EVENTID_APP_LIST_V3	0x15	
d_EMVCL_EVENTID_APP_LIST_V4	0x1D	
d_EMVCL_EVENTID_SHOW_VIRTUAL_PIN_EX	0xC2	
d_EMVCL_EVENTID_SHOW_PIN_DIGIT	0x16	
d_EMVCL_EVENTID_GET_PIN_DONE	0x17	
d_EMVCL_EVENTID_SHOW_PIN_BYPASS	0x18	
d_EMVCL_EVENTID_PIN_BYPASS	0x19	
d_EMVCL_EVENTID_CANCEL_PIN_CONFIRM	0x1A	
d_EMVCL_EVENTID_OFFLINE_PIN_VERIFY_RESULT	0x1B	
d_EMVCL_EVENTID_PIN_TRY_COUNTER	0x1C	

Audio Tone Return Code (public static final short)

Consent Field	Value	Additional Description
d_EMVCL_EVENT_AUDIO_TONE_CARD_READ_OK	0x0001	
d_EMVCL_EVENT_AUDIO_TONE_ALERT	0x0002	

EMVCLSecureDataInfo keyType

Consent Field	Value	Additional Description
KeyType_3DES	0x01	

Castles Technology

KeyType_3DES_DUKPT	0x02	
KeyType_AES	0x03	
KeyType_AES_DUKPT	0x07	
EMVCLSecureDataInfo cipherMethod		
Constent Field	Value	Additional Description
DATA_ENCRYPT_METHOD_ECB	0x00	
DATA_ENCRYPT_METHOD_CBC	0x01	
EMVCLSecureDataInfo macCipherMethod		
Constent Field	Value	Additional Description
MAC_METHOD_CBC	0x00	
MAC_METHOD_X9_19	0x01	
MAC_METHOD_X9_19_START	0x02	
MAC_METHOD_X9_19_UPDATE	0x03	
MAC_METHOD_X9_19_FINAL	0x04	
EMVCLSecureDataInfo checksumType		
Constent Field	Value	Additional Description
ChecksumType_SHA1	0x00	
ChecksumType_MAC	0x01	
ChecksumType_SHA256	0x02	
ChecksumType_NONE	0xFF	
EMVCLSecureDataInfo paddingMethod		
Constent Field	Value	Additional Description
PaddingMethod_00	0x00	
PaddingMethod_FF	0x01	
PaddingMethod_PKCS7	0x02	
EMVCLSecureDataInfo derivedKeyType & macDerivedKeyType		
Constent Field	Value	Additional Description
DerivedKeyType_2TDEA	0x00	
DerivedKeyType_3TDEA	0x01	
DerivedKeyType_AES128	0x02	
DerivedKeyType_AES192	0x03	
DerivedKeyType_AES256	0x04	
EMVCLSecureDataInfo derivedKeyUsage		
Constent Field	Value	Additional Description
DerivedKeyUsage_DataEncryptionEncrypt	0x05	
DerivedKeyUsage_DataEncryptionDecrypt	0x06	
DerivedKeyUsage_DataEncryptionBothWays	0x07	
EMVCLSecureDataInfo macDerivedKeyUsage		
Constent Field	Value	Additional Description

Castles Technology

DerivedKeyUsage_MessageAuthenticationGeneration	0x02	
DerivedKeyUsage_MessageAuthenticationVerification	0x03	
DerivedKeyUsage_MessageAuthenticationBothWays	0x04	
Return Code (public static final int)		
Constant Field	Value	Additional Description
d_EMVCL_NO_ERROR	0x00000000	
d_EMVCL_PENDING	0x80000020	
d_EMVCL_TX_CANCEL	0x80000021	
d_EMVCL_INIT_TAGSETTING_ERROR	0x80000022	
d_EMVCL_INIT_CAPK_ERROR	0x80000023	
d_EMVCL_INIT_PARAMETER_ERROR	0x80000024	
d_EMVCL_INIT_CAPABILITY	0x80000025	
d_EMVCL_INIT_REVOCATION_ERROR	0x80000026	
d_EMVCL_INIT_EXCEPTION_FILE_ERROR	0x80000027	
d_EMVCL_DATA_NOT_FOUND	0x80000030	
d_EMVCL_NOT_OPEN	0x80000050	
d_EMVCL_TRY_AGAIN	0x80000051	
d_EMVCL_FUN_NOT_SUPPORT	0x80000060	
d_EMVCL_PAYMENT_NOT_SUPPORT_COMPLETE	0x80000061	
d_EMVCL_NON_EMV_CARD	0x80000062	
d_EMVCL_APP_LIST_CANCEL	0x80000063	
d_EMVCL_APP_LIST_FAILURE	0x80000064	
d_EMVCL_VISA_NO_AFL	0x80000065	
d_EMVCL_TXN_BUSY	0x80000066	
d_EMVCL_KERNEL_NOT_FOUND	0x80000067	
d_EMVCL_BUF_OVERFLOW	0x80000068	
d_EMVCL_ASRPD_USER_STOP_TXN	0x80000069	
d_EMVCL_AMOUNT_NOT_PRESENT	0x8000006A	
d_EMVCL_RC_DATA	0xA0000001	
d_EMVCL_RC_SCHEME_SUPPORTED	0xA0000004	
d_EMVCL_RC_FAILURE	0xA00000FF	
d_EMVCL_RC_INVALID_DATA	0xA00000F8	
d_EMVCL_RC_INVALID_PARAM	0xA00000F7	
d_EMVCL_RC_INVALID_SCHEME	0xA00000F5	
d_EMVCL_RC_INVALID_VISA_CA_KEY	0xA00000F4	
d_EMVCL_RC_MORE_CARDS	0xA00000F3	
d_EMVCL_RC_NO_CARD	0xA00000F2	
d_EMVCL_RC_NO_EMV_TAGS	0xA00000F1	
d_EMVCL_RC_FALLBACK	0xA00000E9	

Castles Technology

d_EMVCL_RC_EXCEED_OR_EQUAL_CL_TX_LIMIT	0xA00000E8	
d_EMVCL_RC_NO_AP_FOUND	0xA00000E7	
d_EMVCL_RC_DEK_SIGNAL	0xA00000E0	
d_EMVCL_RC_SEE_PHONE	0xA00000E4	
d_EMVCL_RC_AMOUNT_0_NOT_ALLOW	0xA00000D0	

Castles Technology

4.1.1.1 Interface: ISpecialEvent

public interface **ISpecialEvent**

The **ISpecialEvent** interface is tag interface. It has no methods or fields and is used as primary datatype for entire Special Event.

Fields Summary
None

Method Summary	
None	

Castles Technology

4.1.1.1.1 Interface: IEventEMVCLLEDPicShow

public interface **IEventEMVCLLEDPicShow**
extends ISpecialEvent
The **IEventEMVCLLEDPicShow** interface extends ISpecialEvent interface with method that user can implement in the application to show proprietary contactless LED.

Method Summary	
Int	eventEMVCLLEDPicShow (byte index, byte bOnOff) Implement proprietary contactless LED.

Method Detail
int eventEMVCLLEDPicShow(byte index, byte bOnOff); This event will be triggered to indicate the LED signals. User can implement their proprietary LED (physical or virtual) and then flash the LED according to signals. Position : LED4 LED3 LED2 LED1 Color(normal) : Blue Yellow Green Red Color(EUR) : Green Green Green Green **Parameters:** [IN] index – LED Index (Bit0:LED1, Bit1:LED2, Bit2:LED3, Bit3:LED4, Bit4-Bit7:RFU) Bitx = 1: LED(x+1) on/off indicated in bOnOff Bitx Bitx = 0: LED(x+1) with the current state [IN] bOnOff – Turn on or off the LEDs Bitx = 1: control LED(x+1) on if LED index Bitx is set. Bitx = 0: control LED(x+1) off if LED index Bitx is set. **Returns:** 0x00 – Success.

Castles Technology

4.1.1.1.2 Interface: IEventEMVCLAudioIndication

public interface **IEventEMVCLAudioIndication**
extends ISpecialEvent
The **IEventEMVCLAudioIndication** interface extends ISpecialEvent interface with method that user can implement in the application to show proprietary audio indication.

Method Summary	
Int	eventEMVCLAudioIndication (short tone) Implement proprietary audio indication.

Method Detail
int eventEMVCLAudioIndication(short tone); This event will be triggered to indicate tone signal. User can implement their proprietary audio indication and then sound the tone according to signal. **Parameters:** [IN] tone – indicate tone signal. 0x0001 – Success tone. 0x0002 – Alert tone. **Returns:** 0x00 – Success.

Castles Technology

4.1.1.1.3 Interface: IEventEMVCLCancelTransaction

public interface **IEventEMVCLCancelTransaction**
extends ISpecialEvent
The **IEventEMVCLCancelTransaction** interface extends ISpecialEvent interface with method that user can implement in the application to cancel transaction.

Method Summary	
Int	eventEMVCLCancelTransaction () Cancel transaction.

Method Detail
int eventEMVCLCancelTransaction(); Cancel transaction. **Parameters:** None. **Returns:** 0x00 – Continue processing. 0x01 – Current transaction will be canceled.

Castles Technology

4.1.1.1.4 Interface: IEventEMVCLShowMessage

public interface **IEventEMVCLShowMessage**
extends ISpecialEvent
The **IEventEMVCLShowMessage** interface extends ISpecialEvent interface with method that user can implement in the application to show message.

Method Summary	
Int	eventEMVCLShowMessage(byte[] kernelId,byte kernellen ,EMVCLUIReqData uiReqData) Show message during transaction.

Method Detail
int eventEMVCLShowMessage(byte[] kernelId,byte kernellen ,EMVCLUIReqData uiReqData); User show message according to UI request data from EMVCL kernel. **Parameters:** [IN] kernelId –Kernel ID. [IN] kernellen – Length of Kerenl ID. [IN] uiReqData – UI request data. **Returns:** 0x00 – Success

Castles Technology

4.1.1.1.5 Interface: IEventEMVCLAppListV2

public interface **IEventEMVCLAppListV2**
extends ISpecialEvent
The **IEventEMVCLAppListV2** interface extends ISpecialEvent interface with method that user can implement in the application to get candidate list for application selection.

Method Summary	
Int	eventEMVCLAppListV2(EMVCLAppListV2 appListV2) list candidate for application selection

Method Detail
int eventEMVCLAppListV2(EMVCLAppListV2 appListV2); If the interface is registered, the callback will always be triggered every transaction regardless of the candidate is 1 or more than 1. **Parameters:** [IN][OUT] appListV2 – The candidate list for application selection. **Returns:** 0x00 – Success. 0x01 – User cancel 0x02 – Transaction terminates

Castles Technology

4.1.1.1.7 Interface: IEventEMVCLPreNonEMVCard

public interface **IEventEMVCLPreNonEMVCard**
extends ISpecialEvent
The **IEventEMVCLPreNonEMVCard** interface extends ISpecialEvent interface with method that user can implement in the application to implement Non EMV Card Process.

Non EMV card means the card is unrecognized by the emvcl kernel. This card might be used for a local or special payment with proprietary apdu commands and processing, such like Mifare transaction.
After card was detected, this event will be triggered prior to EMV process. Developers can perform the local/specific payment's procedure within the event. For example, perform Mifare transaction within the event.
Finally, developers return action to inform emvcl kerenl the following behavior.

Method Summary	
byte	eventEMVCLPreNonEMVCard (byte cardType) Developer can implement Non EMV Card Process in this event. This event will be triggered before EMV process.

Method Detail
byte eventEMVCLPreNonEMVCard(byte cardType); If the interface is registered, the callback will be triggered prior to starting the EMV process. (before Select PPSE Command) Ex : performTransactionEx() is calling → Non EMV card is tapped and activated → eventEMVCLPreNonEMVCard is triggered → (inside callback) user recognizes the card with cardtype, return 0x01 → (back to kernel) kernel return d_EMVCL_NON_EMV_CARD without UI performTransactionEx() is calling → EMV card is tapped and activated → eventEMVCLPreNonEMVCard is triggered → (inside callback) → user can not recognize the card's cardtype, return 0x00 → (back to kernel) kernel is going on with the process → kernel can recognize the AID → continue the contactless process → EMV transacton completion **Parameters:** [IN] byte cardType – The card type of the card. 0x01 – Unknown card type. 0x02 – MIFARE Ultralight.

Castles Technology

0x03 – MIFARE CLASSIC 1K.
0x04 – MIFARE CLASSIC 4K.
0x05 – MIFARE Desfire.
0x06 – Type-A Layer 3 card / Mifare Classic/ultralight with merged atqa detected.
0x07 – Type-A Layer 4 card / Mifare desfire/plus with merged atqa detected.
0x08 – Type-B Layer 3 card.
0x09 – Type-B Layer 4 card

Returns:

0x00 – Unrecognized card. The kerenl will end transaction with UI behavior, sound alert tone and light alert led.
0x01 – Recognized local or specific card. The kerenl will end transaction without UI behavior

Castles Technology

4.1.1.1.8 Interface: IEventEMVCLRMRDataExchange

public interface **IEventEMVCLRMRDataExchange**
extends ISpecialEvent
The **IEventEMVCLRMRDataExchange** interface extends ISpecialEvent interface with method that user can implement in the application to achieve MIR Data Exchange function.

Method Summary	
int	eventEMVCLRMRDataExchange (EMVCLByteBufData deInData, EMVCLByteBufData deOutData) Data exchange for the MIR transaction.

Method Detail

int eventEMVCLRMRDataExchange(EMVCLByteBufData deInData, EMVCLByteBufData deOutData);

The MIR kernel transmits to the User Application the set of tags, specified in the Data Exchange Tag List. After the User Application checks the tags from Kernel, if need it exchanges new or updated tags to Kernel.

Parameters:

[IN] deInData – Data exchange from kernel.
[IN] dataBuf – The format is TLV1 + TLV2 + …
[IN] dataLen – Length of data from kernel.
[OUT] deOutData – Data exchange to kernel.
[OUT] dataBuf – The format is TLV1 + TLV2 + …
[OUT] dataLen – Length of data to kernel.

Returns:

d_EMVCL_NO_ERROR – Success
d_EMVCL_RC_FAILURE – Terminated transaction

Castles Technology

4.1.1.1.9 Interface: IEventEMVCLAppListV3

public interface **IEventEMVCLAppListV3**
extends ISpecialEvent
The **IEventEMVCLAppListV3** interface extends ISpecialEvent interface with method that user can implement in the application to get candidate list for application selection.

Method Summary	
Int	eventEMVCLAppListV3(EMVCLAppListV3 appListV3) list candidate for application selection

Method Detail
int eventEMVCLAppListV3(EMVCLAppListV3 appListV3); If the interface is registered, the callback will always be triggered every transaction regardless of the candidate is 1 or more than 1. **Parameters:** [IN][OUT] appListV3 – The candidate list for application selection. **Returns:** 0x00 – Success. 0x01 – User cancel 0x02 – Transaction terminates

Castles Technology

4.1.1.1.10 Interface: IEventEMVCLAppListV4

public interface **IEventEMVCLAppListV4**
extends ISpecialEvent
The **IEventEMVCLAppListV4** interface extends ISpecialEvent interface with method that user can implement in the application to get candidate list for application selection.

Method Summary	
Int	eventEMVCLAppListV4(EMVCLAppListV4 appListV4) list candidate for application selection

Method Detail
int eventEMVCLAppListV4(EMVCLAppListV4 appListV4); If the interface is registered, the callback will always be triggered every transaction regardless of the candidate is 1 or more than 1. **Parameters:** [IN][OUT] appListV4 – The candidate list for application selection. **Returns:** 0x00 – Success. 0x01 – User cancel 0x02 – Transaction terminates

Castles Technology

4.1.1.1.11 Interface: IEventEMVCLShowVirtualPIN

public interface **IEventEMVCLShowVirtualPIN**
Extends ISpecialEvent
The **IEventEMVCLShowVirtualPIN** interface extends ISpecialEvent interface with method that user can implement in the application to show the customized virtual PIN pad on the screen.
[Used for girocard kernel only to achieve offline PIN entry]

Method Summary	
Int[][]	eventShowVirtualPINEx () Show virtual PIN pad on the screen then return customized keyboard information.

Method Detail
int[][] eventShowVirtualPINEx(); This event will be triggered when EMVCL kerenl require PIN entry for offline PIN. Please customize the keyboard layout and return keyboard information (x, y, x+width, y+height, key value) back, EMVCL kernel will use these information to get PIN entry. Parameters: **Parameters:** None. **Returns:** int [][] – Customized keyboard layout. Keys with 5 parameters. int[i][j] – i : key index ; j : parameters [][0] – Position x. [][1] – Position y. [][2] – Position x + Width. [][3] – Position y + Height. [][4] – Key value. Ex : Implement 16 keys, please declare buffer as int[16][5]

Castles Technology

4.1.1.1.12 Interface: IEventEMVCLShowPINDigit

public interface **IEventEMVCLShowPINDigit**
extends **ISpecialEvent**
The **IEventEMVCLShowPINDigit** interface extends ISpecialEvent interface with method that user can implement in the application to show PIN digits.
[Used for girocard kernel only to achieve offline PIN entry]

Method Summary	
void	eventShowPinDigit (byte bNoDigits) Show PIN digits on the screen.

Method Detail
void eventShowPinDigit(byte bNoDigits); This event function will be triggered when PINs are entered to show PIN digits. **Parameters:** [IN]bNoDigits – Number of PIN digits **Returns:** None.

Castles Technology

4.1.1.1.13 Interface: IEventEMVCLGetPINDone

public interface **IEventEMVCLGetPINDone**
extends **ISpecialEvent**
The **IEventEMVCLGetPINDone** interface extends ISpecialEvent interface with method that user can implement in the application to get information of completing PIN entry.
[Used for girocard kernel only to achieve offline PIN entry]

Method Summary	
int	eventGetPINDone () Get information of completing PIN entry.

Method Detail
int eventGetPINDone(); This event will be triggered when Offline PIN entry is completed. **Parameters:** None **Returns:** 0x00 – Success

Castles Technology

4.1.1.1.14 Interface: IEventEMVCLShowPinBypass

public interface **IEventEMVCLShowPinBypass**
extends **ISpecialEvent**
The **IEventEMVCLShowPinBypass** interface extends ISpecialEvent interface with method that user can implement in the application to get information of PIN Bypass.
[Used for girocard kernel only to achieve offline PIN entry]

Method Summary	
void	eventShowPinBypass () Show PIN Bypass on the screen.

Method Detail
void eventShowPinBypass(); This event function will be triggered when cardholder bypass the PIN entry. **Parameters:** None **Returns:** None.

Castles Technology

4.1.1.1.15 Interface: IEventEMVCLPinBypass

public interface **IEventEMVCLPinBypass**
extends **ISpecialEvent**
The **IEventEMVCLPinBypass** interface extends ISpecialEvent interface with method that user can give an action when PIN Bypass key is pressed.
[Used for girocard kernel only to achieve offline PIN entry]

Method Summary	
byte	eventPinBypass () Action of PIN Bypass.

Method Detail
byte eventPinBypass(); This event will be triggered when PIN Bypass key is pressed in an offline pin process. The message of confirmation of PIN Bypass can be shown to the cardholder for confirming if they would like to PIN Bypass. The following actions can be brought back to the kernel to bypass PIN or restart a process of getting offline PIN. **Parameters:** None **Returns:** 0 - Action : Bypass PIN 1 - Action : Don't bypass PIN, back to get PIN process 2 - Action : Don't bypass PIN, trigger eventShowVirtualPINEx() again then back to get PIN process

Castles Technology

4.1.1.1.16 Interface: IEventEMVCLCancelPinConfirm

public interface **IEventEMVCLCancelPinConfirm**
extends **ISpecialEvent**
The **IEventEMVCLCancelPinConfirm** interface extends ISpecialEvent interface with method that user can give an action when PIN entry is canceled.
[Used for girocard kernel only to achieve offline PIN entry]

Method Summary	
byte	eventCancelPinConfirm (byte bStatus) Give action when PIN entry is canceled.

Method Detail
byte eventCancelPinConfirm(byte bStatus); When the PIN entry is canceled, this event will be triggered frequently with the status and then request an action. The user can pop a dialog to ask whether the cardholder would like to cancel the PIN entry. **Parameters:** [IN] bStatus – Inform status for cancel PIN confirm process. 0 - First info for canceling PIN entry 1 - Frequently triggered for request an action 2 - Inform the card has been removed **Returns:** 0 - Action : Not receive the confirmation of canceling PIN entry 1 - Action : Back to the PIN entry 2 - Action : Cancel the PIN entry

Castles Technology

4.1.1.1.17 Interface: IEventEMVCLOfflinePinVerifyResult

public interface **IEventEMVCLOfflinePinVerifyResult**
extends **ISpecialEvent**
The **IEventEMVCLOfflinePinVerifyResult** interface extends ISpecialEvent interface with method that user can get the results of verification of Offline PIN.
[Used for girocard kernel only to achieve offline PIN entry]

Method Summary	
void	eventOfflinePinVerifyResult (int result) Get the result of verifying offline PIN.

Method Detail
void eventOfflinePINVerifyResult(int result); This event will be triggered after the verification of offline pin to inform the result of verification. **Parameters:** [IN] result – Result of verification of offline PIN 1 - Success 2 - Fail 3 - Blocked 4 - Fail Blocked **Returns:** None.

Castles Technology

4.1.1.1.18 Interface: IEventEMVCLPinTryCounter

public interface **IEventEMVCLPinTryCounter**
extends **ISpecialEvent**
The **IEventEMVCLPinTryCounter** interface extends ISpecialEvent interface with method to get PIN Try Counter.
[Used for girocard kernel only to achieve offline PIN entry]

Method Summary	
void	eventPinTryCounter (byte bPTC) Get PIN Try Counter.

Method Detail
void eventPinTryCounter(byte bPTC); This event will be triggered to inform the PIN Try Counter. **Parameters:** [IN] bPTC – PIN Try Counter. **Returns:** None.

Castles Technology

4.1.1.2 Class: EMVCLActData

public class **EMVCLActData**

Fields Detail	
int	[IN] version Version. Fixed to 0x00
byte	[IN] start Indicate the status of start transaction. 0x00 – Start A : Start a new transaction 0x02 – Start B : Restart the transaction. Start B may be used by below: (1)Outcome is Try again (2)Transaction is terminated but kernel request restart(See phone, second tap…)
short	[IN] tagNum Number of Transaction related Data
int	[IN] transactionDataLen Transaction related Data Length
byte[]	[IN] transactionData Transaction related Data. (format is TLV1 + TLV2+..+ TLVn)

Castles Technology

4.1.1.3 Class: EMVCLRcDataEx

public class **EMVCLRcDataEx**

Fields Detail	
int	version Version. Fixed to 0x00.
byte	sid Scheme Identifier 0x17 – VISA qVSDC (#1) 0x18 – VISA MSD (#2) 0x20 – Mastercard MagStripe (#2) 0x21 – Mastercard MChip (#1) 0x50 – AMEX EMV (#1) 0x52 – AMEX MagStripe (#2) 0x41 – DISCOVER Zip (#2) 0x42 – DISCOVER D-PAS EMV Mode (#1) 0x43 – DISCOVER D-PAS MS Mode (#2) 0x48 – Interac Flash (#1) 0x63 – JCB EMV (#1) 0x64 – JCB MSD (#2) 0x65 – JCB LEGACY (#1) 0x81 – PURE (#1) 0x91 – UnionPay QuickPass (#1) 0x86 – BANCOMAT Contactless PagoBANCOMATR (#1) 0x87 – MIR PS Contactless terminal kernel (#1) 0x88 – ECPC CPACE (#1) 0x89 – eftpos (#1) 0x8A – SIBS MB (#1) 0x8B – girocard MagStripe (#2) 0x8C – girocard EMV (#1) (#1) the payment is EMV mode (#2) the payment is MSD mode
byte[]	dateTime = new byte[15] YYYYMMDDHHMMSS format
byte	track1Len Track 1 data length
byte[]	track1Data = new byte[100] Track 1 data
byte	track2Len Track 2 data length
byte[]	track2Data = new byte[100] Track 2 data
int	chipDataLen

Castles Technology

	Chip data length (=0: Chip data is not present)
byte[]	chipData = new byte[1024] Chip data Please request your required tags from the Upload DOL Tag DF8F49. Refer to “Castles EMV Contactless Setting Manual” for details.
int	additionalDataLen Additional data length (=0: Additional data is not present)
byte[]	additionalData = new byte[1024] Additional data Please request your required tags from the Upload DOL Tag DF8F4E. Refer to “Castles EMV Contactless Setting Manual” for details.

Castles Technology

4.1.1.4 Class: EMVCLRcDataAnalyze

public class **EMVCLRcDataAnalyze**

Fields Detail	
Short	[OUT] transResult Indicate transaction result. 0x0002 – Approval. 0x0003 – Decline. 0x0004 – Online.
Byte	[OUT] cvmAnalysis Indicate which CVM is required 0x00 – None. 0x01 – Signature. 0x02 – Online PIN. 0x03 – CVM Fail. 0x04 – NO CVM. 0x05 – Confirmation Code Verified.
byte[]	[OUT] cvmResults = new byte[3] CVM Result value
Byte	[OUT] visaAOSAPresent Indicate if Visa Available Offline Spending Amount(AOSA) is present 0x00 – Not present. 0x01 – Present.
byte[]	[OUT] visaAOSA = new byte[6] AOSA value.
Boolean	[OUT] odaFail Indicate if Offline Data Authentication is fail. false – ODA OK. true – ODA Fail.
Boolean	[OUT] completeFunRequired Indicate if Completion function is required.

Castles Technology

4.1.1.5 Class: EMVCLDetectTxnData

public class **EMVCLDetectTxnData**

Fields Detail	
Short	[IN] tagNum Number of transaction related data.
Int	[IN] transactionDataLen Length of transaction related data.
byte[]	[IN] transactionData Transaction related data. (Format is TLV1 + TLV2+...+ TLVn).
Byte	[IN][OUT] selectedAidLen [IN] Buffer size. [OUT] Length of selected AID.
byte[]	[OUT] selectedAid Buffer to get AID.
Int	[IN][OUT] selectAidRspLen [IN] Buffer size. [OUT] Length of response of Select AID.
byte[]	[OUT] selectAidRsp Buffer to get the response of Select AID.
Int	[IN][OUT] selectPpseRspLen [IN] Buffer size. [OUT] Length of response of Select PPSE.
byte[]	[OUT] selectPpseRsp Buffer to get the response of Select PPSE.

Castles Technology

4.1.1.6 Class: EMVCLGetCardData

public class **EMVCLGetCardData**

Fields Detail	
Int	[IN] version Version. Fixed to 0x01.
Int	[IN] tagNum Number of transaction related data.
Int	[IN] transactionDataLen Length of transaction related data.
byte[]	[IN] transactionData Transaction related data (format is TLV1 + TLV2 +...+TLVn).
Int	[IN] requestedCardTagLen Length of requested card tag list.
byte[]	[IN] requestedCardTag Requested card tag list (format is Tag1 + Tag2 +...+Tagn).
Short	[IN][OUT] selectedAidLen [IN] Buffer size. [OUT] Length of selected AID.
byte[]	[OUT] selectedAid Buffer to get AID.
Short	[IN][OUT] selectAidRspLen [IN] Buffer size. [OUT] Length of response of Select AID.
byte[]	[OUT] selectAidRsp Buffer to get the response of Select AID.
Short	[IN] [OUT] selectPpseRspLen [IN] Buffer size. [OUT] Length of response of Select PPSE.
byte[]	[OUT] selectPpseRsp Buffer to get the response of Select PPSE.
Short	[IN][OUT] gotCardDataLen [IN] Buffer size. [OUT] Length of requested card data.
byte[]	[OUT] gotCardData Buffer to get the requested card data (format is TLV1 + TLV2 + …+ TLVn).

Castles Technology

4.1.1.7 Class: EMVCLAidSetTagData

public class **EMVCLAidSetTagData**

Fields Detail	
Byte	[IN] aidLen Length of AID.
byte[]	[IN] aid = new byte[16] AID.
Byte	[IN] kernelIdLen Length of Kernel ID.
byte[]	[IN] kernelId = new byte[8] Kernel ID.
Byte	[IN] transactionType Transaction type.
Int	[IN] tagDataLen Length of tag data.
byte[]	[IN] tagData Tag data (format is TLV1 + TLV2 + …+ TLVn).

Castles Technology

4.1.1.8 Class: EMVCLAidGetTagData

public class **EMVCLAidGetTagData**

Fields Detail	
Byte	[IN] aidLen Length of AID.
byte[]	[IN] aid = new byte[16] AID.
Byte	[IN] kernelIdLen Length of Kernel ID.
byte[]	[IN] kernelId = new byte[8] Kernel ID.
Byte	[IN] transactionType Transaction type.
Int	[OUT] tagDataLen Length of tag data.
byte[]	[OUT] tagData Buffer to get tag data (format is TLV1 + TLV2 + …+ TLVn).

Castles Technology

4.1.1.9 Class: EMVCLAidCombinationList

public class **EMVCLAidCombinationList**

Fields Detail	
int	[OUT] version Version. Fixed to 0x01.
int	[OUT] combinationNum AID.
byte	[OUT] aidLen = new byte[256] Length of AID.
byte[]	[OUT] aid = new byte[256][16] Kernel ID.
byte	[OUT] kernelIdLen = new byte[256] Length of Kernel ID.
byte[]	[OUT] kernelId = new byte[256][8] Kernel ID.
byte	[OUT] transactionType = new byte[256] Transaction type.

Castles Technology

4.1.1.10 Class: EMVCLCAPublicKey

public class **EMVCLCAPublicKey**

Fields Detail	
byte	[IN] action Action. 0x00 –Add one CAPK. 0x01 –Delete one CAPK. 0x02 –Delete All CAPKs.
byte	[IN] index CAPK Index.
int	[IN] modulusLen Length of modulus.
byte[]	[IN] modulus = new byte[248] Modulus.
int	[IN] exponentLen Length of exponent.
byte[]	[IN] exponent = new byte[3] Exponent.
byte[]	[IN] hash = new byte[20] Hash.

Castles Technology

4.1.1.11 Class: EMVCLParameterData

public class **EMVCLParameterData**

Fields Detail	
int	num Number of parameters.
int[]	index = new int[100] List of parameter index.
int[]	len = new int[100] List of length of parameter data.
byte[][]	data = new byte[100][20] List of parameter data.

Castles Technology

4.1.1.12 Class: EMVCLUIReqData

public class **EMVCLUIReqData**

Fields Detail	
byte	[IN] messageIdentifier Indicates the text string to be displayed.
byte	[IN] status Identifies the status of the transaction (for example when the card can be removed).
byte[]	[IN] holdTime = new byte[3] If provided, indicates that the reader is to delay the processing of the Message Identifier in the next User Interface Request until the Hold Time has elapsed. The Hold Time is an integer in units of 100ms; the default value is zero.
byte[]	[IN] languagePreference = new byte[8] If Language Preference (as per EMV Tag '5F2D' according to ISO 639-1) is present and if the language is supported by the reader or terminal, then the message identified and all subsequent messages to the cardholder are displayed in this language until the transaction concludes.
byte	[IN] valueQualifier Value Qualifier is defined as either "Amount" or "Balance".
byte[]	[IN] Value = new byte[6] If Value Qualifier is present and equals "Amount" or "Balance", the Value will be displayed as an amount or balance in conjunction with the message. The representation will be according to the Currency Code with leading zeros suppressed whilst leaving at least one significant digit (so that an amount of 0.09 would be shown as 0.09 and not as .09, 09, or 9).
byte[]	[IN] currencyCode = new byte[2] Numeric value as per ISO 4217.

Castles Technology

4.1.1.13 Class: EMVCLSecureDataInfo

public class **EMVCLSecureDataInfo**

Fields Detail	
int	[IN] version Version. Set to 0x04 which is the latest supported version.
byte	[IN] keyType Key type. 0x01 – 3DES 0x02 – 3DES_DUKPT 0x03 – AES 0x07 – AES_DUKPT
int	[IN] cipherKeySet Key set.
int	[IN] cipherKeyIndex Key index.
byte	[IN] cipherMethod Cipher method. 0x00 – ECB 0x01 – CBC
byte[]	[IN] ICV ICV. Used when cipher method is CBC.
byte	[IN] ICVLen Length of ICV. 8 – When key type is 3DES or 3DES_DUKPT 16 – When key type is AES or AES_DUKPT.
byte[]	[IN] SK Session key
byte	[IN] SKLen Length of session key
byte	[IN] checksumType Checksum type. 0x00 – SHA1 0x01 – MAC 0x02 – SHA256 0xFF – NONE
byte	[IN] paddingMethod Padding method. 0x00 – Padding 0x00. 0x01 – Padding 0xFF. 0x02 – Padding PKCS#7. 0x03 – Padding ISO/IEC 7816-4.(Applicable to version 4)
byte	[IN] LRCIncluded

Castles Technology

	Applicable to track data only. LRC is included in track data or not. 0x01 – Include LRC.
byte	[IN] SS_ESIncluded Applicable to track data only. SS (Start sentinel) and ES (End sentinel) are included in track data or not. 0x01 – Include SS ES.
byte	[IN] isKSNFixed Indicate whether to increase the KSN and generate the session key or not. If this field is TRUE, the KSN won't be increased and session key won't be re-generated, but it is required the session key already generated before.
int	[IN] derivedKeyType Derived key type for AES DUKPT. 0x00 – 2TDEA 0x01 – 3TDEA 0x02 – AES128 0x03 – AES192 0x04 – AES256
int	[IN] derivedKeyUsage Derived key usage for AES DUKPT. 0x05 – DataEncryptionEncrypt 0x06 – DataEncryptionDecrypt 0x07 – DataEncryptionBothWays
byte	[IN] macPara Use parameters macXXX for generating MAC. This item is applicable if checksum type is set to MAC. 0x00 – Generate MAC by using above parameters 0x01 – Generate MAC by using below macXXX paramaters
int	[IN] macCipherKeySet Key set used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01.
int	[IN] macCipherKeyIndex Key index used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01.
byte	[IN] macCipherMethod Cipher method used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01. 0x00 – MAC_METHOD_CBC 0x01 – MAC_METHOD_X9_19

Castles Technology

	0x02 – MAC_METHOD_X9_19_START 0x03 – MAC_METHOD_X9_19_UPDATE 0x04 – MAC_METHOD_X9_19_FINAL
byte[]	[IN] macICV ICV used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01.
byte	[IN] macICVLen Length of ICV used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01. 8 – when key type is 3DES or 3DES_DUKPT 16 – when key type is AES or AES_DUKPT
byte	[IN] macIsKSNFixed Indicate whether to increase the KSN used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01.
int	[IN] macDerivedKeyTyp Derived key type used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01. 0x00 – 2TDEA 0x01 – 3TDEA 0x02 – AES128 0x03 – AES192 0x04 – AES256
int	[IN] macDerivedKeyUsage Derived key usage used for generating MAC. This item is applicable if checksum type is set to MAC and macPara is set to 0x01. 0x02 – MessageAuthenticationGeneration 0x03 – MessageAuthenticationVerification 0x04 – MessageAuthenticationBothWays

Castles Technology

4.1.1.14 Class: EMVCLAppInfo

public class **EMVCLAppInfo**

Fields Detail	
Int	[IN] version Version 0x02 or 0x03 or 0x04
byte[]	[IN] appLabel Application Label.
byte	[IN] appLabelLen Length of Application Label.
byte[]	[IN] aid AID.
byte	[IN] aidLen Length of AID
boolean	[IN] issuerIdentificationNumberPresented true – Issuer identification number is presemted. false –Issuer identification number is not presemted.
byte[]	[IN] issuerIdentificationNumber Issuer Identification Number.
boolean	[IN] issuerCountryCodeAlpha2Presented true – Issuer country code alpha 2 is presented. false – Issuer country code alpha 2 is not presented.
byte[]	[IN] issuerCountryCodeAlfa2 Issuer Country Code Alfa 2.
byte	[IN] applicationPriorityIndicator Application Priority Indicator. **[This item is applied for ver 3 or above]**
boolean	[IN] bKernelIdentifierPresented true – Kernel identifier is presemted. false – Kernel identifier is not presemted. **[This item is applied for ver 4 or above]**
byte[]	[IN] baKernelIdentifier Kernel identifier. **[This item is applied for ver 4 or above]**
byte	[IN] bKernelIdentifierLen Length of Kernel identifier. **[This item is applied for ver 4 or above]**
boolean	[IN] bASRPDPresented true – ASRPD is presemted. false –ASRPD is not presemted. **[This item is applied for ver 4 or above]**
byte[]	[IN] baASRPD Application Selection Registered Proprietary Data. **[This item is applied for ver 4 or above]**

Castles Technology

byte	[IN] bASRPDLen Length of ASRPD. **[This item is applied for ver 4 or above]**
byte[]	[IN] baDirEntry entry data of candidate. **[This item is applied for ver 4 or above]**
byte	[IN] usDirEntryLen Length of DirEntry. **[This item is applied for ver 4 or above]**

Castles Technology

4.1.1.15 Class: EMVCLAppListV2

public class **EMVCLAppListV2**

Fields Detail	
int	[IN] version Version. Fixed to 0x00.
int	[IN] appNum Number of the application (candidate).
EMVCLAppInfo[]	[IN] appInfo Array of application information. Below items will be filled in. version=2 appLabel appLabelLen aid aidLen issuerIdentificationNumberPresented issuerIdentificationNumber issuerCountryCodeAlpha2Presented issuerCountryCodeAlpha2
int	[OUT] appSelectedIndex selected application index, range of value is 0 to AppNum-1.

Castles Technology

4.1.1.16 Class: EMVCLByteBufData

public class **EMVCLByteBufData**

Fields Detail	
byte[]	dataBuf Byte array buffer for storing data.
int	dataLen Data length.

Castles Technology

4.1.1.17 Class: EMVCLCardAcquisitionData

public class **EMVCLCardAcquisitionData**

Fields Detail	
EMVCLByteBufData	[OUT] tlv Include all Tags. Format: tlv + tlv + tlv …
EMVCLByteBufData	[OUT] aid AID.
EMVCLByteBufData	[OUT] pan PAN.
EMVCLByteBufData	[OUT] issuerCountryCode Issuer Counrty Code.

Castles Technology

4.1.1.18 Class: EMVCLConfigData

public class **EMVCLConfigData**

Fields Detail	
int	[IN] version Version.Fixed to 0x00.
String	[IN] configFilename File name of configuration file.
byte	[IN] isSelActive Set to TRUE to Selcet Active.
byte	[IN] selIndex Select index if isSelActive is FALSE
byte	[IN] action RFU.

Castles Technology

4.1.1.19 Class: EMVCLAppListV3

public class **EMVCLAppListV3**

Fields Detail	
int	[IN] version Version. Fixed to 0x00.
int	[IN] appNum Number of the application (candidate).
EMVCLAppInfo[]	[IN] appInfo Array of application information. Below items will be filled in. version=3 appLabel appLabelLen aid aidLen issuerIdentificationNumberPresented issuerIdentificationNumber issuerCountryCodeAlpha2Presented issuerCountryCodeAlpha2 applicationPriorityIndicator
int	[OUT] appSelectedIndex selected application index, range of value is 0 to AppNum-1.

Castles Technology

4.1.1.20 Class: EMVCLAppListV4

public class **EMVCLAppListV4**

Fields Detail	
int	[IN] version Version. Fixed to 0x00.
int	[IN] appNum Number of the application (candidate).
EMVCLAppInfo[]	[IN] appInfo Array of application information. Below items will be filled in. version=4 appLabel appLabelLen aid aidLen issuerIdentificationNumberPresented issuerIdentificationNumber issuerCountryCodeAlpha2Presented issuerCountryCodeAlpha2 applicationPriorityIndicator bKernelIdentifierPresented baKernelIdentifier bKernelIdentifierLen bASRPDPresented baASRPD bASRPDLen baDirEntry usDirEntryLen
int	[OUT] appSelectedIndex selected application index, range of value is 0 to AppNum-1.

Castles Technology

5. EMVEDL

5.1 CAEMVEDL Application Programming Interfaces

Package CTOS
Contains all of the classes for Castles EMVCL user API.

Class Summary	
CtEMVEDL	This class provides the extension functionality of EMV and EMVCL kernels. The purpose is to have common setting/methods used in all the payment entries (e.g. contact chip, contactless chip, msr, manua; entry). For example, the setting of white list in CtEMVEDL is applied to CtEMV, CtEMVCL, CtEMVMSR, CtEMVManualEntry.

Class Summary	
EMVEDLBinInfo	This class consists of a set of variables and is used for getting matched white list information from the white list setting file.
EMVEDLEncryptedData	This class consists of a set of variables and is used for getting encrypted data.

Castles Technology

5.1.1 Class: CtEMVEDL

public class **CtEMVEDL**
The **CtEMVEDL** class provides the extension functionality of EMV and EMVCL kernels. The purpose is to have common setting/methods used in all kind of payment entry (e.g. contact chip, contactless chip, msr, manua; entry). For example, the setting of white list in CtEMVEDL is applied to CtEMV, CtEMVCL, CtEMVMSR, CtEMVManualEntry.

Constructor Summary	
public	CtEMVEDL() Constructs a binder connected with service.

Method Summary	
int	Initialize () Initalize CtEMVEDL environment.
int	setWhiteListFile (InputStream inStream) Send the content of white list file to EMVEDL kernel in the secure module. The EMVEDL kernel will store the setting. Note. The white list setting is applied to all the payment entries, CtEMV, CtEMVCL, CtEMVMSR, CtEMVManalEntry.
int	deleteWhiteListFile () Delete the white list setting.
int	getWhiteListMatchedBinInfo (EMVEDLBinInfo binInfo) Get matched white list information from the white list setting.
int	setEncMode (int encMode) Enc method. Set encryption mode. Note. Enc (Encryption) methods : Per the customer's request to implement the proprietary encryption process. Encrypt data included secured tag (PAN and track) with specific format.
int	setEncKey (int keyset, int keyindex) Enc method. Set encryption key.
int	setCipherMethod (byte cipherMethod) Enc method. Set cipher method.
int	setICV (byte[] ICV, int ICVLen) Enc method. Set ICV.
int	setSessionKey (byte key[], int keyLen) Enc method. Set session key.
int	setTransportKey (byte key[], int keyLen) Enc method. Set transport key.
int	requestSensitiveEmvData (int numOfTags, int tagArray[])

Castles Technology

	Enc method. Request sensitive EMV data.
int	requestToken (int numOfTokens, int tokenArray[]) Enc method. Request token.
int	getEncryption (EMVEDLEncryptedData encData) Enc method. Get encrypted data.
int	changeWDBPath (String path) The EMVDL kernel default set the working database path to the Android Application ID and store the files such as the JSON white list file in it. Each Android App has its seting. This method is used to change the working database path If the developer would like to share the configurations between multiple Android App.
int	getWDBPath () Get the current working database path. If the developer does not change WDB path, the default working database path is the string of Android Application ID.
int	resetWDBPath () Reset the working database path to default.
String	getRtnString () This method is used after specific methods to get String variable.

Constructor Detail
public CtEMVEDL() Create a CtEMVEDL object and connect with binder to invoke service.

Method Detail
public int initialize() This method initializes the EMVEDL environment. **Parameters:** None **Returns:** 0x00 – Success. Other – Error code.
public int setWhiteListFile(InputStream stream); Send the content of white list file to EMVEDL kernel in the secure module. The EMVEDL kernel will store the setting. Note. The white list setting is applied to all the payment entries, CtEMV, CtEMVCL, CtEMVMSR, CtEMVManalEntry.

Castles Technology

Parameters:
[IN] instream – Stream of the white list file.
Returns:
0x00 – Success.
Other – Error code.

public int deleteWhiteListFile()

Delete the white list setting.
Parameters:
None
Returns:
0x00 – Success.
Other – Error code.

public int getWhiteListMatchedBinInfo(EMVEDLBinInfo binInfo)

After transaction with CtEMV or CtEMVCL or CtEMVMSR or CtEMVManalEntry, call this method to check if the PAN is matched to data in the white list file. Get matched white list information from the white list setting if the result is matched.
Parameters:
[OUT] binInfo – Information about white list checking status and result.
Returns:
0x00 – Success.
Other – Error code.

public int setEncMode(int encMode)

Enc method. Set encryption mode.
Note. Enc (Encryption) methods : Per the customer's request to implement the proprietary encryption process. Encrypt data included secured tag (PAN and track) with specific format.
The encryption is applied to all the payment entries, CtEMV, CtEMVCL, CtEMVMSR, CtEMVManalEntry. Collecting requested data from payment entries, packing the data to required format and encrypting them with below Enc parameters.
Parameters:
[IN] encMode – Encryption Mode.
0x01 – Cayan P2PE
Other – RFU
Returns:
0x00 – Success.

public int setEncKey(int keyset, int keyindex)

Enc method. Set encryption key.

Castles Technology

Parameters:
[IN] keyset – Key set.
[IN] keyindex – Key index.
Returns:
0x00 – Success.

public int setCipherMethod(byte cipherMethod)

Enc method. Set cipher method.
Parameters:
[IN] cipherMethod – Cipher method.
0x00 – ECB
0x01 – CBC
Returns:
0x00 – Success.

public int setICV(byte[] ICV, int ICVLen)

Enc method. Set ICV.
Parameters:
[IN] ICV – ICV. Used when cipher method is CBC
[IN] ICVLen – ICV Length. Length of ICV.
8 – When key type is 3DES or 3DES_DUKPT.
16 – When key type is AES.
Returns:
0x00 – Success.

public int setSessionKey(byte key[], int keylen)

Enc method. Set session key.
Parameters:
[IN] key – Session key.
[N] keyLen – Session key length.
Returns:
0x00 – Success.

public int setTransportKey(byte key[], int keylen)

Enc method. Set transport key.
Parameters:
key –Transport key.
keyLen –Transport key length.
Returns:
0x00 – Success.

public int requestSensitiveEmvData(int numOfTags, int tagArray[])

Enc method. Request sensitive EMV data. The EMVEDL kernel will collect the request data from CtEMV or CtEMVCL or CtEMVMSR or

Castles Technology

CtEMVManualEntry.

Parameters:

[IN] numOfTags – Number of Tags.
[IN] tagArray – Array of request tags.
Ex :
tagArray[0] = 0x9F26;
tagArray[1] = 0x9F27;
…

Returns:

0x00 – Success.
Other – Error code.

---

public int requestToken(int numOfTokens, int tokenArray[])

Enc method. Request token.

For Cayan P2PE,
Token 0xC1 : Transport key
Token 0xC2 : track data
Token 0xC3 : Loyalty account data
Token 0xC4 : Sensitive EMV Data
Token 0xC5 : Account number (Manual Entry)
Token 0xC6 : Expiration date (Manual Entry)
Token 0xC7 : CVV (Manual Entry)
Token 0xC8 : Zip code (Manual Entry)

Parameters:

[IN] numOfTokens – Number of Tokens.
[IN] tokenArray – Array of token.

Returns:

0x00 – Success.
Other – Error code.

---

public int getEncryption(EMVEDLEncryptedData encData)

Enc method. Use above Enc parameters to encrypt data.

Parameters:

[OUT] encData – Data Structure to store Encrypted Data.

Returns:

0x00 – Success.
Other – Error code.

---

public int changeWDBPath(Strnig path)

The EMVDL kernel default set the working database path to the Android Application ID and store the files such as the JSON white list file in it. Each Android App has its seting.

Castles Technology

This method is used to change the working database path If the developer would like to share the configurations between multiple Android App.

Parameters:

[IN] path –Working database path.

Returns:

0x00 – Success.
Other – Error code.

public final int getWDBPath ()

Get the current working database path. If the developer does not change WDB path, the default working database path is the string of Android Application ID. After calling getWDBPath(), please call getRtnString() to get the working database path

Parameters:

None.

Returns:

0x00 – Success.
Other – Error code.

public final int resetWDBPath()

Reset the working database path to default.

Parameters:

None.

Returns:

0x00 – Success.
Other – Error code.

public final String getRtnString()

This method is used after specific methods to get String variable.

Parameters:

None

Returns:

value – Get a String variable.

Description of the content of the white list file

```
{
   "start":"47617300",          //start of bin range : if PAN is within the bin range, the following item will be applied
   "end":"47617399",              //end of bin range
   "brand" : "VISA",             //information for brand. kernel don't check this item
   "type"    : "Credit",         //information for type of this bin range. kernel don't check this item
```

Castles Technology

```
   "cfg" : {
      "length": "15:19",        //Valid range of PAN length. min digit : max
digit.
                                //This is only for use in manual data
entry

      "format": "clear",        //two option : clear and cipher
                                //clear : secure data return in plain-text
format
                                //cipher : secure data should be
encrypted

      "mask"    : "6:4:7",      //masked format
                                //first digit indicates how many digits
must be open on masked PAN/Track data
                                //second digit indicates how many digits
must be open at the end of the PAN
                                //third digit indicates how many digits
must be open in the track after the delimiter

//Example: ;123456XXXXXX3445=9901111XXXXXXXXXXXXX?*

      "manual": {               //This is only for use in manual data entry
      "expdate" :     {
                        "prompt":true,      //true (prompt) or false (not
prompt)
                        "length":"4:4",     //require digit for expdate. min
digit : max digit
                        "format":"MMYY"     //information for format of
expdate. kernel don't check this item
                        "label":"EXP"       //information for label shown on
screen. kernel don't check this item
                        },
      "cvv" :           {
                        "prompt":true,      //true (prompt) or false (not
prompt)
                        "length":"3:3",     //require digit for cvv. min digit :
max digit
                        "format":"n"        //information for format of cvv.
kernel don't check this item
                        "label":"CVV"       //information for label shown on
screen. kernel don't check this item
                        },
      "post-code" : {
```

Castles Technology

```
                        "prompt":true,       //true (prompt) or false (not
prompt)
                        "length":"3:5",     //require digit for post code. min
digit : max digit
                        "format":"n",        //information for format of post
cpde. kernel don't check this item
                        "label":"ZIP"        //information for label shown on
screen. kernel don't check this item
                      },
      "addr" :       {
                        "prompt":true,       //true (prompt) or false (not
prompt)
                        "length":"3:20",     //require digit for addr. min digit :
max digit
                        "format":"ans",     //information for format of addr.
kernel don't check this item
                        "label":"…"         //information for label shown on
screen. kernel don't check this item
                        }
                }
        }
},
```

Castles Technology

5.1.1.1 Class: EMVEDLBinInfo

public class **EMVEDLBinInfo**

Fields Detail	
boolean	[OUT] isMatch the PAN in current transaction is matched to setting in the white list file. isMatch is set to true if PAN is within the bin range. true – Matched. Below data will be filled in. false – Not matched. Below data will not be filled in.
int	[OUT] binLen Length of bin.
byte[]	[OUT] binStart Start of bin range.
byte[]	[OUT] binEnd End of bin range.
int	[OUT] brandLen Length of brand.
byte[]	[OUT] brand Information for brand.
int	[OUT] typeLen Length of type.
byte[]	[OUT] type Information for type of this bin range.

boolean	[OUT] gotCipher
	The “format” of white list file is set or not.
	"format": "clear", //two option : clear and cipher
	//clear : secure data return in plain-text format
	//cipher : secure data should be encrypted
	true – set.
	false – Not set. In this case, item isCipher is set to true by default.

boolean	[OUT] isCipher When item isMatch is true, the secured tags are cipher or not. true – The secured tags are cipher. false – The secured tags are allowed to be returned in plaintext format.
boolean	[OUT] gotPanLen The “length” of white list file is set or not. "length": "15:19", //Valid range of PAN length. min digit : max digit.

Castles Technology

	//This is only for use in manual data entry true – set. false – Not set.
int	[OUT] panLenMin Minimum of valid PAN length. "length": "15:19" //"length":"min:max"
int	[OUT] panLenMax Maximum of valid PAN length. "length": "15:19" //"length":"min:max"
boolean	[OUT] invalidPanLen true – Length of PAN is invalid.

boolean	[OUT] gotMaskDigit
	The “mask” of white list file is set or not.
	"mask" : "6:4:7", //masked format
	//first digit indicates how many digits must be open on masked PAN/Track data
	//second digit indicates how many digits must be open at the end of the PAN
	//third digit indicates how many digits must be open in the track after the delimiter
	
	//Example: ;123456XXXXXX3445=9901111XXXXXXXXXXXXXX?*
	true – set.
	false – Not set.

int	[OUT] maskDigit1_BeginOfPan The first digit indicates how many digits must be open on masked PAN/Track data.
int	[OUT] maskDigit2_EndOfPan The second digit indicates how many digits must be open at the end of the PAN.
int	[OUT] maskDigit2_AfterDelimiter The third digit indicates how many digits must be open in the track after the delimiter.
boolean	[OUT] gotExpdate The “expdate” of white list file is set or not. "expdate" : { "prompt":true, //true (prompt) or false (not prompt) "length":"4:4", //require digit for expdate. min digit : max digit "format":"MMYY" //information for format of expdate. kernel don't check this item

Castles Technology

	"label":"EXP" //information for label shown on screen. kernel don't check this item }, true – set. false – Not set.
Boolean	[OUT] expdatePrompt For manual entry only, prompt to get expdate.
int	[OUT] expdateLenMin For manual entry only, required min digit for expdate.
int	[OUT] expdateLenMax For manual entry only, required max digit for expdate.
int	[OUT] expdateFormatLen For manual entry only. Length of format of expdate.
byte[]	[OUT] expdateFormat For manual entry only. Information for format of expdate.
int	[OUT] expdateLabelLen For manual entry only. Length of label of expdate
byte[]	[OUT] expdateLabel For manual entry only. Information for label shown on screen.
boolean	[OUT] gotCVV The “cvv” of white list file is set or not. "cvv" : { "prompt":true, //true (prompt) or false (not prompt) "length":"3:3", //require digit for cvv. min digit : max digit "format":"n" //information for format of cvv. kernel don't check this item "label":"CVV" //information for label shown on screen. kernel don't check this item }, true – set. false – Not set.
boolean	[OUT] cvvPrompt For manual entry only, prompt to get cvv.
int	[OUT] cvvLenMin For manual entry only, required min digit for cvv.
int	[OUT] cvvLenMax For manual entry only, required max digit for cvv.
int	[OUT] cvvFormatLen For manual entry only. Length of format of cvv.
byte[]	[OUT] cvvFormat For manual entry only. Information for format of cvv.

Castles Technology

int	[OUT] cvvLabelLen For manual entry only. Length of label of cvv.
byte[]	[OUT] cvvLabel For manual entry only. Information for label shown on screen.

boolean	[OUT] gotPostcode
	The “post-code” of white list file is set or not.
	"post-code" : {
	"prompt":true, //true (prompt) or false (not prompt)
	"length":"3:5", //require digit for post code. min digit : max digit
	"format":"n", //information for format of post cpde. kernel don't check this item
	"label":"ZIP" //information for label shown on screen. kernel don't check this item
	}
	true – set.
	false – Not set.

boolean	[OUT] postcodePrompt For manual entry only, prompt to get postcode.
int	[OUT] postcodeLenMin For manual entry only, required min digit for postcode.
int	[OUT] postcodeLenMax For manual entry only, required max digit for postcode.
int	[OUT] postcodeFormatLen For manual entry only. Length of format of postcode.
byte[]	[OUT] postcodeFormat For manual entry only. Information for format of postcode.
int	[OUT] postcodeLabelLen For manual entry only. Length of label of postcode.
byte[]	[OUT] postcodeLabel For manual entry only. Information for label shown on screen.
boolean	[OUT] gotAddr The “addr” of white list file is set or not. "addr" : { "prompt":true, //true (prompt) or false (not prompt) "length":"3:20", //require digit for addr. min digit : max digit "format":"ans", //information for format of addr. kernel don't check this item "label":"…" //information for label

Castles Technology

	shown on screen. kernel don't check this item } true – set. false – Not set.
boolean	[OUT] addrPrompt For manual entry only, prompt to get addr.
int	[OUT] addrLenMin For manual entry only, required min digit for addr.
int	[OUT] addrLenMax For manual entry only, required max digit for addr.
int	[OUT] addrFormatLen For manual entry only. Length of format of addr.
byte[]	[OUT] addrFormat For manual entry only. Information for format of addr.
int	[OUT] addrLabelLen For manual entry only. Length of label of addr.
byte[]	[OUT] addrLabel For manual entry only. Information for label shown on screen..

Castles Technology

5.1.1.2 Class: EMVEDLEncryptedData

public class **EMVEDLEncryptedData**

Fields Detail	
int	[OUT] encryptedDataLen Length of encrypted data.
byte[]	[OUT] encryptedData Encrypted data.
int	[OUT] KSNLen Length of KSN.
byte[]	[OUT] KSN KSN.

Castles Technology

6. Appendix

6.1 Error Code List

6.1.1 System、EMV and security error code

#define d_FRD_ACCESS_EXTERNAL_DEVICE_NOT_PERFORMED	0x9001
#define d_FRD_FUNCTION_NOT_SUPPORT	0x9002
#define d_FRD_DESCRYPT_DATA_EXCEED_BUF	0x9004
#define d_FRD_COMMADN_ID_NOT_FOUND	0x9005
#define d_FRD_PACK_CMD_DATA_OVERFLOW	0x9006
#define d_FRD_PACK_DLE_PROTOCOL_DATA_OVERFLOW	0x9007
#define d_FRD_RX_USER_CANCEL	0x9008
#define d_FRD_RX_TIMEOUT	0x9009
#define d_FRD_RX_DATA_OVERFLOW	0x900A
#define d_FRD_PROTOCOL_INVALID	0x900B
#define d_FRD_RX_DATA_PROTOCOL_FORMAT_ERROR	0x900C
#define d_FRD_RX_PARSE_DATA_OVERFLOW	0x900D
#define d_FRD_RSP_DATA_LEN_ERROR	0x900E
#define d_FRD_RSP_DATA_CMD_ID_ERROR	0x900F
#define d_FRD_RSP_DATA_SN_ERROR	0x9010
#define d_FRD_RSP_DATA_VERSION_ERROR	0x9011
#define d_FRD_RSP_DATA_OVERFLOW	0x9012
#define d_FRD_INIT_AUTHEN_KEY_TYPE_NOT_MATCH	0x9013
#define d_FRD_MUTUAL_AUTH_PROCESS_NOT_PERFORMED	0x9014
#define d_FRD_INIT_AUTHEN_CRYPTOGRAM_ERR	0x9015
#define TUAL_AUTHEN_SESSION_KEY_BUF_OVERFLOW	0x9016
#define d_FRD_DEVICE_PARAMETER_NOT_SUPPORT	0x9017
#define d_FRD_TK_CERT_NOT_EXIST	0x9018
#define d_FRD_PARAMETER_INVALID	0x9019

Castles Technology

#define d_FRD_INPUT_DATA_SIZE_EXCEED	0x901A
#define d_FRD_EVENT_CMD_ID	0x901B
#define d_FRD_EVENT_CMD_CANCEL	0x901C
#define d_FRD_CONNECT_NOT_READY	0x901D
#define d_FRD_REBOOT_BUSY	0x901F
#define d_FRD_LOADING_BUSY	0x9020
#define d_FRD_PRINTER_BUSY	0x9021
#define d_FRD_SC_FAIL	0x9101
#define d_FRD_SC_INEFFECTIVE_DATA	0x9102
#define d_FRD_SC_AUTH_NOT_PERFORMED	0x9103
#define d_FRD_SC_AUTH_FAIL	0x9104
#define d_FRD_SC_INEFFECTIVE_PARAMETER	0x9105
#define d_FRD_SC_INEFFECTIVE_MODE	0x9106
#define d_FRD_SC_INEFFECTIVE_CMD	0x9107
#define d_FRD_TASK_BUSY	0x9108
#define d_FRD_RX_BUFFER_OVERFLOW	0x9109

Castles Technology